



Final year project Thesis

To obtain the diploma of : **Engineering degree in Computer Science**

Major: **Intelligent Systems and Data (ISD)**

Theme

Traffic Carbon Footprint Estimation And Forecasting using Federated Learning on The Edge

Realized By
CHOUIKRAT Sabrina
KARIM Meryem Batoul

Supervised By

M. Leila HAMDAD
Mr. Smail NIAR
Mr. Youcef IMINE
Mr. Hamza OUARNOUGH
Mr. Abdelghani BEKRAR

Lcsi(ESI)
LAMIH(UPHF)
LAMIH(UPHF)
LAMIH(UPHF)
LAMIH(UPHF)

Academic Year: 2024/2025

Dedication

To my parents, who have always been the guiding light in my life.
To my father, whom I have always regarded as my greatest role model — your unwavering faith in me has given me the strength to persevere and strive for excellence.
And to my mother, whose boundless love, care, and encouragement — together with her constant prayers — have been my deepest source of comfort and inspiration throughout every stage of this endeavour.

To my siblings, Hiba, Salah, and Romyssa — not only my family, but also my truest friends and steadfast supporters. Thank you for always standing by my side, sharing in both my hardships and joys, and lifting me when I needed it most. Your presence made each step feel lighter.

To my project partner, Sabrina — thank you for sharing this work, the effort, and the moments that made it truly meaningful. Your support, presence, and kindness made all the difference.

To all my dear friends — I would not be who I am today without you.

Finally, I dedicate this work to all of you. Thank you — words can never fully express the depth of gratitude I carry in my heart.

From Meryem

Dedication

To my mother, my greatest source of strength. Your unwavering presence—physically, mentally, and spiritually—has been my greatest source of strength. You built my confidence, ensured my comfort, and stood by me in every endeavor.

To my father, whose tireless support provided the stability upon which I could build my dreams.

Together, you both always prioritized my needs above your own, a sacrifice I can never fully repay. I would not be the person I am today without you. More than anything, it is your example—watching you face life’s challenges with grace and gratitude—that has been my greatest inspiration and the true reason I have reached this milestone.

To my brother, for believing in me with a certainty I often lacked myself, for dismissing my biggest fears with a laugh, and for always reminding me that I was capable of not just succeeding, but excelling.

To my extended family—my grandmothers, aunts, uncles, and cousins. Thank you for your constant encouragement and for sharing in the joy of my achievements. Knowing I have your support has meant so much.

To my project partner, Meryem. Thank you for an unforgettable journey. For the shared laughter that lightened the hardest days, and for the resilience that saw us through every challenge. I am so grateful for this experience and for the chance to have grown alongside you. I could not have asked for a better partner.

To my friends, who stood by me through the most challenging and life-changing years, sharing laughter and resilience.

And to everyone who helped along the way, even with just a word.

I dedicate this work to all of you. Your support made it possible and shaped it into what it is. Thank you, I am forever grateful.

From Sabrina

Acknowledgement

First and foremost, we would like to express our deepest gratitude to Allah for granting us the strength, patience, and perseverance to complete this work.

We are sincerely grateful to our supervisor at ESI, Mrs. Leila Hamdad, for her continuous guidance throughout all stages of this project. Her insightful advice, unwavering support, and constant availability were invaluable in shaping and refining our solution.

We would also like to extend our heartfelt thanks to our supervisors at LAMIH — Prof. Smail Niar, Dr. Youcef Imine, Mr. Hamza Ouarnoughi, and Mr. Abdelghani Bekrar — for warmly welcoming us into the university laboratory and for offering us this enriching opportunity. Their attentive supervision and constructive feedback during the internship made this experience highly instructive and rewarding.

A special note of appreciation goes to Prof. Aït El Kadi from LAMIH, who kindly took the time to listen to our ideas and challenges, and provided us with valuable guidance to overcome them.

Finally, we would like to thank all the professors at ESI who have contributed to our academic journey — from the preparatory cycle to the engineering cycle. Their dedication and knowledge have laid the foundation for our growth .

Abstract

Reducing emissions from road transport is a key challenge in the pursuit of more sustainable and livable cities. As urban areas grow and traffic congestion intensifies, understanding carbon emissions at street-level granularity becomes essential for optimizing infrastructure, improving air quality, and informing policy. Accurate, real-time quantification and prediction of traffic-related emissions can enable data-driven decision-making and smarter traffic management. However, most existing solutions rely heavily on expensive, dedicated sensor deployments, overlooking one of the most widespread and cost-effective sources of data in modern cities: roadside surveillance cameras. The motivation for this work is to bridge this gap by developing an end-to-end framework that exploits this underutilized infrastructure for comprehensive, network-wide emission analysis. We propose a modular design that integrates a computer vision pipeline for per-vehicle emission estimation with an advanced spatio-temporal model for network-level forecasting. The vision pipeline automates the entire process from raw video to structured data, incorporating a fully automatic camera calibration module, fine grained vehicle classification, and robust tracking with occlusion handling. The resulting data feeds a spatio-temporal graph neural network that performs the dual tasks of forecasting future emissions and imputing data for unmonitored road segments.

Furthermore, we explore the integration of Federated Learning, a paradigm widely exploited in the broader traffic domain but still underexplored for emissions, to address key challenges in data privacy and scalability. The results of our study demonstrated the robustness of the proposed video pipeline, the necessity of task-specific calibration, and the effectiveness of clustered Federated Learning in overcoming the non-IID problem.

Keywords— Carbon Footprint Estimation, Traffic Emission Forecasting, Computer Vision, Spatio-temporal Modeling, Graph Neural Networks, Data Imputation, Federated Learning

Résumé

La réduction des émissions provenant du transport routier constitue un défi majeur dans la quête de villes plus durables et plus agréables à vivre. À mesure que les zones urbaines s'étendent et que la congestion du trafic s'intensifie, la compréhension des émissions de carbone à une granularité de rue devient essentielle pour optimiser les infrastructures, améliorer la qualité de l'air et orienter les politiques publiques. Une quantification et une prévision précises et en temps réel des émissions liées au trafic peuvent permettre une prise de décision fondée sur les données ainsi qu'une gestion plus intelligente de la circulation. Cependant, la plupart des solutions existantes reposent fortement sur des déploiements coûteux de capteurs dédiés, négligeant l'une des sources de données les plus répandues et les plus économiques dans les villes modernes : les caméras de surveillance routières.

La motivation de ce travail est de combler cette lacune en développant un cadre de bout en bout qui exploite cette infrastructure sous-utilisée pour une analyse complète des émissions à l'échelle du réseau. Nous proposons une conception modulaire qui intègre un pipeline de vision par ordinateur pour l'estimation des émissions par véhicule, associé à un modèle spatio-temporel avancé pour la prévision des émissions au niveau du réseau. Le pipeline de vision automatise l'ensemble du processus, allant de la vidéo brute aux données structurées, en intégrant un module de calibration automatique des caméras, une classification fine des véhicules et un suivi robuste avec gestion des occultations. Les données ainsi obtenues alimentent un réseau de neurones graphiques spatio-temporel qui réalise deux tâches : la prévision des émissions futures et l'imputation des données pour les segments routiers non surveillés.

En outre, nous explorons l'intégration de l'Apprentissage Fédéré, un paradigme largement exploité dans le domaine du trafic mais encore peu étudié pour les émissions, afin de répondre aux défis clés liés à la confidentialité des données et à la scalabilité. Les résultats de notre étude ont démontré la robustesse du pipeline vidéo proposé, la nécessité d'une calibration spécifique aux tâches, ainsi que l'efficacité de l'Apprentissage Fédéré en clusters pour surmonter le problème des données non-IID.

Mots-clés : Estimation de l'empreinte carbone, prévision des émissions de trafic, vision par ordinateur, modélisation spatio-temporelle, réseaux de neurones graphiques, complétion de données, apprentissage fédéré

ملخص

يُعدّ تبيدّ تقليل الانبعاثات الناتجة عن النقل البري تحدياً أساسياً في السعي نحو مدن أكثر استدامة وقابلة للعيش. ومع توسّع المناطق الحضرية وازدياد ازدحام حركة المرور، تصبح القدرة على فهم انبعاثات الكربون بدقة على مستوى الشوارع أمراً ضرورياً لتحسين البنية التحتية، والارتقاء بجودة الهواء، وصياغة السياسات العامة. إن القياس والتنبؤ الدقيق والفوري بانبعاثات المرور يمكن أن يتيح اتخاذ قرارات مبنية على البيانات وإدارة أكثر ذكاءً لحركة السير. ومع ذلك، فإن معظم الحلول الحالية تعتمد بشكل كبير على نشر مجسات مخصصة ومكلفة، متجاهلةً أحد أكثر مصادر البيانات انتشاراً وفعالية من حيث التكلفة في المدن الحديثة: كاميرات المراقبة على جوانب الطرق.

يتمثل الدافع وراء هذا العمل في سدّ هذه الفجوة من خلال تطوير إطار متكامل يستغلّ هذه البنية التحتية غير المستغلة بشكل كافٍ لإجراء تحليل شامل للانبعاثات على مستوى الشبكة. نقترح تصميمًا معيارياً يدمج خط معالجة مرئية قائم على تقنيات الرؤية الحاسوبية لتقدير الانبعاثات لكل مركبة، مع نموذج زمني-مكاني متقدم للتنبؤ بالانبعاثات على مستوى الشبكة. يقوم خط المعالجة المرئية بأتمتة العملية بأكملها، ابتداءً من الفيديو الخام وصولاً إلى البيانات المنظمة، متضمناً وحدة معايرة تلقائية للكاميرات، وتصنيفاً دقيقاً للمركبات، وتعبئاً متيناً مع معالجة حالات الحجب. وتُغذي هذه البيانات شبكة عصبية بيانية زمنية-مكانية تنجز مهمتين مزدوجتين: التنبؤ بالانبعاثات المستقبلية واستكمال البيانات الخاصة بالمقاطع الطرقية غير المراقبة.

وعلاوةً على ذلك، نستكشف دمج التعلم الفيدرالي، وهو نموذج واسع الاستخدام في مجال المرور ولكنه ما يزال قليل الدراسة في مجال الانبعاثات، وذلك لمواجهة التحديات الأساسية المرتبطة بخصوصية البيانات وقابلية التوسّع. وقد أظهرت نتائج دراستنا متانة خط المعالجة المرئية المقترح، وضرورة إجراء معايرة مخصصة للمهام، فضلاً عن فعالية التعلم الفيدرالي العنقودي في تجاوز مشكلة البيانات غير المتجانسة (DII-non).

الكلمات المفتاحية: تقدير البصمة الكربونية، التنبؤ بانبعاثات المرور، الرؤية الحاسوبية، النمذجة المكانية-الزمانية، الشبكات العصبونية البيانية، استكمال البيانات، التعلم الفيدرالي

Contents

Dedication	i
Acknowledgement	iii
Abstract	iv
Résumé	v
iv	ملخص
List of Acronyms	xiii
General Introduction	1
I State of The Art	3
1 Emission Estimation: Context, Motivation, and Data Challenges	4
1.1 Introduction	4
1.2 GHG emissions	4
1.3 Motivation for using AI-based solutions For Emission Estimation	8
1.4 Traffic Emissions From Estimation to Optimization	8
1.4.1 Urban Planning and Traffic Management	9
1.4.2 Eco Routing Application	9
1.4.3 Optimized Traffic Light Management	10
1.5 Data Collection and Preprocessing	10
1.5.1 Data Sources and Methodologies for Emission Estimation	10
1.5.2 Data Collection Frequency and Duration	16
1.5.3 Data Prepossessing and feature engineering	17
1.6 Challenges in acquiring and preprocessing traffic Emission data	20
1.6.1 Data Availability and Accessibility	20
1.6.2 Heterogeneity of Data Sources and dependency	20
1.6.3 Dynamic Nature of Traffic Data	20
1.6.4 Sensor Reliability and Calibration	21
1.7 Conclusion	21
2 Foundations of Machine Learning and Deep Learning Models for Time Series Analysis	22
2.1 Introduction	22
2.2 Regression and Ensemble Learning	22
2.2.1 The Principle of Regression Analysis	23
2.2.2 What is ensemble learning	23

2.2.3	Architectures of Ensemble Learning	24
2.3	Time Series Modeling for Emission Forecasting	25
2.3.1	Time Series Definition and Formulation	25
2.3.2	Time Series Decomposition	26
2.3.3	Sequence-based deep learning models	27
2.3.4	The Transformer Architecture	33
2.4	Spatio-Temporal Modeling for Emission Estimation	36
2.4.1	Spatial Modeling in Deep Learning Models	36
2.4.2	Learning Paradigms in Graph Neural Networks	42
2.4.3	Spatio-Temporal Modeling: Concepts and Methods	42
2.5	Evaluation Metrics	45
2.5.1	Point-wise Error Metrics.	45
2.5.2	Distribution-based Metrics.	46
2.5.3	Temporal and Spectral Metrics.	46
2.5.4	Task-specific Metrics.	47
2.6	Conclusion	47
3	Applications of Machine and Deep Learning Models for Traffic Emission Estimation and Forecasting	49
3.1	Introduction	49
3.2	Empirical Studies by Model Family	49
3.2.1	ML-based Methods	50
3.2.2	DL-based Methods	53
3.3	General Comparaison between ML and DL methods used for Emission Estimation	63
3.4	Different factors driving traffic emission studies	64
3.5	Discussion and Research Orientations	65
3.6	Conclusion	65
II	Proposed Contribution	67
4	Solution design	68
4.1	Introduction	68
4.2	Motivation and Contribution	68
4.3	General Framework Description	70
4.3.1	Video processing and emission estimation Module	70
4.3.2	Spatio-temporal Module STAD-NET	82
4.3.3	Federated Learning Integration	89
4.4	Conclusion	91
5	Implementation	92
5.1	Introduction	92
5.2	Tools and Libraries Used	92
5.3	Implementation Environment	94
5.4	Video processing pipeline Implementation	94
5.4.1	Data Description and Preprocessing	94
5.4.2	Offline calibration	97
5.4.3	Online inference module	99
5.4.4	Emission estimation implementation	100
5.4.5	Data Aggregation for Forecasting.	101
5.5	Data Generation for STAD-NET Model and Preprocessing	101
5.5.1	Data generation description	102

5.5.2	Data Preprocessing	102
5.5.3	Sliding Window and Data Splitting	104
5.6	Spatio Temporal Model Implementation	104
5.6.1	Mask Generation	104
5.6.2	Model Architecture	105
5.6.3	Task Balancing and Loss Weighting	105
5.7	Federated Learning implementation	106
5.8	Conclusion	107
6	Tests and Solution Evaluation	108
6.1	Introduction	108
6.2	Video processing pipeline evaluation	108
6.2.1	Fine-Grained Classification Evaluation	108
6.2.2	Evaluating Camera Calibration	112
6.2.3	Speed Evaluation	114
6.3	Evaluation of STAD-NET Model	116
6.3.1	Impact of Missing Patterns and Hop Aggregation	116
6.3.2	Forecasting Performance Across Different Horizons	117
6.3.3	Evaluation Metrics for Imputation and Forecasting	118
6.3.4	Study and Comparaison of the Federated Learning approach	119
6.4	Conclusion	121
	General Conclusion	122
	Appendix	131
A	Vision and Geometry Essentials for Traffic Video Analysis	132
A.1	Introduction	132
A.2	Computer vision tasks	132
A.2.1	Image Classification	132
A.2.2	The Vision Transformer (ViT).	133
A.2.3	Object Detection	134
A.2.4	Multi-Object Tracking (MOT)	136
A.2.5	Additional Foundational Algorithms	137
A.3	Camera geometry	138
A.3.1	Camera Intrinsic and Characteristics	138
A.3.2	Perspective Correction: Restoring Metric Scale	139
A.4	Conclusion	142

List of Figures

1.1	2023 Global Greenhouse Gas Emission Levels.	5
1.2	Emissions by sector	6
1.3	Emissions by transport type	6
1.4	Carbon emissions distribution by (a) sector and (b) transport type for 2024 ((United Nations Environment Programme, 2024)).	6
1.5	ECO routing framework example using emission estimation.	9
1.6	Data Collection Methods For Emission Prediction Models.	11
1.7	Figure illustrating Types of V2X connectivity (Farnell, 2025)	14
1.8	Examples of the training approaches used in (Braun et al., 2024).	16
2.1	Ensemble families.	24
2.2	Example decomposition of a time series (Martinez, n.d.).	26
2.3	The structure of a single RNN cell (Amidi and Amidi, 2018).	27
2.4	An RNN reuses the same cell and weights across timesteps.(Amidi and Amidi, 2018)	28
2.5	The architecture of a single LSTM cell (Wikipedia contributors, n.d.).	28
2.6	Architecture of a Bidirectional LSTM.	30
2.7	The internal data flow of a Gated Recurrent Unit (GRU) cell (Wikipedia contrib- utors, n.d.).	31
2.8	Diagram illustrating how 1D convolution works.	32
2.9	Two-stage convolution highlighting the receptive field(from bottom to top). . . .	32
2.10	Dilated convolutions expand the receptive field by skipping inputs.	33
2.11	Scaled dot-product attention mechanism, as presented in (Vaswani et al., 2017). .	34
2.12	Multi-Head Attention as presented in (Vaswani et al., 2017)	34
2.13	The full Transformer architecture (Vaswani et al., 2017).	35
2.14	Undirected Graph.	37
2.15	Directed Graph.	37
2.16	Attributed Graph Representation	38
2.17	Attribute and Structure Dynamic Graphs.	39
2.18	Analogy between CNNs and GCNs (Pham, 2020).	41
2.19	Architecture of Graph WaveNet Model (Wu et al., 2019).	44
3.1	Model-centric taxonomy.	50
3.2	The architecture of the hierarchical heterogeneous graph learning method in the work of (Zeng et al., 2024)	61
4.1	General schema of the proposed framework.	70
4.2	Schema of the General video processing pipeline.	71
4.3	General schema of the calibration (offline) step.	71
4.4	General schema of the feature extraction(online) step.	72
4.5	Perspective transformation.(Kocur and Ftáčnik, 2020).	78
4.6	Example of an output of the modified Yolov6 -3D model.(Kocur and Ftáčnik, 2020). 79	

4.7	Construction of the 3D bounding box from the 3rd VP. (Kocur and Ftáčnik, 2020).	79
4.8	Road Network Graph representation from Roads to Line Graph.	83
4.9	illustration of K-hop neighbor aggregation in graphs while red nodes are missing.	85
4.10	architecture of the Multitask Forecasting Imputation Model.	86
5.1	Top-15 vehicle classes by image count (BoxCars, 107-class subset).	95
5.2	Example of a voting results in the diamond space.	97
5.3	Example result of filtered segments(orange) used for v vanishing point.	98
5.4	Examples of detection (top: 2D box + alpha in BEV) and reconstruction (bottom: 3D box projected onto image)	99
5.5	Road network from OpenStreetMap	102
5.6	SUMO traffic simulation scenario	102
5.7	Comparison of (a) the extracted road network from OpenStreetMap and (b) the simulated traffic scenario generated in SUMO.	102
5.8	Exemple of the Origin Destination Matrix.	103
5.9	Variation of imputation and forecasting losses over epochs using different loss weighting strategies, including fixed λ , confidence-based, and DWA.	106
6.1	Normalized confusion matrix for the DeiT-S classifier on the test set.	110
6.2	Normalized Imputation MAE per Node for Different Hop Aggregations.	117
6.3	Box Plot for Multi-step Ahead Forecasting Error Across Prediction Horizons. . .	117
6.4	True vs imputation values for node 84.	118
6.5	Bar Plot for Per Node RMSE Values for Forecasting Errors.	119
6.6	FedAvg Forecasting Loss	120
6.7	FedAvg Imputation Loss	120
6.8	Clustered FL Forecasting Loss	120
6.9	Clustered FL Imputation Loss	120
6.10	Heatmaps of client evaluation MSE across training rounds. (a)–(b) correspond to FedAvg, while (c)–(d) show the results for Clustered FL.	120
6.11	Client Gradient Projection Evolution Across Federation Rounds.	121
A.1	Example of a simple CNN classification architecture (GeeksforGeeks, 2024) . . .	133
A.2	A simple depiction of a vision transformer architecture.	134
A.3	Example of detection on a traffic image (YOLOv5 output)(GeeksforGeeks, 2024).	135
A.4	Representative illustration of a pinhole camera model(openMVG, n.d.).	138
A.5	illustrative figure of a vanishing point on the image plane vs. the base plane (world plane)) (Mian, n.d.).	142

List of Tables

1.1	Global GHG emission trends by sector and key years (European Commission, Joint Research Centre, 2024).	6
1.2	Change in emissions from 1990 to 2023 in different sectors in the EU (European Commission, Joint Research Centre, 2024).	7
1.3	A Categorized Overview of Features Influencing Traffic Emissions.	19
2.1	Comparison between Transductive and Inductive Learning in Graph Neural Networks	42
3.1	Ensemble studies in traffic-emission estimation: technique, data, and scope (simplified).	53
3.2	Summary of LSTM-based approaches for vehicle emissions prediction.	56
3.3	Categorization of 1D-CNN usage in temporal modeling for traffic emissions forecasting.	59
3.4	Comparative Analysis of Deep Learning Architectures for Traffic Emission Estimation	63
4.1	Comparison of Emission Modeling Frameworks	82
4.2	Summary of Vehicle Classes and Typical Emission Levels	84
5.1	BoxCars 107-class subset: imbalance summary.	95
5.2	List of augmentations used on BoxCars116k.	96
5.3	Fine grained classification models	100
5.4	Snippet of the file containing vehicle classes and their characteristics.	101
5.5	Summary of the model architecture and main hyperparameters.	105
5.6	Client datasets in federated learning, where each client is a subgraph with its feature and adjacency matrices.	107
5.7	Federated Learning Setup	107
6.1	Fine-grained classification performance on the test set.	109
6.2	Top 5 most frequent confusion pairs for the DeiT-S model.	111
6.3	The 5 classes with the lowest F1-scores and their primary misclassification.	111
6.4	Manual sanity check on the first 30 tracks from BrnoCompSpeed Session 1 (center view).	111
6.5	Representative calibration results across multiple sessions.	113
6.6	Distance error along the traffic direction ($\mathbf{u}$).	114
6.7	Tracking coverage per session.	115
6.8	Speed error statistics.	115
6.9	Metrics for Forecasting and Imputing CO2 Emissions	118
A.1	Popular CNN backbones in computer-vision pipelines	133
A.2	A few multi-object trackers	137
A.3	Common approaches to calibrate traffic cameras.	140

List of Acronyms

AADT	Annual Average Daily Traffic	61
ARIMA	AutoRegressive Integrated Moving Average	54
BiLSTM	Bidirectional Long Short-Term Memory	55
BPTT	Backpropagation Through Time	28
CCTV	Closed-Circuit Television	15
CH₄	Methane	5
ChebNet	Chebyshev Graph Convolutional Network	39
CNN	Convolutional Neural Network	31
EV	Electric Vehicle	7
F-gases	Fluorinated Gases	5
GAT	Graph Attention Network	41
GBR	Gradient Boosting Regression	51
GDP	Gross Domestic Product Per Capita	11
GHG	Greenhouse Gas	5
GMAN	Graph Multi-Attention Network	44
GPS	Global Positioning System	12
GRU	Gated Recurrent Unit	28
IoT	Internet of Things	10
IoV	Internet of Vehicles	13
IqR	Interquartile Range	17
MEC	Multi Access Edge Computing	14
MMD	Maximum Mean Discrepancy	13
MOVES	Motor Vehicle Emission Simulator	8
Mtoe	Million Tonnes of Oil Equivalent	11

NO₂ Nitrogen Dioxide 5

OBD On-Board Diagnostics 12

OSM OpenStreetMap 12

PCA Principal Component Analysis 19

PEMS Portable Emission Measurement System 12

RPM Revolutions Per Minute 18

RSU Roadside Unit 13

SOC State of Charge (Battery) 19

SSA Singular Spectrum Analysis 54

VK Vehicle Kilometer 11

VSP Vehicle Specific Power 8

General Introduction

In recent decades, pollution has become one of the most pressing challenges facing our world. Beyond the visible degradation of the environment, pollution—particularly air pollution—has profound consequences for human health, ecosystems, and overall quality of life. Among the many pollutants, greenhouse gases (GHGs), and more specifically CO₂, play a central role in climate change and global warming. CO₂ emissions are now saturating our atmosphere, contributing not only to the planet’s long-term heating but also to short-term urban air quality issues that affect millions daily.

A major contributor to this crisis is road traffic. Urban transport networks, constantly filled with vehicles, are responsible for a substantial share of harmful emissions. As cities grow more congested, this problem worsens. Thus, reducing traffic-related carbon emissions has become a vital objective—whether through long-term infrastructure and policy shifts or through short-term adaptive traffic control and mitigation strategies.

At the heart of such strategies lies the need for **accurate, real-time quantification and prediction of traffic CO₂ emissions**. Being able to measure emissions with high granularity and forecast them over both short and long timeframes is critical to guiding policy, optimizing traffic flow, informing urban planning, and improving public health outcomes.

In response, recent years have witnessed the emergence of various innovative approaches to traffic emission estimation. These solutions use diverse data sources—ranging from vehicle GPS traces, mobile sensors, and satellite imagery to air-quality monitoring stations—and a wide array of methods, including physics-based models, classical machine learning, and hybrid simulations. Some are designed to estimate emissions per vehicle in real time, while others target city- or network-level trends over longer horizons. However, many of these approaches rely on costly infrastructure or suffer from limited spatial coverage.

One underexploited yet highly promising resource is the vast network of **roadside surveillance cameras** already deployed in many cities. Originally installed for security and traffic monitoring, these cameras provide dense and rich visual streams of urban mobility. If harnessed properly, they represent a scalable, cost-effective data source for traffic emission analysis—enabling wide-area coverage without deploying new hardware.

This report proposes a complete framework that transforms raw traffic footage into actionable emissions. The first component is a **vision-based pipeline** that processes raw camera footage to extract structured information—vehicle type, speed, and trajectory—via fully automatic camera calibration, fine-grained vehicle classification, and multi-object tracking. Using this data, the system performs **per-vehicle CO₂ emission estimation** through emission-factor models.

The second component is a **network-level spatio-temporal model** that aggregates emissions from multiple cameras across a road network. It performs two tasks: **forecasting** future emissions and **imputing** emissions for links without direct camera coverage, using graph-based modeling of spatial and temporal dependencies.

Finally, we incorporate **Federated Learning**—a paradigm widely popular in traffic analytics but *underutilized* in emission estimation and forecasting—to tackle data-privacy and scalability constraints. Federated learning further enables different sites to benefit from richer

feature representations explored in other sites, allowing collaborative model training across multiple locations without centralizing edge data.

The structure of this report is as follows:

- **Part I — State of the Art:** Foundations and related work.
 - Chapter 1: Emission Estimation – Context, Motivation, and Data Challenges. In this chapter, we present the key motivations behind our work, explain its relevance for emission optimization, and analyze the different data sources available for use.
 - Chapter 2: Foundations of Machine Learning and Deep Learning Models for Time Series Analysis. In this chapter, we explain the theoretical foundations of machine and deep learning models used for temporal and spatio-temporal forecasting.
 - Chapter 3: Applications of Machine and Deep Learning Models for Traffic Emission Estimation and Forecasting. This chapter reviews the various applications of machine and deep learning models in traffic emission estimation and forecasting, providing a literature review and a comparison of model architectures.
- **Part II — Proposed Contribution:** System design, implementation, and evaluation.
 - Chapter 4: Solution Design. In this chapter, we present the detailed design of our proposed framework, explaining its conception, architecture, and the rationale behind each design choice.
 - Chapter 5: Implementation. In this chapter, we provide details about the implementation of our framework, including the datasets used, preprocessing steps, and hyperparameter tuning process.
 - Chapter 6: Tests and solution Evaluations. In this chapter, we present the results obtained for each component of our framework and evaluate its performance using specific metrics and testing procedures.
 - Appendices: Theoretical background on geometry essentials for computer vision models.

Part I

State of The Art

Chapter 1

Emission Estimation: Context, Motivation, and Data Challenges

1.1 Introduction

Pollution has become one of the most pressing challenges of our era, with profound consequences not only for the environment, (Yang et al., 2025), but also for global public health. Air pollution, in particular, poses a severe and immediate threat. According to the *State of Global Air 2024* report, air pollution is the world’s second-leading risk factor for death¹. The daily combustion of fossil fuels for transport, power, and industry releases a host of harmful agents into the atmosphere, from fine particulate matter to greenhouse gases like carbon dioxide CO₂. The health risks are so significant that in 2013, the International Agency for Research on Cancer classified outdoor air pollution as a Group 1 carcinogen, on par with tobacco smoke (International Agency for Research on Cancer, 2013), (Dockery et al., 1993).

A major contributor to this crisis is road traffic. Urban transport networks are responsible for a substantial share of CO₂ emissions, which, according to the Intergovernmental Panel on Climate Change (IPCC), have already warmed the planet by approximately 1.1°C above pre-industrial levels (Intergovernmental Panel on Climate Change (IPCC), 2021-08-09). Given the slow transition to electric vehicles, it is imperative to implement operational measures to reduce emissions from the existing fleet. The foundation of any effective mitigation strategy lies in the ability to accurately quantify and analyze traffic emissions. Only by measuring emissions with high granularity and understanding their patterns can we make informed decisions, implement targeted policies, and optimize traffic in high emission areas to achieve meaningful reductions.

To that end, this chapter delves deeper into the contribution of road traffic on air pollution. We will begin by analyzing the data that underscores the urgency of the problem. We will then explore why modern Artificial Intelligence (AI) approaches are uniquely suited to model this complex issue and, finally, survey the diverse data sources these techniques can use to transform raw traffic observations into actionable environmental intelligence.

1.2 GHG emissions

Any gas that is released into the atmosphere and traps heat in it is considered to be a greenhouse gas, as it contributes to the greenhouse effect, which in turn is a natural process that maintains the earth’s temperature at an average of 14°C. It works by trapping some of the heat accumulated from the sun during the day in the greenhouse gases, releasing it gradually in the

¹<https://www.stateofglobalair.org/resources/report/state-global-air-report-2024>

atmosphere at night. Without this mechanism, Earth’s average temperature would plummet to approximately -18°C , far too cold to sustain most forms of life.² However, the sudden increase in the release of greenhouse gases into the atmosphere caused by human activities has changed this greenhouse effect from essential to harmful. As we have seen the earth’s global temperatures rising at alarming rates, approximately 1.1°C above pre-industrial levels according to the (Intergovernmental Panel on Climate Change (IPCC), 2023) study. Causing all types of catastrophes and imbalances in various ecosystems.³

The GHG gases are naturally present in the atmosphere in different concentrations. Among those gases, the key gases that see their concentrations changed drastically because of human activity at a global scale include carbon dioxide (CO_2), CH_4 , NO_2 , and F-gases (Intergovernmental Panel on Climate Change (IPCC), 2021). With carbon dioxide (CO_2) being the most present in the atmosphere and the most significant contributor to climate change, primarily due to fossil-fuel combustion and deforestation, and F-gases having the biggest growth overtime due to their widespread use in refrigeration, air conditioning, and electronics manufacturing.⁴ In figure 1.1, we can see a resume of the study conducted by (European Commission, Joint Research Centre, 2024) on global GHG emissions in the atmosphere. These findings highlight the big share that carbon emissions have, which are increasingly harmful to the environment.

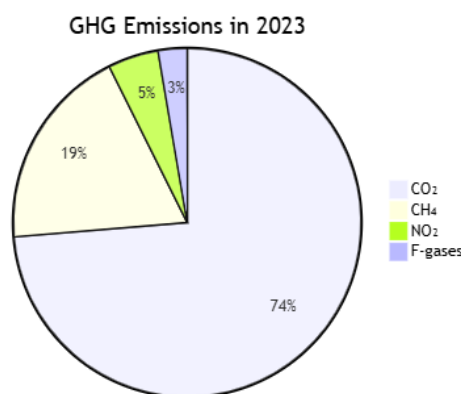


Figure 1.1: 2023 Global Greenhouse Gas Emission Levels.

When we delve deeper into the analysis of carbon emissions across different sectors, reports highlight the significant emissions originating from the energy sector, including those from electricity and heat production. Following closely are sectors such as transportation, industry, buildings, and other fuel-related activities. According to the study, the transportation sector ranks as the second largest source of carbon emissions, following electricity and heat generation. In this same study, it is shown that transportation is the third sector that has seen a very large increase in emissions globally from 1990 and the largest increase compared to all the other sectors in 2023 as we can see from the Table 1.1 (European Commission, Joint Research Centre, 2024).

Within the transportation sector, the majority of emissions come directly from road vehicles as seen in figure 1.4 . This surpasses greatly the emissions generated by the other transport

²NASA, 2023, <https://climate.nasa.gov/causes/>

³World Meteorological Organization, 2022, <https://public.wmo.int/en/our-mandate/climate>

⁴EPA, 2024, <https://www.epa.gov/ghgemissions/overview-greenhouse-gases>

Table 1.1: Global GHG emission trends by sector and key years (European Commission, Joint Research Centre, 2024).

Sector	2023 vs 1990	2023 vs 2005	2023 vs 2022
Power industry	+96% ↑	+36% ↑	+2% →
Industrial combustion/process	+91% ↑	+41% ↑	+2% →
Buildings	+1% →	+3% →	+1% →
Transport	+78% ↑	+26% ↑	+4% →
Fuel exploitation	+48% ↑	+23% ↑	+2% →
Agriculture	+20% ↑	+15% ↑	0% →
Waste	+56% ↑	+37% ↑	+2% →
All sectors	+62% ↑	+28% ↑	+2% →

Key: ↑↑ increase (worse), ↓↓ decrease (better), →→ roughly unchanged . Percentages show relative change vs. the comparison year; arrows indicate direction; color is for emphasis only.

modes for instance railways, shipping, and aviation. The dominance of road transport in emission reflects the impact this specific sector has thus it underlines the importance of developing targeted approaches to specifically reduce road transport emissions.

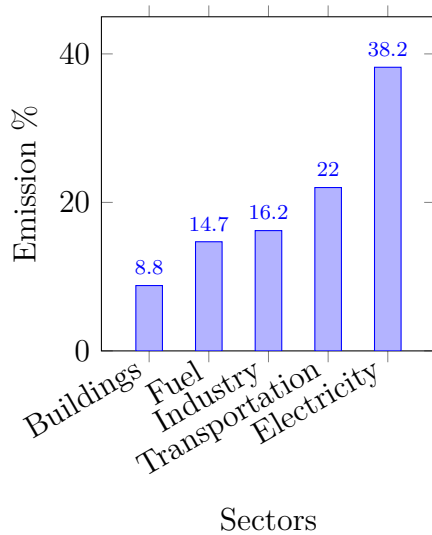


Figure 1.2: Emissions by sector

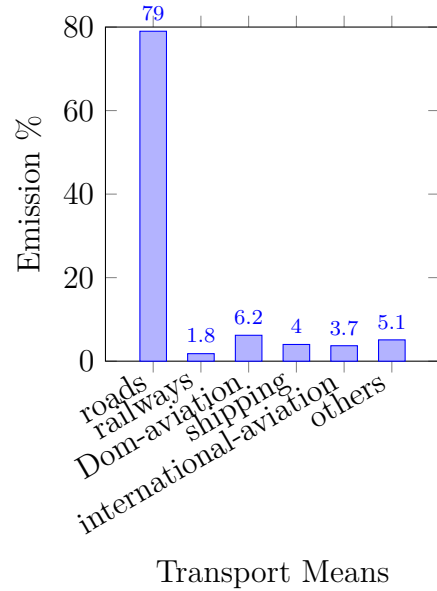


Figure 1.3: Emissions by transport type

Figure 1.4: Carbon emissions distribution by (a) sector and (b) transport type for 2024 ((United Nations Environment Programme, 2024)).

According to the latest climate report published by (European Commission, Joint Research Centre, 2024), the European Union recorded its most significant reduction in greenhouse gas emissions across all sectors in decades during 2023. The most notable reduction was recorded in the power industry, which achieved decreases in emissions throughout the decades in a stable manner. In a similar fashion other sectors have experienced a decrease in emission though not as important, however the transportation sector stands out with being the only sector with a significant increase in emission, even though in the last two decades it experienced a minimal reduction. This stark contrast underscores the effectiveness of targeted policies and measures implemented in the other sector whereas the numerous solutions that have been proposed to reduce emissions from transportation, ranging from biofuels to electric vehicles and solar-powered

cars, are less effective, as the transition to these alternatives is progressing slowly. Those conclusions can be seen in table 1.2 that shows that most sectors have seen a very large decrease from 1990 to 2023 except for the transportation which is still very far behind in effectively reducing GHG emissions.

Table 1.2: Change in emissions from 1990 to 2023 in different sectors in the EU (European Commission, Joint Research Centre, 2024).

Sector	2023 vs 1990	2023 vs 2005	2023 vs 2022
Power industry	−51% ↓	−50% ↓	−20% ↓
Industrial combustion/process	−42% ↓	−30% ↓	−6% ↓
Buildings	−37% ↓	−31% ↓	−6% ↓
Transport	+19% ↑	−6% ↓	−2% →
Fuel exploitation	−46% ↓	−27% ↓	−6% ↓
Agriculture	−27% ↓	−6% ↓	0% →
Waste	−35% ↓	−26% ↓	−2% →
All sectors	−34% ↓	−29% ↓	−7% ↓

Key: ↑↑ increase (worse), ↓↓ decrease (better), →→ roughly unchanged. Percentages are relative to the comparison year in each column; arrows indicate direction; color is for emphasis only.

And one of the main measures taken to reduce transportation emissions is the production of electrical vehicles, but replacing the on-road fleet takes time. Even though electric cars exceeded **20% of global new sales in 2024**, most vehicles in use still burn fuel. Vehicle service lives are long, as the average age of light-duty vehicles in the United States is now about **12–13 years**, and many vehicles remain in service for close to two decades, especially outside early-adopter regions. *Therefore, cutting CO₂ during the transition requires operational measures alongside EV uptake.*

Forecasting CO₂ adds value beyond estimating today’s emissions. First, prediction turns estimates into decisions: if we can forecast when and where emissions will spike (by time of day, weekday, season, weather regime, or special events), agencies can retune signals, coordinate corridors, or meter/reroute flows before stop-and-go patterns lock in. Second, prediction reveals patterns: learned seasonal and spatial signatures, road classes with chronic excess, or sensitivity to meteorology and demand. These patterns guide long-term policies (e.g., which corridors to target for fleet renewal or bus priority, how to schedule construction work and enable scenario testing. Third, forecasts support optimization loops as agencies can use predicted emissions as an objective in signal-timing, eco-routing and freight-scheduling optimizers, evaluate results, and iteratively refine. Evidence from pilots and studies indicates the order of magnitude of potential near-term gains: adaptive AI signal timing has shown up to 30% fewer stops and ≈10% lower intersection emissions in early deployments; eco-driving programmes typically save 5–15% fuel; eco-routing often yields ~5% CO₂ savings per trip. These are local or corridor-level figures, but they illustrate why predict-then-act can deliver meaningful reductions.

To conclude, the increase in global GHG emissions is a threat to both the planet and human life on it, and the transportation sector, and specifically traffic, are one of the biggest contributors in releasing CO₂, which is in turn one of the most harmful gases to the atmosphere, and in order to reduce its release in the atmosphere, it is essential to be able to estimate it in different forms whether globally for instance city-wide or district-wide, or more granularly, at the level of street lanes.

1.3 Motivation for using AI-based solutions For Emission Estimation

Researchers have spent decades trying to pin down on-road carbon emissions, moving from straightforward fuel-consumption equations to intricate models with dozens of traffic- and vehicle-specific inputs. For instance, one of the most prominent methods proposed for carbon emission estimation in traffic is U.S. EPA’s MOVES simulator ⁵, it uses VSP and detailed operational data, such as speed, road gradient, and vehicle acceleration, to estimate emissions with high precision. By integrating detailed traffic and vehicle data. The MOVES model incorporates various variants depending on the type and quantity of features used, allowing it to scale from highly complex models with extensive feature sets to simpler models utilizing a smaller number of features. However, these models heavily depend on domain expertise and the acquisition of data on fleet composition, driving activity, and correction factors which poses a significant challenge due to the complexity and costs needed to gather such data, and the fact that these methods do not capture patterns nor does it detect the relations between features means it is highly sensitive and if the region changes or just the fleet composition changes over the months or the years, new data has to be used for the estimation of emissions, and it does not allow for the prediction of emissions only their quantification.

The availability of new and diverse data sources is one of the reasons researchers are turning to machine-learning (ML) and deep-learning (DL) techniques. Unlike classical formulas that demand exhaustive fleet and activity inventories, modern ML models can mine information from data streams that were once impossible—or too costly—to exploit: dash-cam video, roadside sensors, satellite images, crowdsourced smartphone traces, or even OpenStreetMap geometry. By learning emission patterns directly from these heterogeneous inputs, ML systems sidestep much of the painstaking data collection and hand-tuned factors that traditional models require. Machine learning and deep learning also helps scientists and researchers uncover new trends and maybe new dependencies between data and emissions in varying levels of granularity that would not have been uncovered if left to expertise alone, as well as these methods allow scientists to optimize their time as instead of them having to extract the relationships between the data and the emissions, the model does it for them and sometimes even better, thus the rising interest in employing ML and DL based methods.

1.4 Traffic Emissions From Estimation to Optimization

Estimating traffic emissions is a crucial component of traffic monitoring and control research that aims to lower vehicle emissions. Real-time emission estimation that is accurate is not only a scholarly endeavor; it is a crucial instrument for improving comprehension of how transportation systems affect the environment. This procedure provides important insights into traffic patterns, vehicle behaviors, and the carbon footprints associated with them by gathering data on emissions over time and across different geographies. These insights aid in determining pollution hotspots, detecting wasteful traffic movements, and determining when and where emissions are strongest. Emission estimation can be used to forecast possible future situations based on traffic patterns by using sophisticated modeling approaches. This ability to predict outcomes supports more informed decision-making regarding transportation policies, infrastructure development, and the implementation of measures to reduce emissions. As cities grow and transportation networks become more complex, these estimations will be vital in creating smarter, more sustainable

⁵<https://www.epa.gov/moves/latest-version-motor-vehicle-emission-simulator-moves>

urban environments that balance mobility with environmental responsibility.

1.4.1 Urban Planning and Traffic Management

On-road emission estimation plays a crucial role in traffic management and urban planning by providing insight into the environmental consequences of infrastructure decisions. While new roads and transit projects can sometimes increase emissions—especially in rapidly expanding cities—they may also reduce congestion and improve traffic flow when carefully designed. By incorporating emission estimation models that account for vehicle fleets, traffic patterns, and road characteristics, planners can anticipate these impacts more accurately. This enables the design of infrastructure that balances mobility needs with environmental objectives, guiding route alignment and network planning toward solutions that minimize pollution. Ultimately, such an approach supports long-term sustainability goals by aligning urban growth with cleaner, more efficient transportation systems.

1.4.2 Eco Routing Application

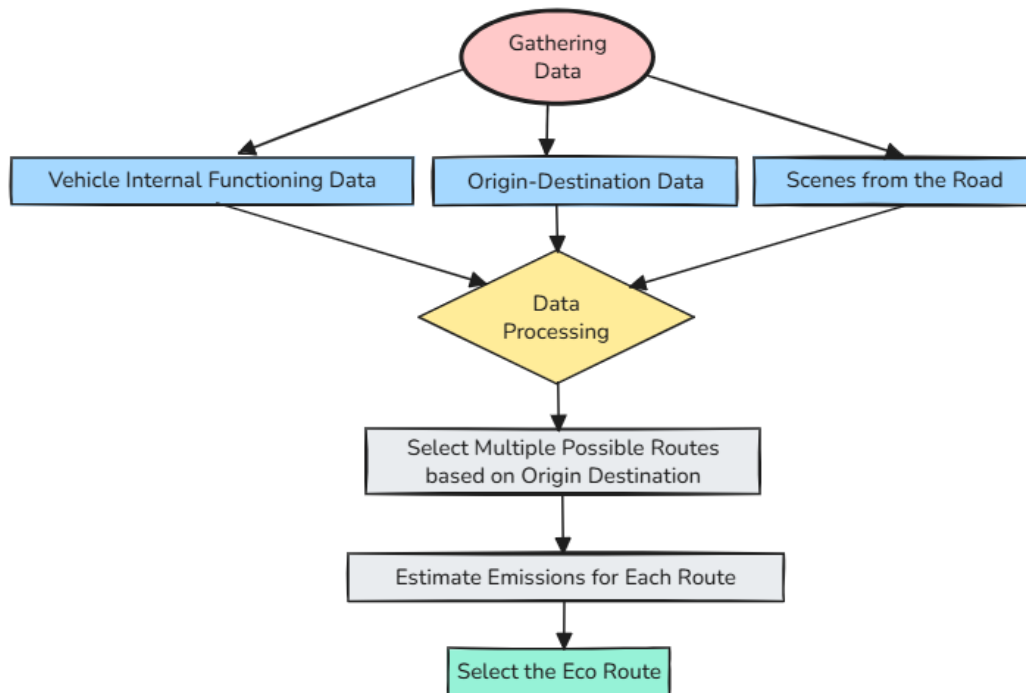


Figure 1.5: ECO routing framework example using emission estimation.

The main goal of suggested eco-routing techniques is emission reduction, as they seek to reduce energy consumption and emissions in fuel-powered vehicles. The development of efficient routing methods based on real-time emission levels is made possible by accurate vehicle emission estimations, which are essential for comprehending present and future emission trends. These help in locating areas with high emissions and provide information on alternate routes with less of an adverse effect on the environment. Eco-routing techniques enable the creation of dynamic, data-driven action plans that give priority to routes with fewer emissions by incorporating such emissions data into routing algorithms ex figure 1.5. Several studies tackled this problem using

real time emission estimation model like in (Pradeep et al., 2024) This is especially important in autonomous vehicles, where such models can be directly integrated into vehicle control systems to optimize driving behaviors for emissions reduction.

1.4.3 Optimized Traffic Light Management

In order to minimize needless stops and lower emissions from frequent acceleration and deceleration, intelligent traffic light systems use emissions estimation to optimize traffic signal synchronization in real time. These systems guarantee more efficient vehicle movement and smoother traffic flow by dynamically modifying signal timings in response to current emissions. This directly lowers overall emissions in metropolitan contexts. According to studies like those in (Santos et al., 2023) , these systems provide a number of important advantages, such as increased urban mobility, less traffic, and lower carbon emissions.

In 2023, Google took a significant step forward in this domain by launching a project that combines AI with Google Maps data to model and optimize Emissions ⁶, with a focus on intersection timings. By synchronizing traffic lights across several junctions, the Green Light system reduces fuel consumption and related pollutants by facilitating smooth traffic flow reducing the stop and go pattern wich contribute the most in increasing traffic emission levels. Cities may reduce the environmental effect of urban transportation systems and maximize traffic management with this proactive strategy.

1.5 Data Collection and Preprocessing

Data serve as the essential fuel for the emission estimation process. Various sources can be leveraged to extract features that are strongly correlated with vehicle emission levels. Depending on their granularity, these data are collected and sampled at different frequencies. Before being used in estimation models, the raw data must undergo several preprocessing steps, including cleaning, formatting, and transformation into suitable representations. In this section, we present the datasets employed in our study, describe their collection process, and detail the preprocessing pipeline applied to prepare them for model development.

1.5.1 Data Sources and Methodologies for Emission Estimation

In the context of emission estimation, various approaches have been proposed in the literature to address the challenge, each tailored to the type of data used. Emission estimation methods can be effectively classified based on the type of data they rely on, with the choice of data often determined by regional characteristics, the specific research focus, and the availability or deployment of IoT sensors for data collection. Different regions may have distinct data availability or infrastructure, such as IoT sensor networks, influencing the choice of method. Additionally, the research objectives, whether they focus on long-term trends or real-time IoT collected data, also guide the selection of data sources. This classification of emission estimation methods can be broadly categorized into three main types: approaches relying on historically averaged data, methods that incorporate spatial components, and real-time prediction techniques see figure 1.6.

1.5.1.1 Historically Avereged Data

Historically averaged datasets pool traffic and fuel-related figures over long periods typically three to eight years, and broad areas, such as an entire city, province, or country. They mix

⁶<https://sites.research.google/gr/greenlight/>

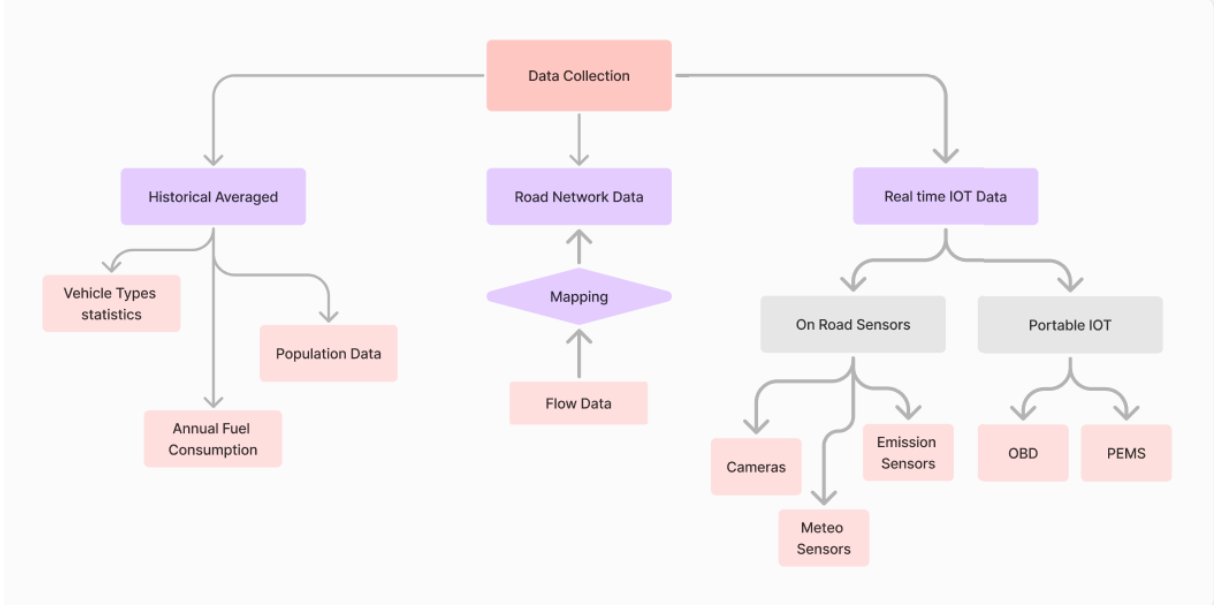


Figure 1.6: Data Collection Methods For Emission Prediction Models.

multiple data streams (vehicle registrations, fuel sales, population, and long-term weather statistics) and report them at monthly or annual intervals. Such data is generally averaged, representing overall trends and emission values rather than granular details, and most of the time collected by governments and provided as open-access resources, for instance the dataset used in ((Cmarer et al., 2024)) was a transportation energy consumption/demand dataset, containing multiple feature for instance Mtoe, VK, Population (POP)in millions, Year (Y), GDP, and CO₂ emissions from the transportation sector between 1970 and 2016 . Another well-known example is Canada’s Open Government Vehicle and Fuel-Consumption Database, which spans seven years and lists make, model, engine size, fuel type, highway/city mileage, and corresponding CO₂ emissions for every registered vehicle class (Government of Canada, 2019) which is used in countless scenarios and studies for instance the study in (Hien and Kor, 2022) used the dataset for analytical purposes mainly, and (Natarajan et al., 2023) used it for estimation and prediction .

Averaging the data over a longer period of time allows for the smoothing of noise and lets researchers exploit the data for various purposes for instance the tracking of decadal emission trends for policy evaluations, the studying of seasonal cycles in pollutants, comparing the emissions per region (urban vs rural), studying the influence of different factors over extended periods.

In order to improve prediction accuracy, some researchers have explored the inclusion of additional features, such as meteorological data, to better capture the impact of climate-related factors on vehicle emissions. For example, (Shakya et al., 2023) experimented with different data configurations, incorporating meteorological parameters, vehicle population data, and emissions data to predict emissions over three time intervals (1 hour, 8 hours, and 24 hours). Their study demonstrated that models incorporating meteorological data outperformed those that excluded it, showing the importance of climate-related features in enhancing prediction accuracy. In a similar manner, (Simić et al., 2020) integrated meteorological factors and temporal dependencies to estimate pollutant concentrations in a street canyon in Zagreb. Using three years of data (2011–2013), their study identified relative humidity, atmospheric pressure, and lag variables as key predictors. Seasonal trends were also observed, with higher pollutant levels in winter and autumn.

The chief limitation of these datasets is resolution. Consequently, historically averaged data

work best for macroscopic analyses and emission patterns identification.

1.5.1.2 Road Network Data

The spatial feature is an important factor in emission estimation because where, along what route, and on what type of road a vehicle travels significantly influences exhaust emission levels. In response, a number of approaches incorporate spatial variables into their feature sets to enable location-dependent emissions to be represented. They rely primarily on road network data to express vehicle flow and emissions as a function of time, considering the road infrastructure, road types, and traffic conditions that naturally assume varied driving patterns and result in varied emission rates. Road networks available from open-source mapping sites, i.e., Google Maps or OSM, are popular in this context. For instance, research studies such as (Zeng and Liu, 2024) and (Rollend and Foster, 2023) effectively utilize OSM data to represent road connectivity and use spatial features and road conditions as valuable predictive features along with traffic flow data. Traffic flow information can be extracted from existing travel demand datasets, e.g., in (Zeng and Liu, 2024), estimated from satellite imagery, e.g., in (Rollend and Foster, 2023), or extracted from historical vehicle trajectory, e.g., in (Shang and Zhang, 2021). These methods have a tendency to combine spatial and flow information for emission estimation, following a mesoscopic approach that provides mid-granularity data.

These data are usually paired with their spatial counterparts, making emissions analysis possible at the road segment level. This method examines the interactions between spatial distribution, traffic flow, various driving patterns by road type, and emissions. The mesoscopic approach is highly appropriate for generating mid-granularity data because it balances between detail and tractability. Emission results of such models can typically be aggregated to larger regional scales and fused with real-time input, allowing for dynamic updating and prediction in real-time applications.

1.5.1.3 Real-Time IoT-Based Emission Data

Recent advances in broadband networks, edge computing, and low-cost sensors now make it practical to capture traffic-emission data in real time. Second-by-second measurements provide a level of spatial and temporal detail that historical inventories simply cannot match, allowing researchers and traffic engineers to pinpoint hot-spots and respond quickly.

Real-time systems draw on two complementary hardware streams:

firstly on vehicle sensors, such as the OBD II which logs engine load, fuel-flow, and catalyst status; PEMS that record tail-pipe gases under real-world driving; GPS and inertial units which add precise speed and location. Together they yield instantaneous emission rates for individual vehicles.

secondly, Roadside and overhead infrastructure such as roadside cameras, lidar, acoustic arrays, and air-quality probes classify passing vehicles, monitor traffic density, and sample exhaust plumes without interrupting flow. In this section we delve deeper into these two categories.

1.5.1.3.1 Data Collected From Portable IoT Devices

This category of methods often uses data gathered from vehicles, making this type of data specific to the vehicle from which it has been collected. These methods frequently utilize (OBD) along with the GPS. Portable IoT devices, such as PEMS, are also employed to capture gas emission levels and construct datasets.

Using Portable Emission Measurement Systems (PEMS)

Those Methods are from first methods Proposed in this field, Researchers used to conduct real scenario test using PEMS installed on vehicles, An example of that are the studies (Zhang and Y.,

2022) Where PEMS are being installed on Light Duty Diesel Trucks and data collected from two test trucks For collecting real time emissions of CO₂ , conducting testes under various conditions to variate and enrich emission data . Or in (Chen and C., 2022), the authors introduced a novel approach that utilizes data collected from sensors installed on vehicle engines to capture features related to engine status along with PEMS data.

Despite the promising results of both studies, their applicability in real-world scenarios is hindered by the requirement for additional IoT sensors installed on vehicles. This full reliance on specialized equipment complicates large-scale deployment, as equipping numerous vehicles with IoT devices during the testing phase would entail significant logistical and financial challenges.

Using On Board Diagnostic System (OBD)

Another approach as a solution for the previous problem , and especially after the increased number of modern vehicles that are equiped with On Board Diagnostic system which often provide data about functioning of vehicles without the need for installing additional iot sensors , because the need for PEMS will be only to collect CO₂ emissions for constructing the training data at a first phase. Other Studies Have come to reduce these dependency. For instance, in (Yavari et al., 2023), the authors utilized data from the OBD system of a delivery van. The data collected was preprocessed on a Raspberry Pi node, involving steps like noise removal, resampling, and interpolation, before being transmitted to a cloud server for model training and prediction. Similarly, (Madziel et al., 2021) combined OBD and PEMS data to develop a microscale CO emission model for full hybrid vehicles,capturing urban, rural, and motorway driving conditions.

Problems With Vehicle Specific Data in Test

The primary limitation of previous approaches lies in the fact that during testing, due to time and resource constraints, studies are often conducted using a very limited number of vehicles. This biases the trained model towards the specific training data used. In reality, there are numerous types of on-road transportation, including buses, trucks, and cars. Even within a single category, there are various makes and models, each exhibiting different emission patterns. Consequently, when training a model using data from a specific test scenario, the model often struggles to achieve high accuracy on other vehicle types due to the non-identical distribution of emission data across these transportation modes.

Proposed Solutions For the Previous Problem

A study in (Xu et al., 2021) demonstrated the potential to address this issue by using data from the OBD system of a Type A vehicle while the model was pre-trained on data from a Type B vehicle. By employing a stacked sparse autoencoder with MMD as a cost function, the authors aligned the data distributions of both vehicle types. This was achieved by learning representations in a high-dimensional space and simultaneously reducing the distance between the variance and the mean of the two distributions. Once the distributions were aligned, any pre-trained deep learning model could be applied for prediction.

Another solution that emerged later is the use of connected vehicles, or IoV. IoV represents a network where vehicles, infrastructure, and other devices are interconnected through wireless communication technologies. This approach enables real-time data collection from multiple vehicles simultaneously. It assumes that vehicles are equipped with wireless communication capabilities, such as Vehicle-to-Everything (V2X) technologies. V2X encompasses various communication modes as shown in figure 1.7:

- **Vehicle-to-Vehicle (V2V):** Communication between vehicles to exchange information like speed, location, and traffic conditions.
- **Vehicle-to-Infrastructure (V2I):** Interaction with roadside infrastructure such as traffic lights and RSU, enabling the collection and transmission of data to central systems.

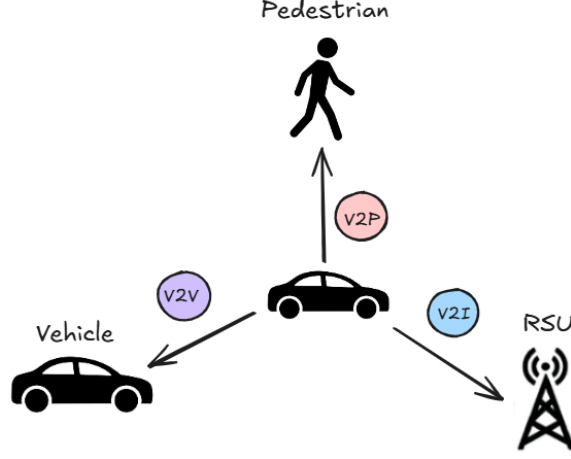


Figure 1.7: Figure illustrating Types of V2X connectivity (Farnell, 2025) .

- **Vehicle-to-Pedestrian (V2P):** Communication between vehicles and nearby pedestrians or cyclists, improving safety in urban environments.

In the IoV framework used for collecting emission data, they generally use the V2I type, where vehicles equipped with these communication technologies transmit data to RSUs, which sample the information and forward it to cloud servers. The cloud servers then preprocess and analyze the data, using it to train and refine emission models.

For instance, (Singh and Dubey, 2023) utilized a dataset collected via OBD-II interfaces from 14 drivers. Vehicle data was transmitted via Bluetooth to an Android app for processing.

Similarly, (Torres Guin et al., 2024) proposed an IoT-based architecture for monitoring vehicle pollutant gas emissions. This architecture comprises several layers: the Perception Layer uses sensors to detect temperature, humidity, and gas concentrations; the Network Layer ensures reliable data transmission via LTE; and the Application Layer leverages the ThingSpeak platform for data storage and management.

In another study, (Saez-Perez et al., 2024a) utilized OBD systems to extract features related to vehicle functionality. The processed data was then transmitted to a MEC server within a 5G network, where an optimized model was deployed to predict CO emissions. The prediction results were sent back to the vehicles, enabling real-time adjustments to effectively reduce emissions.

This decentralized data collection and scalable approach significantly enhances the accuracy and applicability of emission predictions while accommodating a diverse range of vehicles and driving scenarios.

However, the application and efficiency of these methods for real-time prediction tasks can be limited by network connectivity and the bandwidth consumption required for transmitting data collected from vehicles to the server where the model will be trained, highlighting the need for more robust approaches to bring emission models closer to data collection hubs which is known to be the Edge technology.

1.5.1.3.2 Data Collected From On road Installed Devices

This category of methods, instead of relying on portable or vehicle-specific measurements, utilizes infrastructure-based sensors that are installed along roadways to collect and capture area-wide data. These systems gather features from multiple vehicles that are located within a specific road segment, and this allows for a broader and more integrated perspective on traffic and environmental conditions. The strength and reliability of these methods heavily depend on the spatial coverage and density of the deployed sensors. Factors such as the sensor network's

range, field of view, and resolution are what ensure accurate and representative data collection for the target area. The diverse range of employed sensors resulted in a wide set of datasets that researchers have exploited by proposing method after method.

Surveillance Cameras

Despite the omnipresence of roadside CCTV, video streams are still only lightly exploited for emission work. Cameras can supply a rich set of operational variables after feature extraction — for instance the simplest ones to obtain would be vehicle type, count, lane position, and speed — but the process of extracting these features can be complex and, since they do not measure exhaust directly, researchers almost always use them alongside other data sourced from additional sensors depending on the method itself and exploit the easily extractable information. For instance, the study in (Rodriguez Valido et al., 2022) aimed to reduce the cost of installing dedicated IoT devices on roads for real-time emission predictions by exploiting the already installed road cameras. The authors used a deep-learning model to classify vehicles and track their movements, applying perspective correction to estimate real-world speeds. These speeds, combined with vehicle classifications, were used to predict emissions. For this purpose they used the BrnoCompSpeed dataset (Sochor, Juranek, Špaňhel, Maršík, Šíroký, Herout and Zemčík, 2019) which is a computer vision dataset composed of around 18 Full HD videos taken in czech republic highways along with speed measurments, 2D bounding boxes, and camera intrinsics, and is available on demand.

Traffic flow data

A trend that can be noted in this large pool of methods is the use of traffic flow data and traffic flow sensors. Traffic flow refers to the collective motion of vehicles along a road segment and is usually described by three linked variables: flow rate (vehicles per unit time), density (vehicles per unit length), and space-mean speed. Road operators capture these quantities with an array of fixed sensors, or magnetometer detectors embedded in the pavement; side-firing radar and infrared units; Bluetooth and Wi-Fi sniffers; or machine-vision cameras, and aggregate the readings at intervals that range from a few seconds to five minutes. The resulting time series are routinely archived by traffic-management centres and form one of the most accessible open-data resources for emission work.

Because flow data represents how many vehicles occupy a link and how fast they move, they can be combined with speed-dependent emission factors to approximate exhaust without installing tail-pipe probes. (Bilotta and Nesi, 2022) provide a clear example: they fused Florence’s loop-detector counts with air-quality measurements, reconstructed density on unsensed links via the Lighthill–Whitham–Richards traffic-flow model, and then applied polynomial emission curves to estimate link-level CO₂. (Li et al., 2024) followed a similar principle in Chengdu, fusing high-resolution taxi GPS data with multi-source urban features—such as road network data, points of interest, and weather—to create a fine-grained spatiotemporal inference of total traffic emissions.

Yet flow-based estimates inevitably inherit the coarse granularity of their inputs. A mean speed calculated over a 500-metre link smooths away the acceleration-and-braking bursts that dominate real-world exhaust, while macroscopic traffic-flow models such as LWR can distort conditions near intersections and weaving sections. Consequently, flow data remain useful for the cases of city-wide carbon budgets and long-term trend assessments, but they fall short when the task is to pinpoint kerb-side hot-spots or capture second-by-second emission surges.

IOT architectures

A growing amount of studies propose layered IoT architectures in which roadside units (RSUs) or edge servers gather traffic and environmental data in real time, process them locally, and forward summaries to a cloud back-end for long-term analysis. This structure was designed with the purpose of allowing cities to work with the data on the spot and, and eventually trigger quick responses such as rerouting traffic or lowering speed limits in order to keep emissions in check.

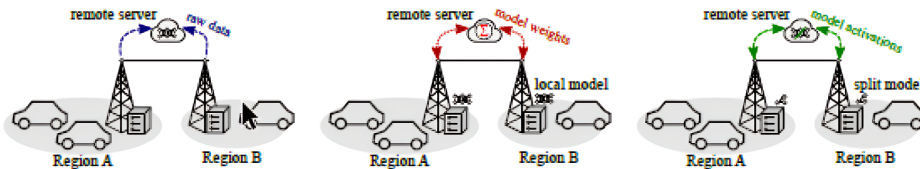


Figure 1.8: Examples of the training approaches used in (Braun et al., 2024).

One three-tier design, presented in (MartínBaos et al., 2021), illustrates the approach. Its physical layer combines particulate-matter sensors, weather probes, and roadside cameras that generate motion vectors. Raw streams of traffic flow, meteorology and provisional emission metrics are pushed to a fog layer where an embedded algorithm converts motion vectors into vehicle counts and speeds, cleans the data, and feeds a lightweight ML model. Output is then relayed to an IBM Cloud instance (cloud layer) that aggregates results city-wide and triggers alerts when emissions exceed legal thresholds.

A second framework, “EcoPredict” presented in (Braun et al., 2024), focuses on balancing prediction accuracy with privacy and bandwidth. RSUs measure fuel-use, speed, and exhaust parameters, then test three learning paradigms : centralised, federated, and split learning, using the FLower communication stack. Centralised training delivered the highest accuracy but incurred heavy data transfer; federated and split learning trimmed bandwidth and enhanced privacy while maintaining near-comparable accuracy and lower latency.

While these architectures promise street-level insights and the ability to act immediately for instance rerouting traffic or capping speeds when thresholds are breached they still face practical hurdles. Most evaluations lack testing in real-world scenarios under diverse conditions, leaving questions about latency under peak loads, wireless back-haul reliability, and sensor noise unresolved. Additionally, the complexity of deploying these architectures is frequently overlooked, with limited discussion on critical factors such as latency, bandwidth requirements, necessary technologies, and the impact of noise and disturbances in real-world environments.

1.5.2 Data Collection Frequency and Duration

The temporal parameters of data collection—specifically the frequency (how often data is sampled) and the duration (the period over which it is gathered)—are critical methodological choices in emission studies. These choices are heavily influenced by the research objectives, the available technology, and the practical constraints of data management.

1.5.2.1 Data Collection Frequency

The sampling rate of data collection varies significantly, often dictated by whether a study utilizes existing infrastructure or deploys its own custom sensors. When relying on pre-existing roadside or vehicle sensors, researchers must contend with heterogeneous data rates; some sensors may report data every second, while others log at intervals of several minutes or even hours. This disparity requires data harmonization strategies, such as aggregating high-frequency data into averages or interpolating low-frequency data to create a consistent time series. Conversely,

studies that install their own equipment, such as PEMS on test vehicles, have the flexibility to define collection intervals that precisely match their research needs, allowing them to capture specific driving events like rapid accelerations with high temporal resolution.

In recent literature, there is a clear trend towards instantaneous, second-by-second data collection. This high-frequency approach is essential for real-time applications such as dynamic traffic routing or adaptive signal control, which aim to mitigate emissions as they happen. Technologies like On-Board Diagnostics (OBD), GPS, and PEMS are central to this paradigm, as they can capture the transient emission spikes associated with specific driving behaviors like acceleration, braking, and idling—details that are lost in averaged or low-frequency data.

1.5.2.2 Data Collection Duration

The duration over which data is collected is determined by the temporal scope of the research question. Long-term studies, often spanning months or years, are designed to analyze seasonal effects, such as the impact of meteorological changes or holiday traffic patterns on emission trends. In contrast, short-term studies may focus on periods of days or weeks to examine daily or weekly variations, such as the pronounced differences in traffic and emissions between weekdays and weekends or during morning and evening rush hours. Researchers often face a trade-off between temporal resolution and duration. Collecting second-by-second data over very long periods can generate massive datasets that are computationally expensive to store and process. Therefore, a careful balance must be struck to ensure the collected data is both comprehensive enough to answer the research question and manageable within the project's resource constraints.

1.5.3 Data Preprocessing and feature engineering

Pre-processing plays a crucial role in the success of any machine learning pipeline, particularly in traffic emission estimation where raw data often originates from heterogeneous sources. Ensuring that this data is clean, coherent, and well-formatted is essential to building reliable models. This section outlines the key steps in this process, from initial data cleaning and integration to feature analysis and selection.

1.5.3.1 Foundational Preprocessing Steps

The initial preprocessing phase focuses on transforming raw, heterogeneous data into a clean, consistent, and coherent dataset ready for analysis. This involves two primary stages: data cleaning and integration.

1.5.3.1.1 Data Cleaning and Quality Assurance The first and most critical step is **data cleaning**, which ensures the quality and reliability of the dataset. Raw data from sources like PEMS, OBD sensors, and GPS units is often imperfect. This stage addresses common issues through several key operations:

- **Handling Missing Values:** Missing data points are either imputed using statistical methods or the entire record is dropped if it's incomplete.
- **Standardization:** All data is converted to consistent units, labels, and timestamps to ensure uniformity.
- **Deduplication and Outlier Removal:** Duplicate entries are removed, and obvious outliers—often identified using statistical methods like z-scores or the IqR—are flagged and handled to prevent them from skewing the model.

This is a mandatory step for any robust modeling pipeline.

1.5.3.1.2 Data Integration Since emission studies almost always rely on multiple data streams (e.g., PEMS + OBD + GPS + weather), **data integration** is necessary to combine them into a single, coherent table. The main challenge is aligning data that was recorded at different frequencies and with different timestamps. Key operations include time alignment, resampling all streams to a common rate (e.g., 1 Hz), and merging data based on common keys, such as a vehicle ID or a precise timestamp. This process must explicitly account for potential issues like clock drift between different sensors or data latency and packet loss in IoT systems.

1.5.3.2 Key Features and Correlation Analysis in Emission Modeling

Once the data is clean, integrated, and consistently formatted, the next step is to understand the predictive power of the individual variables, or features, within it. The performance of any emission model is fundamentally dependent on the relevance of its input features. While the specific features vary, a core set of variables consistently emerges as critical.

1.5.3.2.1 High-Impact and Commonly Used Features Across different methodologies, certain features are almost universally employed due to their direct physical link to the combustion process and vehicle dynamics.

- **Vehicle Dynamics and Fuel Consumption:** Unsurprisingly, speed is a ubiquitous feature, whether it is averaged over a region, measured instantaneously by sensors, or estimated from video. Directly linked to this is fuel consumption, which stands as one of the most powerful predictors due to its direct relationship with CO₂ output. Features like driving time and driving distance are also highly correlated, as they are proxies for total fuel use.
- **Engine Characteristics:** The internal parameters of a vehicle’s engine are paramount. Studies consistently show a strong positive correlation between CO₂ emissions and features like engine size, number of cylinders, and RPM. Larger engines and those operating at higher speeds consume more fuel, leading to higher emissions. Interestingly, while one might expect fuel type (e.g., diesel vs. gasoline) to be a dominant factor, studies suggest its correlation with CO₂ is less significant than engine mechanics, with vehicles of similar characteristics emitting comparable CO₂ levels regardless of fuel.
- **Contextual Factors:** The environment in which a vehicle operates also plays a role. Meteorological data such as temperature and humidity can influence engine efficiency and emissions, though their importance can vary significantly by geographic region and the time scale of the study (e.g., daily vs. seasonal).

1.5.3.2.2 A Categorization of Features To provide a clear overview of the data used in emission research, the wide array of features can be grouped into four primary dimensions: Vehicle, Traffic, Environmental, and Engine/Technical. For instance, models using historically averaged data often focus on Vehicle Characteristics (e.g., engine size, fuel type), while those incorporating road networks emphasize Environmental and Traffic Characteristics. Real-time models, particularly those using portable sensors, rely heavily on granular Engine and Technical Features like throttle position and exhaust temperature. Table 1.3 provides a comprehensive summary of features organized by these categories, offering a clear guide to the data landscape in emission estimation research.

Table 1.3: A Categorized Overview of Features Influencing Traffic Emissions.

Category	Features
Vehicle Characteristics	Vehicle population share (gasoline, diesel), model, vehicle class, engine size, speed, acceleration , VSP (vehicle-specific power; tractive power per unit mass, kW/ton), mileage , fuel consumption .
Traffic Characteristics	Traffic flow (vehicles/hour), traffic density (vehicles/km/lane), average travel time (trip time), temporal patterns (time-of-day / day-of-week / season).
Environmental & Road Characteristics	Air temperature, relative humidity (%), wind speed, atmospheric pressure (hPa), precipitation (e.g., mm/h), road type (highway/arterial/local), road condition (dry/wet), road slope (grade, %), longitude, latitude.
Engine & Technical Features	Engine temperature, engine load (torque demand, %), fuel flow (instantaneous, e.g., L/h), engine RPM (revolutions per minute, rotational speed), coolant temperature, exhaust temperature, hybrid battery SOC (state of charge, %), exhaust flow rate (kg/s).

1.5.3.2.3 The Importance of Feature Analysis The relevance of any given feature is not absolute; it is highly dependent on the dataset, the study’s objectives, and the specific context. Therefore, a critical step in the modeling pipeline is a thorough correlation analysis and feature selection process. Most studies conduct a detailed analysis to identify the most impactful variables, often retaining an average of around 10 key features to build a parsimonious and effective model. This process removes redundant or irrelevant information, reduces the risk of overfitting, and can improve model interpretability.

1.5.3.2.4 Dimensionality Reduction and Feature Selection This analysis of feature importance naturally leads to the next critical step in the pipeline: formally reducing the feature set to its most potent and compact form. After analyzing a wide range of potentially predictive features, the next step is often to reduce the dimensionality of the dataset. This helps to reduce overfitting and decrease model training time.

Feature Selection This process aims to identify and retain a subset of the most informative **original** features. This has the advantage of preserving the physical meaning of the variables. Common techniques include statistical ranking (using correlation or mutual information), embedded methods (like L1/Lasso regularization), and wrapper methods (like Recursive Feature Elimination).

Feature Extraction In contrast, feature extraction creates **new, composite features** from the original ones, which is particularly useful for handling multicollinearity. Common techniques include PCA or autoencoders. A potential drawback is that the resulting latent factors may not have a direct physical meaning.

1.5.3.3 Advanced data preprocessing steps

Beyond the standard cleaning, integration, and scaling routines, some studies introduce task-specific preprocessing. A common extra step is sliding-window segmentation: the raw time series is chopped into overlapping sequences so that temporal models (e.g., RNNs) can learn local patterns. And other different methods, for instance (Al-Nefaie and Aldhyani, 2023) applied a rough k-means clustering step to a roadside-sensor dataset to spot vehicles with erratic emission patterns; removing those records reduced noise and improved downstream model performance. Other authors up-sample minority traffic states to balance the label distribution or

apply Gaussian smoothing to reduce high-frequency sensor noise. These customised steps are added only when they improve model stability or reflect the physics of traffic flow.

1.6 Challenges in acquiring and preprocessing traffic Emission data

Reliable emission-estimation models don't just rely on smart algorithms—they also depend heavily on the data that feeds them. And that's where the real challenge often lies. Collecting and preparing traffic-emission data is rarely straightforward: it comes from many different sources, can be incomplete, and is often recorded at different levels of detail. On top of that, traffic itself is constantly changing, and sensors are not always perfect, which leaves us with gaps, blind spots, or even errors in the records. All of this makes the road to clean, usable data much more complicated than it first appears. In the next sections, we'll go through the main challenges that researchers face when trying to acquire and preprocess traffic-emission data.

1.6.1 Data Availability and Accessibility

Reliable traffic-emission datasets are still hard to come by. Many large collections are proprietary or locked behind government privacy rules, so outside researchers cannot access the raw records. Open-data portals help, but what they provide is often coarse (annual averages rather than second-by-second readings), patchy in coverage, or months out of date, and the other publicly available datasets are limited. Gathering fresh, fine-grained data from IoT sensors can fill those gaps, yet the hardware, connectivity, and licensing fees quickly add up, pricing small teams out of real-time experiments. Until broader data-sharing agreements and lower-cost sensing options mature, these access hurdles will continue to slow progress in emission-modelling research.

1.6.2 Heterogeneity of Data Sources and dependency

Traffic-emission studies draw on IoT sensors, GPS traces, satellite images, and even counts each delivered in different formats, time steps, and units. A roadside CO₂ probe might log at 1 Hz, whereas a hand-compiled inventory reports a single daily mean. Before the data can be fused, researchers must align naming conventions, convert units, and resample to a common resolution, these steps often take more time than the modelling itself.

There's also the problem of dependency of the data to the regions it was recorded and gathered in for instance in large cities, carbon emissions arise from a mix of sources, including industrial, residential, and vehicular activities, making it challenging to isolate the specific contribution of vehicle emissions. The overlap of multiple pollution sources in urban areas can introduce biases in data collection, potentially affecting the accuracy of models trained in these environments. When applied to less populated regions, these models may fail to account for differences in emission patterns, leading to discrepancies in their predictive outputs. This highlights the significant influence of the study area's characteristics on the reliability of emission forecasting.

1.6.3 Dynamic Nature of Traffic Data

Traffic conditions change by the minute: rush-hour surges, accident-induced jams, sudden lane closures, and weather shifts all distort the emission picture. Keeping datasets current therefore demands continuous streaming from high-frequency sensors, which in turn strains bandwidth,

storage, and compute budgets. And because some areas, especially country roads or parts of older cities, don't have enough sensors, no data are collected there. This means we end up with "blind spots": stretches of road or periods of time for which we have no information at all.

1.6.4 Sensor Reliability and Calibration

Emission estimates are only as good as the sensors that feed them. Over months of exposure, electro-chemical cells drift, optical instruments foul, and camera lenses cloud—introducing systematic error unless the gear is recalibrated. Scheduled maintenance is costly for wide-area IoT deployments, and unscheduled outages from power loss or extreme temperatures create holes in the record. Guaranteeing consistent accuracy across hundreds of dispersed nodes remains a major logistical hurdle.

1.7 Conclusion

This first chapter has allowed us to discuss important points first, we traced the problem of traffic-related greenhouse gas emissions from its broader environmental context to the practical challenges of working with real-world data. We examined the major causes and consequences of air pollution, highlighting its devastating effects on both the planet and human health. Among the various pollutants, CO₂ stands out as one of the most harmful gases, released in large quantities through human activity—particularly by the transportation and traffic sector. Reducing emissions in this sector is therefore vital. Although policies are in place to address this issue, they often lack short-term effectiveness. This is where emission estimation becomes a valuable tool, as it enables the detection of emission patterns and hotspots, thus helping decision-makers implement targeted actions to mitigate emissions both in the short and long term.

Second, we conducted a comprehensive survey of the data landscape that underpins modern emissions analysis. We explored the diverse pool of data sources—from historical data to emerging sources like remote sensing and video streams—highlighting the unique strengths and limitations of each. This exploration extended to the critical features that directly influence emission rates, such as vehicle type, speed, acceleration, and roadway characteristics. Furthermore, we detailed the essential data preprocessing methodologies required to clean, fuse, and prepare these heterogeneous inputs for modeling. This process, however, is not without its difficulties. Leading us to identify a few of the challenges this domain faces.

By outlining both the urgency of the problem and the complexity of the data, this chapter has set the stage for examining the computational approaches that can address these challenges. The following chapter introduces the theoretical background of machine learning and deep learning methods, which provide powerful tools for estimating and predicting traffic emissions.

Chapter 2

Foundations of Machine Learning and Deep Learning Models for Time Series Analysis

2.1 Introduction

The analysis of traffic emissions, like many complex environmental phenomena, requires more than data collection—it depends on computational methods capable of capturing intricate patterns and dependencies. The effectiveness of such methods relies not only on their application but also on a solid understanding of the theoretical and mathematical principles that underpin them. This chapter therefore provides the necessary foundation, introducing the machine learning and deep learning approaches most relevant to emission estimation and prediction.

The discussion follows a structured progression. We begin with classical approaches such as Regression and Ensemble Learning, which establish the basis of predictive analysis. From there, we turn to the temporal dimension, addressing the unique challenges of Time Series Modeling. This includes a detailed examination of sequence-based models such as Recurrent Neural Networks (RNN), their variants Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU), as well as the role of 1D Convolutional Neural Networks in processing sequential inputs. The exploration of temporal methods culminates with the Transformer architecture, whose self-attention mechanism has redefined state-of-the-art time series prediction.

Finally, the spatial perspective is introduced through Spatio-Temporal Modeling, which integrates both temporal and spatial dependencies. The chapter concludes with a review of the evaluation metrics commonly used to assess model performance, ensuring a comprehensive framework for comparing and validating approaches.

2.2 Regression and Ensemble Learning

Before diving into deep learning, it is important to first consider the classical machine-learning foundations on which many modern approaches are built. Among these, regression analysis and ensemble methods stand out as some of the most widely used tools in emission-modelling studies. They offer a clear way to link input features with emission outputs, while also providing strategies to reduce prediction errors through model combinations. Although these techniques may seem simple compared to neural networks, they remain highly relevant, both as baselines for comparison and as practical solutions when data are scarce or computational resources are limited. The following subsections outline the principles of regression and ensemble learning.

2.2.1 The Principle of Regression Analysis

At its core, many of the challenges in emission analysis are **regression** problems. Regression is a fundamental supervised learning method used to model the relationship between a dependent variable (the target we want to predict) and one or more independent variables (the features). In its simplest form, this relationship is modeled with a linear equation, such as:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Here, Y is the predicted value (e.g., CO2 emissions), X is the input feature (e.g., traffic speed), β_1 is the coefficient representing the feature's impact, β_0 is the intercept, and ϵ (epsilon) represents the error. The primary **goal of regression** is to find the optimal coefficients (β_0 and β_1) that minimize this error.

This foundational concept has evolved far beyond simple lines into more sophisticated techniques. Methods like **Ridge and Lasso regression**, for instance, introduce regularization to handle multiple features and prevent overfitting. However, to capture the highly complex, non-linear, and temporal patterns found in traffic data, we must turn to the advanced machine learning models discussed throughout the remainder of this chapter.

2.2.2 What is ensemble learning

While a single, finely-tuned model can be effective, its predictions are often limited by its specific view of the data. A powerful strategy to overcome the limitations of any individual model is **Ensemble Learning**. The core principle is that by combining the predictions from a diverse group of models, the collective "wisdom of the crowd" can produce a final prediction that is more accurate and robust than any single model could achieve on its own. This approach is central to modern machine learning for its ability to reduce variance, mitigate overfitting, and improve overall performance on complex regression and classification tasks.

In statistical learning theory an ensemble is defined as a collection of k predictive functions $\{h_1, \dots, h_k\}$ whose individual outputs are merged by an aggregation rule G (for example, a mean, a weighted vote, or a meta-regressor). Formally, the ensemble prediction is

$$H(x) = G(h_1(x), h_2(x), \dots, h_k(x)),$$

and, provided the errors of the h_i are not perfectly correlated, the expected error of H is lower than that of any single h_i because the independent mistakes tend to cancel out.

Translated into machine-learning practice, ensemble learning is the technique of training several separate models, called base learners, and then combining their outputs to obtain one final estimate. When each base learner is purposely kept simple so that, on its own, it performs only marginally better, it is referred to as a weak learner. The aggregated model H , which usually outperforms all of its constituents, is known as a strong learner.

2.2.2.1 The bias variance tradeoff

In supervised learning, bias refers to the systematic error or deviation in a model's predictions from the true values. It represents the simplifying assumptions made by a model to make the target function easier to learn. While variance measures how much a model's predictions change from one training sample to another. High bias leads to under-fitting; high variance causes over-fitting. Ensembles improve this trade-off by averaging several weak learners. If each base model h_i is unbiased with variance σ^2 , the variance of the averaged ensemble drops to σ^2/k as long as the individual errors are not perfectly correlated. Therefore, collections of weak learners

often outperform a single, highly tuned model trained on the same data.

$$H(x) = \frac{1}{k} \sum_{i=1}^k h_i(x)$$

This bias–variance tradeoff offers several advantages. By combining multiple base learners—each of which may make different errors on different parts of the data, an ensemble can reduce overall prediction error more effectively than a single model. In particular, pooling complementary learners helps to “average out” their individual weaknesses. Some models may overestimate a certain outcome, while others underestimate it; their combination often balances these errors, resulting in more accurate predictions overall.

Ensemble methods are well-suited for real-world data, which is often noisy, limited, or drawn from diverse sources like sensors, vehicle logs, and cameras. A single model might overfit to inconsistencies in one input stream, but an ensemble can smooth over these irregularities to produce a more reliable output. Furthermore, techniques like bagging and boosting allow ensembles to make efficient use of small or highly variable datasets by resampling or reweighting the data. This is a critical advantage in emission studies, where obtaining extensive ground-truth data is often a major challenge.

Ensemble gain depends on the error diversity of the base learners. If every model makes the same mistakes, aggregation brings no benefit; if errors are uncorrelated, variance drops while bias rises only marginally, this can be enforced using techniques we will present in the next section.

2.2.3 Architectures of Ensemble Learning

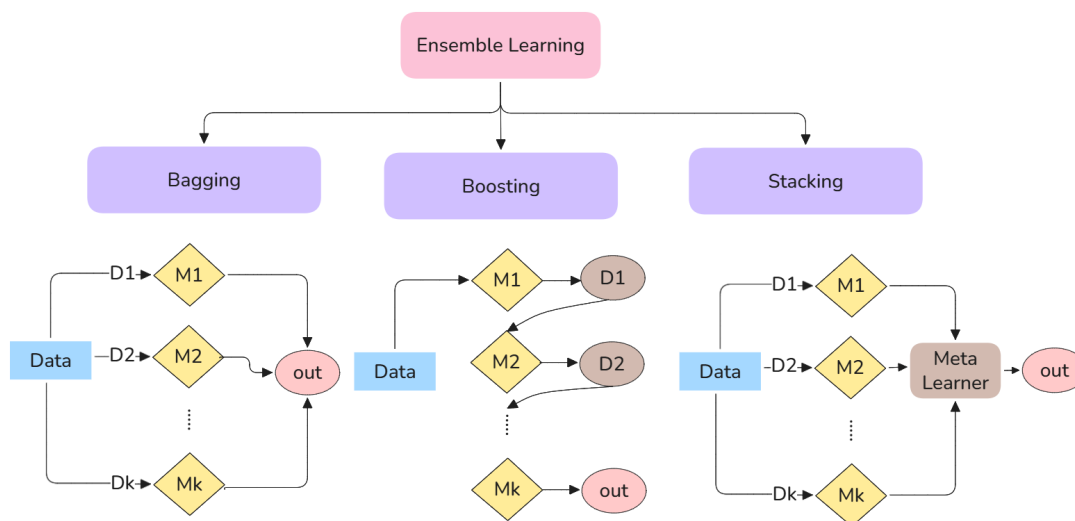


Figure 2.1: Ensemble families.

Ensemble techniques are typically categorized into three main families, distinguished by how they generate and combine base learners. These architectural differences, illustrated in Figure 2.1, fundamentally alter how the models reduce prediction error.

Bagging (Parallel Ensembles): Bootstrap Aggregating, or **bagging**, involves training multiple copies of the same base learner on different bootstrap samples drawn with replacement from the original dataset. Because each learner sees a slightly different subset of the data, their individual errors are only weakly correlated. Consequently, averaging their outputs reduces the variance term of the generalization error, often proportionally to the number of models, while leaving the bias term largely unchanged (Breiman, 1996).

Boosting (Sequential Ensembles): **Boosting** employs the opposite approach: base learners are added sequentially, with each new learner trained to correct the residual errors of the existing ensemble. This method essentially performs a form of gradient descent in function space, greedily minimizing a loss function $L(y, H)$ by adding weak learners h_m weighted by coefficients α_m (Friedman, 2001). This stage-wise procedure systematically converts a collection of high-bias weak learners into a single, low-bias strong predictor.

Stacking and Voting (Meta-Ensembles): These techniques combine predictions from multiple, often heterogeneous, base models. **Voting** is the simplest method, where outputs are aggregated using a fixed rule, such as taking the mean for regression tasks. **Stacking** takes this a step further by training a second-level "meta-learner." The predictions of the first-level models serve as input features for this meta-learner, which then learns the optimal way to combine them into a final prediction (Wolpert, 1992). While voting primarily reduces variance, stacking can reduce both bias and variance by learning the complex relationships between the base models' predictions.

The regression and ensemble methods detailed thus far represent powerful and well-established **classical Machine Learning** techniques. They excel at learning from tabular data where each sample can be treated as an independent entity. However, their core assumption of data independence is precisely their main limitation when analyzing traffic emissions. Real-world traffic data is highly correlated; it is a **time series** with deep, structural dependencies, where the past fundamentally shapes the future.

To capture these complex temporal patterns, a paradigm shift is required—from classical models that rely on manual feature engineering to architectures that can learn hierarchical representations directly from raw data. This marks our transition into the domain of **Deep Learning (DL)**. The following sections will explore the specialized neural network architectures that are purpose-built for sequential and structured data, beginning with the models that have become the cornerstone of modern time series analysis.

2.3 Time Series Modeling for Emission Forecasting

Having established the necessity of moving to architectures that understand temporal dependencies, this section examines the core deep learning models for time series analysis. We will begin by formally defining a time series and its key properties. Following this, we will explore the evolution of specialized neural network architectures designed to capture sequential patterns. The focus will be on the theoretical workings of these models, providing the groundwork for their practical application in emission prediction discussed in the next chapter.

2.3.1 Time Series Definition and Formulation

Formally, a **time series** is an ordered sequence of observations sampled at discrete, uniform time intervals. In the simplest, univariate case, it is a set of scalar values:

$$y = \{ y_t \in \mathbb{R} : t = 1, \dots, T \}$$

where t represents the timestamp.

However, emission data is often multivariate, where several variables (e.g., speed, fuel consumption, CO₂ output) are recorded simultaneously. This is represented as a sequence of m -dimensional vectors:

$$Y = \{ \mathbf{y}_t \in \mathbb{R}^m : t = 1, \dots, T \}, \quad \text{where} \quad \mathbf{y}_t = \begin{pmatrix} y_t^{(1)} \\ \vdots \\ y_t^{(m)} \end{pmatrix}.$$

Because each value depends on what happened just before, time-series observations are temporally autocorrelated, a property that sequence models exploit. An example of a time series in our context is second-by-second tailpipe CO₂ readings from a Portable Emission Measurement System (PEMS).

2.3.2 Time Series Decomposition

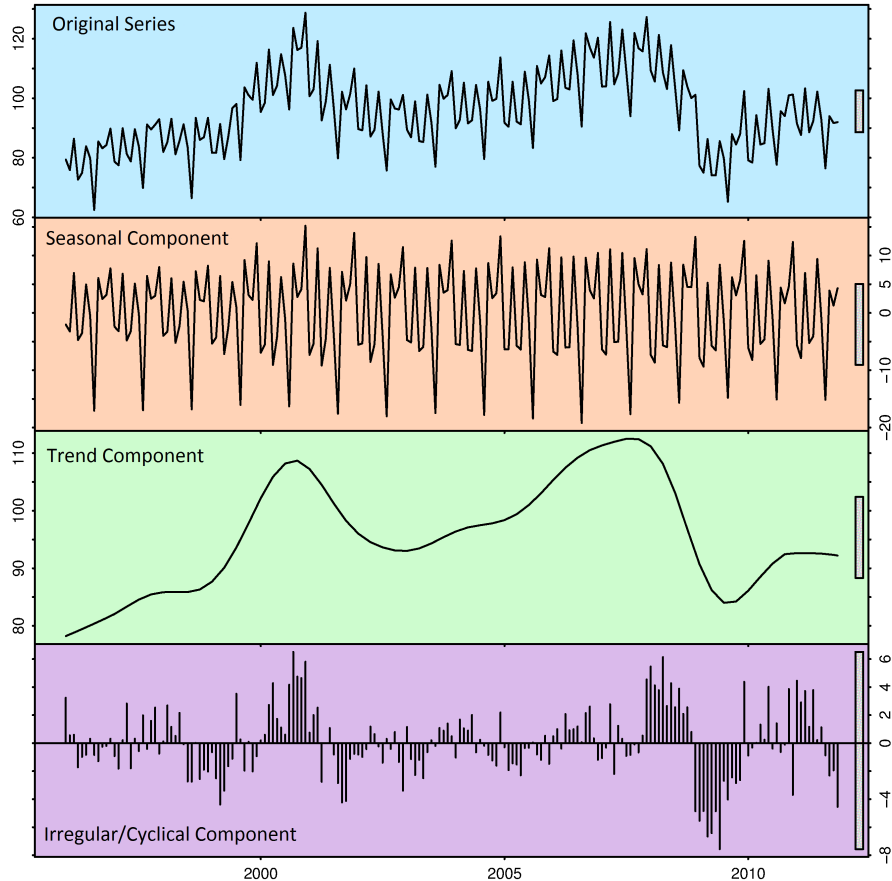


Figure 2.2: Example decomposition of a time series (Martinez, n.d.).

Following the classical decomposition of time series (Hyndman and Athanasopoulos, 2018), its can be written as the sum (additive case) or product (multiplicative case) of three parts:

$$\text{Additive: } y_t = T_t + S_t + R_t \quad \text{Multiplicative: } y_t = T_t \times S_t \times R_t,$$

where

- T_t (trend) is the long-term direction;
- S_t (seasonality) captures fixed-frequency cycles—e.g., the daily rush-hour spike in emissions;
- R_t (residual or irregularity) contains the irregular remainder due to incidents, weather, sensor noise, and other idiosyncratic factors.

An additive model suffices when the magnitude of the seasonal swing is independent of the series level; a multiplicative model is more common in economics, where seasonal amplitude scales with the trend. Figure 2.2 shows an example of the decomposition of a time series into its three seasonal, trend and residual/irregular components.

2.3.3 Sequence-based deep learning models

To effectively model the temporal autocorrelation inherent in time series data, a neural network needs a mechanism to maintain a "memory" of past information. Unlike standard feed-forward networks, which process inputs independently, sequence models require an architecture that can persist information across time steps. This is achieved through the principle of **recurrence**, where the output of the network at one step is fed back as an input for the next. The foundational architecture built on this concept is the Recurrent Neural Network (RNN).

2.3.3.1 Recurrent Neural Networks (RNN)

A Recurrent Neural Network (RNN) is a class of neural network specifically designed for sequential data. Unlike standard feed-forward networks, an RNN maintains an internal "memory" or **hidden state**, allowing it to persist information from previous time steps and influence future predictions.

2.3.3.1.1 The Recurrent Cell The fundamental building block of an RNN is the recurrent cell, illustrated in Figure 2.3. At each time step t , the cell performs two key operations: it updates its internal hidden state and produces an output. It takes two inputs:

- The current input vector from the sequence, $\mathbf{x}_t$.
- The hidden state from the previous time step, $\mathbf{h}_{t-1}$, which acts as the cell's memory.

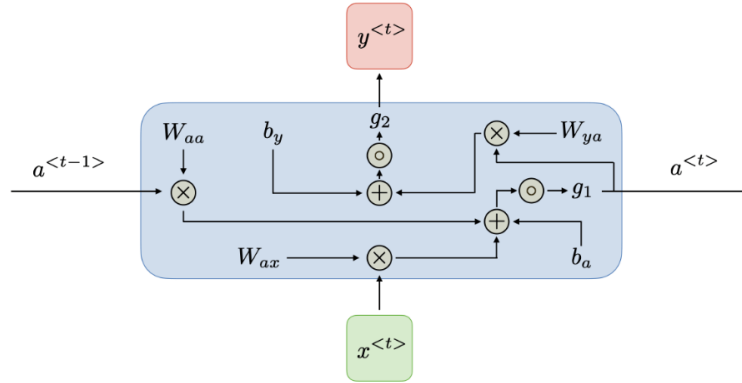


Figure 2.3: The structure of a single RNN cell (Amidi and Amidi, 2018).

These inputs are combined to compute the new hidden state $\mathbf{h}_t$ and the output for the current step, $\hat{\mathbf{y}}_t$, via the following equations:

$$\begin{aligned}\mathbf{h}_t &= \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h) \\ \hat{\mathbf{y}}_t &= \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y\end{aligned}$$

Here, $\mathbf{W}_{hh}$, $\mathbf{W}_{xh}$, and $\mathbf{W}_{hy}$ are weight matrices, and $\mathbf{b}_h$ and $\mathbf{b}_y$ are bias terms. The tanh function is a common non-linear activation function used to keep the hidden state values bounded.

2.3.3.1.2 Operation Through Time and Weight Sharing The true power of the RNN becomes clear when we "unroll" the cell through time, as shown in Figure 2.4. The same recurrent cell, with the exact same set of weights ($\mathbf{W}$), is applied at every time step. This principle of **weight sharing** is crucial: it allows the network to process sequences of any length with a fixed, and often small, number of parameters while applying the same learned logic at

each point in the sequence. If the input dimension is d , the hidden dimension is H , and the output dimension is o , the total number of trainable parameters is $H(d + H) + H + oH + o$.

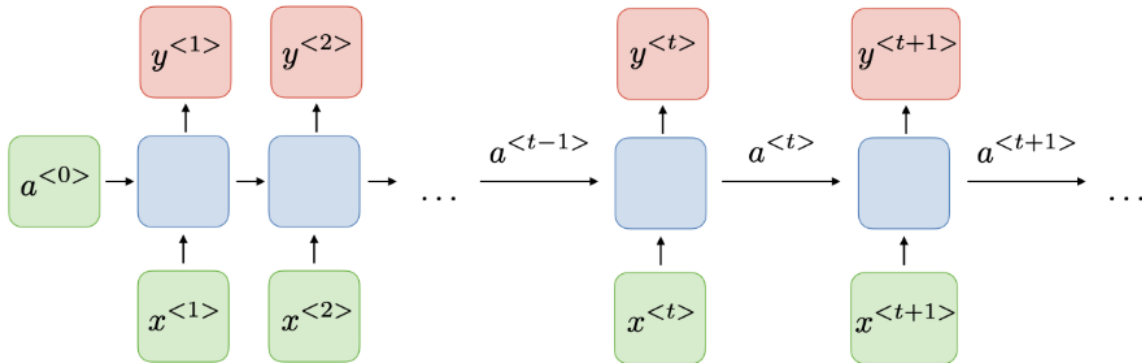


Figure 2.4: An RNN reuses the same cell and weights across timesteps. (Amidi and Amidi, 2018)

2.3.3.1.3 Limitations: The Vanishing Gradient Problem While elegant, the simple RNN architecture struggles to learn long-range dependencies. During training via BPTT, gradients must flow back through the entire sequence, as illustrated in Figure 2.4. The repeated multiplication by the weight matrix W_{hh} can cause the gradients to either shrink exponentially to zero (**vanishing gradients**) or grow uncontrollably (**exploding gradients**).

In practice, this means a simple RNN often fails to capture relationships beyond a few dozen time steps. For traffic emission data, where patterns can span several minutes, this is a significant drawback. This limitation led to the development of more sophisticated gated architectures like LSTMs and GRU, which we will explore next.

2.3.3.2 Long Short-Term Memory (LSTM)

The simple RNN struggles to learn long-term dependencies due to the vanishing gradient problem. The Long Short-Term Memory (LSTM) network, introduced by (Hochreiter and Schmidhuber, 1997), largely mitigates this issue. Its core innovation is a dedicated **cell state** vector, $\mathbf{c}_t$, that acts as a "conveyor belt" for information, allowing it to flow across many time steps with minimal degradation. This flow is meticulously controlled by a set of three **gates** that learn to dynamically add, remove, or read information from the cell state.

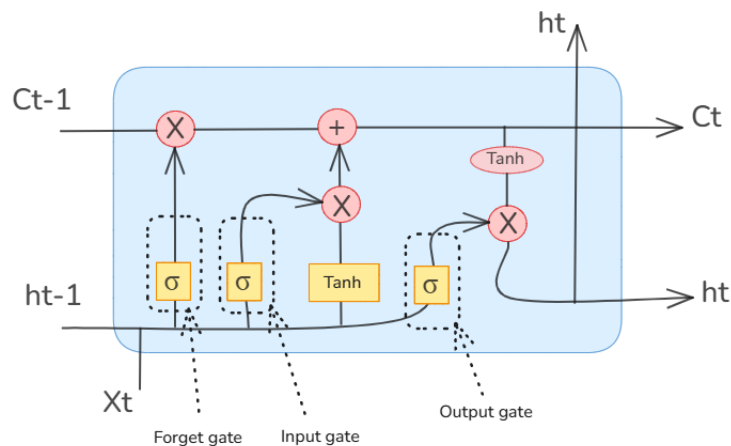


Figure 2.5: The architecture of a single LSTM cell (Wikipedia contributors, n.d.).

2.3.3.2.1 The Gating Mechanism As illustrated in Figure 2.5, the LSTM cell updates its state by passing the previous hidden state $\mathbf{h}_{t-1}$ and the current input $\mathbf{x}_t$ through a series of operations:

First, the **forget gate** (f_t) decides what information to discard from the previous cell state. Its sigmoid activation outputs values close to 0 for information to be forgotten and close to 1 for information to be kept.

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f)$$

Next, the **input gate** (i_t) determines what new information to store. It works in tandem with a tanh layer that creates a vector of new candidate values $\tilde{\mathbf{c}}_t$.

$$\begin{aligned}\mathbf{i}_t &= \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i) \\ \tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_c)\end{aligned}$$

The cell state is then updated by combining the forgotten past with the new candidate information. The element-wise addition in this step is the key to preserving the gradient flow.

$$\mathbf{c}_t = (\mathbf{f}_t \odot \mathbf{c}_{t-1}) + (\mathbf{i}_t \odot \tilde{\mathbf{c}}_t)$$

Finally, the **output gate** (o_t) determines the next hidden state, $\mathbf{h}_t$, which serves as the network's output for the current time step and as an input for the next.

$$\begin{aligned}\mathbf{o}_t &= \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o) \\ \mathbf{h}_t &= \mathbf{c}_t \odot \tanh(\mathbf{c}_t)\end{aligned}$$

2.3.3.2.2 Complexity and Bidirectional LSTMs This gating mechanism makes an LSTM roughly four times more complex than a simple RNN with the same hidden size, increasing computational cost but dramatically improving performance on long sequences.

The analysis can be further enhanced by using a **Bidirectional LSTM (Bi-LSTM)**, which processes the sequence with two separate LSTMs—one forward and one backward—and concatenates their hidden states $\mathbf{h}_t^{\text{bi}} = [\mathbf{h}_t^{\rightarrow}; \mathbf{h}_t^{\leftarrow}]$. This provides a richer representation by using both past and future context, though it makes the model non-causal and thus unsuitable for true real-time prediction, but they are ideal for offline/batch settings, or for streaming with a small *look-ahead* window (accepting latency equal to that window). They also roughly double parameters/compute and output a concatenated feature of size $2H$.

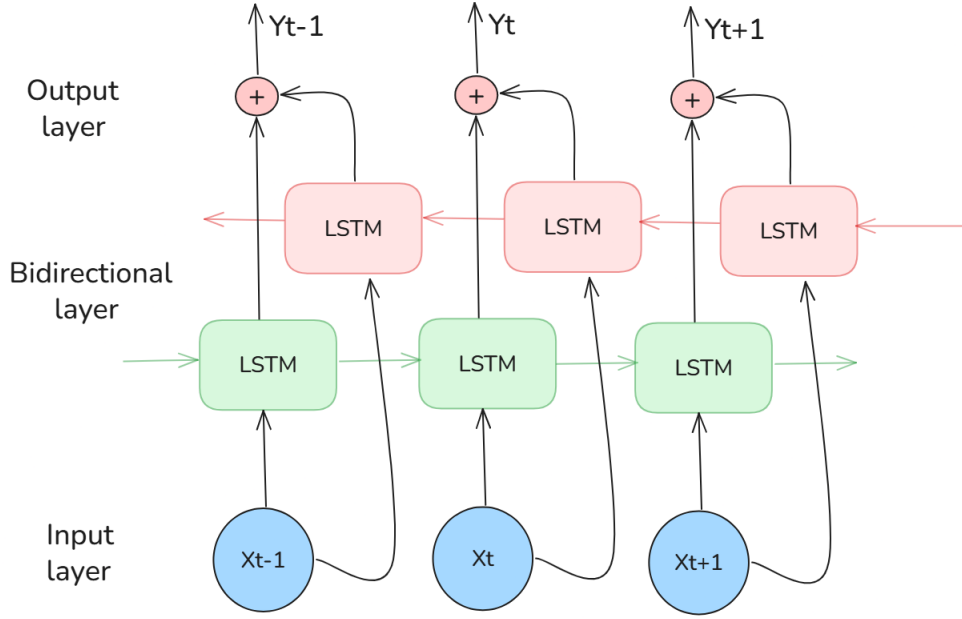


Figure 2.6: Architecture of a Bidirectional LSTM.

2.3.3.3 Gated Recurrent Units (GRU)

The Long Short-Term Memory (LSTM) cell solved the vanishing-gradient problem, but it did so at the price of additional complexity equal to roughly four times as many parameters as a simple RNN. On desktop GPUs that burden is tolerable; on embedded systems with limited processing power, it is not.

A more computationally efficient alternative emerged in 2014, when (Cho et al., 2014) was published. In an appendix they introduced the Gated Recurrent Unit (GRU), a cell that keeps the spirit of the LSTM’s gating mechanism but trims the architecture down to essentials. The GRU abandons the separate cell state and collapses the LSTM’s forget and input gates into a single update gate. Alongside a second reset gate, these two sigmoids control the flow of information with far fewer weights to learn.

As illustrated in Figure 2.7, the GRU cell at time-step t takes the current input x_t and the previous hidden state h_{t-1} and processes them through two primary gates:

2.3.3.3.1 Reset Gate (r_t): First, the reset gate decides how much of the past information to forget. It computes a value between 0 and 1 for each component of the hidden state. A value near 0 means “reset” or ignore the past, while a value near 1 means “keep” it.

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

2.3.3.3.2 Update Gate (z_t): The update gate determines how much of the past information to carry forward to the future. It creates a *candidate* hidden state, $\tilde{h}_t$, which contains the new information from the current time step, blended with the selectively filtered past information from the reset gate.

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h)$$

The update gate then calculates another value, z_t , which acts as a blending control.

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

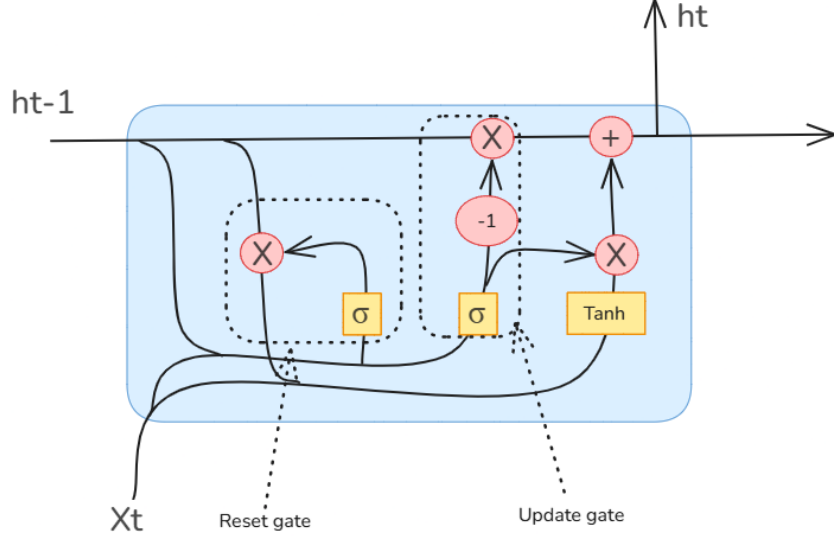


Figure 2.7: The internal data flow of a Gated Recurrent Unit (GRU) cell (Wikipedia contributors, n.d.).

The final hidden state, h_t , is then computed as a convex combination that balances between the old state (h_{t-1}) and the new candidate state ($\tilde{h}_t$). A z_t value near 1 means "keep the old memory," while a value near 0 means "overwrite it with the new candidate."

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$$

This streamlined design is what makes the GRU so effective. It retains the LSTM's ability to manage long-range dependencies but does so with roughly half the parameter count. This leads to faster training, lower memory usage, and makes the GRU a natural fit for real-time, on-line applications like traffic emission prediction.

2.3.3.4 1D Convolutional Neural Networks (CNNs) for Time Series

The recurrent architectures we have explored—RNNs, LSTMs, and GRUs—process sequences step by step. While powerful, this inherently sequential nature can become a computational bottleneck. This raises the question of whether there is a way to process information in parallel and reduce latency. One answer is to adapt CNN, traditionally used for images, to the time domain. The study of (Bai et al., 2018) showed that a simple Temporal Convolutional Network (TCN) can outperform canonical LSTMs and GRUs on a broad suite of sequence-modeling benchmarks, while training faster and modelling longer contexts—further evidence that CNNs can be effective beyond vision tasks.

A key advantage is parallelism: convolutions are evaluated across all time steps simultaneously, making efficient use of modern accelerators. In practice, this *largely alleviates* the long-range vanishing-gradient issues seen in vanilla RNNs (especially when combined with residual/normalization layers), and can enable *fixed-latency* inference under causal padding and fixed stride—useful when emissions must be forecast on low-power roadside hardware.

2.3.3.4.1 The 1D Convolutional Operation A 1D convolution replaces step-by-step recurrence with a sliding, learnable window. The core mechanism, illustrated in Figure 2.8, involves the convolution of an input sequence x with a kernel k of size K . At each time step t ,

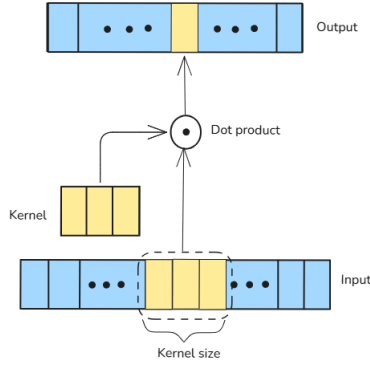


Figure 2.8: Diagram illustrating how 1D convolution works.

the output is the weighted sum of a local patch of the input:

$$(x * k)_t = \sum_{i=0}^{K-1} k_i \cdot x_{t-i}.$$

(Many deep-learning libraries implement the closely related *cross-correlation* operation without kernel reversal; the modeling effect is equivalent up to flipping k .)

With *causal* padding (left padding only), the output at time t depends only on inputs from time t and earlier, making it suitable for real-time prediction. The same kernel slides one step to the right, reuses its weights, and repeats, so a single mini-network is applied to every local patch of the signal. Stacking several such layers allows the model to detect increasingly complex patterns in the driving record: a short kernel might detect throttle blips; a deeper layer can combine them into a stop-and-go cycle. Because the multiply–accumulate operations are performed for all t in parallel, 1D convolutions *typically* train faster than recurrent layers. Unlike RNNs, they do not propagate gradients *through time* via a feedback loop; however, very deep conv stacks can still suffer from vanishing/exploding gradients without residual connections or normalization.

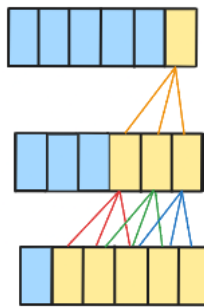


Figure 2.9: Two-stage convolution highlighting the receptive field (from bottom to top).

2.3.3.4.2 Receptive Field and Dilation A single convolutional layer has a limited *receptive field*—the portion of the sequence it can see. Stacking layers expands this field and builds a hierarchy of features, where higher layers combine patterns detected by lower ones.

To grow the receptive field efficiently without a large parameter increase, **dilated convolutions** skip inputs with a fixed step, allowing the kernel to cover a much wider temporal span. By stacking layers with exponentially increasing dilation rates (e.g., 1, 2, 4, 8), a network can achieve a very large receptive field with few layers (Figure 2.10); this is a key idea behind TCN-style architectures.

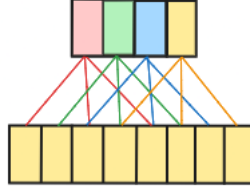


Figure 2.10: Dilated convolutions expand the receptive field by skipping inputs.

2.3.3.4.3 Stride and Padding Two final concepts control the output shape. **Stride** is the step size of the kernel. A stride of 1 produces an output for every input time step; a stride > 1 downsamples the sequence. **Padding** adds zeros so the kernel can apply at the sequence edges. “Same” padding preserves length by padding both ends; *causal* padding pads on the left only to prevent peeking into the future. Choice of padding affects both latency and edge effects.

2.3.4 The Transformer Architecture

The recurrent architectures we have explored—RNNs, LSTMs, and GRUs—all process sequences chronologically. This sequential nature, while intuitive, creates computational bottlenecks and struggles to capture very long-range dependencies. A paradigm shift occurred in 2017 when (Vaswani et al., 2017) published “Attention Is All You Need,” which dispensed with recurrence entirely and introduced the **Transformer**. This architecture relies solely on a mechanism called **self-attention** to process all tokens in a sequence in parallel, revolutionizing the field.

This parallel approach allows the model to directly weigh the influence of every token on every other token, capturing complex dependencies regardless of their distance. The result is a model that not only trains significantly faster on modern GPUs but has also become the foundation for nearly all state-of-the-art models across numerous domains.

2.3.4.1 Scaled Dot-Product Attention and Self-Attention

The fundamental building block of the Transformer is **scaled dot-product attention**. This mechanism allows the model to create a contextual representation for each element in a sequence by calculating its relationship with all other elements. It operates on three learned vectors for each input: a **Query (Q)**, a **Key (K)**, and a **Value (V)**.

The process is captured in a single, powerful equation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

As illustrated in Figure 2.11, this operation involves three steps:

1. **Score:** The dot product of the query matrix Q with the key matrix $K^\top$ is computed to get a matrix of similarity scores.
2. **Scale and Normalize:** The scores are scaled by $\sqrt{d_k}$ to aid training stability. A softmax function is then applied to convert the scores into attention weights.
3. **Output:** The final representation is computed as a weighted sum of the Value vectors V , using the attention weights.

2.3.4.1.1 Self-Attention It is the specific application of this mechanism where the Queries, Keys, and Values are all derived from the *same* input sequence. In essence, this allows the sequence to “attend to itself,” enabling each position to learn a contextual representation based

on its relationship with all other positions in that same sequence. This is the key mechanism that allows the Transformer to understand context without recurrence.

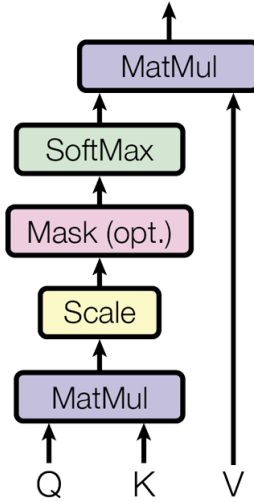


Figure 2.11: Scaled dot-product attention mechanism, as presented in (Vaswani et al., 2017).

2.3.4.2 Multi-Head Attention

Instead of relying on a single attention function, the Transformer enhances its power through **Multi-Head Attention**, shown in Figure 2.12. This module runs the scaled dot-product attention mechanism multiple times in parallel. Each "head" learns a different linear projection of the Q, K, and V vectors, allowing it to focus on different aspects of the sequence's relationships.

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad \text{where} \quad \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

The outputs of all heads are concatenated and passed through a final linear layer, allowing the model to integrate these diverse perspectives.

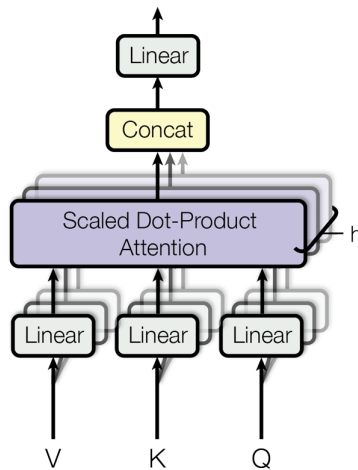


Figure 2.12: Multi-Head Attention as presented in (Vaswani et al., 2017) .

2.3.4.3 Positional Information

Since the self-attention mechanism is permutation-invariant, the model needs an explicit way to understand the sequence's order. This is solved by injecting **positional encodings** into

the input embeddings. While the original paper used sinusoidal functions, subsequent research has explored other effective methods like learned absolute embeddings or relative positional encodings.

2.3.4.4 The Full Encoder-Decoder Architecture

These components are assembled into the Transformer’s classic **encoder-decoder architecture**, shown in Figure 2.13.

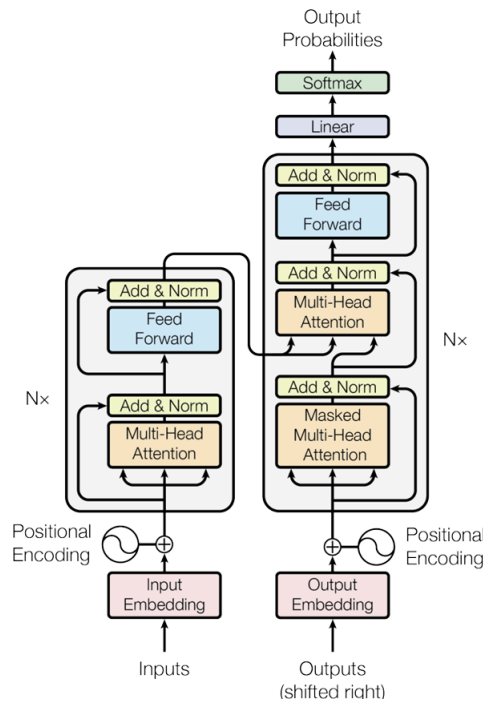


Figure 2.13: The full Transformer architecture (Vaswani et al., 2017).

2.3.4.4.1 The Encoder The encoder builds a rich, contextual representation of the input sequence. It consists of a stack of identical layers, each containing a Multi-Head Self-Attention layer and a Position-wise Feed-Forward Network (FFN), with residual connections and layer normalization around each sub-layer.

2.3.4.4.2 The Decoder The decoder generates the output sequence one element at a time. It contains a stack of similar layers but with a third sub-layer for **cross-attention**, where the decoder focuses on the output of the encoder. The decoder’s self-attention layer is also “masked” to prevent it from attending to future positions.

The Transformer’s ability to model long-range dependencies and its parallelizable nature have made it highly successful for time series forecasting. Unlike RNNs, it can directly relate a point in time to a seasonal pattern from a year ago just as easily as it relates it to the previous time step. This has led to state-of-the-art results on many forecasting benchmarks.

However, this power comes with a significant limitation. The core self-attention mechanism has a computational and memory complexity that scales quadratically with the sequence length ($O(T^2)$). This makes the standard Transformer prohibitively expensive for very long time series, a common scenario in emission modeling. This bottleneck has spurred a wave of research into more efficient, “sparse” attention mechanisms designed to approximate the full attention matrix while reducing the computational burden.

2.4 Spatio-Temporal Modeling for Emission Estimation

The field of traffic analysis naturally lends itself to graph-based data representation. Traffic data can be effectively modeled as a network, where the nodes and edges represent different aspects depending on the context. In one scenario, the road network itself serves as the graph structure, with intersections as nodes and roads as edges. In another, the graph can be formed based on the connectivity between IoT sensors deployed across the urban infrastructure—each sensor acting as a node and the logical or physical communication links between them forming the edges.

In both cases, a strong spatial and temporal correlation exists among the data collected from different locations in the network. Traffic dynamics such as congestion, speed variations, and vehicle density tend to propagate through adjacent roads, forming a flow of information across the network. This interconnected nature of traffic data gives rise to complex relationships that traditional deep learning models, which assume independent and identically distributed inputs, often fail to capture effectively. As noted by (DATA INTELLIGENCE, 2023), such models are not designed to account for the relational structure inherent in many real-world datasets, making them less suitable for traffic network modeling.

Graph Neural Networks (GNNs), on the other hand, are specifically designed to operate on graph-structured data. They excel at capturing the dependencies between connected nodes and can model both local and global interactions across the network. This capability has prompted a shift in the research community—from conventional deep learning techniques to GNN-based approaches—for traffic forecasting, congestion detection, emission predictions and route optimization.

In the following sections, we will delve deeper into how traffic data can be represented as graphs, how GNNs can uncover latent patterns and dependencies within this structure, and how these models are revolutionizing the field of traffic prediction and intelligent transportation systems.

2.4.1 Spatial Modeling in Deep Learning Models

In this section, we'll look at how spatial relationships are taken into account in deep learning models—starting from how data is formatted and represented in graph form, to how this information is actually used within different spatial models from the GNN family.

2.4.1.1 Graph Based Representation Data

In this section, we delve deeper into the definitions, key concepts, and terminologies necessary to establish our understanding of graph modeling. This is followed by an overview of how various types of data can be represented as graphs in network-based systems.

2.4.1.1.1 Definitions and Terminologies

In mathematics, a **graph** is a fundamental structure defined as an ordered pair $G = (V, E)$, where $V = \{v_1, v_2, \dots, v_n\}$ is a set of **nodes (vertices)**, and $E \subseteq \{(v_i, v_j) \mid v_i, v_j \in V\}$ is a set of **edges** representing relationships between nodes. A graph is said to be **undirected** as in figure 2.14 if $(v_i, v_j) \in E$ implies $(v_j, v_i) \in E$, meaning that the edges carry no direction. Graph connectivity can be encoded using the **adjacency matrix** $A \in \mathbb{R}^{n \times n}$, where $A_{ij} = 1$ if $(v_i, v_j) \in E$, and 0 otherwise. The **neighborhood** of a node v_i , denoted $\mathcal{N}(v_i)$, is the set of all nodes directly connected to v_i . The **degree** of a node v_i , given by $\deg(v_i) = |\mathcal{N}(v_i)|$, is used to

build the **degree matrix** $D \in \mathbb{R}^{n \times n}$, a diagonal matrix where $D_{ii} = \deg(v_i)$ and $D_{ij} = 0$ for $i \neq j$. These matrices are crucial in constructing the graph Laplacian and performing operations such as message passing in **Graph Neural Networks (GNNs)**((Bondy and Murty, 2021)).

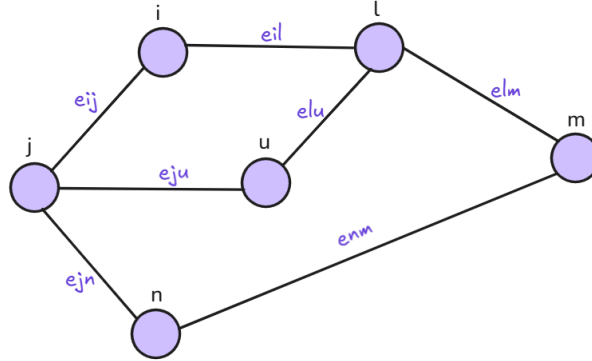


Figure 2.14: Undirected Graph.

2.4.1.1.2 Types of Graphs :

Directed Graphs A graph $G = (V, E)$ is said to be *directed* (or a *digraph*) if each edge has a specified direction. In such graphs, edges are represented as ordered pairs (v_i, v_j) , indicating a connection from node v_i (the source) to node v_j (the target) (Diestel, 2017). This implies that the edge from v_i to v_j , denoted e_{ij} , is not necessarily equal to the reverse edge e_{ji} . Directionality is critical in numerous real-world applications such as web hyperlink structures, citation graphs, and especially traffic networks, where movement often follows fixed paths (e.g., one-way streets).

In directed graphs as we can see in figure2.15, each node has two types of neighbors:

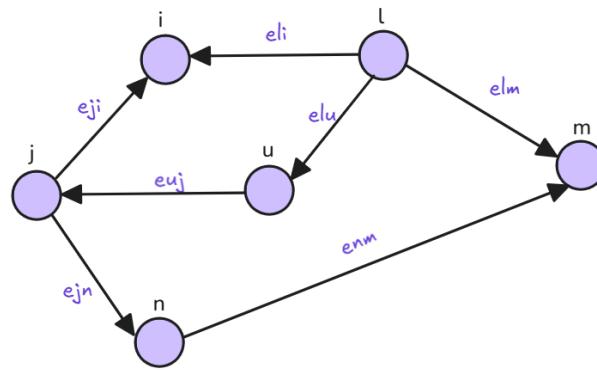


Figure 2.15: Directed Graph.

- **Out-neighborhood:** The set of nodes to which v_i has outgoing edges, denoted $\mathcal{N}_{\text{out}}(v_i) = \{v_j \in V \mid (v_i, v_j) \in E\}$.
- **In-neighborhood:** The set of nodes from which v_i receives incoming edges, denoted $\mathcal{N}_{\text{in}}(v_i) = \{v_j \in V \mid (v_j, v_i) \in E\}$.

Attributed or Featured Graphs In many real-world applications, graphs are not only defined by their structure (i.e., nodes and edges), but also by additional descriptive information associated with these elements. This gives rise to the concept of a *featured graph* (also known as an *attributed graph*) see figure 2.16, in which both nodes and edges are associated with feature vectors capturing properties relevant to the specific task. Formally, a featured graph is defined as $G = (V, E, X, \varepsilon)$, where V denotes the set of nodes, and $E \subseteq V \times V$ represents the set of edges. The function $X : V \rightarrow \mathbb{R}^d$ assigns each node $v_i \in V$ a d -dimensional feature vector $x_i \in \mathbb{R}^d$, while $\varepsilon : E \rightarrow \mathbb{R}^f$ assigns each edge $(v_i, v_j) \in E$ an f -dimensional edge feature vector $\varepsilon_{ij} \in \mathbb{R}^f$.

These features are domain-specific and vary by application. For instance, in a social network, node features may encode user-specific information such as age, interests, or geographic location, whereas edge features may represent interaction strength, frequency of communication, or type of relationship between users.

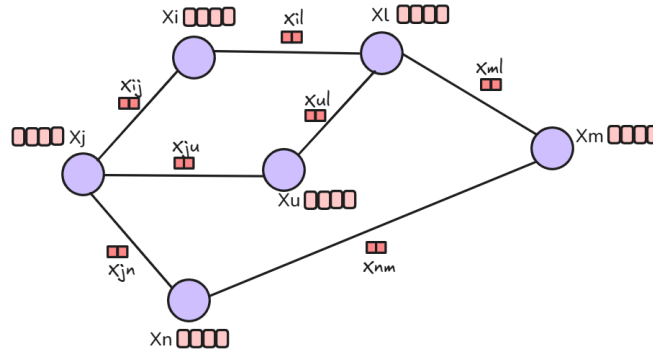


Figure 2.16: Attributed Graph Representation

Heterogeneous Graphs In real-world scenarios, graphs are often *heterogeneous*, meaning they contain multiple types of nodes and/or edges, each representing different entities and relationships. Unlike homogeneous graphs—where all nodes and edges are of a single type—heterogeneous graphs introduced in (Ghosh and Lerman, 2009) enable the modeling of complex systems with semantically rich structures.

Formally, a heterogeneous graph is defined as $G = (V, E, T_V, T_E)$, where V is the set of nodes, $E \subseteq V \times V$ is the set of edges, $T_V : V \rightarrow \mathcal{A}$ assigns each node a type from the set of node types $\mathcal{A}$, and $T_E : E \rightarrow \mathcal{R}$ assigns each edge a relation type from the set of edge types $\mathcal{R}$.

For example, consider a heterogeneous graph constructed from Wikipedia. Nodes represent both authors and articles, so $\mathcal{A} = \{\text{Author}, \text{Article}\}$. Edges encode various relationships such as co-authorship (between two authors who collaborated on the same article), topical similarity (between authors with similar expertise or research domains), and authorship (from an author node to an article node, indicating that the author contributed to the article).

Each relation type $r \in \mathcal{R}$ can be defined as a binary predicate $r : V_a \times V_b \rightarrow \{0, 1\}$, with $V_a, V_b \subseteq V$ being specific node types. Moreover, each node and edge type may have its own feature space: node features are denoted $x_v^{(a)} \in \mathbb{R}^{d_a}$ for node v of type $a \in \mathcal{A}$, and edge features are denoted $e_{uv}^{(r)} \in \mathbb{R}^{k_r}$ for edge (u, v) of relation type $r \in \mathcal{R}$.

Heterogeneous graphs are widely used in knowledge graphs, recommendation systems, bibliographic networks, and biological interaction modeling due to their expressiveness and structural richness.

Static Vs Dynamic Graphs. Graphs can generally be categorized into two main types: static and dynamic, depending on whether their structure or associated attributes evolve over time. A static graph is characterized by a fixed structure and unchanging features. In this case, once the node set V , edge set E , and their corresponding attributes are established, they remain constant throughout the analysis. Such a graph can be represented as $G = (V, E, X, E_f)$, where $X : V \rightarrow \mathbb{R}^d$ assigns feature vectors to nodes and $E_f : E \rightarrow \mathbb{R}^k$ assigns feature vectors to edges. Static graphs are suitable for modeling systems with stable relationships, such as molecular structures.

In contrast, dynamic graphs—also known as temporal graphs figure2.17 capture systems that evolve over time. These graphs can exhibit dynamics in either their attributes or structure. In the case of dynamic attributes, the graph topology remains fixed (i.e., the sets V and E are constant), but the features associated with nodes and edges change over time. This type of graph is typically represented as $G_t = (V, E, X_t, E_{f,t})$, where the subscript t denotes the temporal aspect. A practical example is a computer network in which the physical connections remain the same, while traffic loads or system states fluctuate.

More generally, a dynamic graph can also involve a changing structure, where both the topology and features vary with time. In this setting, the graph at each time step t is expressed as $G_t = (V_t, E_t, X_t, E_{f,t})$. This formulation is suitable for highly flexible systems such as IoT sensor networks, where new sensors (nodes) may be deployed or removed, and communication links (edges) may appear or vanish as the system evolves.

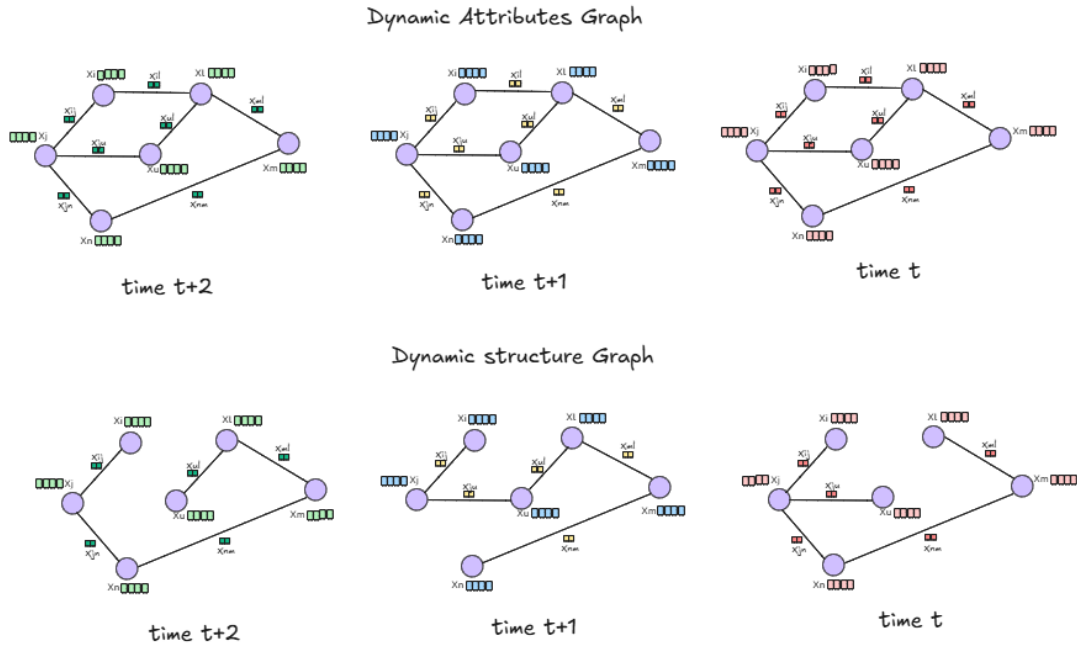


Figure 2.17: Attribute and Structure Dynamic Graphs.

2.4.1.2 Graph Learning :

2.4.1.2.1 ChebNet: CNN on Graphs with Fast Localized Spectral One of the earliest attempts at learning on graph-structured data is the **ChebNet** model, introduced by (Defferrard et al., 2016). The goal of this model is to extend the concept of traditional convolutional neural networks (CNNs) to non-Euclidean domains such as graphs. ChebNet is built upon spectral graph theory and operates in the spectral (frequency) domain of the graph.

This model relies on the **graph Laplacian**, which captures the graph’s structure. The Normalized Laplacian matrix L can be defined as:

$$\mathcal{L} = I - D^{-1/2}AD^{-1/2}$$

where A is the adjacency matrix of the graph, and D is the diagonal degree matrix. The graph Fourier transform is computed by performing eigen value decomposition of the Laplacian:

$$\mathcal{L} = U\Lambda U^\top$$

where U is the matrix of eigenvectors and Λ is the diagonal matrix of eigenvalues. The graph signal $x \in \mathbb{R}^n$ is transformed to the spectral domain using:

$$\hat{x} = U^\top x$$

After applying a spectral filter $g_\theta(\Lambda)$ in the frequency domain, the signal is transformed back using the inverse Fourier transform:

$$x = U\hat{x}$$

The full spectral convolution is then expressed as:

$$g_\theta * x = U g_\theta(\Lambda) U^\top x$$

However, computing the eigendecomposition of the Laplacian is computationally expensive (with complexity $\mathcal{O}(n^3)$). To overcome this, ChebNet proposes an efficient approximation using **Chebyshev polynomials**. The spectral filter is approximated by a K -order Chebyshev expansion:

$$g_\theta * x \approx \sum_{k=0}^K \theta_k T_k(\tilde{\mathcal{L}})x$$

where $T_k(\cdot)$ denotes the Chebyshev polynomial of degree k , $\tilde{\mathcal{L}} = \frac{2}{\lambda_{\max}}\mathcal{L} - I$, and $\lambda_{\max}$ is the largest eigenvalue of $\mathcal{L}$.

2.4.1.2.2 Graph Convolution Networks : GCN Afterwards, a new graph-based neural network was introduced in (Kipf and Welling, 2017), namely the *Graph Convolutional Network (GCN)*, which was proposed as a simpler and more effective alternative to spectral graph convolution methods. Unlike earlier approaches that relied on the full computation or approximation of the eigenvalues of the graph Laplacian, GCN approximates the spectral convolution by employing a first-order Chebyshev polynomial, i.e., setting the polynomial order $K = 1$. This restriction limits information propagation to only the immediate (first-order) neighbors, thus reducing computational complexity while preserving performance.

To include each node’s own features during message passing, self-loops are added to the graph, leading to a modified adjacency matrix:

$$\hat{A} = A + I,$$

where A is the original adjacency matrix and I is the identity matrix. The corresponding degree matrix $\hat{D}$ is computed as $\hat{D}_{ii} = \sum_j \hat{A}_{ij}$. Using this, the normalized adjacency matrix becomes:

$$\tilde{A} = \hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}},$$

which ensures symmetric normalization and stabilizes the training process.

The GCN layer is then defined by the following propagation rule:

$$H^{(l+1)} = \sigma \left(\tilde{A} H^{(l)} W^{(l)} \right),$$

where $H^{(l)}$ denotes the matrix of node features at the l -th layer, $W^{(l)}$ is the trainable weight matrix at that layer, $\sigma(\cdot)$ is a non-linear activation function (such as ReLU), and $\tilde{A}$ is the normalized adjacency matrix with added self-loops. This formulation allows for efficient layer-wise learning on graph-structured data while maintaining locality and computational efficiency.

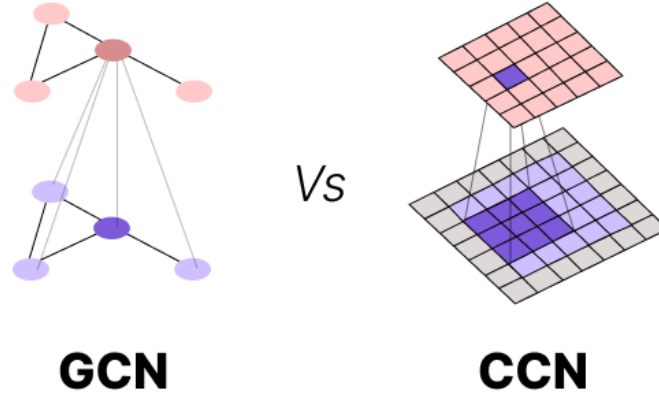


Figure 2.18: Analogy between CNNs and GCNs (Pham, 2020).

2.4.1.2.3 Graph Attention Network : GAT In traditional GCN architectures, the features of neighboring nodes are aggregated either equally or based on the normalized adjacency matrix. However, this uniform treatment limits the model’s ability to distinguish between the relative importance of different neighbors, as not all neighbors contribute equally to the target node’s representation. To address this limitation, the Graph Attention Network (GAT) was introduced (Veličković et al., 2018). GAT employs an attention mechanism to assign a different importance coefficient to each neighboring node during the aggregation process. Specifically, the attention score between node i and its neighbor j is computed as:

$$e_{ij} = \text{LeakyReLU} \left(\vec{a}^\top [\mathbf{W} \vec{h}_i \parallel \mathbf{W} \vec{h}_j] \right)$$

where $\vec{h}_i$ and $\vec{h}_j$ are the input feature representations of node i and its neighbor j , respectively, $\mathbf{W}$ is a learnable weight matrix, $\vec{a}$ is a learnable attention vector, $\parallel$ denotes concatenation, and LeakyReLU is the activation function used to introduce non-linearity. The attention coefficients are then normalized across all neighbors of node i using a softmax function:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})}$$

Finally, the aggregated representation for node i is computed as a weighted sum of its neighbors’ transformed features:

$$\vec{h}'_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W} \vec{h}_j \right)$$

where σ is a non-linear activation function such as ELU. Moreover, GAT uses multi-head attention to stabilize the learning process. In the multi-head setup, K independent attention mechanisms are applied, and their outputs are either concatenated (in intermediate layers) or averaged (in the final layer).

2.4.2 Learning Paradigms in Graph Neural Networks

Following the taxonomy introduced by Hamilton et al. (Chami et al., 2020), learning on graphs can be categorized into two principal paradigms: *inductive* and *transductive* learning. These two paradigms differ fundamentally in how they handle unseen nodes or graphs during inference.

Inductive learning refers to the ability of a model to generalize to completely unseen graphs or nodes that were not part of the training data. In this setting, the model learns a function that can be applied to new graphs without needing to retrain. This is especially useful in real-world scenarios where the graph structure evolves over time or where new nodes appear frequently (e.g., social networks, citation graphs).

Transductive learning, on the other hand, assumes that the entire graph (including test nodes) is known at training time. The model leverages the full graph structure, and the learned embeddings are specific to the nodes seen during training. Consequently, transductive models cannot easily handle new nodes or graphs without retraining.

The following table summarizes the key differences between these two learning paradigms:

Aspect	Transductive Learning	Inductive Learning
Generalization	Only on nodes seen during training	Generalizes to unseen nodes or graphs
Graph Access at Training	Requires access to the full graph including test nodes	Trains on a subgraph or separate graph entirely
Embedding Usage	Node embeddings are learned directly and stored	Learns a function to compute embeddings on the fly
Examples	Label propagation, traditional GCNs (Kipf and Welling, 2017)	GraphSAGE (Hamilton et al., 2017), GAT (Veličković et al., 2018)
Scalability	Limited by size of the full graph	More scalable to large or dynamic graphs

Table 2.1: Comparison between Transductive and Inductive Learning in Graph Neural Networks

2.4.3 Spatio-Temporal Modeling: Concepts and Methods

In many real-world applications, such as traffic networks and sensor networks, data exhibits strong interdependencies between spatial and temporal dimensions. Traditional methods have relied on convolutional neural networks (CNNs) and recurrent neural networks (RNNs) to capture temporal patterns, while graph neural networks (GNNs) have been employed to model spatial interactions between nodes and their neighbors. However, using these architectures in isolation fails to fully capture the intricate spatiotemporal dependencies. This limitation has led to the emergence of a new class of models specifically designed to jointly consider both spatial and temporal dynamics.

2.4.3.1 Spatio-Temporal Graph Convolution Network : STGCN

The concept of jointly modeling spatial and temporal dependencies was first introduced in the seminal work of (Yu et al., 2018) applied to traffic forecasting, where the authors proposed a fully convolution model called the **Spatio-Temporal Graph Convolutional Network (STGCN)**. This architecture eliminates the use of recurrent structures in favor of temporal convolutions, achieving more efficient and faster training.

Each STGCN block is composed of three main components:

1. **Temporal Convolution:** A 1D convolution is applied along the temporal axis to capture temporal patterns within each node. Given input features $\mathbf{X} \in \mathbb{R}^{N \times F \times T}$ where N is the number of nodes, F the number of features, and T the number of time steps, the temporal convolution operates over the last dimension.
2. **Graph Convolution:** A spatial graph convolution is performed using the adjacency matrix $\mathbf{A}$ to model dependencies among connected nodes.
3. **Post-Spatial Temporal Convolution:** A second temporal convolution is applied to refine the features after spatial aggregation.

These spatio-temporal blocks can be stacked to deepen the model and enhance its ability to extract complex patterns. The overall output of the STGCN block can be expressed as:

$$\text{STGCN}(\mathbf{X}) = \text{Conv}_{\text{Time}}(\text{GConv}(\text{Conv}_{\text{Time}}(\mathbf{X})))$$

2.4.3.2 Diffusion convolutional recurrent neural networks : DCRNN

This model was proposed to explicitly handle directed graphs by simulating how information diffuses through space. It was introduced in (Li et al., 2018) for traffic forecasting, which remains one of the most common applications of spatiotemporal models. The core idea of the model lies in the use of a K -step bidirectional diffusion convolution, which captures both incoming and outgoing spatial dependencies over multiple hops. This diffusion process is defined by:

$$\text{DiffConv}(X) = \sum_{k=0}^K \left(D_O^{-1} A \right)^k X W_k^{(f)} + \sum_{k=0}^K \left(D_I^{-1} A^\top \right)^k X W_k^{(b)}$$

where A is the adjacency matrix of the graph, D_O and D_I are the out-degree and in-degree diagonal matrices, $W_k^{(f)}$ and $W_k^{(b)}$ are trainable weights for the forward and backward diffusion processes at step k , respectively.

For the temporal modeling, DCRNN replaces the standard linear transformations in a Gated Recurrent Unit (GRU) with the diffusion convolution operator described above. The resulting architecture, known as the Diffusion Convolutional GRU (DCGRU), enables the model to jointly capture temporal dynamics and spatial diffusion over directed graphs in an end-to-end trainable framework.

2.4.3.3 Graph WaveNet

The study in (Wu et al., 2019) proposed a novel spatio-temporal model, Graph WaveNet, to overcome two main limitations of previous approaches: the assumption of a fixed graph structure, and the difficulty in capturing long-term temporal dependencies. To address the first issue, the authors introduced an adaptive adjacency matrix that is learned during training rather than being pre-defined. This adaptive graph learning mechanism is formulated as:

$$A_{\text{adaptive}} = \text{softmax}(\text{ReLU}(E_1 E_2^\top))$$

where $E_1, E_2 \in \mathbb{R}^{N \times d}$ are learnable node embedding matrices, with N being the number of nodes and d the embedding dimension. This adaptive adjacency matrix is added to the fixed adjacency matrix in the same formula described in the DCRNN architecture for processing the graph convolution.

For modeling temporal dependencies, the model adopts dilated causal convolutions, which allow the receptive field to grow exponentially with depth, enabling effective learning of long-term patterns. The temporal convolution at layer l and dilation rate r is given by:

$$\text{Conv}_{\text{dilated}}(x)_t = \sum_{i=0}^{k-1} w_i \cdot x_{t-r \cdot i}$$

where k is the kernel size, r the dilation rate, and w_i the convolution weights.

The overall model combines spatial graph convolutions (both from the predefined and adaptive adjacency matrices) with dilated temporal convolutions in a residual block structure, as illustrated in Figure 2.19.

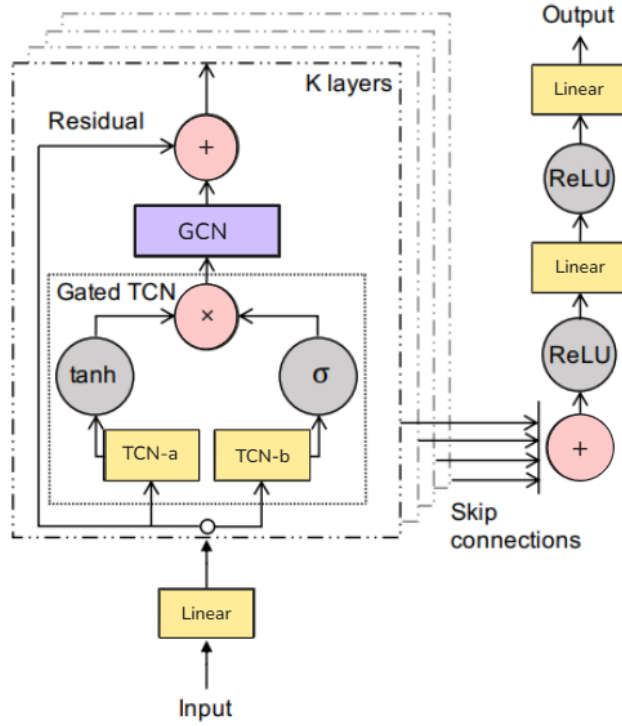


Figure 2.19: Architecture of Graph WaveNet Model (Wu et al., 2019).

2.4.3.4 Graph Multi-Attention Network : GMAN

(Zheng et al., 2020) GMAN is a spatio-temporal traffic forecasting model that leverages a multi-attention mechanism to learn dynamic spatial and temporal dependencies. It consists of three main components: input embedding, encoder/decoder with spatio-temporal attention blocks (STAtt), and output prediction.

- **Input Embedding:** Each node v_i at time t is embedded using spatial and temporal information:

$$H^0 = \text{ReLU}(XW_x + b_x) + \text{ReLU}(TW_t + b_t)$$

where X is the input feature, T is the time embedding (e.g., time-of-day, day-of-week), and W_x, W_t are learnable parameters.

- **Spatio-Temporal Attention Block (STAtt):** Each STAtt block applies multi-head attention in both space and time dimensions:

$$\text{STAtt}(H) = \text{GatedFusion}(\text{SpatialAtt}(H), \text{TemporalAtt}(H))$$

- **Gated Fusion:** Combines spatial and temporal outputs using a gating mechanism:

$$Z = \sigma(W_g \cdot [S; T]) \odot S + (1 - \sigma(W_g \cdot [S; T])) \odot T$$

where S and T are outputs of spatial and temporal attention, σ is the sigmoid function.

- The decoder outputs the future traffic states using stacked STAtt blocks, similar to the encoder.

2.5 Evaluation Metrics

In machine learning and deep learning applications, models must be compared using specific mathematical formulas or functions known as evaluation metrics. These metrics provide a standardized way to assess and compare the performance of different models. In the domain of forecasting, particularly time series forecasting, various evaluation metrics have been developed to capture different aspects of predictive accuracy. Evaluation metrics in time-series forecasting can be grouped according to whether they measure point-wise accuracy, distributional similarity, or the preservation of temporal and spectral structures. This categorization reflects the diverse objectives in emission prediction: capturing local accuracy, modeling uncertainty, and reproducing seasonal traffic patterns.

2.5.1 Point-wise Error Metrics.

These assess the discrepancy between predicted and observed values at each timestep:

- **Mean Squared Error (MSE)**

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Sensitive to large deviations, useful when penalizing outliers is important.

- **Root Mean Squared Error (RMSE)**

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

Expressed in the same units as the target, making it more interpretable.

- **Mean Absolute Error (MAE)**

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Robust to outliers, treats all deviations equally.

- **Coefficient of Determination (R^2)**

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$$

Measures the proportion of variance explained by the model (0 to 1).

2.5.2 Distribution-based Metrics.

These evaluate predictive distributions rather than single values, important for uncertainty-aware forecasting:

- **Quantile (Pinball) Loss**

$$L_\tau(y, \hat{y}) = \begin{cases} \tau(y - \hat{y}), & \text{if } y \geq \hat{y} \\ (1 - \tau)(\hat{y} - y), & \text{if } y < \hat{y} \end{cases}$$

Where $\tau \in (0, 1)$ is the quantile level. Assesses how well predicted quantiles (e.g., 10th, 50th, 90th percentiles) match observations. It enables asymmetric evaluation of under- and over-prediction.

- **Continuous Ranked Probability Score (CRPS)**

$$\text{CRPS}(F, y) = \int_{-\infty}^{+\infty} (F(z) - \mathbf{1}_{\{y \leq z\}})^2 dz$$

Measures the distance between the predicted cumulative distribution function (CDF) and the observed outcome, generalizing MAE to probabilistic forecasts.

- **Kullback–Leibler Divergence (KL)**

$$D_{KL}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

Measures divergence between predicted distribution Q and true distribution P . It quantifies how much information is lost when using the predicted distribution instead of the true one, asymmetric and sensitive to underestimation.

- **Jensen–Shannon Divergence (JS)**

$$D_{JS}(P \parallel Q) = \frac{1}{2} D_{KL}(P \parallel M) + \frac{1}{2} D_{KL}(Q \parallel M), \quad M = \frac{1}{2}(P + Q)$$

A symmetric and bounded version of KL divergence, commonly used to compare distributions in a more stable way.

2.5.3 Temporal and Spectral Metrics.

These evaluate the ability of forecasts to capture periodic and temporal structure:

- **Fourier / Spectral Loss**

$$L_{\text{Fourier}} = \|\mathcal{F}(y) - \mathcal{F}(\hat{y})\|^2$$

Where $\mathcal{F}$ is the Fourier transform. Compares the Fourier transforms of predicted and observed sequences, penalizing discrepancies in frequency components. This ensures that cyclic behaviors such as rush-hour peaks or seasonal variations are accurately reproduced.

- **Dynamic Time Warping (DTW)**

$$DTW(y, \hat{y}) = \min_{\pi} \sqrt{\sum_{(i,j) \in \pi} (y_i - \hat{y}_j)^2}$$

Measures similarity between sequences by finding an optimal alignment π that allows for shifts and distortions in time. It captures temporal dynamics even when patterns occur at slightly different times.

2.5.4 Task-specific Metrics.

Certain metrics are tailored to practical forecasting needs:

- **Mean Absolute Percentage Error (MAPE)**

$$MAPE = \frac{100}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Expresses prediction errors as percentages of actual values, making results easy to interpret across different scales. However, it becomes unstable when true values are close to zero.

- **Symmetric Mean Absolute Percentage Error (sMAPE)**

$$sMAPE = \frac{100}{N} \sum_{i=1}^N \frac{|y_i - \hat{y}_i|}{(|y_i| + |\hat{y}_i|)/2}$$

A modified version of MAPE that normalizes errors by the average of predicted and actual values, improving stability when observations are small.

- **Wasserstein Distance (Earth Mover's Distance)**

$$W(P, Q) = \inf_{\gamma \in \Gamma(P, Q)} \int |x - y| d\gamma(x, y)$$

Measures the cost of transforming predicted distribution Q into observed distribution P .

2.6 Conclusion

In this chapter, we have constructed a comprehensive theoretical foundation for the predictive models central to time series analysis. Our exploration systematically progressed from the foundational principles of Regression and Ensemble Learning to more sophisticated architectures, beginning with a detailed examination of sequence-based deep learning models for time series analysis, including Recurrent Neural Networks (RNNs), their gated variants like LSTM and GRU, and the powerful Transformer architecture. Recognizing that traffic is an inherently networked phenomenon, we then incorporated the spatial dimension by introducing Graph Neural Networks (GNNs) to model road network topology, culminating in a review of state-of-the-art hybrid architectures that concurrently capture spatio-temporal dependencies. Finally, we surveyed the critical evaluation metrics required to rigorously assess the performance of these diverse models.

Having established this extensive theoretical toolbox, the logical next step is to move from theory to practice. While we now understand the mechanics of these models, it is crucial to examine how they are empirically applied to the specific domain of traffic emission estimation

and forecasting. The next chapter will therefore survey the existing literature to analyze real-world implementations of these techniques. We will review empirical studies organized by model family, from classical machine learning to deep learning, and provide a comparative analysis of their performance and use cases. This will allow us to understand the practical strengths and weaknesses of each approach and identify the key factors and research orientations that currently drive the field.

Chapter 3

Applications of Machine and Deep Learning Models for Traffic Emission Estimation and Forecasting

3.1 Introduction

The previous chapters outlined the theoretical background, data landscape, and methodological tools required to understand traffic–emission modelling. We now turn to their concrete applications in the literature. This chapter reviews how different machine learning and deep learning families have been applied to estimate and predict traffic-related emissions, highlighting both methodological innovations and practical limitations.

Our goal is not simply to catalogue models, but to compare how they perform under diverse conditions: What types of data do they require? Which pollutants or temporal horizons can they reliably predict? How do they balance accuracy, interpretability, and computational cost?

To answer these questions, we adopt a model-centric taxonomy (Figure 3.1) and group studies by their methodological family—from regression and ensemble approaches to recurrent, convolutional, and attention-based architectures. For each group, we summarize representative studies, discuss their strengths and shortcomings, and point out common challenges in real-world deployment. This structured review provides a comparative view of the state of the art and sets the foundation for identifying promising research directions in traffic–emission forecasting.

3.2 Empirical Studies by Model Family

Following the model-centric taxonomy shown in Figure 3.1 (and introduced in the organisation fo the previous Chapter), this section begins our systematic review of the literature. We examine key studies grouped by the family of models they employ—from classical machine-learning approaches to modern deep-learning architectures. To ensure clear and consistent comparison, each study is analyzed through the same lens: *(i)* data sources and feature sets used, *(ii)* target pollutants and prediction horizons, *(iii)* performance against strong baselines, and *(iv)* principal contributions or limitations, including deployment notes when available. This structured approach allows us to form a balanced view of each family’s strengths and weaknesses for traffic-emission estimation and prediction.

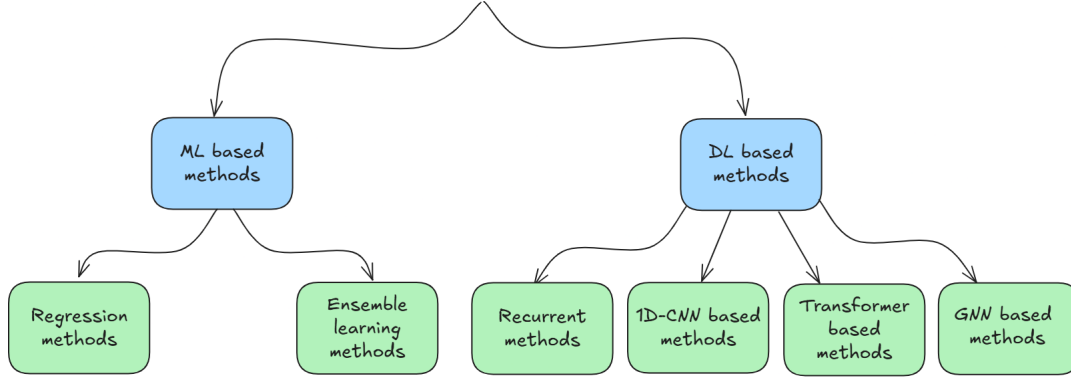


Figure 3.1: Model-centric taxonomy.

3.2.1 ML-based Methods

As outlined in our taxonomy, this first section is dedicated to applications using classical Machine Learning (ML) models. We will focus on two primary families: regression-based approaches, which model direct statistical relationships, and ensemble methods, which combine multiple models to improve predictive power. These techniques are frequently used in the literature for their robustness on tabular data and will provide a solid baseline for understanding the challenges of emission estimation.

3.2.1.1 Regression-based methods

In the emission literature, regression models serve three primary roles: as fast **baselines** on tabular data, as **calibration components** inside larger pipelines, and in **lightweight sensing setups** where computational resources are limited. While powerful for these tasks, regression is less often the final, primary model for fine-grained estimation. The following examples illustrate these distinct applications.

Regression models are powerful tools for establishing performance benchmarks in emission estimation. A study by (Vu and Ko, 2024) on Canada’s official vehicle dataset compares standard linear models against **Generalized Additive Models (GAMs)**. Unlike linear models that fit a single straight line, GAMs are more flexible, using smooth functions to capture complex, non-linear relationships between vehicle features and emissions.

The study found that while linear models perform exceptionally well when fuel consumption data is available, GAMs gain a slight predictive edge when this data is excluded. This highlights their ability to find more subtle patterns in the remaining features. However, this flexibility comes at a cost: GAMs require careful tuning through cross-validated smoothing to avoid overfitting to the training data. And in broader benchmarks, such as in (?), they occupy a middle ground between simpler, faster linear models and more computationally intensive but highly accurate tree-based ensembles like XGBoostpeerj, 18 models were evaluated (Linear/Ridge/LASSO/SVR/k-NN/Decision Tree; RF/XGB/LightGBM; and several DL models); tree-based ensembles achieved the best accuracy, while linear/regularized regression models offered the shortest runtimes.

Moving from baselines to practical application, regression is frequently used as a calibration component. An example in the study (Bilotta and Nesi, 2022) in which they estimate street-level CO₂ by splitting traffic into two regimes—*free-flow* and *congested/stop-go*—and fitting a small multilinear regression near each air-quality sensor:

$$\text{CO}_2 = a \cdot (\text{free-flow cars}) + b \cdot (\text{congested cars}) + \text{offset},$$

where a and b act as regime-specific emission factors. If a CO_2 probe has no co-located counter, local flows are first reconstructed from the road network and then used in the regression. Factors are re-estimated by *season*, and once learned, applied to traffic counts citywide to build a CO_2 map. In practice, the regressions were statistically strong and produced coherent patterns aligned with traffic and seasons, but they require periodic recalibration and careful time/space alignment; they are also less suited to second-by-second dynamics without richer inputs.

The third common use case arises in resource-constrained settings. Although not tail-pipe CO_2 , (MartínBaos et al., 2021) show why regression remains attractive: they proposed a low-cost IoT setup where a Raspberry Pi counts traffic using motion vectors, and simple ML models (Linear Regression, Random Forest, Gaussian Process) estimate AQI (Air Quality Index) from on-device sensors, all within a device→fog→cloud pipeline. In multi-city tests, the ML step cuts AQI error by about 3–4×, showing that cheap cameras plus basic sensors and lightweight ML can deliver useful real-time air-quality estimates without heavy calibration.

Another study with the same intention is (Park et al., 2023) developed a model to predict the *hourly on-road CO_2 enhancement* in Seoul, which they define as the traffic-induced CO_2 measured by a mobile sensor minus the urban background CO_2 measured at a fixed tower. Their model, a *regression-enhanced Random Forest*, first uses a Lasso regression and then applies a Random Forest to the residuals. The inputs include hourly traffic volume and speed, road and land-use classifications, and meteorological data such as wind speed. The model demonstrated good predictive skill, producing emission maps that clearly matched known traffic patterns and the diluting effects of wind. However, the model’s performance is highly dependent on dense sensor coverage and would be degraded by missing data. Consequently, transferring the model to other cities would likely require significant local retraining and recalibration.

While simple regression models are useful for establishing a baseline, they have inherent limitations for predicting complex phenomena like traffic emissions. Real-world emissions are the result of many interacting, non-linear processes that change over both space and time.

Classical regression models are not designed to understand these dynamics. They cannot learn hierarchical features from raw data, and they have no built-in way to understand the structure of a road network or the “memory” of traffic flow over time. While techniques like sliding windows can approximate these dynamics, they often miss long-range dependencies (like the effect of an event 10 minutes ago) and spillover effects between connected roads.

3.2.1.2 Ensemble learning methods

In the field of traffic emissions estimation, *ensemble learning methods* have gained significant attention. These methods are particularly appealing because they can handle noisy data, reduce overfitting, and provide better generalization. In the literature, most ensemble-based studies tend to fall into two main categories: those that rely on **boosting-based models**, and those that build more complex **stacking or hybrid ensemble architectures**. Each approach offers distinct advantages and is often chosen depending on the type of data, prediction goals, and computational constraints.

Several works use *gradient boosting methods* as their primary predictive tool. These models build strong learners by combining many weak learners (usually shallow decision trees) in a sequential manner, focusing on correcting previous errors at each step.

For example, (Wen et al., 2021) employed GBR on real-world emissions data collected via Portable Emission Measurement Systems (PEMS). The study systematically compared performance across different road types (urban, suburban, and highway) and found that boosting models consistently outperformed other techniques like Random Forests and simple neural networks. The study’s key contribution lies in its careful evaluation under different driving scenarios, showing the robustness of boosters across contexts.

Similarly, (Pradeep et al., 2024) developed an approach called *VER-XGBoost*, which used XGBoost to estimate emissions from road scenes. What makes this study notable is how it incorporated visual cues from road images—such as traffic signs and intersections—directly into the boosting pipeline alongside environmental and PEMS data. XGBoost’s built-in regularization and early stopping mechanisms proved useful for avoiding overfitting, especially in real-time, resource-constrained applications.

While boosting alone often performs well, some researchers have explored more complex ensemble strategies like *stacking*, where multiple models are trained in parallel and their outputs are combined by a final model (called a meta-learner).

(Yavari et al., 2023) proposed such an architecture in their *ArtEMon system*, using a weighted stacking ensemble of three boosting models—GB, XGBoost, and LightGBM. A fourth Gradient Boosting model acted as a meta-learner to combine their outputs. This setup was embedded in an Internet-of-Things (IoT) pipeline for real-time emissions estimation. The main advantage here is the end-to-end real-time capability with a robust fusion of predictions, making it more stable than any single model.

Taking things further, (Liu, Tang, Zhou and et al., 2024) introduced a hybrid stacking ensemble that combined traditional tabular models with deep learning. In the first layer, XGBoost was used alongside LSTM and attention-based neural networks to capture both structured and sequential patterns. The outputs of these models were then passed to a Random Forest meta-learner. This architecture effectively handled both static features and temporal dynamics, though it introduced higher computational cost and required careful tuning to avoid overfitting.

Finally, (Fei et al., 2019) proposed a three-layer stacking model, starting with base models like Random Forest, CatBoost, and Gradient Boosting. These were followed by a LASSO regression model to reduce overfitting, and then a final weighted average of predictions from LightGBM and XGBoost. While this design offered small accuracy gains, it required considerable engineering effort and cross-validation to perform reliably.

Despite their strengths, ensembles come with trade-offs. Large ensembles can be computationally expensive to train and deploy, especially when multiple models run in parallel or when deep stacks are used. Interpretation also becomes more difficult as model complexity increases. Most critically, bagging and boosting assume that each input record is independent—an assumption that fails in time-series or spatial contexts, where emissions often depend on previous states or neighbouring locations. As a result, researchers increasingly turn to sequence models (e.g., RNNs, Transformers) for time-aware tasks, and to graph neural networks when modelling road network interactions. Ensembles still provide a powerful starting point, but their limitations motivate the more specialised approaches explored in the next sections.

Table 3.1: Ensemble studies in traffic–emission estimation: technique, data, and scope (simplified).

Study	Ensemble technique	Inputs / Data	Scope (plain language)
(Wen et al., 2021)	Gradient Boosting Regression (boosting)	PEMS tailpipe signals + driving context	Predict emissions for trips/road segments in <i>urban, suburban, and highway</i> driving; compare against RF/ANN
(Pradeep et al., 2024)	XGBoost (boosting) within a visual eco-routing pipeline	Camera scene cues + environmental + PEMS/GPS	Estimate emissions per road segment using what the camera sees, then pick the <i>lowest-emission route</i> between two points
(Yavari et al., 2023)	Weighted stacking of GB, XGBoost, LightGBM	OBD/vehicle + ambient; analyzer-in-the-loop; IoT (vehicle→edge→cloud)	Provide <i>real-time</i> emissions for a vehicle (or fleet) from cheap signals, suitable for live monitoring
(Liu, Tang, Zhou and et al., 2024)	Heterogeneous stacking: base = XGBoost + LSTM + Attention; meta = Random Forest	Canadian tabular + temporal signals	Predict <i>per-segment</i> CO emissions on Canadian roads by combining time-series and static features
(Fei et al., 2019)	Three-layer stacking: RF/GBoost/CatBoost → LASSO → weighted fusion	Urban monitoring + meteorology (two cities)	Estimate <i>tailpipe CO and HC concentrations</i> for city roads using monitoring and weather data

3.2.2 DL-based Methods

Deep learning methods are more widely used than classical ML in recent traffic–emission work. Their appeal is twofold: they have delivered stronger results in practice, and they can represent temporal and spatial dependencies—while fusing heterogeneous inputs—that ML models usually approximate via heavy feature engineering. Following Figure 3.1, the next subsections focus on DL families.

3.2.2.1 Recurrent neural networks (RNN) for emission prediction

Although vanilla RNNs appeared in early traffic–emission studies, their use has remained limited. A simple RNN’s hidden state typically carries information only across a few dozen time steps before gradients vanish; by contrast, minute-scale emission dynamics often require hundreds (or thousands, for second-by-second data) of consecutive readings to capture stop-and-go cycles, gear shifts, and thermal lags. As a result, recent work tends to deploy plain RNNs only as baselines against gated variants. For example, (Singh and Dubey, 2021) compare a plain RNN with an LSTM on second-by-second telematics and report markedly lower CO₂ error for the LSTM; similarly, (Gurcan, 2024) include a simple RNN in a broad benchmark and find it near the bottom, with ensemble methods and LSTM variants preferred. The consensus is practical: vanilla RNNs are useful for pedagogical comparison, but long sequences and non-stationary regimes favor gated recurrent networks or attention-based models that mitigate vanishing gradients and model longer temporal dependencies.

3.2.2.2 LSTM Applications In Emissions Prediction

With the shift from static features to high-frequency traffic and environmental measurements, Long Short-Term Memory (LSTM) networks have emerged as a natural progression from plain recurrent neural networks (RNNs). Their gated cell architecture preserves gradients over longer sequences, allowing the model to capture dependencies that unfold over multiple timesteps. This capability is particularly important in emission forecasting, where patterns are shaped by stop-and-go traffic cycles, gear changes, and gradual thermal effects in engines. By processing sequences as a whole rather than in isolation, LSTMs can integrate both short- and long-term influences and performing better on the last one .

In the work of (Alfaseeh et al., 2020), an LSTM-based framework was developed to forecast link-level greenhouse gas (GHG) emission rates across the downtown Toronto road network. The authors generated their dataset using a calibrated traffic microsimulation coupled with the MOVES emission model, producing one-minute resolution sequences of speed, traffic density, in-link speeds, and historical GHG emission rates. Various LSTM configurations were tested, with the best-performing setup using the previous three minutes of these variables as inputs and two hidden layers. Results showed that the LSTM achieved higher correlation and lower RMSE than the ARIMA baselines with and without the clustering.

(Zhang et al., 2023) The study used second-by-second emission measurements from a chassis dynamometer, combined with information on ambient temperature, vehicle speed, and a binary indicator representing the use of a gasoline detergent fuel additive. The LSTM was configured to process sequences of these variables to predict CO₂ emissions in real time, with a particular focus on the initial minutes of operation when cold-start effects dominate. Results showed that including the fuel-additive indicator significantly improved prediction accuracy during the cold-start phase, reducing transient CO₂ error relative to the same architecture without the indicator. This highlights a key strength of LSTMs: their ability to integrate both dynamic driving variables and discrete categorical factors, thereby learning complex, data-driven relationships .

However, one of the main drawbacks of LSTM models is their computational complexity. A standard four-gate LSTM scales with both input dimensionality and hidden size, which increases memory usage, latency, and energy consumption. On embedded targets (e.g., vehicle microcontrollers, roadside Raspberry Pi), (Tena-Gago et al., 2023) highlight that CO₂ concentration prediction in hybrid vehicles is considerably more challenging than in conventional cars due to frequent power-train switch-overs. To address this, they propose UWS-LSTM, a lightweight recurrent model designed to capture hybrid mode transitions. For real-time deployment, the network is pruned to a single LSTM layer with 32 hidden units, demonstrating that this **tiny** model can still preserve switch-over dynamics and achieve nearly 97% accuracy while running efficiently on low-power IoT devices.

Tuning remains sensitive: short lookback windows fail to capture important temporal patterns, while long ones increase computational cost, slow convergence, and risk overfitting by incorporating redundant or noisy history. To address this, (Tang et al., 2023) embed a SSA around the LSTM to automatically optimize hyperparameters such as learning rate, sequence length, and hidden size based on validation loss. The SSA-LSTM achieved notable gains in forecasting accuracy—raising R^2 above 99% and cutting MAE and RMSE by nearly 40% and 28%, respectively.

To further enhance forecasting accuracy on vehicular emission data, many studies have included CNNs and attention along with the LSTM model architecture . (Braun et al., 2024) propose EcoPredict, a CNN-LSTM hybrid model evaluated under centralized, federated, and split learning deployments. Their results show that centralized training achieves the highest accuracy but raises communication and privacy concerns: federated learning preserves privacy

with only a modest accuracy drop, and split learning offers near-centralized accuracy but suffers from sequential training delays. Using real vehicular traces (Luxembourg and Cologne scenarios on SUMO), EcoPredict demonstrates that CNN–LSTM models with distributed training can predict urban CO₂ emissions with high fidelity—though the approach requires strict time alignment of inputs and imputation of missing ticks to maintain consistent sequences.

On the other hand, from attention–LSTM combinations we highlight the work of (Xie et al., 2021), who proposed a Parallel Attention-Based LSTM (PA-LSTM) to predict CO, NO, NO₂, and THC emissions. Their model employs temporal attention at three levels, in a parallel way using OBD data, and PEMS data, and across their fusion enabling the model to distinguish the contribution of each source. These weighted features are then encoded by an LSTM and combined with past emissions in a decoder to forecast future values.

Bidirectional LSTMs are less frequently applied in forecasting tasks, particularly in real-time scenarios, because they require maintaining two LSTMs in parallel, which increases computational cost. Nevertheless, Al-Nefaie and Aldhyani (Al-Nefaie and Aldhyani, 2023) demonstrated their effectiveness on the Canadian government’s vehicle CO₂ emissions dataset. The authors trained both LSTM and BiLSTM models on the data. Their results showed that the BiLSTM achieved the lowest RMSE values compared to the one-way LSTM.

At the same time, several limitations emerge consistently. Standard LSTMs are computationally expensive, making them difficult to deploy at scale or in resource-constrained IoT environments without pruning or simplification. Long input sequences, while beneficial for capturing dependencies, increase training cost and risk overfitting. Many works also depend on strict temporal alignment and data imputation, which limits applicability in real-world deployments where sensor data are often irregular or incomplete. Distributed approaches such as federated or split learning address privacy and communication challenges, but at the expense of either reduced accuracy or higher latency the table 3.2 present a summary for the previous works

Overall, LSTMs remain a powerful backbone for emission forecasting, but the literature suggests that their practical success depends on hybridization with other mechanisms (CNNs, attention) and on architectural adaptations to meet the constraints of real-time, distributed, and resource-limited deployment scenarios.

3.2.2.3 Gated Recurrent Units (GRUs)

Theoretically, GRUs are attractive for second-by-second emissions: they keep the key gating idea of LSTMs but with fewer parameters and faster convergence, which helps on small datasets and resource-constrained hardware. In practice, however, GRUs are *less common* than LSTMs in traffic-emission studies. The reasons are mostly pragmatic rather than fundamental: LSTMs are the incumbent choice with well-worn recipes, and when sequences are very long or effects are multi-modal, the extra capacity of an LSTM sometimes yields slightly better fits. In practice, GRUs appear either as the *main sequence model* when compute/data are tight, or as a *lightweight branch* inside hybrid pipelines.

(Shakya et al., 2023) build a GRU encoder–decoder to forecast PM_{2.5} at **1 h, 8 h, and 24 h** horizons, using two years of hourly records that combine air-quality observations, meteorology, and traffic volume. After basic EDA and scaling (min–max normalisation), the data are split **80/20** for training/testing. The encoder compresses the past 24 hours into a context vector; the decoder unrolls predictions for the chosen horizon. The GRU is evaluated against common baselines (Random Forest, XGBoost, a feedforward ANN, and an LSTM of matched depth/width). Results show the GRU *trains faster and generalises slightly better* than the matched LSTM on this seasonal, multi-source dataset; adding meteorology alongside traffic consistently improves accuracy across horizons. the set-up aggregates by site/hour and does not explicitly model spa-

Table 3.2: Summary of LSTM-based approaches for vehicle emissions prediction.

Study	Data	Model	Target	Strengths	Limitations
Alfaseeh & Tu (2020)	Microsimulation + MOVES	LSTM	GHG	Outperforms ARIMA, captures road-level dynamics	limited generalization
Zhang & Chen (2023)	Dynamometer + additive	LSTM	CO ₂	Captures cold-start, integrates categorical inputs	Lab setup, limited variability
Tena-Gago (2023)	Hybrid on-board	UWS-LSTM	CO ₂	Lightweight, runs on IoT.	specific, simplified model
Tang & Gong (2023)	Real driving emissions	SSA-LSTM	CO ₂	Hyperparameter auto-tuning, $R^2 > 0.99$, High fidelity, distributed deployment	High compute cost from SSA retraining
Chen (2024)	SUMO Lux/Cologne	CNN-LSTM + Fed/Split	CO ₂		Needs alignment, imputation, trade-offs in FL/SL
Xie & Zhang (2021)	OBD + PEMS	PA-LSTM	CO, NO, NO ₂ , THC	Multi-source integration, interpretable attention	Computationally heavy
Al-Nefaie (2023)	Canadian gov. dataset	BiLSTM	CO ₂	captures bidirectional deps.	Double cost, hard real-time use

tial spillovers between locations; generalisation to other cities depends on similar monitoring coverage and consistent sensor calibration.

To predict noisy, real-time NO_x emissions from diesel vehicles, (Li et al., 2021) developed a sophisticated "divide and conquer" strategy. Instead of feeding the entire complex signal into a single model, they first process it in several stages to isolate its different components.

First, the raw emission data is cleaned using **Singular Spectrum Analysis (SSA)** to reduce noise. Next, the cleaned signal is broken down into a set of simpler, more stable wave-like components using a technique called **ICEEMDAN** (Improved Complete Ensemble Empirical Mode Decomposition with Adaptive Noise).

These components are then sorted by their complexity. Using a metric called **sample entropy**, the signal is split into two streams: a "high-frequency" stream, which captures sharp, transient spikes caused by events like gear changes, and a "low-frequency" stream, which represents the slower, underlying emission trends.

This separation allows them to use the right tool for the right job. A **Gated Recurrent Unit (GRU)** is used to predict the complex, high-frequency parts. Simultaneously, a more efficient **Support Vector Regressor (SVR)** is used to model the simpler, low-frequency trends. By recombining the predictions from both models, this hybrid approach achieves higher accuracy and is more computationally efficient than using a single, heavy recurrent model for the entire signal. The key insight is that the GRU excels as a specialist on the most challenging parts of the signal when the simpler baseline is handled separately.

Gated Recurrent Units (GRUs) are a great choice when the sequence-modeling power of an LSTM is needed but with a lighter footprint, making them faster and less memory-intensive.

However, they still have limitations. Because they process data one step at a time, they can be slow for very long-term forecasting. For tasks involving extremely long patterns or highly unpredictable data, more advanced models like Temporal Convolutions or Transformers often perform better.

3.2.2.4 1-D CNN Applications In Emissions Prediction

More recently, an increasing number of studies have explored the use of one-dimensional Convolutional Neural Networks (1D-CNNs) for time-series analysis tasks, owing to their strong capability in capturing short-term temporal dependencies and their computational efficiency. In the context of traffic emission forecasting, 1D-CNNs have been widely adopted as a core component for extracting a variety of temporal dependency patterns from sensor data. Their strength lies in detecting fine-grained, localized patterns—such as short throttle bursts, transient sensor spikes, or abrupt wind gusts—while processing all time steps in parallel and requiring relatively few parameters. This combination of local pattern sensitivity, parallelizable architecture, and lightweight computational footprint makes 1D-CNNs particularly attractive for real-time or resource-constrained deployment scenarios.

In a first study, (Fei et al., 2021) introduce a Multi-Component Fusion Temporal Networks (MCFT-Net) architecture that explicitly models three types of temporal dependencies: **short-term (closeness)**, which captures relationships between emissions at time t and those in recent prior intervals (e.g., emissions at 9:00 AM with those at 8:00 AM); **daily (period)**, which captures dependencies at the same time slot across consecutive days (e.g., emissions at 8:00 AM today and yesterday); and **long-term (trend)**, which models seasonal variations in temporal dependencies (e.g., morning emission patterns from winter to summer). These dependencies are captured via three parallel one-dimensional CNN branches. External variables such as weather conditions and weekday type are modeled through a separate ANN. The outputs of the temporal branches are combined using a cascade fusion strategy, followed by an attention mechanism that adaptively weighs their contributions. The final emission prediction is obtained by merging the

weighted temporal representation with the external features.

While MCFT-Net provides a rich temporal modeling framework, several domain-specific limitations remain. First, the parallel CNN branches operate independently without explicit feature interaction before fusion, which may limit the ability to capture cross-scale temporal patterns. Second, the model assumes fixed periodicity for daily and long-term components, which can lead to degraded performance under nonstationary conditions such as traffic disruptions, special events, or abrupt policy changes. Finally, the evaluation is conducted on a single-city dataset without cross-domain validation, making it difficult to assess the generalizability of the learned temporal dependencies to other stations .

While the previous study employed a 1D-CNN as a stand-alone base model, other works have explored hybrid architectures that combine CNNs with additional temporal dependency extraction models, particularly LSTMs and temporal attention mechanisms. For example, (Nguyen et al., 2024) proposed a framework that integrates a CNN layer with an LSTM and a Bayesian Neural Network (BNN). The CNN component applies 1D convolution filters to time-series data—such as air pollutant levels and meteorological variables—to extract localized spatial features. This design effectively captures abrupt variations in pollution levels and weather patterns, thereby enhancing the model’s spatial sensitivity. To address data sparsity and quantify predictive uncertainty, the authors incorporated a BNN for probabilistic forecasting and employed Bayesian optimization for hyperparameter tuning. This hybrid approach reduced RMSE by 11 % compared to a stand-alone LSTM, while the Bayesian layer provided valuable uncertainty estimates (useful for setting alert thresholds), albeit at the cost of tripling memory usage and slowing down training.

Similarly, in (Xie et al., 2024), the authors tackled the challenges of carbon emission forecasting, including complex and imbalanced datasets as well as the nonlinear, non-stationary dynamics of meteorological and traffic variables. They introduced the MSCL-attention model, which integrates multi-scale feature extraction using 1D-CNNs with temporal modeling via LSTMs. The architecture trains CNN and LSTM layers in parallel: the CNN module extracts local features at three different scales, while a three-layer stacked LSTM captures long-term temporal dependencies. These representations are then fused using a multi-head self-attention mechanism, enabling the model to adaptively weigh features across scales and time horizons before passing them through a dense layer for final prediction.

*Taken together, 1D-CNNs act like a fast temporal **microscope**: they sharpen local fluctuations and offload raw pattern detection, while longer-range or relational reasoning is typically delegated to LSTMs, attention layers. They pay off most on noisy, second-by-second multivariate streams; on cleaner, low-frequency inventories, strong tree ensembles can match the gain without added complexity.*

However, 1D-CNNs come with a distinct set of limitations that are important to consider. Their most significant constraint is their fixed receptive field, meaning they can only **see** a fixed-length history. If this window is too short to capture a full traffic cycle, the model will miss crucial long-range dependencies unless its depth and dilation are increased, which in turn adds complexity and can make training less stable. Furthermore, these models assume that data arrives in regularly sampled sequences. This means that any gaps, irregular measurements, or misaligned data sources must be fixed beforehand through imputation or resampling to avoid confusing the model. It is also critical to use causal padding to prevent the model from **cheating** by looking at future data, which would lead to inflated performance scores. The following **table 3.3** resume the different usecases for 1D-CNN in traffic emission forecasting .

Table 3.3: Categorization of 1D-CNN usage in temporal modeling for traffic emissions forecasting.

Usage Type	Description	Examples
Temporal Decomposition + CNN	Time series is decomposed into predefined temporal components (short-term, daily, seasonal), each processed by a separate CNN branch before fusion. it explicitly models different temporal patterns and is effective for periodic or seasonal data. but relies on fixed decomposition and may struggle with irregular events or anomalies.	MCFT-Net (Fei et al., 2021)
Multi-Scale / Dilated CNN	Uses multiple CNN layers with varying kernel sizes or dilation rates to capture temporal dependencies at different scales in a single architecture. it captures multiple temporal ranges without explicit decomposition and allows efficient parallel processing. but requires careful tuning of dilation/kernels and may miss fine-grained temporal transitions.	(Xie et al., 2024)
CNN + RNN Hybrid	Combines CNNs for short-term/local feature extraction with RNNs (e.g., LSTM, GRU) for long-term dependencies, sometimes enhanced with attention for feature weighting. it leverages complementary strengths of CNNs and RNNs for complex temporal relationships. but higher computational cost, longer training time, and increased optimization complexity.	(Nguyen et al., 2024)

3.2.2.5 Transformer and attention based methods

While attention mechanisms have been integrated into a wide variety of deep learning models for emission estimation—such as LSTMs and 1D-CNNs—their conceptual foundations and role in these architectures have already been detailed in previous sections. Here, we focus specifically on recent developments that rely on Transformer-based architectures, particularly those tailored for time-series emission prediction.

Transformers, originally designed for natural language processing, have increasingly been adapted to handle environmental sensor data and vehicular telemetry. Their ability to model long-range dependencies through self-attention makes them suitable for capturing the complex temporal structure of exhaust emissions, especially when traditional RNNs or CNNs fall short due to limited receptive fields or vanishing gradients.

A recent study by (Liu, Zhang, Lv, Gao, Cai and Chen, 2024) proposes *ETSformer*, a Transformer variant tailored to forecast CO, HC, NO_x, and PM emissions from buses. The model is built upon two novel attention blocks: Exponential Smoothing Attention (ESA), which captures long-term growth trends in the data, and Frequency Attention (FA), which identifies seasonal or periodic patterns in the frequency domain. These components allow ETSformer to

explicitly decompose the input signal into trend, seasonality, and noise—a strategy inspired by classical time-series decomposition methods.

The model was trained on sensor data collected through a collaborative perception setup combining onboard and roadside units. By maintaining a computational complexity of $\mathcal{O}(L \log L)$, it achieves a balance between scalability and expressiveness. *This interpretability is a valuable feature, allowing domain experts to trace back prediction components to underlying signal dynamics. However, the relatively small dataset used for evaluation may limit the generalizability of the findings, particularly across different cities, vehicle types, or sensor configurations.*

In a distinct line of work, (Saez-Perez et al., 2024b) adapt the Vision Transformer (ViT) for predicting CO₂ emissions in self-driving vehicles. The architecture retains only the encoder layers from the ViT and replaces the traditional feed-forward network with a Convolutional Neural Network (CNN) block to better capture localized spatial cues. Additionally, the standard dot-product attention is swapped with a location-based attention mechanism that constrains each token’s attention scope to its neighbors, thereby enhancing the model’s ability to capture fine-grained spatial and temporal dependencies.

This modified ViT was trained using both visual data (from onboard cameras) and time-series data (from environmental and vehicle sensors). *Notably, the best performance was achieved using just two encoder layers—an interesting outcome that suggests deep hierarchies may not always be necessary for regression tasks, particularly when the input is noisy or lacks hierarchical structure. However, as with ETSformer, the study relies on relatively small datasets, which could limit its application in more varied or larger-scale deployment scenarios.*

Finally, (Li et al., 2023) address the challenge of modeling irregular and noisy emissions data from heavy-duty vehicles with their hybrid architecture, *TSF-Transformer*. Unlike the previous studies, this model employs a two-stage encoding pipeline. First, the raw sensor readings are passed through short LSTM modules that act as local temporal encoders. These LSTM outputs are then treated as tokens and passed to a Transformer encoder, which models more global dependencies across the LSTM-encoded time slices. The decoder is Transformer-based and uses autoregressive feedback to make multi-step forecasts.

This layered design is notable for splitting the modeling task between local and global components—short-term emissions spikes are handled by the LSTM modules, while long-term contextual patterns are captured by the Transformer. The result is a model well-suited for handling second-by-second telemetry with both sharp fluctuations and trend components. However, the architecture comes with increased training complexity and stricter requirements on data synchronization, which may pose difficulties in practical deployment across heterogeneous vehicle fleets.

Taken together, these studies showcase three distinct approaches to integrating Transformers into emission forecasting pipelines. ETSformer leans toward interpretability and signal decomposition, the modified ViT emphasizes spatial locality in autonomous contexts, and TSF-Transformer blends local LSTM encoding with Transformer-based sequence modeling. All three benefit from attention mechanisms to dynamically focus on informative time steps or spatial patches.

Despite their promising performance, a recurring limitation among these models is their reliance on relatively constrained datasets, often collected under controlled conditions. Future work should investigate how well these models generalize to more diverse driving environments and sensor modalities, possibly leveraging transfer learning or continual adaptation.

3.2.2.6 GNN Applications In Emissions Prediction

Graph Neural Networks (GNNs), particularly spatio-temporal variants, have found widespread application in traffic forecasting and traffic emission prediction tasks due to their natural alignment with the graph-structured nature of road networks. In this section, we review several prominent studies that apply Spatio-Temporal Graph Neural Networks (STGNNs) to the do-

main of traffic emission estimation, analyze their methodologies, and examine how they align their data with the structure of GNN models. Motivated by the fact that traffic emissions exhibit both spatial and temporal correlations, we highlight how these properties influence model design and performance for the previously published works.

3.2.2.6.1 Flow-Based Emissions Estimation Models A common and intuitive pipeline consists of first predicting traffic flow on the network, and then translating this information into emission estimates using either empirical or regulatory emission factors.

In the work of (Rollend, 2023), the authors developed a graph-based framework that first predicts AADT and subsequently uses it to estimate emissions. The road network is constructed from OpenStreetMap (OSM), where intersections are represented as nodes and road segments as edges. Each edge is enriched with structural attributes such as road type, segment length, and number of lanes, along with the associated AADT values.

To better model the connectivity between road segments, the network is transformed into its line graph, enabling the learning of inter-segment dependencies. A GATv2 (Graph Attention Network v2) is employed to predict the log-transformed AADT values. To account for the sparsity of labeled data, the authors introduce a regularization loss that penalizes imbalances in traffic inflow and outflow at each intersection.

After the GNN is trained, the predicted AADT values are converted into emission estimates using the formula:

$$\text{Emissions}_i = \text{AADT}_i \times \text{Length}_i \times \text{EF}_i \quad (3.1)$$

where EF_i is a localized emission factor that accounts for road type, fleet composition, and fuel usage.

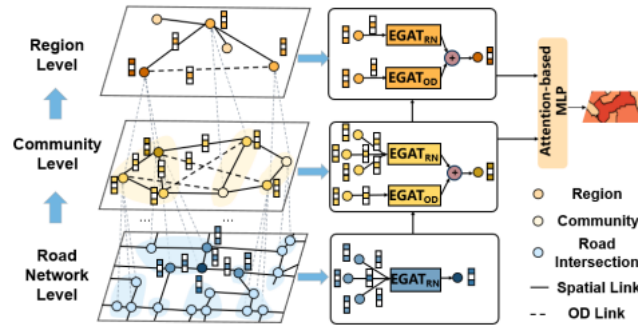


Figure 3.2: The architecture of the hierarchical heterogeneous graph learning method in the work of (Zeng et al., 2024)

Similarly, the work in (Zeng et al., 2024) adopts a hierarchical heterogeneous GNN to estimate traffic-related emissions as it is shown in figure 3.2. Unlike the previous approach, this model incorporates both physical connectivity and origin-destination (OD) demand data as edge types, capturing richer mobility patterns. The framework is structured into three hierarchical levels:

- **Road Network Level:** Nodes represent intersections; edges represent road segments. Physical attributes such as segment length and number of lanes are embedded.
- **Community Level:** Nodes represent aggregated spatial zones, with edges modeling either local road connectivity or local OD-based travel demand.

- **Regional Level:** Communities are further aggregated into larger regions. Physical edges at this level typically represent highway connections with distinct driving behaviors while the second type of edges represent OD demands cross regions.

An attention mechanism is applied to weigh the relative contributions of each level to the final emission output. These weighted embeddings are passed to a multilayer perceptron (MLP) to produce final emission estimates.

The study (Chen et al., 2022) represents a more detailed modeling approach by incorporating additional traffic dynamics such as speed and acceleration. It proposes a multi-channel Spatiotemporal Graph Convolutional Network (ST-GCN) where different traffic attributes—such as volume, average speed, and acceleration rate—are treated as separate input channels. This allows the model to process diverse temporal features in parallel, capturing richer motion patterns.

The ST-GCN is used to model both spatial interactions across road segments and temporal dependencies over time. The outputs of this network are then combined with emission factors provided by the MOVES (Motor Vehicle Emission Simulator) model, ensuring that the emission predictions are grounded in real-world regulatory standards.

Collectively, these three studies illustrate the progressive sophistication of flow-based emissions estimation models. The approach in (Rollend, 2023) demonstrates the feasibility of graph-based traffic flow prediction for emission estimation, but relies on aggregated AADT values, which may overlook intra-day variability and instantaneous driving dynamics. The hierarchical heterogeneous model in (Zeng et al., 2024) enriches spatial representation by incorporating multi-scale connectivity and road types along with OD demand, enabling the capture of different mobility patterns beyond immediate physical adjacency. Finally, (Chen et al., 2022) pushes the modeling boundary by integrating high-resolution temporal dynamics such as speed and acceleration, providing a more mechanistic link between traffic states and emissions especially given that emissions are highly correlated with sudden changes in speed, acceleration, and deceleration rates, which increase significantly in stop-and-go traffic conditions. While this progression enhances predictive accuracy and realism, it also reflects an inherent trade-off between model complexity, data requirements, and scalability to regions with limited sensing infrastructure.

3.2.2.6.2 Direct Emission Modeling via Sensor-Based Graphs Departing from traditional flow-based paradigms, the study (Sahoo et al., 2023) applies graph-based learning to directly model the complex interactions among engine and sensor signals for predicting transient NOx emissions. In this setting, nodes represent sensors, and the graph is constructed dynamically using a self-attention mechanism that computes pairwise similarity scores between feature embeddings.

The network includes a Maximum Mean Discrepancy (MMD) loss to ensure statistical regularization and distributional smoothness. Additionally, a Graph Fourier Transform (GFT) is employed to analyze the signal in the spectral domain, capturing high-frequency temporal variations associated with transient driving behaviors.

This method provides a more fine-grained and dynamic understanding of emissions by leveraging low-level vehicle data, bypassing the need for flow estimation. While highly precise, its applicability is limited to scenarios where detailed sensor data is available, such as controlled experiments or onboard diagnostics systems.

Across these models, we observe a clear trade-off between data granularity and scalability. Flow-based models offer broader applicability but rely on indirect inference of emissions using specific emission factors. sensor-based approaches improve accuracy and temporal resolution but at the cost of requiring richer datasets. A unified framework that balances data accessibility with modeling precision remains an open challenge in spatiotemporal GNN-based emission estimation models.

3.3 General Comparaison between ML and DL methods used for Emission Estimation

At the conclusion of this overview for the different architectures and their use in traffic emission estimation, we can observe that the choice of model strongly depends on both the type of data available and the specific objectives of the study. For instance, when analyzing the relationship between internal vehicle combustion functions and emissions, time series models and traditional ML are often combined with engine-level sensor data. In contrast, at the mesoscopic level, where the goal is to understand how traffic flow influences emissions throughout different times of the day, days of the week, or months, time series or spatio-temporal models are typically the most effective. The following table 5.3 provides a general comparison of the advantages and limitations of each model type.

Table 3.4: Comparative Analysis of Deep Learning Architectures for Traffic Emission Estimation

Model	Training Time	Data Requirements	Long-Term Dependency Modeling	Spatial Structure Support	Comments / Suitability
RNN	Moderate	Medium	Limited (due to vanishing gradient)	No	Simple to implement but less effective for long sequences
LSTM	High	Medium-High	Good	No	Suitable for emission time series; better handling of temporal context
GRU	Moderate	Medium	Good	No	Comparable to LSTM with fewer parameters; faster convergence
1D-CNN	Fast	Low-Medium	Local Dependencies	No	Good for local patterns, low latency systems
Attention-based Models	High	High	Excellent	No	Requires large data and compute; good for interpreting feature importance
Time-Series Transformer	Very High	Very High	Excellent	No	Best for long sequences, needs GPU and pretraining for performance
STGNN with LSTM	Very High	High	Good	Yes	Captures both spatial and temporal features; complex to train but good for long term dependencies .

Model	Training Time	Data Requirements	Time Dependency Modeling	Spatial Structure Support	Comments / Suitability
STGNN with 1D-CNN	High	Medium	Moderate	Yes	Faster than LSTM variants; good trade-off for edge use
STGNN with Cross-Attention	Very High	Very High	Excellent	Yes	Best performance in spatio-temporal reasoning; computationally intensive

From the comparison of different deep learning architectures for temporal and spatio-temporal forecasting (Table 5.3), it is clear that no single model is optimal for all scenarios. LSTMs are well-suited for capturing long-term temporal dependencies but are computationally intensive, while GRUs reduce the number of parameters and converge faster, yet still struggle with complex sequences. The new trend emphasizes CNN-based models, which efficiently capture short-term temporal patterns and are suitable for low-latency and edge applications; however, they alone cannot model long-term dependencies, necessitating complementary methods. Attention mechanisms and Transformers excel at identifying important temporal features and modeling long-range dependencies, but require large datasets and significant computational resources. Equally important is the spatial component: spatio-temporal models, such as STGNNs, combine temporal and spatial information to account for interactions across neighboring road segments, enabling more accurate and realistic emission predictions. Integrating these temporal and spatial modeling approaches allows for capturing both short-term fluctuations and long-term trends, while also providing resilience to missing data and heterogeneous sensor coverage, guiding the design of robust forecasting frameworks for traffic emissions.

3.4 Different factors driving traffic emission studies

Comparing all the different models presented in this chapter directly would not be fair or very useful. Each study was designed with different goals, types of data, and constraints in mind. There is no “best” model in general—what works best depends on the situation. In this section, we reflect on how the context of a study shapes the choice of method, the complexity of the emissions prediction problem, and how these factors help explain the wide variety of model architectures seen in the literature.

First, the design of a research study is always guided by its specific context. For example, the scope of the study has a major influence: a project that tries to predict emissions across an entire city will usually use aggregated data (like hourly values), while a study focused on emissions at a specific street corner will require very detailed, high-frequency data. This also affects the temporal resolution of the dataset: if the goal is to capture rapid changes during rush hour, the model needs second-by-second data, but if the focus is on seasonal trends, a lower resolution may be fine.

Other practical factors play a role too. The hardware environment is a major one—some models are deployed on powerful servers, while others must run on simple devices inside vehicles. The type of vehicles involved (e.g., buses, taxis, trucks) also matters, because different fleets have different driving patterns. Even cost considerations can affect model design—for instance, some studies use existing traffic cameras to avoid installing expensive new sensors. All of these real-world constraints push researchers toward certain methods and limit others.

Another major challenge is the complexity of the emissions problem itself. Emissions are affected by many different factors that vary across time, space, and people. These include:

- Temporal variation: Emissions can change drastically from one minute to the next, especially during traffic jams or rush hour. They also follow daily, weekly, and seasonal patterns.
- Spatial and environmental variation: Emissions depend heavily on location. A vehicle driving through a quiet neighborhood behaves very differently than one in stop-and-go traffic downtown. Weather conditions like rain or heat can also change how cars perform.
- Human factors: Drivers have different styles, which can influence fuel use and emissions significantly.

Because of all these influences, no study can include every possible factor. Researchers have to choose which ones to focus on. This decision often determines which modeling approach makes the most sense. For example, if short-term fluctuations are most important, a model like a 1D-CNN might be a good fit. If the goal is to understand long-term trends, a model based on transformers or LSTMs may be better.

In short, the methods used in emissions forecasting are shaped not just by technical performance but by many other practical and contextual factors. Understanding these helps explain why so many different models are used—and why no single method dominates the field. A direct, one-to-one comparison of the diverse methods reviewed in this chapter would be misleading. The choice of an optimal model is not absolute; it is deeply contingent on a constellation of factors, including the specific research question, the nature of the available data, and the practical constraints of the study.

3.5 Discussion and Research Orientations

To orient our research and identify potential contributions, it is essential to highlight some of the current conditions, limitations, and gaps in both research and practical applications. Starting from the data acquisition phase, most studies rely on the installation of additional sensors, either on vehicles themselves or along the roads. Some approaches assume that these sensors are already deployed, while others consider emissions only at the level of monitoring stations. A clear and persistent challenge is that most actual road networks lack the additional sensors needed for accurate prediction models. In practice, the most commonly available devices are traffic CCTVs. This motivates the exploration of their potential to capture features strongly correlated with emissions, such as vehicle visual characteristics, speed, make and model, and acceleration or deceleration rates, which are key inputs in widely used emission estimation models like MOVES and COPERT.

Another important limitation is that even the existing CCTVs are unevenly distributed. Many roads lack cameras entirely, while others may experience intermittent connectivity with the central server due to distance or communication disruptions. This situation raises the question of whether emission levels can be predicted on unmonitored roads by using their spatial correlations with monitored nodes, an approach that is largely unexplored in the literature.

Furthermore, traffic emission estimation often suffers from missing values due to the high communication costs associated with centralized modeling frameworks, as well as privacy concerns related to data sharing. While only a few studies have applied federated learning to emission estimation, numerous traffic forecasting frameworks have already demonstrated its potential to preserve data privacy and reduce transmission overhead. This highlights the importance of designing frameworks that can integrate spatial reasoning, handle missing data effectively, and leverage federated learning strategies for accurate, secure and scalable emission prediction.

3.6 Conclusion

This chapter has examined the landscape of machine learning and deep learning methods applied to traffic emission estimation, highlighting both their diversity and their situational nature. The

comparison of architectures shows that each family of models offers unique strengths—ranging from the simplicity of regression approaches to the capacity of spatio-temporal neural networks to capture complex dependencies across time and space. No single approach can be deemed universally superior; instead, the optimal choice depends on the characteristics of the data, the temporal and spatial resolution required, and the practical constraints of deployment.

Beyond technical performance, our review underscored the importance of context in shaping methodological choices. The scope of a study, the availability and quality of data, the computational environment, and even cost considerations all determine which models can be realistically adopted. This explains the wide methodological variety in the literature and highlights the need for adaptable frameworks rather than rigid prescriptions.

Looking forward, three directions appear particularly strategic for advancing the field. First, the use of existing urban infrastructures, such as CCTV networks, represents a promising avenue for scalable and cost-efficient data acquisition. Second, graph-based spatio-temporal models are key to exploiting the inherent structure of road networks, enabling predictions even in areas with limited direct monitoring. Third, privacy-aware and distributed learning paradigms, such as federated learning, offer a pathway to reconcile accuracy with data governance and scalability. Together, these orientations point to a future where emission estimation frameworks are not only more accurate but also more practical, equitable, and sustainable in real-world deployments.

Part II

Proposed Contribution

Chapter 4

Solution design

4.1 Introduction

The previous chapters reviewed how emissions are estimated and forecast in the literature, including the data sources used, common machine learning and deep learning models, and how these choices vary with scale and purpose. Building on that review, this chapter presents the conception and overall design of our proposed solution.

Our objective is to estimate and predict traffic-related carbon emissions at an area-wide scale using only existing roadside surveillance cameras. Estimating emissions in urban areas presents additional challenges compared to highways, where vehicle flow is generally more predictable and vehicles take longer to traverse from one road segment to another. Urban traffic, in contrast, is more complex and less controllable due to dense intersections, frequent stops, and fluctuating flows, making accurate emission estimation more difficult.

From these urban traffic videos, we extract essential visual features and convert them into emission estimates per location. On top of these estimates, we develop a graph-based forecasting model to predict short- and mid-term emissions across the traffic network. This model learns the spatiotemporal evolution of emissions, enabling it to propagate localized changes (e.g., a surge in emissions on one road segment) through connected links, thus generating consistent, network-level forecasts.

In addition to centralized deployment, we explore the potential of federated learning for decentralized model training. This approach addresses many challenges related to data transmission in traffic monitoring by enabling collaborative learning without sharing raw data.

The design prioritizes practicality and portability by using pre-installed cameras, avoiding additional roadside hardware, and maintaining a workflow that is easily adaptable to new cities with minimal recalibration.

The remainder of this chapter is structured as follows: Section 4.2 introduces the foundational concepts and application context of our solution. Section 4.3 outlines the general framework for traffic carbon emission forecasting and estimation first, then details each component of the solution: (i) video processing and emission estimation per location, (ii) the graph-based predictor for area-wide forecasting, and (iii) the application of federated learning. Finally, Section 4.5 summarizes the chapter and provides pointers to the implementation and evaluation detailed in subsequent chapters.

4.2 Motivation and Contribution

Road networks, especially urban ones, generally lack dedicated sensors for gas emissions monitoring, unlike highways where vehicle flows are more predictable and easier to track. This limitation

motivated us to leverage the most widely available existing sensors in urban environments: roadside video surveillance cameras. Roadside video surveillance networks represent a rich yet largely untapped resource for environmental analysis. The video streams captured by these cameras contain the precise traffic dynamics essential for granular emissions modeling—including vehicle presence, classification, traffic flow, and speed. Despite their widespread deployment in many urban centers, these camera networks are significantly underutilized for emission estimation. The primary advantage of our approach lies in exploiting this existing infrastructure, which enables extensive spatial coverage of the road network without incurring significant costs or logistical challenges associated with installing new hardware.

However, the distribution of these cameras is often uneven, with some road segments lacking coverage entirely and other segments experiencing intermittent data loss due to connectivity issues. This creates heterogeneous missing data patterns across the network. Current spatiotemporal models in traffic emission forecasting predominantly focus on representing areas with available data, either assuming full sensor coverage across the network or modeling only the observed sensor nodes. These assumptions limit their applicability for real-world, non-uniform sensor deployments. Hence, it is essential to extend deep learning models beyond forecasting to also perform imputation tasks, filling in missing data to provide a more complete and accurate emission picture.

Furthermore, to tackle privacy and transmission challenges inherent to traffic monitoring systems, we investigate the use of federated learning. By enabling decentralized model training directly on distributed camera nodes, federated learning minimizes sensitive data transmission and reduces latency between sensors and servers. This exploration is critical for developing scalable and privacy-aware traffic emission estimation solutions.

Our key contributions in this work include:

1. Developing a computer vision pipeline for automatic vehicle traffic feature extraction and emission estimation.
2. Proposing a multitask spatiotemporal neural network that simultaneously performs forecasting and imputation of traffic emissions.
3. Applying federated learning methodologies to enable decentralized training while preserving data privacy and communication efficiency.

4.3 General Framework Description

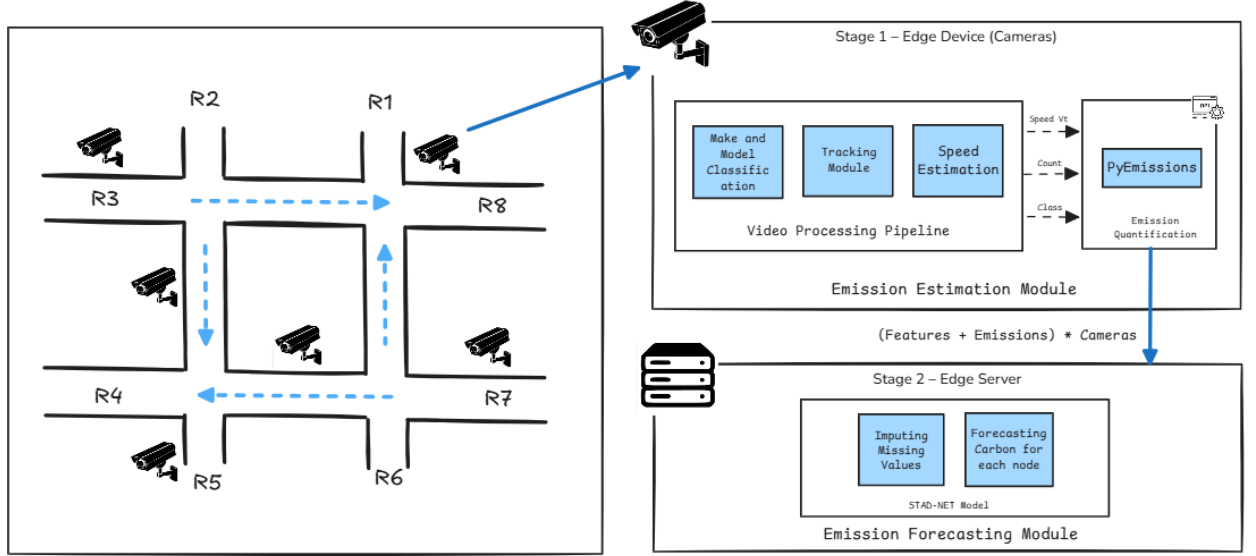


Figure 4.1: General schema of the proposed framework.

The scope of the proposed framework is the estimation and prediction of traffic-related carbon emissions across a defined road network. The methodology is intended for networks of a sufficient scale to allow the predictive model to effectively extract spatio-temporal patterns. The operational effectiveness of the model relies on a fundamental prerequisite: a significant percentage of the road network’s segments must be monitored by existing surveillance cameras. This density of observation is essential to provide the necessary ground-truth data for the model to learn the network’s dynamics and accurately perform imputation on segments that lack direct sensor coverage. Furthermore, the current design is constrained to cameras installed on road segments; it does not account for those placed at intersections, as their distinct traffic dynamics would necessitate a separate modeling approach.

The general workflow of the solution, operates as follows: for a given road network with cameras deployed on a known subset of segments, our video processing pipeline is applied to each independent video stream. This initial stage extracts critical traffic parameters—such as vehicle counts, classifications, and second-by-second speeds—and computes a preliminary emission estimate for each monitored segment. Subsequently, the time-series data from all cameras are aggregated and serve as input to a centralized, graph-based forecasting and imputation model. This model then exploits the learned spatio-temporal dependencies across the network to simultaneously forecast future emissions on the observed segments and extend these predictions to unmonitored segments, thereby generating a comprehensive, network-wide overview of the traffic carbon footprint.

4.3.1 Video processing and emission estimation Module

Our primary objective is to design a highly automated video processing pipeline that can operate on any surveillance camera footage with minimal human intervention. The pipeline’s core function is to transform raw video frames into a structured set of features for each tracked vehicle,

including its make(vehicle brand) and model, estimated speed, and calculated emissions on a given road segment.

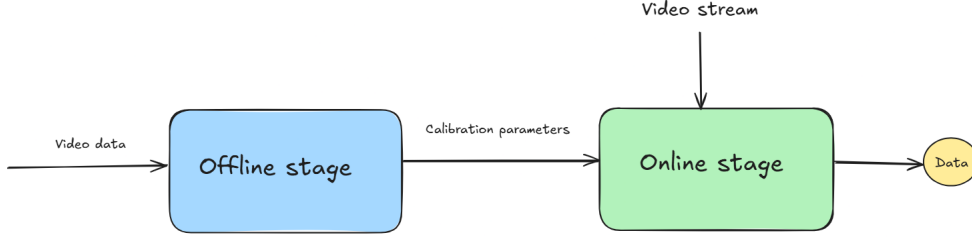


Figure 4.2: Schema of the General video processing pipeline.

Our system runs in two stages as shown in figure 4.2. First, an offline calibration step which is illustrated in 4.3, where we use a short clip from the fixed camera to understand the scene geometry: the two road directions seen in the image (vanishing points), the basic camera settings (intrinsic), and how many meters one pixel represents (the scale). These values are saved as the calibration parameters. Second, an online stage that works in real time on the live video as illustrated in 4.4: it detects vehicles, assigns a fine-grained class, keeps a track ID for each vehicle over time, projects motion onto the road plane using the saved calibration, and from that motion computes speed per track. The outputs (timestamp, track id, class, speed) are the data we use for speed-based emission estimation. We will be detailing all of these steps in the upcoming sections, and for optimal understanding we refer the readers to Appendix A for fundamental concepts linked to computer vision and camera geometry which are essential for the upcoming sections.

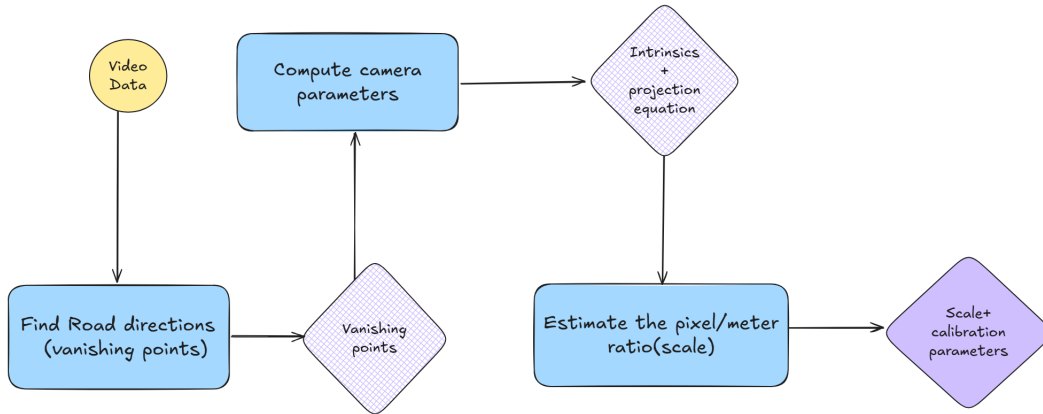


Figure 4.3: General schema of the calibration (offline) step.

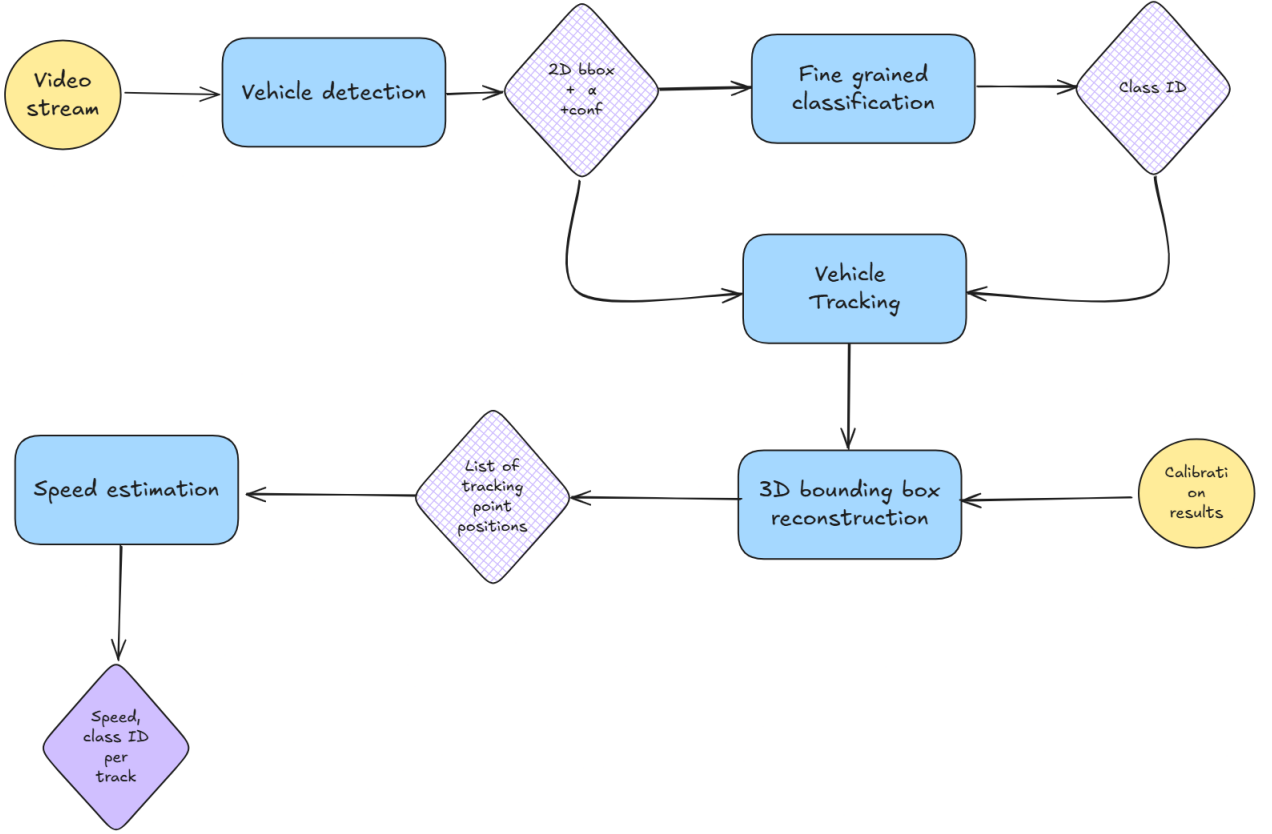


Figure 4.4: General schema of the feature extraction(online) step.

4.3.1.1 Camera Calibration Module

Monocular traffic videos suffer from perspective distortion: objects nearer to the camera appear larger, and image distances are not uniform across the field of view. As a result, pixel displacements cannot be interpreted as real-world distances, yet speed estimation requires displacements on the road plane in physical units (meters).

operational settings, relying on factory intrinsics/extrinsics or surveyed control points is rarely feasible. For many surveillance cameras, parameters are unknown, outdated, or inaccessible; cameras may be re-aimed due to weather, maintenance, or replacement, and older infrastructure often lacks reliable documentation. A design that depends on prior calibration data is therefore brittle and hard to deploy at scale.

Therefore, camera calibration is a crucial preliminary step in our pipeline. Its objective is to solve this distortion problem by mapping the 2D pixel coordinates of the camera’s view into a normalized 3D space. In this normalized space, the geometry of the road is consistent, allowing for accurate measurement of vehicle displacement. To automate this process and avoid manual setup, our framework relies on a vanishing point-based calibration method, an approach closely inspired by the study (Sochor et al., 2017) and follows a similar structure. The entire process is fully automatic and does not require manual measurements, making it suitable for real-world deployments using standard traffic camera footage. Once calibrated, we can estimate accurate speeds for each tracked vehicle.

To implement this calibration, our pipeline adopts the traffic camera model and methodology originally proposed in (Dubská et al., 2014), (Dubská et al., 2015) and reused in (Sochor et al., 2017). This model is specifically designed for the task of speed measurement, focusing on what is necessary to accurately measure distances on the road plane. Before detailing the calculations, it’s important to outline the model’s core assumptions and notation.

4.3.1.1.1 Camera Model and Assumptions : The calibration adopts standard, practical assumptions for fixed traffic cameras:

- The principal point $\mathbf{c} = (c_x, c_y)$ is assumed to be at the center of the image.
- The pixel skew is assumed to be zero.
- The monitored road section is locally flat and straight, an assumption that holds true for most highway and urban road segments.

Image points are written in homogeneous coordinates as $\mathbf{p} = [p_x, p_y, 1]^\top$. Vanishing points of three orthogonal scene directions are denoted by $\mathbf{u}$ (along traffic flow), $\mathbf{v}$ (across lanes, on the road plane), and $\mathbf{w}$ (zenith, perpendicular to the road plane).

Focal length from orthogonal vanishing points. The camera’s focal length, f , is the first intrinsic parameter to be recovered. It is derived from the geometric relationship between the two orthogonal road-plane vanishing points, $\mathbf{u}$ and $\mathbf{v}$, and the camera’s principal point, $\mathbf{c}$. The principal point $\mathbf{c}$ represents the optical center of the image, which, as per the model’s assumptions, is taken to be the image’s geometric center. The focal length is computed using the following formula:

$$f = \sqrt{-(\mathbf{u} - \mathbf{c}) \cdot (\mathbf{v} - \mathbf{c})}.$$

This equation enforces the dot product of $(\mathbf{u} - \mathbf{c})$ and $(\mathbf{v} - \mathbf{c})$ to be negative, which in turn will be a valuable condition later on in our proposed method.

Road-plane normal and plane equation. With the known vanishing points and focal length, the third vanishing point $\mathbf{w}$ can be calculated and thus the road plane can be calculated as follows :

$$\begin{aligned}\bar{\mathbf{u}} &= \begin{bmatrix} u_x \\ u_y \\ f \end{bmatrix} \\ \bar{\mathbf{v}} &= \begin{bmatrix} v_x \\ v_y \\ f \end{bmatrix} \\ \bar{\mathbf{w}} &= \bar{\mathbf{u}} \times \bar{\mathbf{v}} \\ \mathbf{n} &= \frac{\bar{\mathbf{w}}}{\|\bar{\mathbf{w}}\|}\end{aligned}$$

The road plane ρ is represented as

$$\rho = \begin{bmatrix} \mathbf{n}^\top \\ \delta \end{bmatrix}, \text{ where}$$

δ is an arbitrary scale (we set $\delta=1$ during calibration; the absolute metrics scale is introduced later).

Projection of image points onto the road plane. After having estimated the road plane ρ , the final step is to project any 2D image point $\mathbf{p}$ onto it to find its real-world 3D coordinates, $\mathbf{P}$.

This process begins by constructing the 3D direction vector $\bar{\mathbf{p}}$. This is done by taking the 2D pixel coordinates of the image point, (p_x, p_y) , and adding the camera’s focal length, f , as

the third component. This vector, $\bar{\mathbf{p}}$, represents the 3D line of sight from the camera’s origin out through the specific pixel $\mathbf{p}$ on the image sensor.

$$\bar{\mathbf{p}} = \begin{bmatrix} p_x \\ p_y \\ f \end{bmatrix}$$

The projection of this point onto the plane is then calculated using the following equation:

$$\mathbf{P} = -\frac{\delta}{\begin{bmatrix} \bar{\mathbf{p}}^\top & 0 \end{bmatrix} \cdot \boldsymbol{\rho}} \bar{\mathbf{p}}$$

Let’s dissect this formula based on the vector operations. The term in the denominator, $\begin{bmatrix} \bar{\mathbf{p}}^\top & 0 \end{bmatrix} \cdot \boldsymbol{\rho}$, is a compact notation for performing a dot product that effectively isolates the geometric relationship between our line-of-sight vector and the road plane. This dot product calculates a single scalar value.

The entire fraction, $-\frac{\delta}{\begin{bmatrix} \bar{\mathbf{p}}^\top & 0 \end{bmatrix} \cdot \boldsymbol{\rho}}$, therefore computes a crucial **scaling factor**. This number represents the precise distance one must travel from the camera’s origin along the line-of-sight vector $\bar{\mathbf{p}}$ to intersect with the road plane.

Finally, this scaling factor is multiplied by the original line-of-sight vector $\bar{\mathbf{p}}$. This operation effectively “stretches” or “shrinks” the direction vector to the exact length needed to touch the plane, yielding the final 3D point $\mathbf{P}$ which lies on the road surface.

Metric Scaling To convert the distances on this computed plane to meters, we introduce a single scene scale λ (meters per unit in the normalized road plane). The true metric distance between two projected points is the euclidean distance times the scaling factor:

$$D_m = \lambda \|\mathbf{P}_1 - \mathbf{P}_2\|.$$

In our pipeline, λ is estimated automatically downstream by aggregating per-vehicle geometric cues (dimensions inferred from 3D bounding boxes) against canonical vehicle lengths/widths. Once λ is fixed, the camera is fully calibrated for metric roadway measurements (speed).

4.3.1.1.2 Estimation of vanishing point $\mathbf{u}$ The vanishing point $\mathbf{u}$ corresponds to the dominant direction of vehicle motion on the road plane. Estimating $\mathbf{u}$ from these motion cues is therefore a natural and robust approach. Vehicles provide a multitude of coherent trajectories, which allows us to suppress background noise—such as moving trees, shadows, or camera jitter—by restricting the analysis to vehicle regions and filtering out short or inconsistent tracks.

Our methodology for estimating this vanishing point is a multi-stage process designed to reliably extract and aggregate these motion cues. The procedure begins by isolating relevant features to track. We first employ a YOLO detector (Varghese and M., 2024) on the initial video frame to identify all vehicles. Within the resulting bounding boxes, the Shi–Tomasi corner detector (Shi and Tomasi, 1994) is applied to select high-quality feature points. This crucial step ensures that our analysis is focused exclusively on vehicle motion, effectively discarding potential distractors from the background.

Once these feature points are identified, we track their movement across subsequent frames using the Lucas–Kanade (LK) optical flow algorithm (Lucas and Kanade, 1981). This process yields a set of short motion vectors, each representing the instantaneous direction of a point on a vehicle. To enhance the reliability of this data, we apply several filtering steps. Any vectors that are too short are discarded, as they are likely the result of tracking errors or minor jitter.

As vehicles enter and leave the scene, this feature detection and tracking process is periodically repeated to maintain a consistent flow of data.

The resulting set of filtered motion vectors, referred to as "edgelets," still requires a final outlier rejection step to handle non-dominant traffic flows, such as vehicles turning or on cross-ramps. We accomplish this by computing an orientation histogram of all edgelet angles and retaining only those aligned with the dominant direction of traffic.

Finally, with a clean set of coherent motion vectors, we aggregate them to find their common point of convergence. For this, we employ a robust voting scheme in Diamond Space (Dubská and Herout, 2013). Each filtered edgelet casts a vote in this space, and the most prominent peak in the resulting accumulator provides a stable and accurate estimate of the vanishing point, u . This entire procedure is formally outlined in Algorithm 1

Algorithm 1 Motion-Based Estimation of the Vanishing Point

```

1: Input: Video  $V$ , Number of frames  $N_{frames}$ 
2: Output: Estimated Vanishing Point  $\hat{u}$ 

3: //– Initialization –
4:  $I_{prev} \leftarrow \text{READFRAME}(V)$ 
5:  $Boxes \leftarrow \text{DETECTVEHICLES}(I_{prev})$ 
6:  $TrackedPoints \leftarrow \text{SEEDPOINTSINBOXES}(I_{prev}, Boxes)$ 
7:  $AllMotionVectors \leftarrow \emptyset$ 

8: //– Main Tracking Loop –
9: for  $t = 1$  to  $N_{frames}$  do
10:    $I_{current} \leftarrow \text{READFRAME}(V)$ 
11:   if no frame then break
12:   end if
13:    $NewVectors \leftarrow \text{TRACKPOINTS}(I_{prev}, I_{current}, TrackedPoints)$ 
14:    $FilteredVectors \leftarrow \text{FILTERVECTORS}(NewVectors)$ 
15:    $AllMotionVectors \leftarrow AllMotionVectors \cup FilteredVectors$ 
16:   if  $\text{SHOULDRESEED}(t, TrackedPoints)$  then  $\triangleright$  Periodically or if points are lost
17:      $Boxes \leftarrow \text{DETECTVEHICLES}(I_{current})$ 
18:      $NewPoints \leftarrow \text{SEEDPOINTSINBOXES}(I_{current}, Boxes)$ 
19:      $TrackedPoints \leftarrow \text{UPDATEPOINTS}(FilteredVectors, NewPoints)$ 
20:   end if
21:    $I_{prev} \leftarrow I_{current}$ 
22: end for

23: //– Voting and Final Estimation –
24:  $\theta_{dominant} \leftarrow \text{FINDDOMINANTANGLE}(AllMotionVectors)$ 
25:  $FinalVectors \leftarrow \text{FILTERBYANGLE}(AllMotionVectors, \theta_{dominant})$ 
26:  $Lines \leftarrow \text{VECTORTOLINES}(FinalVectors)$ 
27:  $\hat{u} \leftarrow \text{VOTEINDIAMONDSpace}(Lines)$   $\triangleright$  Select best peak
28: return  $\hat{u}$ 

```

4.3.1.1.3 Estimation of vanishing point v Estimating the second vanishing point, v , which corresponds to the direction across the road lanes, requires a different approach that relies on static scene geometry rather than motion. The process is more constrained, as it must effectively isolate features like road markings or vehicle edges while rejecting a significant amount

of noise. To achieve this, our method employs a robust, multi-stage filtering pipeline to distill a clean set of candidate lines before a final, geometrically-aware selection is made.

The procedure begins by processing sampled video frames through a Canny edge detector (Canny, 1986) to produce a binary edge map, which strengthens structural lines and suppresses ambiguous textures. A Line Segment Detector (LSD) (Von Gioi et al., 2012) is then applied to this map to extract an initial set of line segments. These raw segments are then passed through a rigorous filtering cascade designed to purify the pool of candidates. First, we apply a u-alignment filter to remove any segments that are oriented towards the previously computed vanishing point u , thereby eliminating lines parallel to the direction of traffic. From the remaining lines, a vertical line suppression step removes any nearly vertical edgelets to prevent confusion with the vertical vanishing point, w . Finally, a v-orientation filter identifies the dominant orientation and keeps only those segments that fall within a wide angular band of this consensus direction.

After this cascade, the surviving edgelets represent a high-confidence set of lines aligned with the v direction. These lines are used to vote for candidate vanishing points in Diamond Space. Unlike the simpler process for u , the final selection of v from the top voting candidates is governed by a sophisticated scoring function. This function evaluates each candidate based on a weighted combination of multiple criteria: its geometric plausibility (ensuring it produces a focal length within a realistic field of view), its support from the filtered edgelets, its membership in a dense cluster of peaks in the voting space, and its raw voting prominence. The candidate with the highest combined score is selected as the final estimate, ensuring a geometrically sound and data-supported result. This entire procedure is summarized in Algorithm 2.

Algorithm 2 Estimation of the Second Vanishing Point ($\hat{v}$)

```

1: Input: Video Stream  $V$ , Estimated Vanishing Point  $\hat{u}$ 
2: Output: Estimated Vanishing Point  $\hat{v}$ 

3: //– Phase 1: Collect and Filter Candidate Lines –
4:  $SampledFrames \leftarrow SAMPLEFRAMES(V)$ 
5:  $AllEdgelets \leftarrow \emptyset$ 
6: for each frame in  $SampledFrames$  do
7:    $Edgelets \leftarrow DETECTLINESEGMENTS(frame)$  ▷ Using Canny + LSD
8:    $Edgelets \leftarrow FILTEROUTULINES(Edgelets, \hat{u})$ 
9:    $Edgelets \leftarrow FILTEROUTVERTICALLINES(Edgelets)$ 
10:   $Edgelets \leftarrow FILTERBYDOMINANTORIENTATION(Edgelets)$ 
11:   $AllEdgelets \leftarrow AllEdgelets \cup Edgelets$ 
12: end for

13: //– Phase 2: Vote for Vanishing Point Candidates –
14:  $CandidatePeaks \leftarrow VOTEINDIAMONDSPACE(AllEdgelets)$ 

15: //– Phase 3: Select the Optimal Candidate –
16:  $\hat{v} \leftarrow SELECTBESTCANDIDATE(CandidatePeaks, \hat{u}, AllEdgelets)$  ▷ Scores candidates
   based on geometric plausibility, edgelet support, and voting metrics
17: return  $\hat{v}$ 

```

After identifying both u and v , all that's left is estimating the road plane using the previously defined equations, and obtain a homography that converts points from the 2D image coordinates to the normalized 3D road plane.

4.3.1.1.4 Scale estimation The next step in our pipeline is to estimate a real-world scale factor to convert normalized 3D road plane distances into metric units (meters). A common approach for this is to manually measure a reference object with known dimensions in the scene, such as a lane marker or a pole. However, this method requires manual intervention for each new camera deployment and relies on objects that lack universal standard dimensions. To create a fully automated and generalizable solution, we use the most common objects in the scene: the vehicles themselves. By using the average dimensions of a standard vehicle as our reference, we can create a robust, data-driven calibration process. This method begins by detecting vehicles and reconstructing their 3D bounding boxes. From these cuboids, we extract the length and width segments, which are then projected onto the 2D road plane to measure them in road plane units. Finally, these road plane measurements are mapped to their assumed average real-world values. We collect this meters-per-pixel ratio from many vehicles over a number of frames and apply a filtering process. We detail this whole procedure in the next paragraphs starting with a detailed explanation on how we retrieve the 3D bounding boxes through image transformation and vehicle detection and then how we estimate the scale .

3D Bounding Box Projection In this section, we describe the method used to reconstruct 3D bounding boxes of vehicles from a monocular camera feed and estimate the real-world scene scale required for speed computation. We follow the approach proposed in (Kocur and Ftáčnik, 2020), and in (Macko et al., 2025). Since the theory and reasoning are detailed in the original papers, we will focus here on the high-level process and practical steps applied in this work.

Image transformation. The approach start by transforming the image using the process proposed in (Kocur, 2019), which works as follows : First, one of the two admissible vanishing-point pairs, either (u, v) or (v, w) , is selected. For each vanishing point in the chosen pair, two lines are drawn such that they pass through the vanishing point and are tangent to the road mask, resulting in four tangent lines in total. The intersections formed by each pair of tangents from different vanishing points yield four points, which are then matched to the corners of a target image transform rectangle of predefined size. This correspondence is made so as to preserve the original vertical vehicle movement direction (e.g., cars moving from top-left to bottom-right in the input image should move from top to bottom in the image transform). A perspective transformation H is computed from these four point pairs. This transform is applied to the road mask. The final homography aligns the across-lane direction with the x -axis and the vertical (zenith) direction with the y -axis while preserving perspective consistency. These steps are illustrated in figure 4.5 from steps a to f.

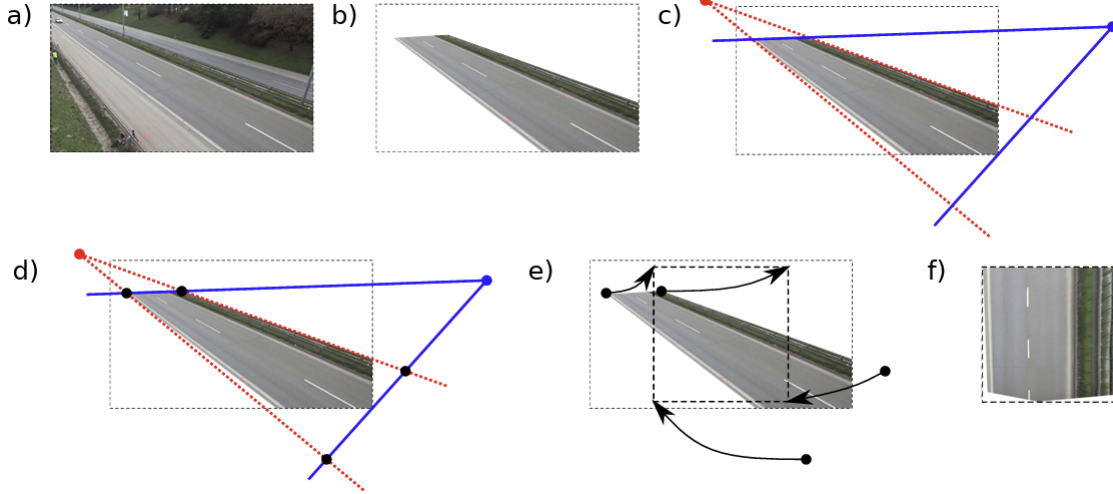


Figure 4.5: Perspective transformation.(Kocur and Ftáčnik, 2020).

Vehicle Detection Once the image transform has been established, vehicle detection is carried out using a modified YOLOv6 model — referred to as YOLOv6–3D — that has been trained specifically on rectified frames from the BrnoCompSpeed dataset and proposed in (Macko et al., 2025).

The model is built upon the original YOLOv6 architecture (Li et al., 2022), which is a single-stage object detection framework designed for high-speed industrial applications. Its efficiency and real-time performance make it an excellent foundation for traffic monitoring. The architecture consists of three main components: an EfficientRep Backbone for multi-scale feature extraction, which uses structural re-parameterization to maintain high accuracy at inference speed; a Rep-PAN Neck that enables the effective fusion of the multi-scale features; and three Efficient Decoupled Heads that use separate branches for classification and bounding box regression.

The key addition in YOLOv6–3D is a lightweight regression head that predicts a single scalar $\alpha \in [0, 1]$ for each detected vehicle. This α encodes the *normalized position of the top-front edge of the 3D cuboid inside the 2D box*, providing the extra cue needed to resolve geometric ambiguity in monocular 3D reconstruction.

This prediction is performed by a small, efficient convolutional block that is appended at each detection scale. The block processes the feature maps from the main YOLOv6 neck through a series of operations designed to distill the rich spatial information down to a single, precise value:

$$\alpha_{\text{pred}} = \sigma\left(\text{Conv}_{1 \times 1}(\text{Conv}_{1 \times 1}(\text{SiLU}(\text{BN}(\text{Conv}_{3 \times 3}(\text{features}))))\right)$$

The data flows through this block as follows:

1. A **3x3 Convolution** (Conv_3x3) first processes the input features to capture local spatial patterns relevant to the vehicle’s orientation.
2. **Batch Normalization** (BN) is then applied to stabilize the learning process, followed by a **SiLU activation** function which introduces non-linearity, allowing the network to learn more complex relationships.
3. Two consecutive **1x1 Convolutions** (Conv_1x1) act as efficient, fully-connected layers that progressively transform these features and reduce their dimensionality down to a single channel.
4. Finally, a **Sigmoid activation** function (σ) is applied. It constrains the final output value to be strictly within the range of $[0, 1]$.

Training augments the standard detection loss with an L_α term for α :

$$L = L_{\text{cls}} + L_{\text{box}} + \lambda_\alpha L_\alpha,$$

with λ_α balancing the contribution of the α regression against classification (which only classifies objects into vehicles or not) and bounding box terms.

An example of an image output from the model is provided in figure 4.6. The inferred alpha parameter, along with the third vanishing point, enables the estimation of full 3D bounding boxes. Specifically, for each detection, we define four base points in the image transform (front-left, front-right, rear-left, rear-right), and project rays from the third vanishing point through these points to simulate depth. And top corners are obtained by offsetting the base points along the ray directions. All eight 3D corners are then projected back into the original image using the inverse homography. The midpoint between the bottom front corners is also computed to provide a stable reference point for tracking. This results in the list of projected 3D cuboids we need.

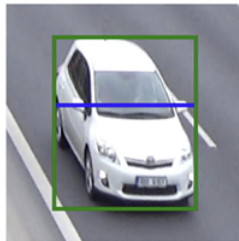


Figure 4.6: Example of an output of the modified Yolov6 -3D model.(Kocur and Ftáčnik, 2020).

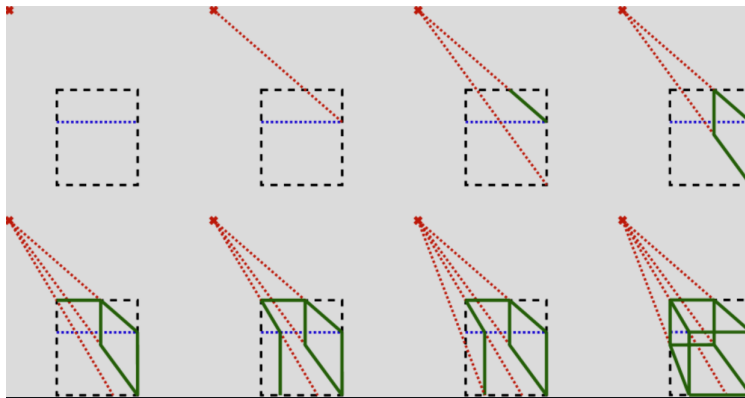


Figure 4.7: Construction of the 3D bounding box from the 3rd VP. (Kocur and Ftáčnik, 2020).

Scale estimaion After rectifying frames and projecting the 3D bounding boxes of the vehicles, we still have one last thing to rectify. Distances along and across the road are not guaranteed to be isotropic (i.e., that a pixel corresponds to the same physical distance whether measured along or across the road) due to projective stretching; therefore, measuring directly in the previous image transform can bias speed. To obtain metric distances, we operate on the *normalized road plane* we defined previously, where the road axes are orthonormal up to a single global scale λ .

For each reconstructed 3D vehicle cuboid, we extract (i) the bottom-front edge (vehicle width) and (ii) one bottom side edge (vehicle length), project their endpoints to the normalized

plane, and measure their lengths in normalized units. Comparing these to real world dimensions (length/width priors) yields per-frame scale candidates. We then (a) reject candidates with implausible shape ratios (length/width outside a realistic band), (b) trim statistical outliers by percentiles, and (c) simply taking the median as the final scene scale $\hat{\lambda}$. This produces a robust, marker-free metric calibration driven by the observed traffic.

By the end of this process, all the needed camera parameters are retrieved and online inference can start.

4.3.1.2 Vehicle detection and fine grained Classification

Following the offline calibration phase, which provides geometric parameters such as vanishing points and homographies, the system transitions to its online mode. In this stage, all operations are performed on live video streams, and the calibration outputs are reused directly for real-time inference.

We reuse the yolov6-3D model in this stage for vehicle detection as well as 3D bounding box reconstruction, and it is the first step of the online processing, for each vehicle we detect the 2D bounding box, the alpha parameter and the detection confidence.

Once a vehicle has been detected in the rectified frame, the system must assign a fine-grained appearance label (make/model/variant) to enable downstream emissions estimation. In practice, fine-grained classification is a demanding task: subtle cues must be recognized under varying viewpoints, scale, and partial occlusion. Preliminary trials indicated that coupling this task with detection in a single model degrades recognition quality, consistent with the well-known difficulty of multi-task interference on subtle categories. Although a dedicated classifier adds a second forward pass, the accuracy gain is critical for reliable class-to-emission mapping.

We evaluated several candidate classifiers under our deployment constraints (fine-grained accuracy and per-frame throughput). Based on these tests, *DeiT-Small non-distilled* was selected as the best fit for our use case. Full selection results are reported later. The **DeiT (Data-efficient Image Transformer)** (Touvron et al., 2021) model follows the same architectural paradigm as the original Vision Transformer (ViT), which is a strong image classifier, but in its plain form it tends to be data-hungry. Therefore the **DeiT** changes the *training recipe* to work well with standard-sized datasets. The key difference is *distillation*: during pretraining, DeiT learns not only from the labels but also from a teacher model’s “soft” targets (via an extra distillation token), which injects useful biases about local patterns and helps the transformer generalize with less data. This makes DeiT pretrains especially effective when classes are fine-grained.

For our pipeline, we selected the **DeiT-Small** variant, as it offers a favorable balance between accuracy and the low latency required for processing video streams. Crucially, we use the publicly available pre-trained weights but opt for the **non-distilled version** for fine-tuning. This choice simplifies both training and deployment by removing the complexity of the extra distillation token and teacher model. Even without using the distillation head, the model’s weights retain the significant benefits of their original training process. They provide a much stronger starting point than a standard ViT, leading to higher fine-tuning accuracy and more stable training, especially on specialized, fine-grained datasets like ours.

4.3.1.3 Vehicle tracking and speed estimation

Having obtained per-frame detections and the geometric cue α from YOLOv6-3D, the next step is to convert these instantaneous observations into *trajectories* (one stable ID per physical vehicle) and to compute speeds from their motion on the road plane. Reliable identity continuity is essential: inter-frame displacements must be measured along the same physical vehicle to yield meaningful metric speeds.

We employ *ByteTrack*, a tracking-by-detection method that associates detections across frames using only bounding boxes and confidences, without requiring appearance embeddings. This keeps the pipeline light and real-time which is the primary reason for our choice. ByteTrack runs a two-stage association per frame: (i) high-confidence detections are matched to predicted track positions using IoU; (ii) remaining tracks are given a second chance to match with lower-confidence detections, improving robustness to short occlusions or temporary detector drops. A Kalman filter predicts each track’s position before matching; unmatched tracks are kept alive for a short buffer to avoid premature termination.

The integration into our pipeline is simple. The detected vehicles and confidence scores produced by the detector are fed directly into ByteTrack. The output is a set of active tracks, each with a stable ID. However, a 2D tracked box in a rectified view is still insufficient for accurate metric speed calculation. To obtain a stable, physically meaningful anchor point, we use the (box,) pair to reconstruct the vehicle’s full 3D cuboid.

From this 3D cuboid, we extract a single reference point: the midpoint of the front bottom edge. This ”bottom-front center” was deliberately chosen for several reasons. As a point on the road plane, it is compatible with our planar projection and is not affected by vertical jitter. It is also geometrically stable, meaning it is less sensitive to small perturbations in the 2D box or perspective changes. Finally, as it aligns with the vehicle’s direction of travel, it provides a kinematically meaningful point for tracking. This reference point is then projected onto the normalized road plane for each vehicle at each frame. The final output of this stage is a collection of clean trajectories, ready for the final speed computation.

4.3.1.3.1 Speed computation. With a clean, temporally consistent trajectory of reference points established for each vehicle, the final step in this module is to compute its speed. The calculation is a straightforward conversion of the displacement between consecutive points on the normalized road plane into a metric velocity. Given the isotropic scene scale λ (in meters per pixel) and the camera’s frame rate (fps), the instantaneous speed v_t in km/h between two points separated by Δf frames is given by:

$$v_t = 3.6 \frac{\lambda \|\mathbf{X}_t - \mathbf{X}_{t-\Delta f}\|_2}{\Delta f/\text{fps}}$$

where $\|\mathbf{X}_t - \mathbf{X}_{t-\Delta f}\|_2$ is the Euclidean distance in pixels between two reference points, λ is the scene scale that converts this distance to meters, and the denominator $\Delta f/\text{fps}$ represents the elapsed time in seconds. The constant 3.6 is the final conversion factor from meters per second to kilometers per hour.

4.3.1.4 Emission estimation

We require an emissions model that operates at the same temporal and spatial scales as our measurements: per-vehicle and second-by-second. This ensures that important behaviors—idling, stop-and-go, hard accelerations and decelerations—are preserved rather than averaged out. PyEmission meets this requirement by mapping instantaneous speed and acceleration, together with basic vehicle descriptors (technology, mass...), to instantaneous fuel-use and pollutant mass rates; CO₂ is then obtained from fuel consumption. In other words, the model is microscopic and event-sensitive, aligning with the fidelity of our vision-derived trajectories.

By contrast, tools such as COPERT and MOVES target different design goals. COPERT is built around average-speed factors and is therefore optimized for aggregated segments and

¹Idaho National Laboratory, <https://inl.gov/>

²European Environment Agency, <https://www.eea.europa.eu/en>

³U.S. Environmental Protection Agency, <https://www.epa.gov/moves>

Table 4.1: Comparison of Emission Modeling Frameworks

Criteria	PyEmission	COPERT	MOVES
Model type	Microscopic	Macroscopic	Mesosopic
Designed for	Programmable use in pipelines	Emission factor estimation (manual use)	Regulatory scenario simulation
Ease of integration	Python-native, lightweight	GUI software (Windows only)	Java-based complex framework
Input re- quirements	Speed, acceleration, vehicle specs	Average speed, vehicle category, fleet mix	Detailed U.S. region + fleet data
Geographic dependence	None	EU-specific	U.S.-only
Temporal resolution	Per-second	Averaged	Time-step (not microscopic)
Credibility	INL ¹ research project	EEA-endorsed ²	EPA-endorsed ³
Supported pollutants	CO ₂ , CO, NO _x , HC, Fuel	CO ₂ , NO _x , HC, PM, etc.	CO ₂ , CO, NO _x , PM, HC, etc.
Main limitation	Vehicle specs must be manually mapped	Poor temporal granularity	Complexity

planning inventories, where intra-trajectory dynamics are intentionally smoothed. MOVES, while comprehensive, is a mesoscopic/regulatory framework whose inputs, temporal resolution, and geographic assumptions are tailored to fleet-level inventories and policy analysis rather than per-vehicle, instantaneous estimation. For a pipeline that prioritizes fine-grained kinematics and road-segment realism, a microscopic, instantaneous-speed approach is the most faithful choice—hence the selection of PyEmission. For a clearer view of the advantages and the different factors influencing our choice, we have compiled a comparison of these three major tools for emission estimation in Table 4.1.

We note a current limitation: PyEmission’s public models primarily cover gasoline vehicles, with reduced coverage for other powertrains (e.g., diesel, hybrid, electric). This constrains absolute comparability across mixed fleets. Nonetheless, for our objective—capturing the emissions impact of transient traffic dynamics—the microscopic, instantaneous formulation provides the right behavioral sensitivity, and the model choice can be revisited as broader powertrain modules become available.

4.3.2 Spatio-temporal Module STAD-NET

In this section, we delve deeper into the design and conception of the spatiotemporal model architecture. We begin by detailing the structure of the data used as input to the model, focusing on how raw traffic information collected from distributed cameras is transformed into a graph representation that captures both spatial and temporal dependencies within the urban

road network. Traffic data—such as vehicle counts, speeds, and classification features—are first aggregated and mapped onto the nodes of a graph, where each node corresponds to a road segment. The edges of the graph encode the physical connectivity and traffic flow relationships between these locations, enabling the model to effectively capture spatial interactions and dependencies. A particular challenge addressed in this architecture is the handling of missing data, which naturally arises due to inconsistent camera coverage, sensor failures, or transmission losses. Rather than discarding incomplete data or relying solely on imputation prior to modeling, the spatiotemporal graph neural network is designed to jointly perform emission forecasting and missing data imputation as a multitask learning problem. This integrated approach allows the model to leverage spatial correlations and temporal patterns to predict missing values dynamically while forecasting future emissions. Through this architecture, the spatiotemporal relationships inherent in traffic emissions are explicitly modeled, enabling more reliable emission estimates and predictions even in the presence of data sparsity.

4.3.2.1 Graph Conception and Representation

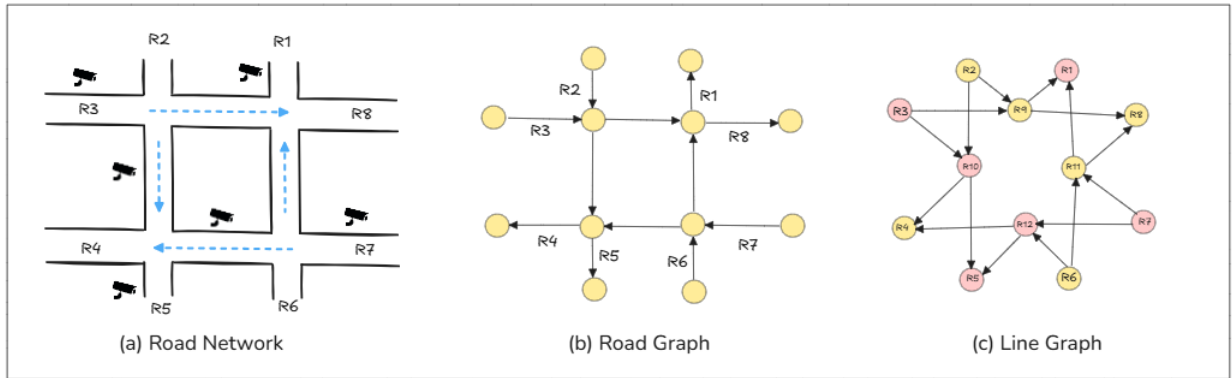


Figure 4.8: Road Network Graph representation from Roads to Line Graph.

In this study, we represent the urban road network as a directed graph $G = (V, E)$, where the vertices V correspond to intersections, and the directed edges E represent road segments with specific travel directions. This directed graph structure accommodates both one-way and two-way streets by allowing edges to convey distinct traffic flow directions.

Given that most traffic cameras in our study are installed along road segments rather than intersections, the essential traffic information is naturally associated with edges instead of nodes. However, many graph neural network (GNN) architectures primarily operate on node features. To bridge this representation gap, we apply a *line graph transformation* (Hemaspaandra et al., 2015) to the original graph G , resulting in a transformed graph $L(G) = (V_L, E_L)$ where each node in V_L corresponds to an edge in G . In this line graph, edges E_L represent feasible transitions where a vehicle can move from one road segment to another.

This transformation effectively converts edge-centric traffic data into node-centric features compatible with standard GNN frameworks, thereby enabling spatial relational learning at the granularity of road segments. In $L(G)$, we distinguish between two categories of nodes:

- **Nodes with information:** nodes corresponding to road segments equipped with surveillance cameras, providing direct observational data.
- **Nodes without information:** nodes corresponding to non-monitored road segments lacking direct sensor data.

This distinction is crucial for modeling and imputing missing information within the network. Overall, the line graph approach provides a principled method to capture the spatial and directional dependencies of traffic flow and emissions while aligning the data representation with GNN processing requirements. We can see the transformation steps more in details in figure 4.8.

4.3.2.2 Selected Features

This section provides a comprehensive overview of the key traffic features utilized within the spatiotemporal graph model, as well as the vehicle classification categories relevant to emission estimation.

4.3.2.2.1 Features Used in the Model The primary features incorporated into the model include:

- **Vehicle Speed:** Average speed of vehicles recorded at each monitored road segment, measured in km/h.
- **Vehicle Count per Class:** Number of vehicles passing through each road segment, distinguished by vehicle class.
- **Carbon Emission Estimate:** amount of measured emissions per road segment

These features form the attributes of the nodes in the graph, enabling the model to capture spatial and temporal variations in traffic emissions.

4.3.2.2.2 Vehicle Classes and Emission Characteristics Vehicle classes differ significantly in their emission profiles due to factors such as engine size, fuel type, and vehicle weight. Table 4.2 summarizes the typical vehicle categories used in this study, along with associated descriptions and general emission levels.

Table 4.2: Summary of Vehicle Classes and Typical Emission Levels

Vehicle Class	Description	CO ₂ Emission Level (g /km)
Sedan	Compact and mid-size cars	120 – 180
SUV	Sport Utility Vehicles	180 – 250
Light Trucks / Pickups	Light commercial vehicles	200 – 300
Vans / Minivans	Passenger and cargo vans	160 – 220
Heavy-duty Trucks	Freight trucks	800 – 1200
Motorcycles	Two-wheelers	50 – 100

This vehicle classification refines emission estimates on a per-class basis, enhancing both the accuracy and granularity of the overall emission model.

4.3.2.3 Imputation Technique

To effectively address missing data within the traffic network graph, we adopt a dynamic masking strategy inspired by masked language models such as BERT. The core idea is to train the spatiotemporal graph neural network to reconstruct missing node attributes by leveraging information from neighboring nodes.

In our approach, the context of each node is represented by the embeddings of its neighbors. Because the underlying graph is directed, incoming edges to a node provide information about

the node’s future states, while outgoing edges reflect its past states. This bidirectional information flow is crucial for accurate imputation, as it allows the model to propagate dependencies both forward and backward along the traffic network.

During training, we dynamically mask a percentage of observed nodes in each batch, simulating missing data scenarios. The model is then tasked with reconstructing the masked node features solely from the unmasked neighboring nodes’ embeddings. This process encourages the network to learn robust spatial and temporal dependencies and to fill in gaps effectively. Additionally, a substantial portion of missing nodes may themselves be neighbors, necessitating

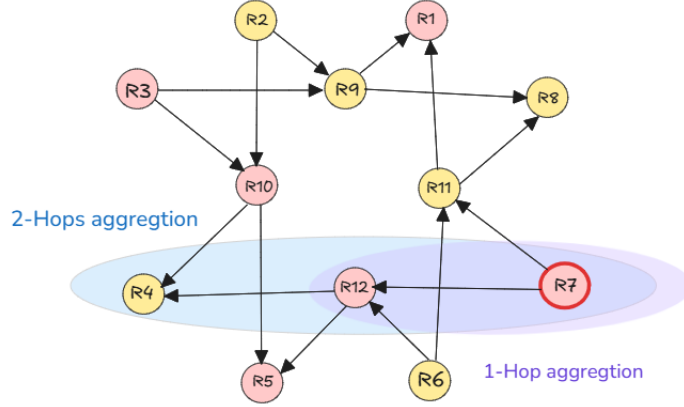


Figure 4.9: illustration of K-hop neighbor aggregation in graphs while red nodes are missing.

multi-hop aggregation within the graph convolutional layers see figure 4.9. To accommodate varying influences among neighbors and transition heterogeneity in the traffic flow, we employ an attention mechanism that computes attention scores to form the transition matrix $\mathbf{P}$ guiding information diffusion across the graph.

This dynamic masking framework both improves the network’s robustness against data sparsity and enables joint learning of imputation and forecasting tasks, which will be detailed in the subsequent architecture section.

4.3.2.4 Spatio-Temporal Model Architecture

In multitask learning architectures, it is common to employ a *shared backbone* network that extracts general features useful across all tasks, followed by multiple *specialized task-specific heads* that focus on individual subtasks. This design facilitates the efficient sharing of representations while allowing each task to optimize for its particular output requirements (Ruder, 2017).

In our proposed model, the shared backbone is responsible for learning rich spatiotemporal representations from the graph-structured traffic data. The extracted features are then fed into separate heads dedicated to the forecasting and imputation tasks, respectively. This architecture not only promotes representational efficiency and improved generalization but also allows each task to benefit from complementary information learned jointly. Moreover, employing this multitask framework with a shared backbone significantly reduces the overall memory footprint compared to training separate models for each task. Since most parameters reside within the shared layers, this approach avoids redundancy and leads to lower memory consumption and faster inference (Ruder, 2017).

A distinctive aspect of our multitask framework lies in the differing information requirements of the two tasks. The imputation task necessitates *bidirectional* information flow within the

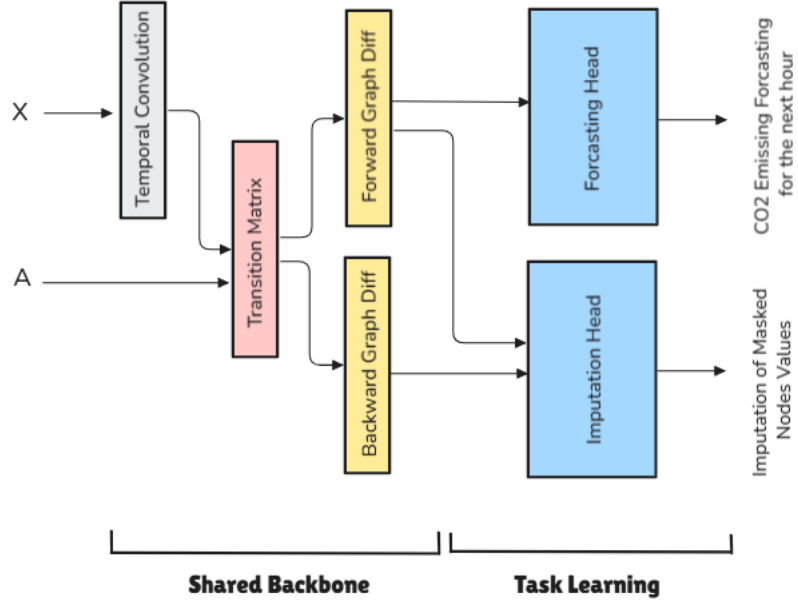


Figure 4.10: architecture of the Multitask Forecasting Imputation Model.

directed graph to accurately reconstruct missing node features, as both past and future neighbor states provide valuable context for inferring the missing data (Cao et al., 2018). In contrast, the forecasting task can operate effectively with *unidirectional* forward temporal information, predicting future emissions based on historical and current states alone.

The model design is illustrated in Figure 4.10 where we mention the following Components :

- **Temporal Convolution Layer:** This layer employs gated causal one-dimensional convolutional neural networks (CNNs) designed to capture temporal dependencies within the feature sequences of each node independently. Specifically, a gating mechanism is integrated to act as a dynamic filter that suppresses irrelevant or noisy signals, enhancing the quality of extracted temporal features.

The gated temporal convolution can be expressed as:

$$\text{Output} = \tanh(\text{Conv}(x)) \odot \sigma(\text{Gate}(x))$$

where Conv and Gate are convolution operations with kernel size k , $\tanh$ denotes the hyperbolic tangent activation, σ is the sigmoid activation, and $\odot$ represents element-wise multiplication.

- **Dynamic Transition Matrix Calculation:** In our model, the transition probability matrix, denoted as $\mathbf{P}$, is dynamically constructed using a self-attention mechanism. For each node, we define query $\mathbf{Q}$, key $\mathbf{K}$, and value $\mathbf{V}$ matrices derived from the node embeddings. Formally, letting $\mathbf{H} \in \mathbb{R}^{N \times d}$ be the matrix of node features:

$$\mathbf{Q} = \mathbf{H}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{H}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{H}\mathbf{W}_V,$$

where $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d \times d'}$ are learnable parameters.

The attention score matrix $\mathbf{W}$ is computed via scaled dot-product:

$$\mathbf{W} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d'}}.$$

To preserve the original adjacency structure of the road network, we apply a mask to $\mathbf{W}$ that assigns $-\infty$ to entries corresponding to non-adjacent node pairs. This masking ensures that attention weights are only computed between connected nodes, preventing spurious transitions:

$$\widetilde{\mathbf{W}}_{ij} = \begin{cases} \mathbf{W}_{ij}, & \text{if } (i, j) \in \mathcal{E}, \\ -\infty, & \text{otherwise.} \end{cases}$$

where $\mathcal{E}$ is the set of edges in the graph.

Finally, the transition probability matrix $\mathbf{P}$ is obtained by applying the softmax function row-wise:

$$\mathbf{P}_{ij} = \frac{\exp(\widetilde{\mathbf{W}}_{ij})}{\sum_k \exp(\widetilde{\mathbf{W}}_{ik})}.$$

This dynamic transition matrix contrasts with conventional approaches like DCRNN (Li et al., 2018), which rely on a static adjacency matrix or the graph Laplacian matrix. These static matrices assume fixed spatial relationships and stationary graph structures, which do not account for the temporal variability of transitions between road segments. Our dynamic construction adapts to changing traffic conditions by learning transition probabilities that vary over time, providing a more realistic and flexible representation of spatial dependencies.

- **Graph Diffusion Layer:** To capture rich spatial dependencies within the directed traffic graph, the model employs a bidirectional diffusion mechanism over the dynamically computed transition matrix $\mathbf{P}$. This process aggregates information from multi-hop neighborhoods, where k denotes the maximum number of hops considered.

Let $\mathbf{X} \in \mathbb{R}^{B \times T \times N \times H}$ represent the input node features, with batch size B , time steps T , number of nodes N , and hidden feature dimension H .

The diffusion is performed iteratively for each hop $k = 1, \dots, K$, separately in forward and backward directions:

For *backward diffusion* over $\mathbf{P}$ (which represents transitions from nodes to their neighbors), the state updates as:

$$\mathbf{v}_b^{(k)} = \mathbf{P} \mathbf{v}_b^{(k-1)}, \quad \mathbf{v}_b^{(0)} = \mathbf{X}.$$

For *forward diffusion* the transpose $\mathbf{P}^\top$ is used to propagate information:

$$\mathbf{v}_f^{(k)} = \mathbf{P}^\top \mathbf{v}_f^{(k-1)}, \quad \mathbf{v}_f^{(0)} = \mathbf{X}.$$

The aggregated forward and backward representations are computed by linearly combining the multi-hop states using learnable scalars α_k^f and α_k^b :

$$\mathbf{f}_{\text{sum}} = \sum_{k=0}^K \alpha_k^f \mathbf{v}_f^{(k)}, \quad \mathbf{b}_{\text{sum}} = \sum_{k=0}^K \alpha_k^b \mathbf{v}_b^{(k)}.$$

Finally, nonlinear transformations project these sums into output embeddings:

$$\mathbf{Fwd} = \tanh(\mathbf{W}_f \mathbf{f}_{\text{sum}}), \quad \mathbf{Bwd} = \tanh(\mathbf{W}_b \mathbf{b}_{\text{sum}})$$

where $\mathbf{W}_f$ and $\mathbf{W}_b$ are learnable linear projection matrices.

This bidirectional multi-hop diffusion enables the effective integration of contextual information from both upstream and downstream nodes in the directed graph, with the learnable α parameters adjusting the influence of different neighborhood depths on the final node representations.

- **Imputation Head:** The imputation head utilizes the outputs of the bidirectional graph diffusion module—representations aggregated from both forward and backward directions—to recover missing data in the traffic sequence. To further capture temporal dependencies within these diffusion-enhanced embeddings, separate temporal convolutional layers are applied to the forward and backward diffusion outputs.

Formally, let $\mathbf{Fwd}$ and $\mathbf{Bwd}$ denote the diffusion outputs. Each is passed through a temporal convolution layer that models sequential dependencies, producing $\mathbf{Fwd}_{conv}$ and $\mathbf{Bwd}_{conv}$.

These temporally convolved features are then fused to integrate complementary forward and backward information. The fused representation is fed into a stack of dense (fully connected) layers that transform the features and produce the final imputed sequences for the missing nodes in the graph.

- **Forecasting Head:** The forecasting head utilizes the output from the forward diffusion process to predict future traffic states. The forward diffusion embeddings, which aggregate spatial information propagated along the direction of traffic flow, are passed through an additional temporal convolution layer to model sequential dependencies relevant for future state predictions.

Formally, let $\mathbf{Fwd}$ denote the forward diffusion output. This tensor is input to a second temporal convolutional layer, enhancing its ability to capture pertinent temporal patterns extending into the forecast horizon.

The temporally convolved features are then fed through a stack of fully connected feed-forward networks (FFNs) that transform the learned representations into final predictions. These predictions correspond to the estimated traffic emission sequences up to one hour into the future, reflecting the model’s capacity for multi-step forecasting.

- **Loss Function:** The overall training loss $\mathcal{L}$ is a weighted combination of the imputation loss and the forecasting loss, balanced by a hyperparameter $\lambda \in [0, 1]$.

We define two types of masks:

The *dynamic mask* $m^{dy} \in \{0, 1\}^{B \times T \times N \times F}$, where $m^{dy} = 0$ indicates nodes temporarily masked for imputation, and $m^{dy} = 1$ indicates available nodes.

The *static mask* $m^{st} \in \{0, 1\}^{B \times T \times N \times F}$, where $m^{st} = 0$ indicates permanently missing nodes, and $m^{st} = 1$ indicates available nodes.

Given imputed outputs $\hat{\mathbf{X}}$, forecasting predictions $\hat{\mathbf{Y}}$, and ground truth sequences $\mathbf{X}, \mathbf{Y}$, the losses are defined as:

$$\mathcal{L}_{\text{imp}} = \frac{\sum (1 - m^{dy}) \odot (\hat{\mathbf{X}} - \mathbf{X})^2}{\sum (1 - m^{dy})},$$

$$\mathcal{L}_{\text{foc}} = \frac{\sum (m^{dy} \cdot m^{st}) \odot (\hat{\mathbf{Y}} - \mathbf{Y})^2}{\sum (m^{dy} \cdot m^{st})}.$$

where $\odot$ denotes element-wise multiplication and summations are over all dimensions.

The total loss optimized during training is then:

$$\mathcal{L} = \lambda \mathcal{L}_{\text{imp}} + (1 - \lambda) \mathcal{L}_{\text{foc}}.$$

This formulation ensures that the model learns to accurately reconstruct masked node features through imputation, while simultaneously minimizing forecasting error on unmasked nodes. For imputed nodes, forecasting is achieved by reinjecting their reconstructed values

into the forecasting model. This design enables a balanced and adaptive training process, as specified in the following pseudo-algorithm describing the model’s training 3.

Algorithm 3 Training with Dynamic and Static Masking for Imputation and Forecasting

```

1: Input: DataLoader with batches  $(\mathbf{X}, \mathbf{Y}, m^{dy}, m^{st})$ , model, adjacency matrix  $\mathbf{A}$ , loss
   weight  $\lambda$ , number of epochs  $E$ 
2: Initialize accumulators:  $\text{running\_loss}, \text{impute\_loss}, \text{forecast\_loss} \leftarrow 0$ 
3: for epoch in  $1 \dots E$  do
4:   Set model to train mode
5:   for each batch index  $i$ ,  $(\mathbf{x}_b, \mathbf{y}_b, m^{dy}, m^{st})$  in DataLoader do
6:     Expand masks to match feature dimension:  $m_{exp}^{dy}, m_{exp}^{st}$ 
7:      $\mathbf{x}_{b\_masked} \leftarrow \mathbf{x}_b \odot m_{exp}^{dy}$ 
8:      $(\text{imputed}, \text{pred}) \leftarrow \text{model}(\mathbf{x}_{b\_masked}, \mathbf{A}, m^{dy})$ 
9:     Imputation target mask:  $\text{mask}_{imp} \leftarrow 1 - m_{exp}^{dy}$ 
10:    Forecasting target mask:  $\text{mask}_{foc} \leftarrow m_{exp}^{dy} \odot m_{exp}^{st}$ 
11:     $\mathcal{L}_{imp} \leftarrow \frac{\sum (\text{mask}_{imp} \odot (\text{imputed} - \mathbf{x}_b)^2)}{\sum \text{mask}_{imp}}$ 
12:     $\mathcal{L}_{foc} \leftarrow \frac{\sum (\text{mask}_{foc} \odot (\text{pred} - \mathbf{y}_b)^2)}{\sum \text{mask}_{foc}}$ 
13:     $\mathcal{L} \leftarrow \lambda \mathcal{L}_{imp} + (1 - \lambda) \mathcal{L}_{foc}$ 
14:    Backpropagation:  $\mathcal{L}.\text{backward}()$ , optimizer step, reset gradients
15:    Accumulate  $\text{running\_loss}, \text{impute\_loss}, \text{forecast\_loss}$ 
16:   end for
17:   Report average epoch losses
18: end for
19: Output: Trained model

```

4.3.3 Federated Learning Integration

In urban areas, traffic emission monitoring often relies on a network of traffic surveillance cameras installed at multiple positions and road segments. These cameras continuously capture traffic flow and vehicle activity data that are essential for estimating pollutant levels. However, transmitting the raw, high-volume sensing data from these dispersed cameras directly to a central monitoring system poses significant challenges. The traffic data captured by cameras inherently contain sensitive information about vehicle movements and locations, raising serious privacy and confidentiality concerns, especially when cameras are owned and managed by different municipal authorities or private organizations. For instance, surveillance cameras operated by city government, private parking operators, and transportation agencies may each collect data that cannot be shared openly due to regulatory or competitive restrictions. These factors—communication bottlenecks, data heterogeneity, and privacy requirements—strongly motivate the adoption of distributed learning approaches such as federated learning, where models can be trained locally at the camera nodes without exposing raw data, thereby enabling cooperative and privacy-preserving traffic emission monitoring across the urban road network.

In federated learning (FL) scenarios, a significant challenge arises from the non-independent and identically distributed (non-IID) nature of data across clients. Each client, in our case representing edge servers belonging to different organizations monitoring distinct parts of an urban traffic network, often holds data with distinct distributions reflecting local traffic patterns, sensor types. This heterogeneity limits the predictive ability and generalizability of global models trained through traditional federated averaging methods (FedAvg), which assume client data are roughly IID. Early averaging approaches and conventional regularization techniques tend to

enforce a single global model that inadequately captures client-specific nuances, resulting in poor overall performance and inability to generalize well across all clients. The discrepancies between data distributions cause conflicting updates during aggregation, often slowing convergence and degrading accuracy. This limitation has catalyzed the emergence of a new paradigm in federated learning—personalized federated learning—that seeks a calibrated balance between global generalization and client-specific adaptation, enabling models better suited to the diverse tasks and distributions inherent to each client. This evolution addresses the fundamental shortcoming of early FL methods by explicitly factoring in client heterogeneity and task specificity, which are crucial in complex real-world applications such as urban traffic emission monitoring.

To better position our approach within the broader landscape of personalized federated learning (pFL), we examined several representative personalization strategies. These methods differ in how they balance global knowledge sharing with client-specific adaptation, aiming to alleviate the challenge of non-IID data. While initial experiments with fine-grained personalization techniques such as FedPer showed limited improvements in our setting, this motivated us to explore clustering as an alternative form of personalization.

In clustered FL (Sattler et al., 2019), clients with similar gradient dynamics are grouped together, and aggregation is performed within each cluster. This design allows models to better adapt to the heterogeneity of client data without overfitting to individual clients or collapsing into a single global model. Thus, clustered FL can be interpreted as an effective personalization strategy that lies between fully global and fully local training.

Dynamic Clustering Process in Federated Learning

Algorithm 4 Clustered Federated Learning (pseudocode)

```

1: Input:  $\theta^{(0)}, K, \eta, T$ 
2: for  $t = 0$  to  $T - 1$  do
3:   for all clients  $i$  in parallel do
4:     compute  $g_i^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_i(\theta^{(t)})$ 
5:   end for
6:   collect  $\{g_i^{(t)}\}$ 
7:   apply KMeans to clusters  $C_1^{(t)}, \dots, C_K^{(t)}$ 
8:   for  $k = 1$  to  $K$  do
9:      $\Delta_k^{(t)} \leftarrow \frac{1}{|C_k^{(t)}|} \sum_{i \in C_k^{(t)}} g_i^{(t)}$ 
10:     $\theta_k^{(t+1)} \leftarrow \theta^{(t)} - \eta \Delta_k^{(t)}$ 
11:   end for
12:   distribute  $\theta_k^{(t+1)}$  to clients in cluster  $C_k^{(t)}$ 
13: end for
14: Output: cluster models  $\{\theta_k^{(T)}\}$ 

```

In our clustered FL setup, personalization emerges by dynamically grouping clients that exhibit similar training dynamics. At each communication round, clients train locally and send their gradient updates to the server. Instead of averaging all updates globally (as in FedAvg), the server applies a clustering algorithm (K-Means) to partition clients into groups according to the similarity of their gradient directions. Within each cluster, updates are aggregated to produce a specialized model, which is then redistributed back to the clients. This procedure allows clusters to adapt dynamically from one round to another, enabling clients with similar learning behaviors to benefit from shared training while mitigating heterogeneity see algorithm 4.

4.4 Conclusion

This chapter has outlined the conception of our proposed framework for traffic emission estimation and prediction from roadside surveillance video. The design integrates three complementary components: a video processing and emission estimation pipeline that converts raw footage into structured emission data, a graph-based spatio-temporal model capable of both forecasting and imputing emissions across the network, and a federated learning strategy aimed at enabling scalable, privacy-aware deployment.

Together, these modules form a coherent architecture that directly addresses the limitations identified in earlier chapters—namely, the underutilization of urban surveillance cameras, the challenges of incomplete sensor coverage, and the need for privacy-preserving solutions. By bridging raw video data with network-level forecasting, the framework sets the foundation for both accurate estimation and practical adaptability across diverse urban contexts.

Having established this architectural blueprint, the next chapter turns to its concrete realization. We detail the implementation choices, algorithms, and technical considerations required to translate the conceptual design into a functioning system.

Chapter 5

Implementation

5.1 Introduction

The preceding chapter detailed the conceptual blueprint for our end-to-end framework, justifying the architectural choices and the logical flow of the proposed system. This chapter transitions from design to execution, providing a comprehensive account of the practical implementation of each component. computing platform used for development. Following this, the chapter is structured around the implementation of the two core modules of our framework. First, we will detail the implementation of the video processing pipeline, covering everything from the preparation and augmentation of the BoxCars116k and BrnoCompSpeed datasets to the specific configurations of the offline calibration and online inference components.

Subsequently, we will provide an equally thorough account of the spatio-temporal forecasting module. This includes the data generation and preprocessing steps that transform it into a suitable format for graph-based learning, as well as the specific implementation of the model’s architecture, its multi-task loss function, and finally the clustered federated learning setup. Ultimately, this chapter serves as a detailed technical record, bridging the gap between the conceptual design and the empirical results that will follow.

5.2 Tools and Libraries Used

- **OpenStreetMap (OSM):** An open-source, crowdsourced map database utilized to extract detailed road network structures and their characteristics, including turning conditions. OSM provides foundational geographic data representing the road network topology.
- **TOMTOM API:** A commercial API service providing real-time traffic data. It was used to extract Origin-Destination (OD) matrices indicating volumes of flows in each direction, as well as segment-level speed estimations, enabling realistic traffic modeling.
- **SUMO (Simulation of Urban MObility):** An open-source microscopic traffic simulator employed to generate realistic traffic circulation scenarios. SUMO simulates individual vehicle movements allowing detailed assessment of traffic dynamics and emissions.
- **TraCI (Traffic Control Interface):** A Python API for interfacing with SUMO simulations, enabling real-time control and data extraction from the traffic simulator. It allows executing simulation steps, querying vehicle states, and manipulating traffic elements programmatically.

- **PyTorch:** A powerful deep learning framework used for designing, implementing, and training neural network architectures. It offers dynamic computation graphs and GPU acceleration.
- **NetworkX:** A Python library for creation, manipulation, and analysis of complex graph structures. NetworkX facilitated the representation and operations on the road network graph including adjacency and traversal computations.
- **Optuna:** An open-source hyperparameter optimization framework that automates the search for optimal parameters using techniques such as random search, TPE (Tree-structured Parzen Estimator), and pruning strategies. It integrates seamlessly with deep learning frameworks like PyTorch and TensorFlow to improve model performance efficiently.
- **Scikit-learn (sklearn):** A comprehensive machine learning library used primarily for data preprocessing, including normalization and data splitting, enhancing data quality for model training.
- **Matplotlib:** A 2D plotting library utilized for visualizing data, model outputs, and traffic and emission patterns to support exploratory analysis and presentation.
- **Pandas:** A data manipulation library enabling efficient handling of tabular data via DataFrames, used extensively for organizing and processing traffic and emission datasets.
- **Flower:** A federated learning framework facilitating distributed training across decentralized data sources, enabling privacy-preserving training experiments for traffic modeling.
- **pyEmission:** An emission calculation library that estimates vehicle emissions based on traffic parameters, integrated with simulation outputs to produce accurate emissions assessments.
- **Neptune:** A metadata and experiment tracking tool used for monitoring model parameters, metrics, and changes in real time during training, supporting experiment reproducibility and analysis.
- **OpenCV:** A fundamental open-source computer vision library used for a wide range of image and video processing tasks, including reading video streams, image manipulation for data augmentation, and geometric transformations required for camera calibration.
- **NumPy:** The foundational package for numerical computation in Python. It was used extensively for handling multi-dimensional arrays, matrices, and performing mathematical operations that underpin both the computer vision and data processing workflows.
- **Timm (PyTorch Image Models):** A comprehensive library of state-of-the-art computer vision models and utilities. It was instrumental for accessing pre-trained weights and implementing the various classifier architectures (DeiT, ConvNeXt, etc.) benchmarked in our evaluation.
- **Ultralytics:** The framework providing the YOLOv8 model implementation. It was used for training and evaluating our baseline one-stage detector-as-classifier, as well as for the object detection front-end of our two-stage pipeline.

5.3 Implementation Environment

The entire implementation of our framework, from data preparation and model training to the final experiments, was conducted on the Grid’5000 distributed testbed¹. Grid’5000 is a large-scale and versatile experimental platform for research in all aspects of computer science, spread across multiple sites in France. We were granted access to the platform’s extensive resources, allowing us to reserve and deploy experiments on a variety of computing nodes. While we had access to multiple sites, our work was primarily concentrated on the Lille site, where we had a larger allocated storage quota of **200 GB**.

Resources on the platform were allocated on-demand via a job reservation system. This allowed us to request specific hardware tailored to our needs, such as powerful GPU nodes for computationally intensive tasks like model training and heavy inference, or standard CPU nodes for less demanding processes like data preprocessing, calibration utilities, and evaluation scripts. The specific details and technical specifications for all available nodes and GPU types can be found on the official Grid’5000 website. This flexible, high-performance computing environment was essential for managing the project’s significant computational load and data storage requirements.

5.4 Video processing pipeline Implementation

We implement the pipeline in three stages: (i) *offline calibration* to estimate vanishing points and focal length as well as the scale; (ii) *online inference* with detection, 3D box reconstruction, fine-grained classification, tracking and speed estimation; and (iii) *emission estimation* on per-track summaries. .

5.4.1 Data Description and Preprocessing

Before going into details on our implementation of the video processing pipeline we start by presenting the used datasets as well as the preprocessing.

5.4.1.1 BoxCars116k for Fine-Grained Vehicle Classification

An important prerequisite for accurate, class-dependent emission modeling is the ability to perform fine-grained vehicle classification from real-world surveillance imagery. To this end, we have selected the BoxCars116k dataset (Sochor, Špaňhel and Herout, 2019), obtained from the Kaggle platform². Its primary advantage lies in its specific relevance to our application domain; the dataset was curated from traffic surveillance viewpoints, capturing approximately 116,286 images of 27,496 unique vehicles across a wide array of poses, lighting conditions, and partial occlusions. This provides a significant advantage over datasets composed of sanitized, web-sourced images or surveillance footage limited to a single angle, as it ensures our model is trained on data that mirrors the challenging conditions of its deployment environment. For our work, we adopt the **Hard** configuration, which treats each unique combination of make, model, and model variant as a distinct class. While this preserves the richest possible signal for emissions modeling, it introduces a significant challenge: a severe long-tailed class distribution.

To quantify this imbalance, we conducted a detailed statistical analysis of the dataset. The class-frequency distribution exhibits a pronounced long-tail structure, a characteristic visually confirmed in Figure 5.1 and detailed in Table 5.1. Our analysis reveals that a small fraction of vehicle models dominate the dataset, with a few popular classes containing thousands of

¹<https://www.grid5000.fr/w/Grid5000:Home>

²<https://www.kaggle.com/datasets/igorlashkov/boxcars-dataset>

samples while many others are represented by only a few hundred images or less. The significant disparity between the mean and the much lower median number of images per class further underscores this severe skew. Such an imbalance can bias a model toward over-represented classes and degrade generalization on rare ones—precisely the categories that matter in fine-grained recognition—thereby motivating the class-aware strategies described next.

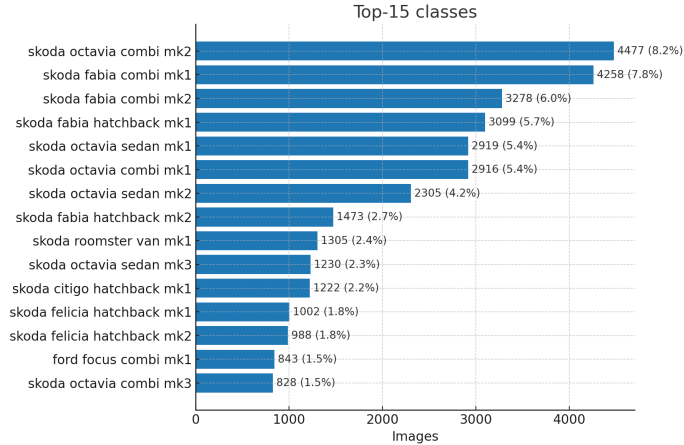


Figure 5.1: Top-15 vehicle classes by image count (BoxCars, 107-class subset).

Table 5.1: BoxCars 107-class subset: imbalance summary.

Metric	Value
Total classes	107
Total images	54454
Top-10 share	50.1%
Top-20 share	64.8%
Max class count	4477
Min class count	63
Imbalance ratio (max/min)	71.1×
Mean images/class	508.9
Median images/class	217.0
classes < 200	47
Fraction classes < 200	43.9%

5.4.1.2 Preparing and augmenting BoxCars116k

Before any augmentation, we convert BoxCars116k into a classifier-ready format. For each annotated vehicle we crop the 2D bounding box with a fixed **+8% context margin** (to avoid cutting distinctive parts such as badges and headlights) and clamp to image bounds. We split the dataset into 80% for training, 10% for validation and 10% for tests.

5.4.1.2.1 Class-aware augmentation (operational policy). The dataset is strongly long-tailed, so we adopt a two-tier policy applied *only* to the training split. We set a target per-class count $T = 600$. For any class with $n_c < T$, we generate $T - n_c$ augmented samples (*minority/heavy* pipeline). Classes with $n_c \geq T$ receive a light, on-the-fly augmentation during training (*majority/light*). Offline augmented files, when materialized, are tagged with the `_aug`

token in the filename and are never added to validation/test. The cap at $T = 600$ increases diversity for rare classes without distorting the overall class prior; heavy transforms simulate real surveillance artifacts (illumination, haze), whereas the light pipeline avoids unrealistic deformations for well-represented classes. All these augmentations were applied with the Albumentations python library, we resume them in Table 5.2

Table 5.2: List of augmentations used on BoxCars116k.

Transform	Purpose
HorizontalFlip	Exploit left-right symmetry of vehicles/lanes.
HueSaturationValue	Model color/illumination drift from camera.
Rotate	Small pose noise without unrealistic rotations.
RandomBrightness Contrast	Day/night and exposure variations.
Perspective	Mild viewpoint/geometric changes from mounting/road grade.
RandomFog	Low-visibility conditions common in traffic scenes.
Cutout / RandomErasing	Robustness to partial occlusions (other cars, poles).

5.4.1.3 BrnoCompSpeed

An essential component of our pipeline is the ability to accurately estimate vehicle speed from monocular traffic footage. To rigorously validate this module under realistic conditions, we adopt the BrnoCompSpeed dataset (Sochor, Juranek, Špaňhel, Maršík, Šíroký, Herout and Zemčík, 2019)—a benchmark specifically curated for single-camera visual speed measurement. We obtained access directly from the authors via email after several months, a detail that highlights the challenge in acquiring such specialized academic datasets.

The primary strength of BrnoCompSpeed lies in its meticulous data collection. The dataset comprises 18 full-HD videos, each approximately one hour long, recorded at six different roadside locations from three distinct viewpoints (frontal, left, and right). Crucially, each vehicle is annotated with a precise ground-truth speed measured by instrument-grade optical gates (LiDAR) and cross-checked with GPS, enabling a robust quantitative evaluation of our monocular pipeline. The dataset is further enriched with extensive scene metadata, including road-area masks, annotated measurement lines, and road-plane distance measurements. The official evaluation tooling is designed for reproducible comparisons, expecting standardized camera-calibration parameters (such as vanishing points and the principal point) and per-vehicle trajectories.

In our framework, BrnoCompSpeed serves exclusively as a benchmark for the evaluation of our speed estimation pipeline. It is important, however, to note its limitations: the dataset contains no vehicle class labels (make/model) or any emission-related annotations. Furthermore, its highway setting may not fully capture the lower-speed dynamics or heavier occlusions typical of dense urban CCTV. Despite these constraints, it remains one of the few publicly available datasets offering annotated surveillance videos with highly accurate speeds for thousands of vehicle trajectories. Finally, its shared origin with the BoxCars116k dataset ensures a high degree of domain compatibility in terms of capture geometry and vehicle fleet characteristics, strengthening the coherence of our overall data strategy.

5.4.2 Offline calibration

5.4.2.1 vanishing points detection

Our implementation is grounded in well-vetted, open-source libraries. Core low-level operations such as the Shi–Tomasi corner detector, Lucas–Kanade optical flow, the canny edge detector and the Line Segment Detector (LSD) were implemented using the OpenCV library. For the crucial voting step to identify the point of convergence, we utilized the original and unmodified Diamond Space implementation provided by the authors (Dubská and Herout, 2013), ensuring a faithful application of their robust method.

In practice, the estimation of the two primary vanishing points presented different challenges and required distinct implementation strategies.

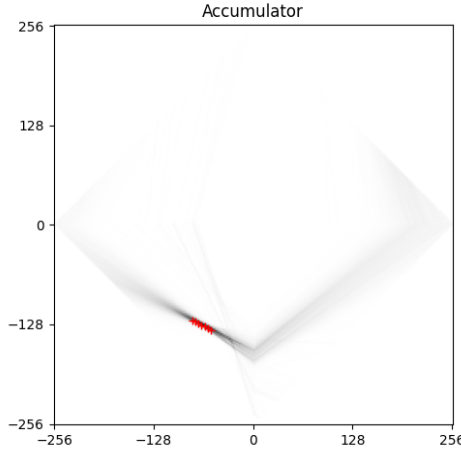


Figure 5.2: Example of a voting results in the diamond space.

Estimating the motion-based vanishing point, u , proved to be a relatively stable and automated process. A reliable estimate was typically achieved using 500 to 1500 video frames. The procedure begins with a lightweight YOLOv8-nano model to identify vehicles (cars, motorcycles, trucks, buses), retaining only detections with a confidence of at least 0.75. Within each of these bounding boxes, we then seed approximately 800 Shi–Tomasi corners. These corners are tracked across frames using Lucas–Kanade optical flow, and any resulting motion vectors with a displacement of less than 3 pixels are discarded to filter out noise. To ensure a continuous supply of high-quality tracks, corners are re-seeded every 100 frames or whenever the active track count falls below 300, we also noticed that a larger portion of the time, the resulting peaks in the diamond space after voting of the motion lines are very prominent and close to each other, as we can see in figure 5.2, which shows that selecting the most prominent peak leads to the dominant motion direction. We found that the default parameters for this motion-tracking procedure performed well across a wide variety of scenes, making the estimation of u a largely automated task.

In contrast, estimating the static vanishing point, v , was significantly more challenging and highly scene-dependent. The primary difficulty arises from the reliance on static line features (e.g., road markings), which can be sparse or visually ambiguous as we can see in figure 5.3. While a stable estimate could sometimes be achieved with just 150-500 frames, more complex scenes required accumulating data over 1000-2000 frames and often necessitated careful tuning of the Canny and LSD parameters. After line detection, we apply a cascade of geometric filters: segments aligned within 30° - 40° of the previously found u are discarded, as are those within 10° - 20° of the vertical axis. The surviving line segments then vote in Diamond Space to produce the final estimate for v .

This inherent scene dependency, particularly for v , motivates a natural future direction for this work: the design of an auto-tuning module. Such a system could automatically optimize key parameters to balance the trade-off between computational cost and estimation accuracy, which remains an open and worthwhile extension.



Figure 5.3: Example result of filtered segments(orange) used for v vanishing point.

5.4.2.2 Scale Estimation

To estimate the scene scale without introducing another fine-grained classification stage (and the maintenance burden of a per-model mapping file), we adopt global priors for typical vehicle dimensions and found them sufficient in practice: a *mean vehicle length* of **4.8** m and a *mean vehicle width* of **2.0** m. We implement the standard equations to recover the road plane and camera parameters from vanishing points (intrinsic, plane, projector from image to road), and we use the YOLOv6-3D projection and geometry utilities to reconstruct a 3D cuboid for each detected vehicle. On the modified frames we run the YOLOv6-3D detector with **confidence** = 0.6 and **intersection over union** = 0.85; for each detection we project its 3D cuboid then we take the cuboid’s footprint edges that lie on the road plane and measure the *width* from points 1–2 (bottom-front segment) and the *length* from points 1–5 (bottom side) using the euclidean distance (these points can be seen clearly in the bottom row of the 3D projected bounding box examples in figure 5.4). Endpoints are projected to the normalized road plane before measuring, so distances are orthonormal up to a single global scale. Denoting the projected points by $\mathbf{a}_n, \mathbf{b}_n, \mathbf{c}_n$, we compute

$$\ell_{\text{len}} = \|\mathbf{a}_n - \mathbf{b}_n\|, \quad \ell_{\text{wid}} = \|\mathbf{b}_n - \mathbf{c}_n\|.$$

We immediately reject implausible cuboids using a shape-ratio gate $2.0 \leq \ell_{\text{len}}/\ell_{\text{wid}} \leq 3.5$, then convert every accepted vehicle into two candidate scales,

$$s_{\text{len}} = \frac{4.8}{\ell_{\text{len}}}, \quad s_{\text{wid}} = \frac{2.0}{\ell_{\text{wid}}} \quad (\text{m/px}),$$

apply a symmetric 5–95% percentile trim to remove outliers, and require at least 20 valid vehicles to stabilize aggregation. For the final scene scale $\hat{\lambda}$ we keep the pipeline simple and reproducible: we compute the trimmed medians $\hat{s}_\ell = \text{median}(s_{\text{len}})$ and $\hat{s}_w = \text{median}(s_{\text{wid}})$, and report $\hat{\lambda} = \frac{1}{2}(\hat{s}_\ell + \hat{s}_w)r$. Defaults worked well across scenes; denser traffic allows tightening these gates, while sparse scenes benefit from loosening them to accumulate enough candidates. We finally accept $\hat{\lambda}$ only if $\hat{s}_\ell$ and $\hat{s}_w$ agree within a small tolerance (a few percent); otherwise we gather more frames and tighten the filtering parameters then re-aggregate. The resulting $\hat{\lambda}$ (in m/px) is saved with per-axis estimates and used later on directly by the speed module to convert pixel displacements into meters.

5.4.3 Online inference module

5.4.3.1 Vehicle detection and 3D bounding box reconstruction

For the monocular 3D vehicle detection and reconstruction, we adopted the publicly available implementation of the YOLOv6-3D model³.

We integrated this implementation into our pipeline with only one minor but crucial modification. The original code did not expose the detector’s confidence score, a value essential for the subsequent tracking module (ByteTrack). We therefore adapted the inference function to return the confidence alongside the 2D bounding box and the α value.

These outputs serve two downstream purposes: the (box, α) pair is used by the geometry module to reconstruct the 3D cuboid, while the confidence score is passed to ByteTrack to manage identity assignment. Figure 5.4 provides a visual illustration of this process, showing the 2D detection in the transformed image and the corresponding 3D bounding box that is reconstructed from it for multiple vehicles.

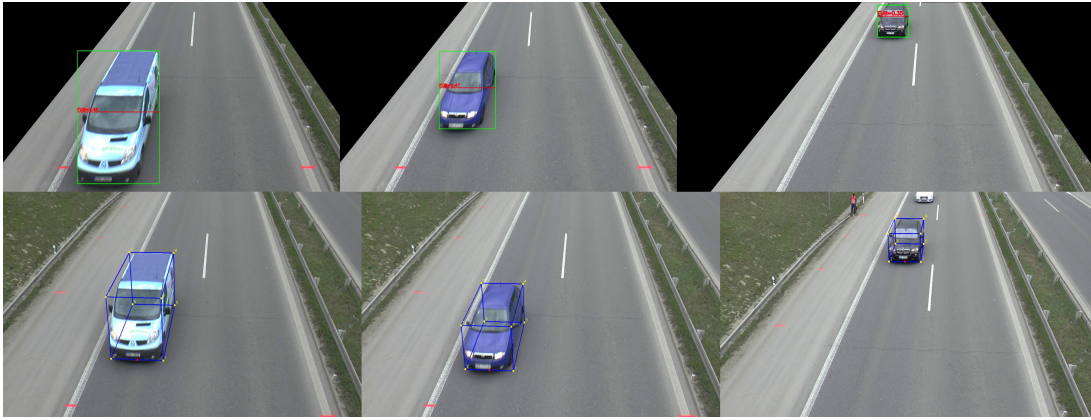


Figure 5.4: Examples of detection (top: 2D box + alpha in BEV) and reconstruction (bottom: 3D box projected onto image)

5.4.3.2 Fine grained vehicle classification model

To ensure a fair, rigorous, and reproducible comparison of the candidate classifiers, we established a unified training protocol that was applied consistently to each model. This protocol standardizes everything from data preparation to optimization and model selection.

The training process for all models utilized the preprocessed and cropped vehicle images from the BoxCars116k dataset. Images were resized to match the native input dimensions of each architecture— 224×224 for DeiT and 320×320 for the convolutional classifiers—to ensure compatibility with their pretrained weights while preserving essential visual detail. A consistent set of data augmentations was applied to introduce meaningful diversity, including small random rotations ($\pm 8^\circ$), color jitter, horizontal flipping, and random erasing. This standardized pipeline ensures that each model was trained on data with the same distribution and level of challenge.

Each model was trained using a two-phase strategy to ensure stable convergence. We began with a **20-epoch warm-up phase**, where only the final classification head was trained while the pretrained backbone remained frozen. This allows the classifier to adapt to the new domain without destabilizing the rich, pre-learned features. Following this, the entire model was unfrozen for a full **fine-tuning phase** of 80 to 100 epochs, allowing the backbone to specialize to the nuances of vehicle classification.

³<https://github.com/gajdosech2/Vehicle-Speed-Estimation-YOLOv6-3D>

Optimization was performed using the **AdamW** optimizer with a **cosine learning rate decay**. We used a **Cross-Entropy loss** with **label smoothing** to mitigate model overconfidence, a common issue in datasets with long-tailed class distributions. Throughout this process, we monitored performance on a validation set and retained the checkpoint with the highest Top-1 accuracy for final evaluation.

The candidate models subjected to this protocol are summarized in Table 5.3.

Table 5.3: Fine grained classification models

Model	Pretrained weights (ID / filename)	Pretraining dataset	Library
DeiT-Small (patch16)	deit_small_patch16_224.fb_in1k	ImageNet-1k	timm
ConvNeXt-Tiny	convnext_tiny.fb_in22k_ft_in1k	ImageNet-22k → ImageNet-1k (ft)	timm
MobileNetV3-Large (1.0x)	mobilenetv3_large_100	ImageNet-1k	timm
EfficientNetV2-S	EfficientNet_V2_S_Weights.IMAGENET1K_V1	ImageNet-1k	torchvision

5.4.3.3 Vehicle tracking and speed estimation

This section details the implementation of our tracking and post-processing pipeline. The goal is to convert the frame-by-frame 3D vehicle detections into stable trajectories, each with a single, reliable fine-grained class label.

For the real-time tracking, we use the official implementation of the ByteTrack algorithm. While our detector provides a full 3D cuboid, we track the vehicle’s bounding box on the rectified frame, using the center of the cuboid’s bottom-front edge as the kinematic reference point. We set the initial detection confidence threshold to 0.6 to ensure only high-quality detections are used to initialize new tracks, while a lower threshold of 0.1 allows the algorithm to rescue existing tracks with lower-confidence detections during brief occlusions. To minimize identity switches, the IoU matching threshold was set to a strict 0.9, and a track is only considered lost if it remains unassociated for a buffer of 20 frames. For each frame, we log the real-time kinematic data for every active track ID, including its image and world-plane coordinates and its instantaneous speed.

A single vehicle’s passage may generate multiple classification events with varying confidence scores. To assign one definitive label to the entire track, we implemented an aggregation logic. First, all predictions with a confidence below 0.4 are discarded. The final label is then determined primarily by the total accumulated confidence for each unique class across the trajectory. This choice can be overridden by a special condition that favors long, stable runs of a single class, prioritizing periods of high certainty that often occur when the vehicle is clearly visible.

The process concludes by logging the processed data for downstream analysis. We save both a detailed, per-frame log containing kinematic data for debugging and a per-track summary file. This summary file, which contains the median speed and the final, aggregated classification label for each vehicle, serves as the primary input for the emission estimation module.

5.4.4 Emission estimation implementation

With the vehicle trajectories and classifications established, the final step in our video processing pipeline is to estimate the carbon emissions for each track. This is accomplished by feeding the detailed kinematic data into the **PyEmission** library.

The process begins by preparing two key inputs for each tracked vehicle. The first is a set of physical vehicle parameters. Since the BoxCars116k dataset does not include these, we compiled a supplementary mapping file (a snippet is shown in Table 5.4). This file links each of the 107 fine-grained vehicle classes to its essential characteristics: mass, frontal area, and drag coefficient, which were gathered from official and third-party automotive sources.

The second, and most critical, input is the vehicle’s **1 Hz driving cycle**. This is a time series of the vehicle’s speed, recorded once per second. We construct this cycle from the high-frequency data we previously logged. For each second of a vehicle’s trajectory, we take the *median* of all recorded speed samples, convert it to m/s, and use this value for that second’s data point. Any seconds where the vehicle was not tracked (e.g., due to brief occlusion) are filled with zeros.

The library processes this information to produce a detailed report of per-vehicle pollutants (CO₂, CO, NO_x, HC) and fuel consumption per tracked vehicle.

Table 5.4: Snippet of the file containing vehicle classes and their characteristics.

class_id	class_name	vehicle_type	mass_kg	frontal_area_m2	cd
0	audi a6 combi mk4	passenger_car	1590.000	2.260	0.310
1	audi a6 sedan mk4	passenger_car	1790.000	2.260	0.290
2	audi q7 suv mk1	suv	2240.000	2.870	0.340
3	bmw 1 hatchback mk1	passenger_car	1375.000	2.160	0.300
4	bmw x3 suv mk2	suv	1880.000	2.650	0.360

5.4.5 Data Aggregation for Forecasting.

The per-vehicle emission data is the final output of our estimation pipeline, but it must be aggregated before it can be used by the downstream forecasting model. To prepare this data, we process the results in discrete time steps (e.g., 10 seconds). For each time step, we aggregate all vehicles that were present on the road segment and compute several key features: the total vehicle count for each major category (SUV, sedan, truck, Light duty vehicle, Heavy duty vehicle and motorcycles), the sum of their CO₂ emissions, and the median speed of all vehicles. This creates a structured time-series dataset that perfectly matches the input requirements for our spatio-temporal emission prediction model.

5.5 Data Generation for STAD-NET Model and Pre-processing

Realistic mobility scenarios are essential for accurate traffic emission estimation. Existing traffic datasets predominantly focus on flow or speed estimation tasks, or are derived from taxi trajectory data. However, such datasets often lack several important features required for emission modeling, including direct emission measurements and detailed vehicle category information.

Therefore, it is essential to generate training data that incorporates the necessary attributes for training robust prediction models. Simply generating random traffic scenarios is inadequate, as it fails to capture the crucial spatiotemporal dependencies present in real-world urban traffic.

To address this, we simulate near-realistic mobility scenarios that preserve the intrinsic spatiotemporal relationships between traffic elements. This approach ensures that the resulting data represents realistic traffic dynamics necessary for high-fidelity emission estimation.

In the following sections, we detail the process of generating these mobility scenarios using microscopic traffic simulation tools, particularly SUMO, and explain the subsequent feature extraction and data preprocessing steps to prepare the dataset for model training.

5.5.1 Data generation description



Figure 5.5: Road network from OpenStreetMap

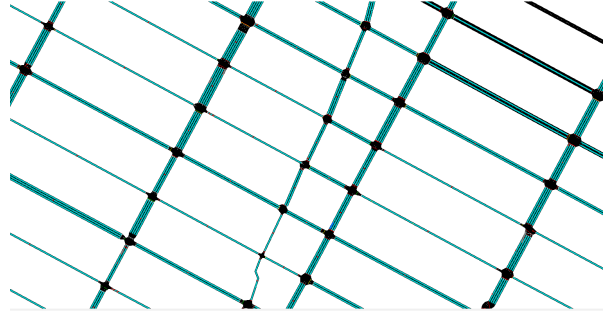


Figure 5.6: SUMO traffic simulation scenario

Figure 5.7: Comparison of (a) the extracted road network from OpenStreetMap and (b) the simulated traffic scenario generated in SUMO.

The process begins by selecting a target region for traffic simulation. We chose the Manhattan urban area, a widely recognized benchmark in traffic scenario studies, particularly in urban settings where grid network structures dominate. Manhattan is a particularly suitable testbed due to the availability of comprehensive trip and speed data collected within the area.

Starting with OpenStreetMap (OSM), the road network data for Manhattan is extracted and formatted to be compatible with the SUMO traffic simulation platform see figure 5.7. However, a road network alone is insufficient to run realistic simulations; route definitions are required to model vehicle movement accurately.

To generate these routes, we utilize traffic trip data collected via the TOMTOM API. This dataset comprises hourly Origin-Destination (OD) see figure 5.8 matrices for Manhattan from August 1 to August 28, 2024, along with corresponding speed measurements within the same period. The OD matrices provide the volume of trips between various zones, while speed data reflects actual traffic conditions. Using these two data sources, routes are generated and automatically mapped from real-world geographic coordinates into SUMO’s coordinate system, simulating nearly a full month of traffic activity within Manhattan. To introduce heterogeneity, SUMO’s vehicle class distribution feature is employed, allowing the inclusion of multiple vehicle categories, each characterized by unique parameters influencing emission profiles.

Within the simulation environment, 2D detectors are strategically placed across road segments to capture traffic metrics, such as vehicle counts per class, average speeds, and cumulative emissions per vehicle over specified time intervals. This information is recorded and compiled into CSV files, effectively simulating the type of data captured by real-world camera sensors.

Finally, these CSV files, along with the graph adjacency matrix derived from the road network, undergo preprocessing and become the input datasets for the modeling pipeline.

5.5.2 Data Preprocessing

While the process of generating the data has been described in the previous section, here we focus on transforming the obtained raw data into a structured and meaningful format suitable for

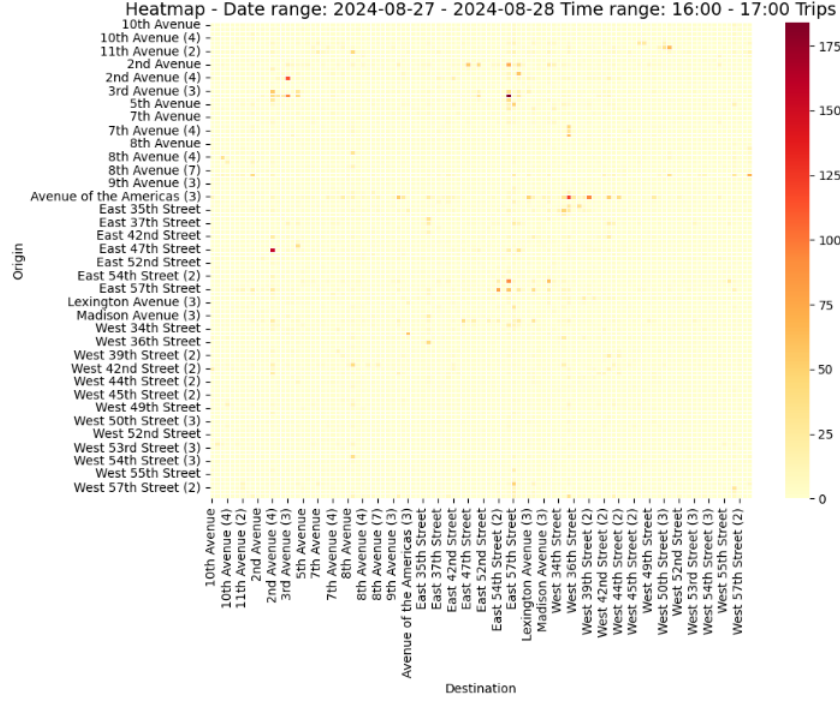


Figure 5.8: Example of the Origin Destination Matrix.

forecasting models. The traffic data used in this work is generated from mobility scenarios and collected in real time at intervals of two seconds using the `traci` library to interact with SUMO. However, such a fine temporal granularity produces data that is not well organized and often prevents the identification of clear patterns. In particular, the raw distribution obtained directly from the generation is characterized as zero-inflated, which requires several preprocessing steps in order to improve usability and robustness. These steps are detailed below:

- **Sampling:** To adapt the data to real-world settings and highlight broader temporal patterns, raw values recorded every two seconds are aggregated over 15-minute intervals. This sampling reduces noise, smooths out highly granular fluctuations, and aligns the data with realistic traffic monitoring practices, where such fine-grained measurements are typically unavailable.
- **Count data transformation:** Vehicle count data often exhibits heavy-tailed, skewed distributions with high variability across time intervals. To stabilize variance and reduce skewness, a logarithmic transformation is applied. This transformation makes the data more Gaussian-like, facilitating pattern detection and improving the performance of forecasting models that are sensitive to scale.
- **Emission data smoothing:** Raw emission data represents aggregated sums over short intervals. When extended to 15-minute windows, this may introduce sudden variations due to localized spikes, which are not representative of gradual atmospheric dispersion. To address this, an Exponential Moving Average (EMA) is introduced as an additional feature alongside the raw signal. The EMA acts as a noise-reducing indicator that emphasizes seasonality and smoother trends without discarding the original data.
- **Normalization:** Finally, a min-max normalization is applied to scale all variables into a common range, typically $[0, 1]$. This step ensures that features with different magnitudes contribute proportionally to the learning process and avoids dominance of high-scale variables (e.g., emissions) over lower-scale ones (e.g., counts).

The combination of these preprocessing steps transforms the zero-inflated, noisy raw measurements into well-structured time series that both reflect realistic traffic monitoring conditions and enhance the ability of the forecasting model to learn meaningful spatiotemporal patterns.

5.5.3 Sliding Window and Data Splitting

Before applying the sliding window technique, the dataset is structured as a three-dimensional tensor of shape (T, N, F) , where T denotes the number of timesteps, N the number of nodes in the road network, and F the number of features per node. In our case, the data contains $T = 1887$ timesteps, $N = 189$ nodes, and $F = 8$ features. This representation ensures that at each time step, every node is associated with a feature vector of dimension F , capturing the traffic characteristics of the road segment it represents. Such a structure provides the foundation for applying temporal learning strategies.

To capture temporal dependencies in traffic dynamics, we adopt a sliding window strategy with stride=1 for constructing input-output pairs. At each step, we use the previous 8 timesteps (corresponding to 2 hours of historical data, with a 15-minute sampling interval) as the input sequence, and forecast the subsequent 6 timesteps, which correspond to a one-hour and a half prediction horizon. This formulation follows a sequence-to-sequence prediction paradigm, where inputs are represented as multivariate time series over all graph nodes.

After constructing the input-output samples, the dataset is divided into training, validation, and test subsets with a chronological split to prevent information leakage. Specifically, the first 70% of the samples are used for training, the following 15% for validation, and the remaining 15% for testing.

5.6 Spatio Temporal Model Implementation

5.6.1 Mask Generation

To improve the robustness of our model to incomplete observations, we generate a dynamic mask during training. While the static mask corresponds to the real missing nodes (nodes for which no sensor data is available across all timesteps), the dynamic mask simulates partial and random unavailability of data. This strategy forces the model to learn spatiotemporal dependencies and leverage information from neighbors and historical values to reconstruct missing observations.

Concretely, the dynamic mask is defined as a binary tensor of the same shape as the input sequence (B, T, N, F) , where B is the batch size, T the input length, N the number of nodes, and F the number of features. Each entry is set to 0 if the corresponding node-feature is masked (artificially hidden), and 1 otherwise. The proportion of masked values is controlled by a masking ratio $\rho \in [0, 1]$. In our experiments, we set $\rho = 0.15$, meaning that 15% of the available node-feature values are masked at each training iteration. At each training batch, we sample a new mask by randomly selecting nodes or features according to ρ .

Importantly, this dynamic masking process is applied only on nodes/features that are originally available in the dataset (i.e., not permanently missing). In this way, the model learns to impute missing information by exploiting correlations across both space (neighboring nodes) and time (previous timesteps). During inference, only the static mask with ratio of 20% corresponding to the real missing nodes is applied.

This procedure can be summarized as:

- Initialize a binary mask with all ones.
- Randomly sample a subset of available node-feature pairs according to the masking ratio ρ .

- Set the corresponding entries to zero to simulate missing values.
- Multiply the mask with the input sequence to obtain the final masked input.
- Then feed it into the model as an additional channel, enabling the model to be mask-aware.

5.6.2 Model Architecture

The proposed model is organized into several layers and modules, each responsible for handling missing values, capturing temporal dependencies, and modeling spatial diffusion. Table 5.5 summarizes the main components of the architecture along with their parameters.

For the training of our Spatio-Temporal Attention Diffusion Network (STAD-Net), we optimized the model using the Adam optimizer with a learning rate of 1×10^{-3} , which has shown strong stability in spatio-temporal forecasting tasks. The model was trained for a maximum of 70 epochs with a batch size of 32. To prevent overfitting and reduce unnecessary computations, we applied an early stopping criterion: training was stopped if the validation loss did not improve for 5 consecutive epochs.

Module	Description	parameters
Projection Layer	Linear projection of concatenated input features and one mask channel	Input dim $F = 8+1$, Hidden dim $H = 32$
Temporal Encoding	Sinusoidal positional encoding	Max sequence length = 8
Initial Temporal Conv	Gated grouped temporal CNN	Kernel size = 5, Hidden dim $H = 32$
Transition Matrix	Attention-based adjacency refinement	Hidden dim = 32, Seq length = 8,
Bidirectional Diffusion	K-hop message passing (forward/backward)	k , Learnable α_f, α_b , Hidden dim = 32
Temporal Conv (forward)	Gated CNN for forward diffusion output	Kernel sizes = 3 and 5
Temporal Conv (backward)	Gated CNN for backward diffusion output	Kernel sizes = 3 and 5
Decoder (Imputation)	Layers: $[2 \times \text{Linear} + \text{ReLU}]$	Sequence length = 8, Hidden dim = 32
Decoder (Forecasting)	Layers: $[2 \times \text{Linear} + \text{ReLU}]$	Forecasting length = 8, Hidden dim = 32

Table 5.5: Summary of the model architecture and main hyperparameters.

5.6.3 Task Balancing and Loss Weighting

In our implementation, we address the challenge of balancing multiple tasks, namely imputation and forecasting, by carefully designing the loss function. The total loss for training the model is defined as a weighted sum of the imputation loss $\mathcal{L}_{\text{imp}}$ and the forecasting loss $\mathcal{L}_{\text{fore}}$:

$$\mathcal{L}_{\text{total}} = \lambda \mathcal{L}_{\text{imp}} + (1 - \lambda) \mathcal{L}_{\text{fore}}, \quad (5.1)$$

where $\lambda \in [0, 1]$ represents the relative importance of each task. One of the main challenges when using a fixed balancing parameter is deciding the appropriate value for each task, in order

to prevent the shared backbone from being biased towards the easier task—in our case, the forecasting task. To overcome this limitation, we employ a dynamic weighting strategy.

To automatically adjust these weights during training and ensure balanced learning of both tasks, we adopt the *Dynamic Weight Averaging (DWA)* method proposed by Liu *et al.* (Liu et al., 2019). In DWA, the task weights are dynamically computed based on the relative rate of loss change between consecutive epochs, allowing the model to focus more on tasks that are learning slower, thus ensuring a balanced optimization. Formally, the DWA weights are calculated as:

$$\lambda_k(t) = \frac{K \cdot \exp\left(\frac{L_k(t-1)}{L_k(t-2)}\right)}{\sum_{i=1}^K \exp\left(\frac{L_i(t-1)}{L_i(t-2)}\right)} \quad (5.2)$$

where K is the number of tasks, and $L_k(t)$ denotes the loss of task k at epoch t .

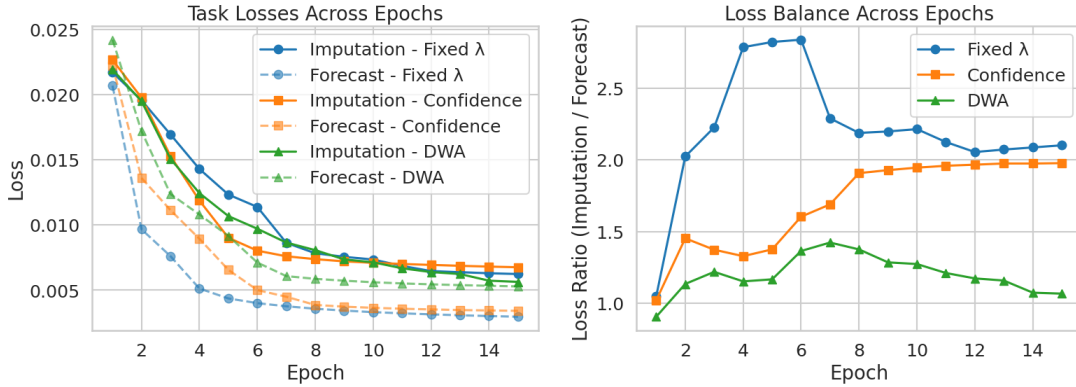


Figure 5.9: Variation of imputation and forecasting losses over epochs using different loss weighting strategies, including fixed λ , confidence-based, and DWA.

We found DWA to be particularly suitable for our scenario, as imputation and forecasting have different convergence rates. As illustrated in Figure 5.9, we compared three cases: (i) a fixed unweighted loss with $\lambda = 0.5$, (ii) a confidence-based weighting, and (iii) DWA. DWA effectively balances the contribution of each task throughout training, preventing one task from dominating the optimization and resulting in improved overall performance.

5.7 Federated Learning implementation

This section aims to explain how federated learning is integrated into our STAD-Net model. For this purpose, we partitioned the data horizontally among 9 different clients. In this study, we did not focus on system heterogeneity; instead, we considered clients with varying data sizes according to their assigned subgraphs. The description and characteristics of each client dataset are summarized in Table 5.6.

5.7.0.1 Federated Learning Setup

The experiments were conducted with $N = 9$ clients, where at each communication round, 80% of clients were randomly selected to participate in training. Each selected client trained locally for $E = 5$ epochs using mini-batches of size 32 before sending updates to the server. The server performed aggregation using either standard FedAvg or our proposed Clustered Federated Learning (CFL). For CFL, client gradients were clustered dynamically at each round using K-Means with cosine similarity. The number of clusters was fixed to $K = 3$. The complete configuration is summarized in Table 5.7.

Table 5.6: Client datasets in federated learning, where each client is a subgraph with its feature and adjacency matrices.

Client (Subgraph)	Nodes	Edges	Avg Degree
0	15	26	3.47
1	21	29	2.76
2	23	27	2.35
3	26	36	2.77
4	24	33	2.75
5	23	29	2.52
6	19	18	1.89
7	28	40	2.86
8	10	10	2.00

Table 5.7: Federated Learning Setup

Parameter	Value
Number of clients (N)	9
Fraction of clients per round	0.8
Number of clusters (K)	3
Communication rounds (T)	10
Local epochs (E)	5
Batch size	16
Optimizer	Adam
Learning rate (η)	0.001
Aggregation strategy	FedAvg / CFL
Clustering algorithm	K-Means (cosine similarity)

5.8 Conclusion

In this chapter, we have transitioned from architectural design to the realization of a concrete and fully functional system. The necessary tools, datasets preprocessing steps, and models were carefully implemented, configured, and integrated into a coherent pipeline. With the implementation phase completed, the focus now shifts toward a comprehensive performance assessment. The subsequent chapter is dedicated to the rigorous evaluation of each system component, testing its reliability, accuracy, and efficiency. Through this analysis, we aim to validate the effectiveness of the proposed solution and to quantify its performance with respect to the initial research objectives, thereby bridging the gap between design and measurable outcomes.

Chapter 6

Tests and Solution Evaluation

6.1 Introduction

The previous chapters have detailed the conceptual design and technical implementation of our end-to-end system for traffic carbon footprint estimation and prediction. The purpose of this chapter is to move from theory and construction to a rigorous, empirical validation of the proposed solution. To this end, we will conduct a series of experiments designed to systematically assess the performance of each core component and demonstrate the system’s overall efficacy. The evaluation is structured to mirror the modular design of our framework, assessing each primary component in a systematic manner. We will begin with a comprehensive validation of the video processing pipeline. This involves a series of targeted experiments to individually evaluate its key sub-modules: we first assess the performance of the fine-grained classification model the accuracy of the offline camera calibration, then the accuracy of the offline camera calibration, and finally, the reliability of the tracking, scale, and speed estimation results. Following this, we will evaluate the spatio-temporal forecasting module, assessing its effectiveness in the dual tasks of imputation and forecasting under various conditions. We will also present the results obtained from the employed federated learning technique and compare them with established baselines. This component-wise validation allows us to not only measure the performance of the system as a whole but also to understand the specific contribution and reliability of each stage of our proposed solution.

6.2 Video processing pipeline evaluation

This section presents the empirical evaluation of the video processing pipeline. Our analysis is structured to follow the logical flow of the pipeline, assessing each key component sequentially. We begin with an extensive evaluation of the fine-grained vehicle classification model, then we move on to the calibration process, where the primary goal is to validate the correctness of the estimated camera parameters—specifically the vanishing points and focal length—as these are critical for all subsequent metric calculations. With the camera geometry established, we then evaluate the speed estimation module, which concurrently tests the performance of our metric scale estimation and the final speed computation.

6.2.1 Fine-Grained Classification Evaluation

In this section, we evaluate our fine grained classification model. The goal is to demonstrate its performace not only on images but also on video. The evaluation proceeds in two steps:

1. First, we conduct a **comparative analysis of several dedicated classification models** to select the best-performing one based on accuracy and efficiency.
2. Second, although our videos lack official fine-grained annotations, we run a small **qualitative validation** on real footage to confirm that the chosen classifier also behaves well in a real-world context.

6.2.1.1 Comparative Analysis of different fine grained Classifiers

To select the optimal classifier for our two-stage pipeline, we adopt a unified evaluation protocol. Using the shared setup described earlier, we train each candidate model and keep the checkpoint that achieves the highest validation Top-1 accuracy. We then evaluate that checkpoint on the held-out test set. The results are benchmarked not only on classification performance but also on practical characteristics to provide a holistic view of their deployment trade-offs.

6.2.1.1.1 Evaluation Metrics. To ensure a comprehensive comparison, we evaluate the models using a combination of accuracy and efficiency metrics:

- **Top-1 and Top-5 Accuracy:** Top-1 accuracy is the standard classification metric, measuring the percentage of predictions where the model’s single highest-probability output is the correct class. Top-5 accuracy, which is particularly relevant for fine-grained tasks with many classes, considers a prediction correct if the true label is among the model’s top five predictions. It provides insight into whether the model is ”close” even when it is not perfectly correct.
- **Macro-F1 Score:** To account for potential class imbalance in the dataset, we report the Macro-F1 score. It is the unweighted mean of the F1 scores calculated for each class, ensuring that the performance on rare vehicle models is given the same importance as performance on common ones. An F1 score is the harmonic mean of precision and recall.
- **Parameters (M):** The number of trainable parameters (in millions) indicates the model’s size and complexity.
- **Inference Speed (FPS):** Measured in Frames Per Second (FPS) with a batch size of one, this metric quantifies the model’s throughput and is critical for real-time application performance.

Table 6.1: Fine-grained classification performance on the test set.

Model	Top-1 (%)	Top-5 (%)	Macro-F1	Params (M)	FPS (bs=1)
DeiT-S	98.99	99.98	0.99	21.79	252.08
ConvNeXt-T	98.85	99.98	0.99	27.90	202.18
EfficientNetV2-S	98.70	99.96	0.99	20.32	53.57
MobileNetV3-L	97.94	99.93	0.98	4.34	193.12

The results summarized in Table 6.1 reveal a clear hierarchy among the models, allowing us to analyze the trade-offs between accuracy and efficiency. While the top three models—**DeiT-S**, **ConvNeXt-T**, and **EfficientNetV2-S**—all achieve excellent and highly competitive accuracy scores, **DeiT-S** consistently holds a slight lead in both Top-1 and Macro-F1 metrics. This suggests it maintains a marginal but important advantage, particularly on the less frequent vehicle classes.

The choice of the model therefore depends on the target deployment scenario. If the primary constraint were minimal model size for edge deployment, MobileNetV3-L would be the logical choice. However, for the scope of this project, our goal is to achieve the highest possible accuracy while still maintaining real-time processing capabilities. Given that modern edge devices are increasingly powerful, we prioritize the superior accuracy and speed of **DeiT-S**. We accept its larger parameter count as a reasonable trade-off for its state-of-the-art performance, and thus select it for the remainder of the pipeline.

6.2.1.2 Detailed results of the DeiT-S

Having selected DeiT-S as the optimal classifier, we now conduct a detailed analysis of its performance on the held-out test set. We evaluate the model using Top-1/Top-5 accuracy, Macro-F1 score, and a normalized confusion matrix (Figure 6.1) to provide a comprehensive view of its fine-grained recognition capabilities.

The model demonstrates outstanding overall accuracy, the corresponding confusion matrix, shown in Figure 6.1, is almost perfectly diagonal, visually confirming the model’s high precision. The performance is remarkably consistent across the entire label space, with a median class F1-score of 0.996. Impressively, 102 of the 107 classes achieve a recall of at least 99% , and all classes surpass 95% recall, indicating that the model’s strength is not limited to just the most frequent vehicle types.

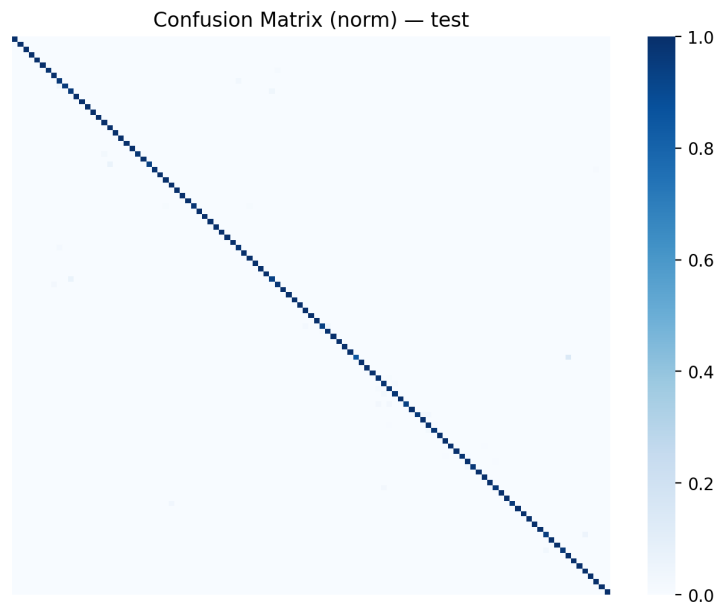


Figure 6.1: Normalized confusion matrix for the DeiT-S classifier on the test set.

A closer examination reveals that the few errors the model makes are not random but are semantically plausible, occurring almost exclusively between "sibling" vehicles. The most common confusions, detailed in Table 6.2, involve vehicles of the same model with different body styles (e.g., Combi vs. Hatchback) or adjacent generations (mk2 vs. mk1).

We also analyzed the classes with the lowest F1-scores. As shown in Table 6.3, even these "worst-performing" classes maintain a very high level of accuracy, with the lowest F1-score still being 92.3%. Furthermore, the main source of error for these classes remains confusion with their respective platform twins (e.g., SEAT Alhambra vs. VW Sharan, Peugeot Partner vs. Citroën Berlingo), reinforcing the pattern of semantically plausible errors.

In summary, this detailed analysis confirms that DeiT-S is an exceptionally robust classifier for our fine-grained task. Its high overall accuracy, consistent performance across all classes, and

Table 6.2: Top 5 most frequent confusion pairs for the DeiT-S model.

True Class	Predicted Class	Count
Skoda Fabia Combi mk2	Skoda Fabia Hatchback mk2	11
Skoda Fabia Hatchback mk1	Skoda Fabia Combi mk1	10
Skoda Fabia Combi mk1	Skoda Fabia Hatchback mk1	8
Skoda Fabia Hatchback mk2	Skoda Fabia Combi mk2	7
Skoda Felicia Hatchback mk2	Skoda Felicia Hatchback mk1	5

Table 6.3: The 5 classes with the lowest F1-scores and their primary misclassification.

Class Name (True Class)	F1-Score	Mainly Confused With
SEAT Alhambra MPV mk1	0.923	Volkswagen Sharan MPV mk1
Fiat Doblo Van mk1	0.939	Peugeot Partner Van mk1
Peugeot Partner Van mk1	0.951	Fiat Doblo Van mk1
Peugeot Partner Van mk2	0.955	Citroën Berlingo Van mk2
Renault Megane Combi mk2	0.958	Renault Megane Combi mk3

the understandable nature of its rare errors make it a highly reliable component of our pipeline.

6.2.1.3 Sanity Check on Track-Level Label Aggregation

While the BrnoCompSpeed dataset lacks official fine-grained annotations, a qualitative sanity check is essential to confirm that our label aggregation logic performs well on real-world video.

Table 6.4: Manual sanity check on the first 30 tracks from BrnoCompSpeed Session 1 (center view).

Category	Count	Share
Correct	22	73.3%
Incorrect (In-Vocabulary)	2	6.7%
Out-of-Vocabulary (OOV)	6	20.0%
Total	30	100%
In-Vocabulary Accuracy	91.7% (22/24)	

We conducted a small manual audit on the **first 30 vehicle tracks** from Session 1 (center view). For each track, we visually identified the ground-truth class and compared it to the final, aggregated label produced by our pipeline. Each track was then categorized as:

1. **Correct:** The final label exactly matches the ground truth.
2. **Incorrect:** The final label is wrong, but the true class exists in our training dataset.
3. **Out-of-Vocabulary (OOV):** The true vehicle model does not exist in our training dataset.

Out of the 30 tracks, our pipeline correctly identified 22 vehicles. Of the remaining 8, 6 were Out-of-Vocabulary, and only 2 were incorrectly classified. This yields an in-vocabulary accuracy of 91.7% (22 correct out of 24 in-vocabulary vehicles). The results are summarized in Table 6.4.

The track-level aggregation produces stable and accurate labels, achieving a 91.7% success rate on vehicles that exist in its knowledge base. An analysis of the two genuine errors reveals that one was a semantically plausible "sibling" confusion (a Volkswagen Touran mistaken for a Volkswagen Sharan), while the other was a clear mismatch (A renault megan msitaken for a ford focus) even though the two vehicles were close in build from the perspective of the camera.

The six out of vocabulart vehicles highlight the model's behavior when faced with unseen classes. One notable case involved a BMW sedan model not present in our dataset, for which the pipeline predicted "Skoda Octavia sedan." While the make and model are incorrect, the model successfully identified the correct body style (sedan). This demonstrates a degree of generalization; when faced with an unknown object, the model maps it to the most visually similar category it knows rather than making a random guess, downstream emissions also benefit from this as same vehicle categories are very in terms of emissions, and the final aggregation for the forecasting model is based on vehicle category.

Ultimately, these results confirm that our pipeline correctly identifies the right label for each track with good accuracy, and its failure modes are both understandable and often logically consistent.

6.2.2 Evaluating Camera Calibration

Before any scale, speed, or emissions estimation, we must verify the three *gatekeepers* of the camera model: the two orthogonal vanishing points, $\hat{\mathbf{u}}$ (along traffic flow) and $\hat{\mathbf{v}}$ (cross-road), and the focal length $\hat{f}$. If any of these are wrong, downstream quantities cannot be trusted. We therefore first evaluate the performance of our pipeline in estimating these three crucial parameters.

6.2.2.1 Protocol and Metrics

Our evaluation is performed on the BrnoCompSpeed dataset, which provides ground-truth calibration for all its videos. For each tested video, we run our automated pipeline to estimate $(\hat{\mathbf{u}}, \hat{\mathbf{v}}, \hat{f})$ and compare these estimates directly against the provided ground truth. To quantify the accuracy of our calibration, we use the following standard metrics:

- **VP Angular Error** (θ_u, θ_v): The angular difference in degrees between the estimated and ground-truth vanishing point vectors. This directly measures the accuracy of the estimated road directions. For a homogeneous VP $\hat{p} = [\hat{x}, \hat{y}, 1]^\top$ and ground-truth $p = [x, y, 1]^\top$, we define the error as:

$$\theta(\hat{p}, p) = \arccos\left(\frac{(\hat{p} - \mathbf{c}) \cdot (p - \mathbf{c})}{\|(\hat{p} - \mathbf{c})\| \|(p - \mathbf{c})\|}\right) \times \frac{180}{\pi}$$

- **Orthogonality Error** ($\varepsilon_\perp$): The deviation from a perfect 90° angle between the estimated $\hat{\mathbf{u}}$ and $\hat{\mathbf{v}}$. This measures the geometric consistency of the result, which is a key assumption for road scenes.

$$\varepsilon_\perp = |90^\circ - \angle(\hat{\mathbf{u}}, \hat{\mathbf{v}})|$$

- **Focal Length Error**: The relative error between the estimated focal length $\hat{f}$ and the ground truth, which is critical for accurate metric scale recovery.

$$\text{Relative Error} = \frac{|\hat{f} - f_{\text{gt}}|}{f_{\text{gt}}} \times 100\%$$

6.2.2.2 Results

Due to the large size of the BrnoCompSpeed dataset, a comprehensive evaluation across every video was not feasible. We instead selected a diverse set of eight videos from sessions 1, 4, 5, and 6 to test our pipeline’s robustness across various camera viewpoints and conditions.

Our automated method successfully converged on seven of these eight videos. To provide a clear and representative summary of the pipeline’s performance, Table 6.5 presents the results of these runs.

Table 6.5: Representative calibration results across multiple sessions..

Session	View	$\hat{\mathbf{u}}$ (θ_u)	$\hat{\mathbf{v}}$ (θ_v)	$\varepsilon_{\perp}$	$\hat{f}$	Rel. Error
1	Center	0.16°	1.86°	0.83°	2477.8	2.22%
1	Left	2.17°	1.19°	0.64°	2089.0	8.66%
4	Left	0.42°	0.66°	0.04°	4318.5	0.14%
5	Center	1.77°	2.70°	1.64°	1586.4	13.80%
5	Left	0.45°	8.98°	8.24°	1700.7	19.11%
6	Right	1.85°	1.08°	0.02°	3834.5	0.15%
4	right	2.2°	2.03°	0.33°	2783.78	0.82%
5	right	2.24°	25.32°	23.03°	852.6	45.38%

6.2.2.3 Discussion and Interpretation

The calibration results reveal a clear and consistent pattern across the different test videos. The estimation of the traffic-flow vanishing point, $\hat{\mathbf{u}}$, is highly robust, while the accuracy of the cross-road vanishing point, $\hat{\mathbf{v}}$, is heavily dependent on the scene’s geometry. Across all converged runs, the angular error for $\mathbf{u}$ remains consistently low (typically below 2.3°), a stability that can be attributed to the dense, temporally coherent motion cues provided by the vehicles themselves. These cues create a strong and unambiguous signal for the Diamond Space voting algorithm, allowing $\hat{\mathbf{u}}$ to be estimated reliably even in visually sparse environments.

In contrast, the performance for the cross-road vanishing point $\mathbf{v}$ varies significantly with the scene content. In structured environments with clear geometric features orthogonal to the road—such as curbs or fences—the pipeline successfully extracts clean line segments, leading to accurate $\hat{\mathbf{v}}$ estimates. This is evident in the results for Session 4/Left and Session 6/Right, where low angular errors in $\hat{\mathbf{v}}$ correspond naturally to accurate focal length estimations, with relative errors as low as 0.14%. However, in open settings like those in Session 5, which lack strong static edges, the pool of candidate lines for $\mathbf{v}$ becomes sparse and unreliable. This leads to a low-contrast or biased vote in Diamond Space, resulting in significant errors in both $\hat{\mathbf{v}}$ and the orthogonality constraint, as seen in the Session 5/Left results.

This direct coupling between the vanishing points and the focal length is a critical finding. Because f is recovered from the orthogonality constraint between $\mathbf{u}$ and $\mathbf{v}$, any errors in the estimation of $\mathbf{v}$ directly propagate to and inflate the focal length error. This relationship is clear in the results: cases with low θ_v and $\varepsilon_{\perp}$ yield low focal error, while the largest errors in $\mathbf{v}$ coincide with the largest focal length deviations.

These findings have direct implications for any practical deployment. The reliability of downstream speed and emission estimations hinges on the quality of this initial geometric calibration. Our results show that while $\hat{\mathbf{u}}$ can be trusted across most views, the accuracy of $\hat{\mathbf{v}}$ needs to be improved. Therefore, a practical application of this pipeline should include a quality screening step, where only calibrations meeting certain geometric thresholds are accepted for metric analysis.

6.2.3 Speed Evaluation

This section evaluates the ultimate output of our geometric pipeline: the per-vehicle speed estimations. The accuracy of these speeds serves as the final validation for all preceding stages, from vanishing point estimation to scene scaling.

Our evaluation is performed on three representative videos from the BrnoCompSpeed dataset: *Session 1 / Center*, *Session 4 / Right*, and *Session 5 / Center*. For each video, we use the previously computed camera calibration to run the full scale estimation then detection and classification, tracking, and projection pipeline. The resulting system tracks are then matched to the ground-truth vehicle data using the official BrnoCompSpeed evaluation procedure¹, which is based on temporal and geometric consistency. Our primary metrics for speed error, comparing our estimated speed $\hat{v}$ to the ground truth v_{gt} , are the **Mean Absolute Error (MAE)**, **median absolute error**, and the **95th percentile error**, reported in both km/h and as a relative percentage. We also report the tracker’s coverage via **recall** and the number of **false positives**, and we start our evaluation with reporting scaling errors.

6.2.3.1 Scale evaluation

To validate the accuracy of our scale estimation, we rely on BrnoCompSpeed’s evaluation of distances along the vehicle motion axis (u-axis), comparing predicted distances against ground-truth reference markers, we log the results in Table 6.6. Across all sessions, the distance errors remained consistently low, with the worst absolute error reaching only 0.30 m in session 5 (center view). Relative errors were also minimal, with median deviations between 1.02% and 1.43%, depending on the session. Session 4, with seven reference markers, achieved the most accurate results, highlighting the robustness of our pipeline when sufficient reference spans are available.

These results confirm that our calibration and scale estimation pipeline is both stable and reliable. The vanishing-point and homography components deliver geometrically sound projections, while the final metric scaling step ensures that object lengths are preserved accurately in the ground plane space. Even with just a few reference points, for instance session 1 has only 2 markers, the consistency of error metrics indicates good generalization and robustness.

Table 6.6: Distance error along the traffic direction (**u**).

Session	Mean [m]	Median [m]	P95 [m]	Worst [m]	Mean [%]	Median [%]	P95 [%]
S1 / Center	0.29	0.29	0.29	0.29	1.02	1.02	1.02
S4 / Right	0.07	0.09	0.15	0.17	0.99	1.23	2.13
S5 / Center	0.12	0.10	0.26	0.30	1.62	1.43	3.79

6.2.3.2 Tracking evaluation

We record all of the results for the three sessions in Table 6.7. We can observe that tracking performance is strongest on session 1 center view (recall = 0.974), indicating that the detector-tracker stack recovers nearly all ground-truth trajectories. The higher false-positive count (59) is largely explained by evaluation window mismatch (we can notice that the ground truth for all sessions in the brnocompspeed dataset starts a little later than the actual video) and occasional short spurious tracks at scene borders; both can be mitigated with an initial warm-up ignore window and a minimum track-length filter.

Session 5 center view also achieves high coverage (recall = 0.866), though with more false positives (72). This can be explained by the high vehicle count which suggests more traffic

¹<https://github.com/JakubSochor/BrnoCompSpeed>

density making these scene more susceptible to ID switched, but overall it retrieves a high majority of the tracks.

However, session 4 right viewpoint exhibited the lowest recall (0.631), perhaps due to the different perspective, as this session is captured from a right angle which can introduce more occlusions leading to fragmented or missed tracks. This goes to show the importance of perspective in such tasks, and that there is room for improvement, perhaps a better strategy in these cases would be to incorporate vehicle re-identification techniques, for example embeddings that would help distinguish physical appearances of the vehicles, or just a few rules that would help stitch a vehicle’s separated tracks together.

Table 6.7: Tracking coverage per session.

Session	matched	Recall	False Positives
S1 / Center	826	0.974057	59
S4 / Right	743	0.631266	39
S5 / Center	1755	0.865812	72

6.2.3.3 Speed Evaluation

From the results in Table 6.8, we observe that Session 1 Center view yields the most accurate speed estimates. This is largely due to the clarity of the viewpoint: the camera is well-centered, vehicles are fully visible, and the traffic flow is relatively smooth with minimal lane changes. These conditions allow for stable tracking and more precise distance measurements, which in turn lead to more accurate speed estimation. The small number of tracks in this session may also slightly reduce the average error.

Session 4 Right view follows closely in performance. It shares many of the positive traits of Session 1, such as minimal occlusions and consistent motion patterns. However, its side-facing angle introduces a bit more perspective, and even minor occlusions can have a larger impact due to the tighter field of view. These factors contribute to a slight increase in error, but the performance remains strong overall.

On the other hand, Session 5 Center view exhibits the highest speed estimation errors. The larger number of tracks in this session suggests denser traffic and a higher frequency of lane changes and overlaps. These dynamic interactions complicate both detection and tracking, leading to noisier speed calculations. Despite this, the error margins remain within acceptable bounds for downstream tasks like emission estimation, where small inaccuracies in speed do not significantly affect the overall prediction accuracy.

Table 6.8: Speed error statistics.

Session	Mean [km/h]	Median [km/h]	P95 [km/h]	Worst [km/h]	Mean [%]	Median [%]
S1 / Center	1.04	0.70	2.58	24.03	1.36	0.96
S4 / Right	2.60	2.46	3.86	32.19	2.62	2.54
S5 / Center	4.00	3.95	5.46	17.38	5.15	5.14

Overall, the results are satisfactory and sufficient for downstream use, however, this evaluation allowed us to recognize the strengths of our pipeline, and its weaknesses on which we can improve in future work. We used the calibrated geometry, tracking, and speed estimates to compute per-vehicle and aggregate emissions with the `pyEmission` library. Unfortunately, no

annotated ground-truth emissions are available for a quantitative end-to-end evaluation. Nevertheless, by validating each upstream feature—camera calibration and scale, trajectory geometry, vehicle class, and especially speed, we minimize error propagation and increase confidence that the derived emissions are consistent. In short, the pipeline produces reliable inputs for emission estimation, and the outputs are suitable for analysis and planning; establishing an emissions ground truth remains an important direction for future work. Thus from here on we can start evaluating the next module for forecasting and imputation.

6.3 Evaluation of STAD-NET Model

In this section, we present a comprehensive evaluation of the STAD-NET model from multiple perspectives. Our analysis is designed to assess both the forecasting and imputation capabilities of the model, while also examining its robustness under varying levels and patterns of missing data. A particular focus is placed on analyzing how spatially structured missing patterns affect the model’s ability to perform multi-hop aggregation—an essential mechanism for capturing long-range dependencies within the traffic network.

In addition, we investigate the prediction errors across different forecasting horizons, providing a detailed view of the model’s temporal accuracy. We also report evaluation metrics for both the imputation and forecasting tasks. Finally, we conduct a comparative study of federated learning strategies, contrasting the standard FedAvg approach with the proposed CLSFL method, and analyzing their respective impacts on the quality of the final client models.

6.3.1 Impact of Missing Patterns and Hop Aggregation

In graph-based imputation, the ability to reconstruct missing node values strongly depends on the information available from neighboring nodes. When some neighbors are also missing, the local structural context becomes less informative, which negatively impacts the imputation capacity of the model. To mitigate this issue, multi-hop aggregation is employed: by extending the receptive field beyond immediate neighbors, the model can capture information from a broader set of nodes, thus reducing the effect of localized missing patterns.

To analyze the effect of hop aggregation on imputation performance, we simulate missing data by masking contiguous nodes in the graph. We then evaluate the resulting imputation error under different hop aggregation settings. This approach allows us to investigate how the choice of hop size influences the model’s robustness to structured missing patterns and its ability to accurately reconstruct node representations.

For visualization see figure 6.2, we examine the imputation loss using normalized values for the following nodes:

- **Nodes 117, 118, 119:** All three nodes are included in the mask, with each node having missing neighbors only at the first hop.
- **Node 139:** Missing neighbors were introduced at both the first and second hops, allowing us to study the effect of multi-hop aggregation.

From the analysis of the aggregation example, we observe that using only one-hop aggregation results in relatively high imputation errors across all nodes, likely due to insufficient information from neighboring nodes. Introducing two-hop aggregation improves the imputation performance for all nodes. When three-hop aggregation is applied, node 139 benefits further, reflecting its need for information from more distant neighbors. Node 117 also shows some improvement. While nodes 118 and 119 experience a slight degradation due to noise from aggregating distant neighbors with potentially different distributions. However, the overall imputation error across all nodes is reduced, demonstrating the net benefit of including higher-order hops.

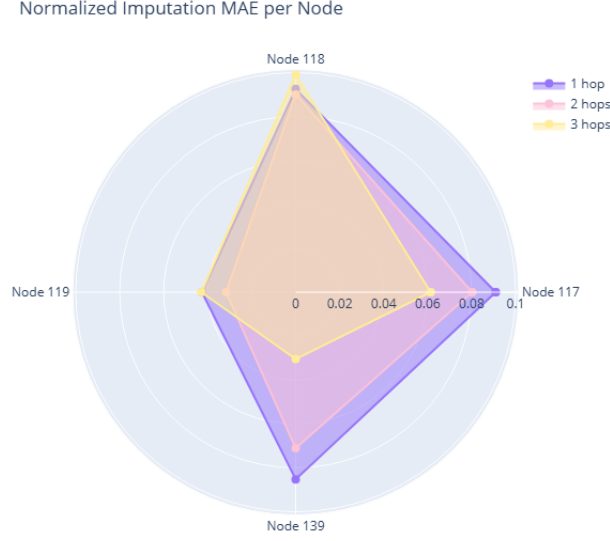


Figure 6.2: Normalized Imputation MAE per Node for Different Hop Aggregations.

6.3.2 Forecasting Performance Across Different Horizons

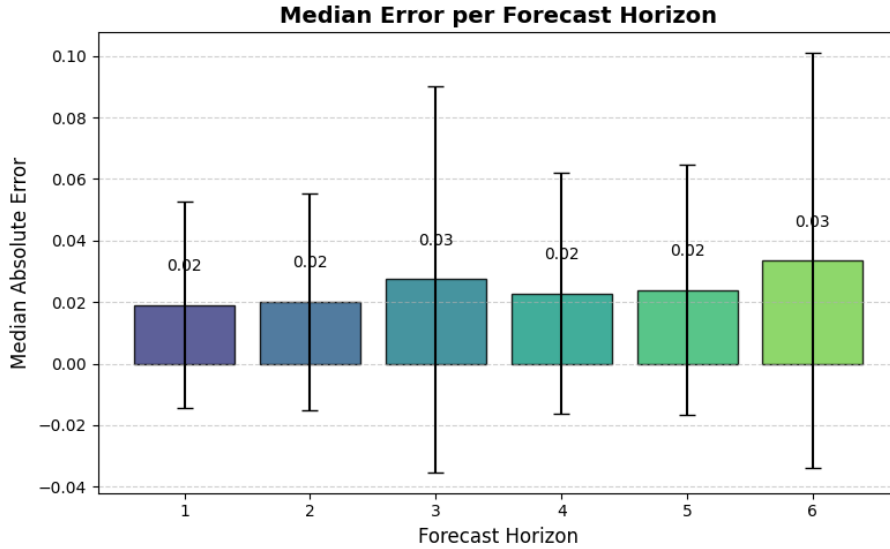


Figure 6.3: Box Plot for Multi-step Ahead Forecasting Error Across Prediction Horizons.

To assess the forecasting capability of our model, we report the RMSE values across different prediction horizons, as shown in Figure 6.3. The results indicate that the model provides accurate short-term forecasts, reflecting its strong ability to capture immediate temporal dependencies. As the prediction horizon extends, the error generally increases. An interesting observation is the localized increase in error at the third horizon (corresponding to the third quarter-hour), which can be attributed to abrupt changes in traffic patterns occurring at this time scale. Unlike autoregressive models, where errors accumulate recursively, our CNN-based architecture predicts all horizons directly; thus, the performance variations across horizons are primarily due to temporal non-stationarities in the data rather than error propagation. Overall,

these findings confirm the robustness of the model for short-term forecasting while highlighting the challenges associated with capturing regime shifts at specific horizons.

6.3.3 Evaluation Metrics for Imputation and Forecasting

In order to rigorously assess the performance of the proposed model on both the imputation and forecasting tasks, it is essential to rely on well defined quantitative evaluation metrics. These metrics provide objective criteria for measuring the accuracy and robustness of model predictions when compared to the ground truth. While imputation and forecasting are distinct tasks, they share the common goal of reconstructing missing or future values in a time series, and therefore many evaluation measures can be applied to both. In what follows, we present the most relevant metrics employed in the literature.

- **SMAPE (Symmetric Mean Absolute Percentage Error):** Defined as:

$$\text{SMAPE} = \frac{100}{n} \sum_{t=1}^n \frac{|y_t - \hat{y}_t|}{(|y_t| + |\hat{y}_t|)/2}$$

It expresses the prediction error as a percentage, normalized by the average magnitude of actual and predicted values. Unlike MAPE, SMAPE remains stable even when the true values are close to zero.

Table 6.9: Metrics for Forecasting and Imputing CO2 Emissions

Feature	RMSE	MAE	SMAPE (%)	R ²
Forecasting Co2 Emissions (kg)	0,5201	0.3629	31.43	0.9439
Forecasting Normalized Emissions	0.02622	0.01669	31.43	0.9439
Imputing Normalized Emissions	0.11689	0.08259	68.66	0.4327

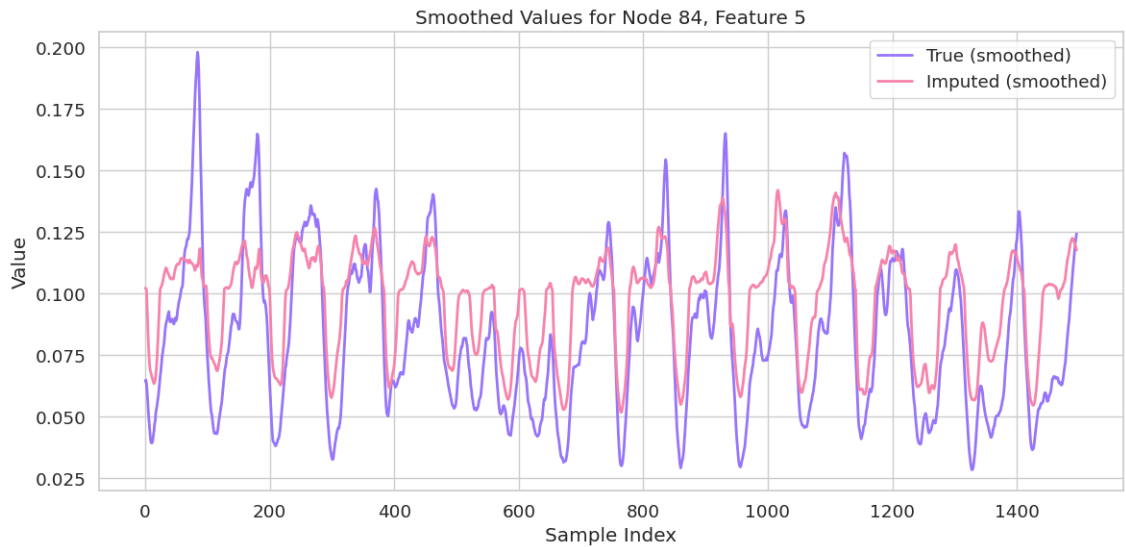


Figure 6.4: True vs imputation values for node 84.

The forecasting metrics for CO₂ emissions presented in Table 6.9 assess the accuracy of the model in predicting future emission values. The Root Mean Squared Error (RMSE) of 0.5201 kg and the Mean Absolute Error (MAE) of 0.3629 kg (denormalized values) indicate that

the model predictions are on average very close to the true emissions. In the normalized scale, the corresponding RMSE and MAE values are 0.02622 and 0.01669, respectively, confirming that the model performs well relative to the feature scale. The Symmetric Mean Absolute Percentage Error (SMAPE) of 31.43% shows that the relative deviation between predicted and actual emissions is moderate, while the R^2 score of 0.9439 indicates that the model captures approximately 94% of the variance in the CO₂ emissions. Overall, these metrics demonstrate that the model provides reliable and accurate forecasts of CO₂ emissions across the road network. On the other hand, we can observe that the imputation loss is around 0.10609, measured using MSE calculated only on the fully missing nodes. This value is higher compared to the forecasting task, which is expected since no direct information is available for these nodes, and their values are imputed solely based on neighboring nodes. In general, the model captures the variations of the specific node emissions, but the imputed value can sometimes be higher or less than the true emission of the node, especially when the node's emission quantity differs significantly from those of its neighbors, where other nodes are captured well see figure 6.4

We showcase the forecasting ability per node in Figure 6.5. The results highlight that RMSE values remain relatively low for the observed nodes, indicating that the model effectively captures their temporal dynamics when sensor data is available. In contrast, the missing nodes exhibit noticeably higher errors. This discrepancy arises because the forecasting process relies on previously imputed values by reinjecting them to the forecasting part, which inevitably introduce additional uncertainty. Consequently, the error propagation from imputation affects the subsequent forecasting step.

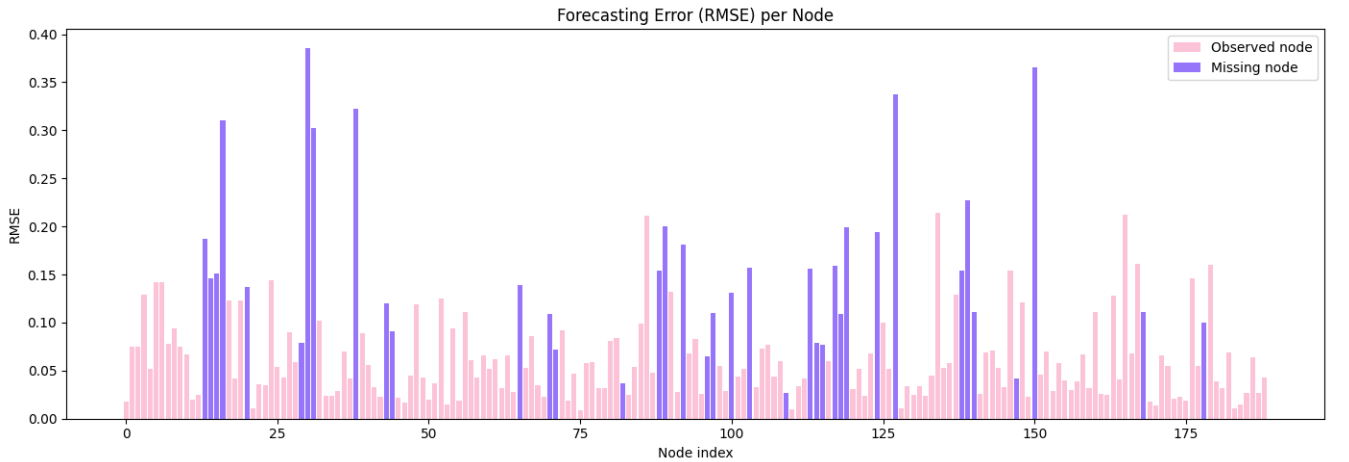


Figure 6.5: Bar Plot for Per Node RMSE Values for Forecasting Errors.

6.3.4 Study and Comparison of the Federated Learning approach

In order to demonstrate the effectiveness of the clustered FL method compared to standard FedAvg, we conducted experiments using the same model under two settings: (i) simple FedAvg, where all client parameters are averaged together, and (ii) clustered FL, where client gradients are projected and grouped into clusters before aggregation. For our study, we defined 3 clusters. Using fewer clusters would reduce the impact of clustering, while using too many clusters would result in very small groups, approaching the case of no aggregation at all.

Figure 6.10 presents the heatmap of the evaluation MSE for all clients across different rounds. Subfigures (a) and (b) correspond to forecasting and imputation losses under FedAvg, whereas subfigures (c) and (d) show the corresponding results for K-Clustered FL. From the loss matrices, it is evident that clustering mitigates the effect of client heterogeneity, enabling clients

to obtain better-performing models by the final training rounds. Although dynamic clustering introduces some instability in the intermediate rounds—since clients may move between clusters or occasionally end up isolated—the training process gradually stabilizes. Once the clustering structure becomes consistent, the aggregated models converge more effectively, ultimately surpassing the performance of standard FedAvg. A closer inspection of client-level errors confirms this trend. Under FedAvg, some clients—particularly Clients 0, 3, 5, and 6—consistently exhibited the highest forecasting losses, while Clients 0, 1, 3, and 6 recorded the highest imputation losses. These discrepancies reflect the uneven impact of data heterogeneity in the global averaging process. In contrast, clustered FL reduces this imbalance by grouping similar clients together, ensuring that even those with initially higher error rates benefit from more tailored aggregation.

Regarding cluster stabilization, we present Figure 6.11, which illustrates the t-SNE projection of client gradients within their respective clusters across training rounds, where the initially dispersed gradients progressively converge to form well-defined clusters.

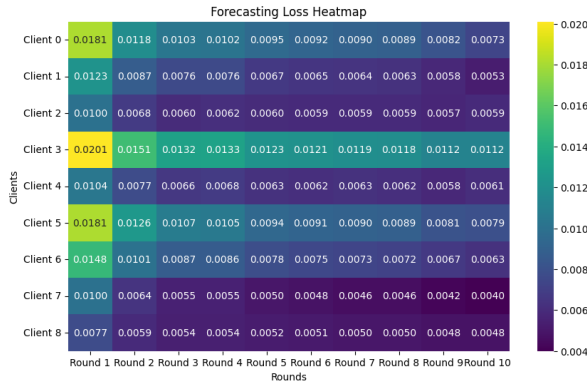


Figure 6.6: FedAvg Forecasting Loss

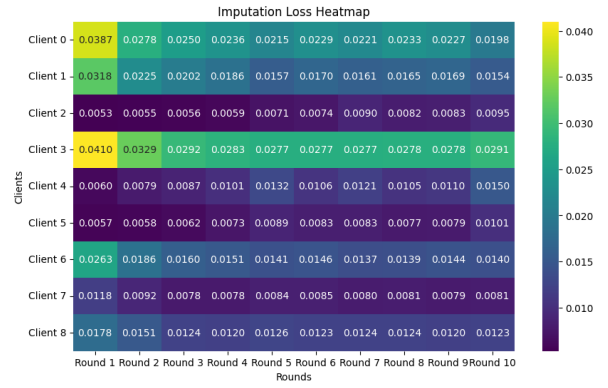


Figure 6.7: FedAvg Imputation Loss

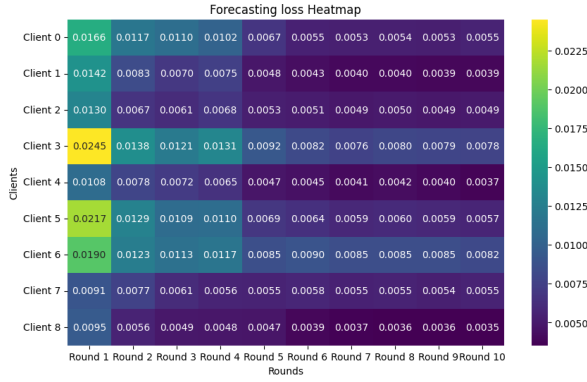


Figure 6.8: Clustered FL Forecasting Loss

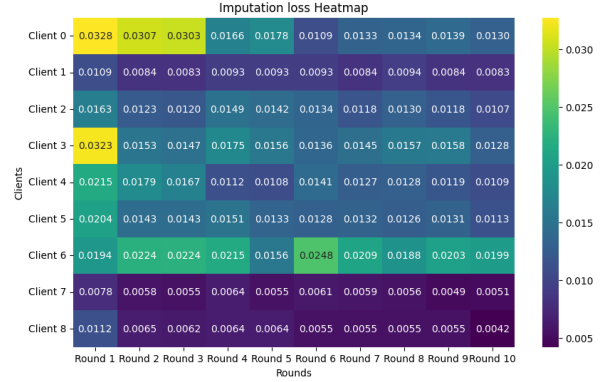


Figure 6.9: Clustered FL Imputation Loss

Figure 6.10: Heatmaps of client evaluation MSE across training rounds. (a)–(b) correspond to FedAvg, while (c)–(d) show the results for Clustered FL.

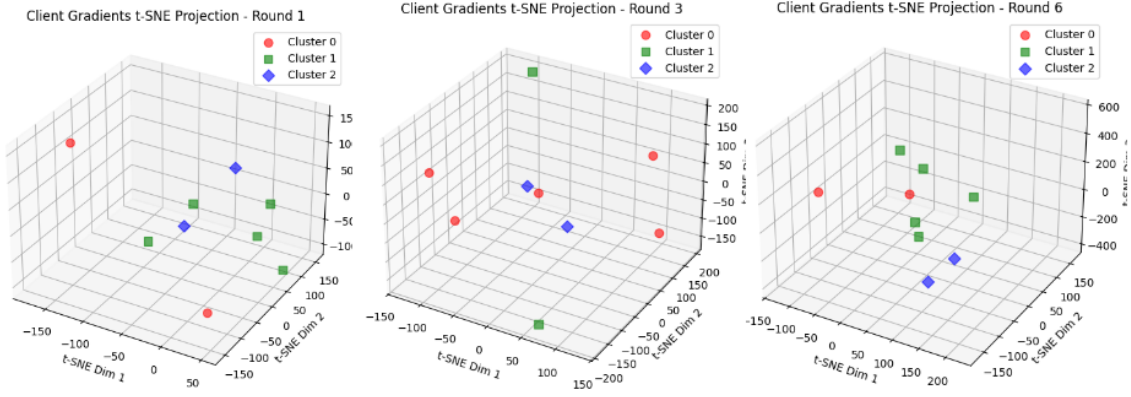


Figure 6.11: Client Gradient Projection Evolution Across Federation Rounds.

6.4 Conclusion

In this chapter, we tested the different components of our proposed framework, varying from video processing pipeline to imputation, prediction, and federated learning. The evaluation of the video pipeline confirmed the reliability of our calibration, fine-grained classification, and tracking modules so that vehicle classification and speed estimates can be used safely in downstream emission estimation. Similarly, the scale and tracking analysis confirmed that our pipeline is working strongly under different traffic scenarios. Proceeding to the forecasting experiment, our experiments demonstrate the significance of capturing both temporal and spatial interdependencies in the face of missing data. The performance results indicated that the suggested spatio-temporal model was able to efficiently balance imputation and forecasting accuracy as the performance increases with the use of multi-hop neighborhood information. Finally, the federated learning study highlighted the benefits of clustering methods. Compared to baseline FedAvg, cluster-based FL reduced client heterogeneity issues more efficiently and performing better models in each training round. In the upcoming chapter, we discuss the remaining shortcomings of our approach and propose possible strategies to address them before drawing our final conclusions.

General Conclusion

At the end of this study, we emphasize that our work has tightly interwoven the different stages of conception, implementation, and evaluation within the domain of traffic carbon footprint estimation. Starting from visual feature extraction, through emission estimation, and extending to forecasting tasks, the proposed pipeline represents a complete and coherent framework that bridges computer vision, spatio-temporal modeling, and federated learning.

Nevertheless, several shortcomings remain and highlight potential avenues for improvement. First, the imputation module still struggles under highly irregular missing patterns, suggesting that more robust models or physics-informed regularization could further enhance reconstruction accuracy. Second, the video processing pipeline remains highly dependent on camera quality and environmental conditions (e.g., lighting, weather, and occlusions), which may affect the reliability of detection, calibration, and speed estimation. Third, the lack of large-scale, diverse, and publicly available datasets in this domain still limits the generalization and benchmarking of emission estimation methods.

Looking toward future directions, several opportunities can be identified. For instance, the integration of license plate detection would enable unique vehicle identification and re-identification across different cameras, providing stronger consistency checks and the ability to detect malfunctioning or high-emission vehicles. Such an enhancement would also support trajectory reconstruction, thereby reinforcing the imputation task through physics-based regularization and enabling more reliable forecasting. Moreover, as more datasets in this field become publicly available, richer training and evaluation scenarios will emerge, allowing the proposed framework to evolve and scale effectively.

In conclusion, this work demonstrates the feasibility and potential of a holistic traffic carbon footprint estimation framework. By combining video-based vehicle analysis, spatio-temporal forecasting, and federated learning, we have laid the groundwork for scalable, privacy-preserving, and data-driven solutions to monitor traffic-related emissions. Future research building upon these directions will not only address current limitations but also extend the scope and impact of this contribution toward sustainable urban mobility.

Bibliography

- Abdel-Aziz, Y. I. and Karara, H. M. (1971), Direct linear transformation from comparator coordinates into object space coordinates in close-range photogrammetry, *in* ‘Proceedings of the Symposium on Close-Range Photogrammetry’, American Society of Photogrammetry, Urbana, IL, USA.
- Al-Nefaie, A. H. and Aldhyani, T. H. H. (2023), ‘Predicting CO₂ emissions from traffic vehicles for sustainable and smart environment using a deep learning model’, *Sustainability* **15**(9), 7615.
- Alfaseeh, L., Tu, R., Farooq, B. and Hatzopoulou, M. (2020), ‘Greenhouse gas emission prediction on road network using deep sequence learning’, *Transportation Research Part D: Transport and Environment* **88**, 102593.
- Amidi, S. and Amidi, A. (2018), ‘Cs 230 - recurrent neural networks cheat sheet’.
URL: <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks>
- Bai, S., Kolter, J. Z. and Koltun, V. (2018), ‘An empirical evaluation of generic convolutional and recurrent networks for sequence modeling’.
URL: <https://arxiv.org/abs/1803.01271>
- Bewley, A., Ge, Z., Ott, L., Ramos, F. and Upcroft, B. (2016), Simple online and real-time tracking, *in* ‘2016 IEEE International Conference on Image Processing (ICIP)’, IEEE, pp. 3464–3468.
- Bilotta, S. and Nesi, P. (2022), ‘Estimating CO₂ emissions from IoT traffic flow sensors and reconstruction’, *Sensors* **22**(9), 3382.
- Bogdan, O., Eckstein, V., Rameau, F. and Bazin, J.-C. (2018), Deepcalib: A deep learning approach for automatic intrinsic calibration of wide field-of-view cameras, *in* ‘Proceedings of the 15th ACM SIGGRAPH European Conference on Visual Media Production (CVMP ’18)’, Association for Computing Machinery.
- Bondy, J. and Murty, U. (2021), *Graph Theory*, Springer.
- Braun, C., Da Costa, J. B. D., Villas, L. A. and de Souza, A. M. (2024), Ecopredict: Assessing distributed machine learning methods for predicting urban emissions, *in* ‘2024 IEEE 100th Vehicular Technology Conference (VTC2024-Fall)’, pp. 1–5.
- Breiman, L. (1996), ‘Bagging predictors’, *Machine Learning* **24**(2), 123–140.
- Canny, J. (1986), ‘A computational approach to edge detection’, *IEEE Transactions on Pattern Analysis and Machine Intelligence* **PAMI-8**(6), 679–698.
- Cao, W. et al. (2018), ‘Brits: Bidirectional recurrent imputation for time series’, *Advances in neural information processing systems* **31**.

- Caprile, B. and Torre, V. (1990), ‘Using vanishing points for camera calibration’, *International Journal of Computer Vision* **4**(2), 127–140.
- Chami, I., Abu-El-Haija, S., Perozzi, B., Ré, C. and Murphy, K. (2020), ‘Machine learning on graphs: A model and comprehensive taxonomy’, *arXiv preprint arXiv:2005.03675*. Accessed: 2025-07-20.
URL: <https://arxiv.org/abs/2005.03675>
- Chen, W., Zhang, W., Yu, B., Zhang, H. and Yang, C. (2022), ‘Multi-channel spatiotemporal graph convolutional network with moves emission model for urban traffic emissions prediction’, *Atmospheric Environment* **267**, 118766.
- Chen, Y. and C. (2022), ‘Embedding-graph-neural-network for transient nox emissions prediction’, *Energies* **16**(1), 3.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H. and Bengio, Y. (2014), ‘Learning phrase representations using rnn encoder–decoder for statistical machine translation’.
URL: <https://arxiv.org/abs/1406.1078>
- Cınarler, G., Yeşilyurt, M. K., Ağbulut, Ü., Yılbaşı, Z. and Kılıç, K. (2024), ‘Application of various machine learning algorithms in view of predicting the CO₂ emissions in the transportation sector’, *Science and Technology for Energy Transition* **79**, 15.
- DATA INTELLIGENCE (2023), ‘Leveraging gnn for non-iid and non-euclidean data’.
URL: <https://data-intelligence.hashnode.dev/leveraging-gnn-cora-non-iid-non-euclidean>
- Defferrard, M., Bresson, X. and Vandergheynst, P. (2016), Convolutional neural networks on graphs with fast localized spectral filtering, in ‘Advances in Neural Information Processing Systems (NeurIPS)’, Vol. 29, pp. 3844–3852.
- Diestel, R. (2017), *Graph Theory*, 5 edn, Springer.
- Dockery, D. W., Pope, C. A., Xu, X., Spengler, J. D., Ware, J. H., Fay, M. E., Ferris, B. G., J. and Speizer, F. E. (1993), ‘An association between air pollution and mortality in six u.s. cities’, *New England Journal of Medicine* **329**(24), 1753–1759.
- Dubská, M. and Herout, A. (2013), Real projective plane mapping for detection of orthogonal vanishing points, in ‘Proceedings of the British Machine Vision Conference (BMVC)’, BMVA Press, Bristol, UK, pp. 1–10.
- Dubská, M., Herout, A., Juránek, R. and Sochor, J. (2015), ‘Fully automatic roadside camera calibration for traffic surveillance’, *IEEE Transactions on Intelligent Transportation Systems* **16**(3), 1162–1171.
- Dubská, M., Herout, A. and Sochor, J. (2014), Automatic camera calibration for traffic understanding, in ‘Proceedings of the British Machine Vision Conference (BMVC)’, BMVA Press.
- European Commission, Joint Research Centre (2024), ‘Edgar global greenhouse gas emissions database 2024 report’.
URL: <https://edgar.jrc.ec.europa.eu/report2024>
- Farnell (2025), ‘Transforming Mobility: The Architecture and Solutions of V2X Communication in Electric Vehicles’.
URL: <https://si.farnell.com/transforming-mobility-the-architecture-and-solutions-of-v2x-communication-in-electric-vehicles-trc-ar>

- Fei, X., Long, F., Li, F. and Ling, Q. (2021), ‘Multi-component fusion temporal networks to predict vehicle exhaust based on remote monitoring data’, *IEEE Access* **9**, 42358–42369.
- Fei, X., Zhang, Q. and Ling, Q. (2019), ‘Vehicle exhaust concentration estimation based on an improved stacking model’, *IEEE Access* **7**, 179454–179463.
- Friedman, J. H. (2001), ‘Greedy function approximation: A gradient boosting machine’, *Annals of Statistics* **29**(5), 1189–1232.
- GeeksforGeeks (2024), ‘Convolutional Neural Network (CNN) in Machine Learning’.
URL: <https://www.geeksforgeeks.org/deep-learning/convolutional-neural-network-cnn-in-machine-learning/>
- Ghosh, R. and Lerman, K. (2009), ‘Structure of heterogeneous networks’, *arXiv preprint arXiv:0906.2212*. Introduced mathematical framework for multi-type node/link networks.
- Girshick, R. (2015), Fast R-CNN, in ‘Proceedings of the IEEE International Conference on Computer Vision (ICCV)’, pp. 1440–1448.
- Girshick, R., Donahue, J., Darrell, T. and Malik, J. (2014), Rich feature hierarchies for accurate object detection and semantic segmentation, in ‘Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)’, pp. 580–587.
- Government of Canada (2019), ‘Open Government Vehicle and Fuel-Consumption Dataset’.
URL: <https://open.canada.ca/data/en/dataset/98f1a129-f628-4ce4-b24d-6f16bf24dd64>
- Gurcan, F. (2024), ‘Forecasting co₂ emissions of fuel vehicles for an ecological world using ensemble learning, machine-learning and deep-learning models’, *PeerJ Computer Science* **10**, e2234.
- Hamilton, W. L., Ying, R. and Leskovec, J. (2017), Inductive representation learning on large graphs, in ‘Advances in Neural Information Processing Systems (NeurIPS)’, Vol. 30, pp. 1024–1034.
URL: <https://arxiv.org/abs/1706.02216>
- He, K., Zhang, X., Ren, S. and Sun, J. (2015), ‘Deep residual learning for image recognition’, *arXiv preprint arXiv:1512.03385*.
URL: <https://arxiv.org/abs/1512.03385>
- Hemaspaandra, J., Grenig, T. and Tall, P. (2015), Hierarchical Line Graph Neural Networks and Graph Theory, PhD thesis, Università degli Studi di Padova.
URL: <https://thesis.unipd.it/retrieve/1d5fbb1e-a284-4b8d-92da-6ca6115e00e8/thesis.pdf>
- Hien, N. L. H. and Kor, A. L. (2022), ‘Analysis and prediction model of fuel consumption and carbon dioxide emissions of light-duty vehicles’, *Applied Sciences* **12**(2), 803.
- Hochreiter, S. and Schmidhuber, J. (1997), ‘Long short-term memory’, *Neural Computation* **9**(8), 1735–1780.
- Huang, G., Liu, Z., Van Der Maaten, L. and Weinberger, K. Q. (2016), ‘Densely connected convolutional networks’, *arXiv preprint arXiv:1608.06993*.
URL: <https://arxiv.org/abs/1608.06993>
- Hyndman, R. J. and Athanasopoulos, G. (2018), *Forecasting: Principles and Practice*, 2nd edn, OTexts, Melbourne, Australia.
URL: <https://otexts.com/fpp2/>

- Intergovernmental Panel on Climate Change (IPCC) (2021), ‘Climate change 2021: The physical science basis. contribution of working group i to the sixth assessment report of the intergovernmental panel on climate change’.
URL: <https://www.ipcc.ch/report/ar6/wg1/>
- Intergovernmental Panel on Climate Change (IPCC) (2021-08-09), ‘Climate change widespread, rapid, and intensifying — ipcc’.
URL: <https://www.ipcc.ch/2021/08/09/ar6-wg1-20210809-pr/>
- Intergovernmental Panel on Climate Change (IPCC) (2023), ‘Climate change 2023: Synthesis report. contribution of working groups i, ii and iii to the sixth assessment report of the intergovernmental panel on climate change’.
URL: <https://www.ipcc.ch/report/ar6/syr/>
- International Agency for Research on Cancer (2013), ‘Outdoor air pollution a leading environmental cause of cancer deaths’.
URL: <https://www.iarc.who.int/wp-content/uploads/2018/07/pr221E.pdf>
- Kipf, T. N. and Welling, M. (2017), Semi-supervised classification with graph convolutional networks, *in* ‘5th International Conference on Learning Representations (ICLR)’.
URL: <https://arxiv.org/abs/1609.02907>
- Kocur, V. (2019), Perspective transformation for accurate detection of 3D bounding boxes of vehicles in traffic surveillance, *in* ‘Proceedings of the 24th Computer Vision Winter Workshop (CVWW)’, Graz University of Technology, Stift Vorau, Austria, pp. 33–41.
- Kocur, V. and Ftáčnik, M. (2020), ‘Detection of 3d bounding boxes of vehicles using perspective transformation for accurate speed measurement’, *Machine Vision and Applications* **31**(7–8), 1–15.
- Li, C., Li, L., Jiang, H., Weng, K., Geng, Y., Li, L., Ke, Z., Li, Q., Cheng, M., Nie, W., Li, Y., Zhang, B., Liang, Y., Zhou, L., Xu, X., Chu, X., Wei, X. and Wei, X. (2022), ‘Yolov6: A single-stage object detection framework for industrial applications’.
- Li, J., Jiang, C., Han, K. and et al. (2024), ‘High-resolution spatiotemporal inference of urban road traffic emissions using taxi GPS and multi-source urban features data: a case study in Chengdu, China’, *Urban Informatics* **3**(1), 17.
- Li, J., Yu, Y., Wang, Y., Zhao, L. and He, C. (2021), ‘Prediction of transient NO_x emission from diesel vehicles based on deep-learning differentiation model with double noise reduction’, *Atmosphere* **12**(12), 1702.
- Li, Y., Yu, R., Shahabi, C. and Liu, Y. (2018), Diffusion convolutional recurrent neural network: Data-driven traffic forecasting, *in* ‘International Conference on Learning Representations (ICLR)’.
URL: <https://arxiv.org/abs/1707.01926>
- Li, Z., Zhang, X. and Dong, Z. (2023), ‘Tsf-transformer: A time series forecasting model for exhaust gas emission using transformer’, *Applied Intelligence* **53**, 17211–17225.
- Liu, Q., Zhang, L., Lv, C., Gao, H., Cai, Y. and Chen, L. (2024), ‘New perspective of environmental impact research: predicting bus exhaust emissions using the etsformer based on collaborative perception’, *Sustainable Horizons* **11**, 100105.

- Liu, S., Johns, E. and Davison, A. J. (2019), End-to-end multi-task learning with attention, in ‘Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)’, pp. 1871–1880.
- Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C. and Berg, A. C. (2016), SSD: Single shot multibox detector, in B. Leibe, J. Matas, N. Sebe and M. Welling, eds, ‘Computer Vision – ECCV 2016’, Vol. 9905 of *Lecture Notes in Computer Science*, Springer, pp. 21–37.
- Liu, Y., Tang, C., Zhou, A. and et al. (2024), ‘A novel ensemble approach for road traffic carbon emission prediction: a case in Canada’, *Environmental Development and Sustainability* **27**, 15977–16013.
- Lucas, B. D. and Kanade, T. (1981), An iterative image registration technique with an application to stereo vision, in ‘Proceedings of the 7th International Joint Conference on Artificial Intelligence (IJCAI-81)’, pp. 674–679.
- Macko, A., Gajdošech, L. and Kocur, V. (2025), ‘Efficient vision-based vehicle speed estimation’, *Journal of Real-Time Image Processing* **22**(3).
- Madziel, M., Jaworski, A., Kuszewski, H., Woś, P., Campisi, T. and Lew, K. (2021), ‘The development of co2 instantaneous emission model of full hybrid vehicle with the use of machine learning techniques’, *Energies* **15**(1), 142.
- MartínBaos, J. A., RodríguezBenitez, L., GarcíaRódenas, R. and Liu, J. (2021), ‘Iot based monitoring of air quality and traffic using regression analysis’, *Applied Soft Computing* **115**, 108282.
- Martinez, V. (n.d.), ‘Using decomposition to improve time-series prediction’.
URL: <https://quantdare.com/decomposition-to-improve-time-series-prediction/>
- Mian, A. (n.d.), ‘Cits 4402 computer vision: Lecture 09 – projective geometry’. Lecture material, Accessed : 2025-08-30.
URL: <https://teaching.csse.uwa.edu.au/units/CITS4402/lectures/Lecture09-ProjectiveGeometry.pdf>
- Natarajan, Y., Wadhwa, G., Sri Preethaa, K. R. and Paul, A. (2023), ‘Forecasting carbon dioxide emissions of light-duty vehicles with different machine learning algorithms’, *Electronics* **12**(10), 2288.
- Nguyen, H. A., Le, T. H., Azzi, M. and Ha, Q. P. (2024), ‘Monitoring and estimation of urban emissions with low-cost sensor networks and deep learning’, *Ecological Informatics* **82**, 102750.
- openMVG (n.d.), ‘cameras — openmvg library’. Accessed: 2025-08-30.
URL: <https://openmvg.readthedocs.io/en/latest/openMVG/cameras/cameras/>
- Park, C., Jeong, S., Kim, C., Shin, J. and Joo, J. (2023), ‘Machine learning based estimation of urban on-road CO2 concentration in Seoul’, *Environmental Research* **231**, 116256.
- Pham, C. (2020), ‘Graph Convolutional Networks (GCN)’.
URL: <https://www.topbots.com/graph-convolutional-networks/>
- Pradeep, V., Khemmar, R. and Jendoubi, F. (2024), ‘Visual eco-routing (ver): Xgboost based eco-route selection from road scenes and vehicle emissions’, *IEEE Access*.
- Redmon, J., Divvala, S., Girshick, R. and Farhadi, A. (2016), You only look once: Unified, real-time object detection, in ‘Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)’, pp. 779–788.

- Ren, S., He, K., Girshick, R. and Sun, J. (2015), Faster R-CNN: Towards real-time object detection with region proposal networks, *in* ‘Advances in Neural Information Processing Systems (NeurIPS)’, Vol. 28.
- Rodriguez Valido, M., Gomez-Cardenes, O. and Magdaleno, E. (2022), ‘Monitoring vehicle pollution and fuel consumption based on AI camera system and gas emission estimator model’, *Sensors* **23**(1), 312.
- Rollend, D. and Foster (2023), ‘Machine learning for activity-based road transportation emissions estimation’, *Environmental Data Science* **2**, e38.
- Rollend, Kevin, K. R. M. (2023), ‘Machine learning for activity-based road transportation emissions estimation’, *Environmental Data Science* p. e38.
- Ruder, S. (2017), ‘An overview of multi-task learning in deep neural networks’, *arXiv preprint arXiv:1706.05098* .
- Saez-Perez, J., Benlloch-Caballero, P., Tena-Gago, D., Garcia-Rodriguez, J., Calero, J. M. A. and Wang, Q. (2024a), ‘Optimizing ai transformer models for co2 emission prediction in self-driving vehicles with mobile/multi-access edge computing support’, *IEEE Access* . [Accessed:2024-12].
- Saez-Perez, J., Benlloch-Caballero, P., Tena-Gago, D., Garcia-Rodriguez, J., Calero, J. M. A. and Wang, Q. (2024b), ‘Optimizing ai transformer models for co₂ emission prediction in self-driving vehicles with mobile/multi-access edge computing support’, *IEEE Access* .
- Sahoo, D., Jiang, Y., Song, Y., Zhang, K., Guan, Y. and Li, Z. (2023), ‘Embedding-graph-neural-network for transient nox emissions prediction’, *IEEE Transactions on Intelligent Transportation Systems* .
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.-C. (2018), ‘Mobilenetv2: Inverted residuals and linear bottlenecks’, *arXiv preprint arXiv:1801.04381* .
URL: <https://arxiv.org/abs/1801.04381>
- Santos, O., Ribeiro, F., Metrôlho, J. and Dionísio, R. (2023), ‘Using smart traffic lights to reduce co emissions and improve traffic flow at intersections: Simulation of an intersection in a small portuguese city’, *Applied System Innovation* .
- Sattler, F., Wiedemann, S., Müller, K.-R. and Samek, W. (2019), ‘Clustered federated learning: Model-agnostic distributed multi-task optimization under privacy constraints’, *arXiv preprint arXiv:1910.01991* .
- Shakya, D., Deshpande, V., Goyal, M. K. and Agarwal, M. (2023), ‘Pm_{2.5} air pollution prediction through deep learning using meteorological, vehicular, and emission data: A case study of new delhi, india’, *Journal of Cleaner Production* **427**, 139278.
- Shang, R. and Zhang (2021), ‘Analyzing the effects of road type and rainy weather on fuel consumption and emissions: A mesoscopic model based on big traffic data’, *IEEE Access* **9**, 62298–62315.
- Shi, J. and Tomasi, C. (1994), Good features to track, *in* ‘1994 Proceedings of IEEE Conference on Computer Vision and Pattern Recognition’, Seattle, WA, USA, pp. 593–600.
- Simić, I., Lovrić, M., Godec, R., Kröll, M. and Bešlić, I. (2020), ‘Applying machine learning methods to better understand, model and estimate mass concentrations of traffic-related pollutants at a typical street canyon’, *Environmental Pollution* **263**, 114587.

- Singh, M. and Dubey, R. (2021), ‘Deep learning model-based co₂ emissions prediction using vehicle telematics sensors data’, *IEEE Transactions on Intelligent Vehicles* .
- Singh, M. and Dubey, R. K. (2023), ‘Deep learning model based co₂ emissions prediction using vehicle telematics sensors data’, *IEEE Transactions on Intelligent Vehicles* **8**(1), 768–777.
- Sochor, J., Juranek, R., Špaňhel, J., Maršík, L., Šíroký, A., Herout, A. and Zemčík, P. (2019), ‘Comprehensive data set for automatic single camera visual speed measurement’, *IEEE Transactions on Intelligent Transportation Systems* **20**(5), 1633–1643.
- Sochor, J., Juránek, R. and Herout, A. (2017), ‘Traffic surveillance camera calibration by 3d model bounding box alignment for accurate vehicle speed measurement’, *Computer Vision and Image Understanding* **161**, 87–98.
- Sochor, J., Špaňhel, J. and Herout, A. (2019), ‘Boxcars: Improving fine-grained recognition of vehicles using 3-d bounding boxes in traffic surveillance’, *IEEE Transactions on Intelligent Transportation Systems* **20**(1), 97–108.
- Tan, M. and Le, Q. V. (2019), ‘Efficientnet: Rethinking model scaling for convolutional neural networks’, *arXiv preprint arXiv:1905.11946* .
URL: <https://arxiv.org/abs/1905.11946>
- Tang, J., Gong, R., Wang, H. and Liu, Y. (2023), ‘Scenario analysis of transportation carbon emissions in China based on machine learning and deep neural network models’, *Environmental Research Letters* **18**(6), 064018.
- Tena-Gago, D., Golcarenenji, G., Martinez-Alpiste, I., Wang, Q. and Alcaraz-Calero, J. M. (2023), ‘Machine-learning-based carbon dioxide concentration prediction for hybrid vehicles’, *Sensors* **23**(3), 1350.
- Torres Guin, W., Sánchez Aquino, J., Bustos Gaibor, S. and Coronel Suarez, M. (2024), ‘Iot architecture for vehicle pollutant gas emission monitoring and validation through machine learning’, *Ingenius* (32), n/a.
- Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A. and Jégou, H. (2021), Training data-efficient image transformers & distillation through attention, in M. Meila and T. Zhang, eds, ‘Proceedings of the 38th International Conference on Machine Learning (ICML)’, Vol. 139 of *Proceedings of Machine Learning Research*, PMLR, pp. 10347–10357.
- United Nations Environment Programme (2024), Emissions gap report 2024: No more hot air ... please! with a massive gap between rhetoric and reality, countries draft new climate commitments, Technical report, UNEP, Nairobi.
URL: <https://wedocs.unep.org/20.500.11822/46404>
- Varghese, R. and M., S. (2024), Yolov8: A novel object detection algorithm with enhanced performance and robustness, in ‘2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS)’, Chennai, India, pp. 1–6.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. and Polosukhin, I. (2017), Attention is all you need, in ‘NeurIPS’.
URL: <https://arxiv.org/abs/1706.03762>
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P. and Bengio, Y. (2018), Graph attention networks, in ‘International Conference on Learning Representations (ICLR)’.

- Von Gioi, R. G., Jakubowicz, J., Morel, J.-M. and Randall, G. (2012), ‘Lsd: A line segment detector’, *Image Processing On Line* **2**, 35–55.
- Vu, H. T. T. and Ko, J. (2024), ‘Effective modeling of CO₂ emissions for light-duty vehicles: Linear and non-linear models with feature selection’, *Energies* **17**(7), 1655.
- Wen, H. T., Lu, J. H. and Jhang, D. S. (2021), ‘Features importance analysis of diesel vehicles’ nox and co₂ emission predictions in real road driving based on gradient boosting regression model’, *International Journal of Environmental Research and Public Health* **18**(24), 13044.
- Wikipedia contributors (n.d.), ‘Long short-term memory’.
URL: <https://en.wikipedia.org/wiki/Longshort-termmemory>
- Wojke, N., Bewley, A. and Paulus, D. (2017), Simple online and realtime tracking with a deep association metric, in ‘2017 IEEE International Conference on Image Processing (ICIP)’, IEEE, pp. 3645–3649.
- Wolpert, D. H. (1992), ‘Stacked generalization’, *Neural Networks* **5**(2), 241–259.
- Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C. and Yu, P. S. (2019), Graph wavenet for deep spatial-temporal graph modeling, in ‘Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)’, pp. 1907–1913.
- Xie, H., Zhang, Y., He, Y., You, K., Fan, B., Yu, D., Lei, B. and Zhang, W. (2021), ‘Parallel attention-based LSTM for building a prediction model of vehicle emissions using PEMS and OBD’, *Measurement* **185**, 110074.
- Xie, Y., Liu, L., Han, Z. and Zhang, J. (2024), ‘Mscl-attention: A multi-scale convolutional long short-term memory (lstm) attention network for predicting CO₂ emissions from vehicles’, *Sustainability* **16**(19), 8547.
- Xu, Z., Wang, R., Kang, Y., Zhang, Y., Xia, X. and Wang, R. (2021), ‘A deep transfer nox emission inversion model of diesel vehicles with multisource external influence’, *Journal of Advanced Transportation* **2021**(1), 4892855.
- Yang, J., Kang, S., Chen, D. et al. (2025), ‘Reduced solid water storage over the Tibetan Plateau caused by black carbon’, *Communications Earth & Environment* **6**, 430.
- Yavari, A., Mirza, I. B., Bagha, H., Korala, H., Dia, H., Scifleet, P. and Shafiei, M. (2023), ‘Artemon: Artificial intelligence and internet of things powered greenhouse gas sensing for real-time emissions monitoring’, *Sensors* **23**(18), 7971.
- Yu, B., Yin, H. and Zhu, Z. (2018), Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting, in ‘Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)’, AAAI Press, pp. 3634–3640.
- Zeng, J. and Liu (2024), Estimating on-road transportation carbon emissions from open data of road network and origin-destination flow data, in ‘Proceedings of the AAAI Conference on Artificial Intelligence’, Vol. 38, pp. 22493–22501.
- Zeng, L., Wang, S., Zhang, L., Wang, X., Zheng, Y., Zhang, J., Zhang, X., Lin, Y., Liu, C. and Lin, F. (2024), ‘Estimating on-road transportation carbon emissions from open data of road network and origin-destination flow data’, *arXiv preprint arXiv:2402.05153*.
URL: <https://arxiv.org/abs/2402.05153>

- Zhang, R., Chen, H., Xie, P., Zu, L., Wei, Y., Wang, M. and Zhu, R. (2023), ‘Exhaust emissions from gasoline vehicles with different fuel detergency and the prediction model using deep learning’, *Sensors* **23**(17), 7655.
- Zhang, R. and Y. (2022), ‘A deep learning micro-scale model to estimate the co₂ emissions from light-duty diesel trucks based on real-world driving’, *Atmosphere* **13**(9), 1466.
- Zhang, Y., Sun, P., Jiang, Y., Yu, D., Yuan, Z., Luo, P., Liu, W. and Wang, X. (2021), ‘Bytetrack: Multi-object tracking by associating every detection box’, *arXiv preprint arXiv:2110.06864* .
URL: <https://arxiv.org/abs/2110.06864>
- Zhang, Y., Wang, C., Wang, X., Zeng, W. and Liu, W. (2020), ‘Fairmot: On the fairness of detection and re-identification in multiple object tracking’, *arXiv preprint arXiv:2004.01888* .
URL: <https://arxiv.org/abs/2004.01888>
- Zhang, Z. (2000), ‘A flexible new technique for camera calibration’, *IEEE Transactions on Pattern Analysis and Machine Intelligence* **22**(11), 1330–1334.
- Zheng, C., Fan, X., Wang, C. and Qi, J. (2020), Gman: A graph multi-attention network for traffic prediction, in ‘Proceedings of the AAAI Conference on Artificial Intelligence’, Vol. 34, pp. 1234–1241.

Appendix A

Vision and Geometry Essentials for Traffic Video Analysis

A.1 Introduction

The core of our proposed system is a pipeline that extracts meaningful, metric information from raw video streams. To fully appreciate the design and evaluation of this pipeline, a foundational understanding of several key concepts is essential. This appendix serves as a self-contained guide to these principles.

We will begin by presenting the primary computer vision tasks that form the application layer of our system, such as object detection, classification, and multi-object tracking. Subsequently, we will delve into the fundamental principles of camera geometry. Since video is our main source of data, understanding the characteristics of the image formation process—including the pinhole camera model, projective geometry, and perspective correction—is crucial for translating pixel-level observations into the accurate, real-world measurements that our project requires.

A.2 Computer vision tasks

Computer vision, as an area of artificial intelligence, enables computers to interpret images and video, turning raw pixels into meaningful, measurable information. In the following sections, we will cover the core computer vision tasks relevant to our work, namely image classification, object detection, and object tracking.

A.2.1 Image Classification

Image classification is the task of assigning a single label to an entire image (e.g., *car*, *truck*). In the modern deep-learning era, it became the flagship benchmark that drove many advances in computer vision, and it remains the simplest entry point for extracting fleet composition from traffic images.

A.2.1.1 Standard CNN Layout

A typical classifier has three named parts:

- **Backbone** : a convolutional network that converts raw pixels into a multi-scale feature pyramid on which task-specific heads operate using a stack of convolutional layers and non-linearities.

- **Neck** : a small feature aggregator; for classification this is often just *global average pooling* (or attention pooling) that reduces the spatial map to a single vector.
- **Head** : a lightweight predictor (usually a linear or small MLP layer) that maps the vector to class scores, followed by softmax.

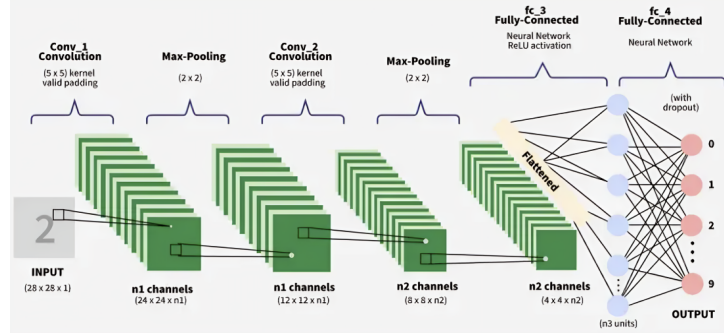


Figure A.1: Example of a simple CNN classification architecture (GeeksforGeeks, 2024) .

Table A.1 summarises the families of backbones encountered most often in traffic-related computer-vision works.

Table A.1: Popular CNN backbones in computer-vision pipelines

Architecture	Key idea(s)	Typical usage / strength
ResNet (He et al., 2015)	Deep residual learning: identity skip-connections ease training of very deep stacks.	Default backbone in Faster/Mask R-CNN, RetinaNet, DeepSORT. Strong accuracy with moderate compute.
MobileNet (Sandler et al., 2018)	Depthwise separable convolutions and inverted bottlenecks for mobile efficiency.	Lightweight one-stage detectors (e.g., MobileNet-SSD, YOLO-Nano) and embedded MOT pipelines.
EfficientNet (Tan and Le, 2019)	<i>Compound scaling</i> : jointly scales depth, width, and resolution for Pareto-optimal accuracy vs. FLOPs.	High-accuracy detector backbones where GPU memory is limited; good balance for medium-speed inference.
DenseNet (Huang et al., 2016)	Dense connectivity: each layer receives all preceding feature maps, improving gradient flow and feature reuse.	Enhances small-object sensitivity in detectors and segmentation nets; often used where fine detail matters.

A.2.2 The Vision Transformer (ViT).

The Vision Transformer (ViT) architecture offers an alternative to convolutions by applying self-attention over a sequence of image patches. The process begins with **patch embedding**, where the image is split into N non-overlapping $P \times P$ patches. Each patch is flattened and linearly projected to a d -dimensional token, producing a sequence $\mathbf{Z}_0 \in \mathbb{R}^{N \times d}$. To this sequence, a special classification token, $[\text{CLS}]$, is prepended. Because the model needs to know where each patch came from, learnable or sinusoidal **positional embeddings** $\mathbf{E}_{\text{pos}}$ are added to the tokens.

The core of the ViT consists of **Encoder blocks**, which are repeated L times. Each block contains two main sublayers with residual connections. The first is *Multi-Head Self-Attention (MHSA)*, where queries, keys, and values are computed from the tokens, and attention weights

allow each patch to attend to all others, capturing global context. The second is a simple two-layer *MLP* feed-forward network, often with a GELU activation, which is applied token-wise. Layer Normalization is used before these sublayers, a common practice known as *pre-norm*. Finally, for the **classification head**, the final state of the [CLS] token is passed to a linear layer to produce the class logits.

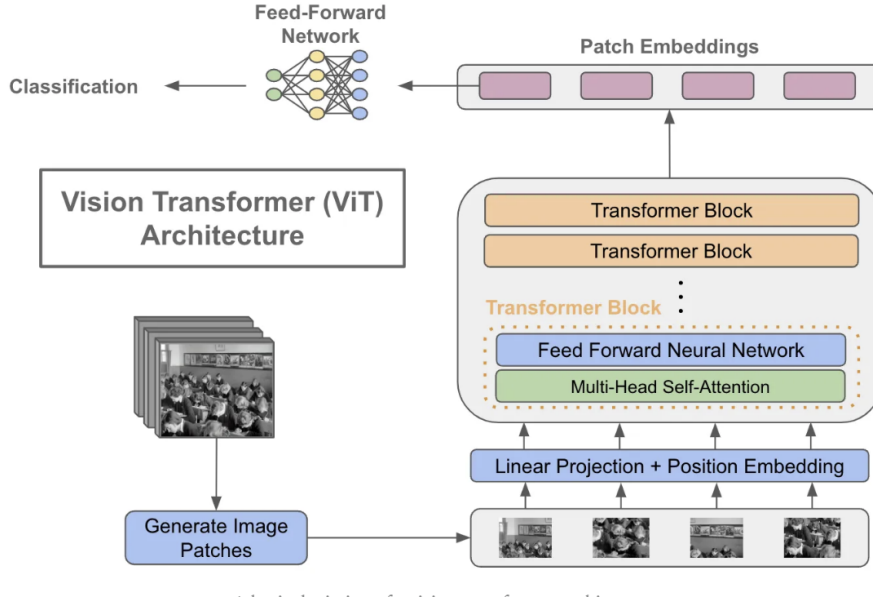


Figure A.2: A simple depiction of a vision transformer architecture.

A.2.2.1 Outputs and Evaluation Metrics.

The output of a classifier is a prediction, $\hat{y}$, for a given ground-truth class, y . To evaluate performance, we use several standard metrics. **Top-1 accuracy** measures the fraction of samples where $\hat{y}$ exactly matches y . **Top- k accuracy** is a more lenient metric that counts a prediction as correct if y appears among the k classes with the highest scores. For a more detailed view, especially under class imbalance, we use per-class precision, recall, and the F1-score, defined as:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}, \quad \text{F1}_c = \frac{2 \text{Precision}_c \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}.$$

A.2.3 Object Detection

Moving beyond classifying the whole image, Object Detection aims to localize and classify individual object instances within it. For each detected object, the model outputs a tight, axis-aligned bounding box (bbox), a class label, and a confidence score this is illustrated in figure A.3, which shows the output of a detection and classification task run on a traffic image using the YOLOv5 model. Bounding boxes are typically parameterized by their corner coordinates or by a center point with width and height.

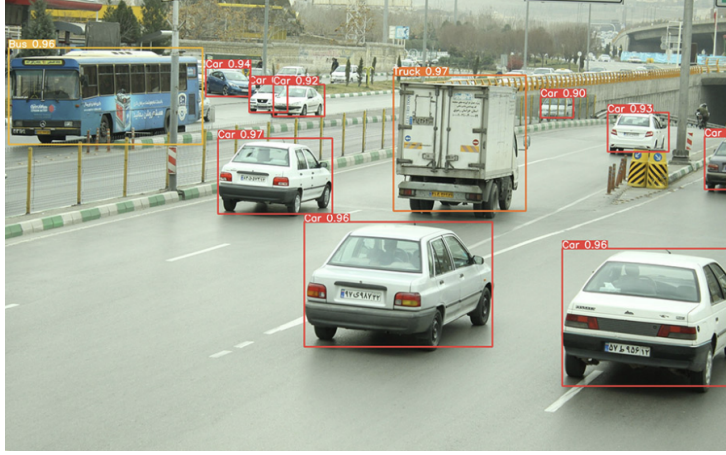


Figure A.3: Example of detection on a traffic image (YOLOv5 output)(GeeksforGeeks, 2024).

A.2.3.1 Model Architectures:

The architectural design of a detector dictates its approach to finding objects. The two main philosophies are:

- **Two-Stage Models:** These models, such as the R-CNN (Girshick et al., 2014) family (including Fast R-CNN (Girshick, 2015) and Faster R-CNN (Ren et al., 2015)), break detection into a sequence of two distinct steps. This approach is often highly accurate but can be computationally slower. The process is as follows:
 1. *Region Proposal:* In the first stage, the network scans the image and proposes a set of candidate regions—areas that are likely to contain an object.
 2. *Classification and Refinement:* In the second stage, each proposed region is individually classified (e.g., as "car," "truck," or "background"), and the coordinates of the bounding box are refined to fit the object more precisely.
- **One-Stage Models:** In contrast, models like YOLO (You Only Look Once) (Redmon et al., 2016) and SSD (Single Shot MultiBox Detector) (Liu et al., 2016) treat object detection as a single, unified regression problem. They skip the region proposal step and instead predict bounding boxes and class probabilities directly from the full image in a single forward pass. This makes them significantly faster and ideal for real-time applications.

Regardless of the architecture, a detector's raw output often includes multiple, overlapping bounding boxes for the same object. To clean this up, a crucial post-processing step is required before the model's performance can be evaluated and most models apply the Non-Maximum Suppression (NMS): NMS retains the highest-score box and suppresses others whose IoU (Intersection Over Union) with it exceeds a threshold.

A.2.3.2 Outputs and evaluation.

We evaluate detectors with *overlap-based* and *precision-recall* metrics.

Intersection-over-Union (IoU). For a predicted box B and ground-truth box G ,

$$\text{IoU}(B, G) = \frac{|B \cap G|}{|B \cup G|} \in [0, 1].$$

A detection is a *true positive (TP)* if its class is correct and $\text{IoU} \geq \tau$ (e.g., $\tau = 0.5$); otherwise it is a *false positive (FP)*. Ground-truth boxes not matched by any prediction are *false negatives (FN)*.

Average Precision (AP). Sorting predictions by confidence yields a precision–recall (PR) curve; AP is the area under this curve for one class at a fixed IoU (e.g., AP@0.5).

Mean AP (mAP). Mean of AP across classes. The *COCO* metric mAP@[.5:.95] averages AP over IoU thresholds 0.50, 0.55, ..., 0.95, rewarding both correct classification and precise localization.

We use mAP@0.5 for an intuitive score and mAP@[.5:.95] for stricter localization, plus per-class precision/recall to show class imbalance effects. We also state the score and IoU thresholds used for NMS (e.g., $\text{conf}=0.25$, $\text{IoU}=0.5$).

A.2.4 Multi-Object Tracking (MOT)

While the previous vision tasks operate on static images, Multi-Object Tracking (MOT) adds the critical dimension of time. Its goal is to go beyond single-frame detections and create a cohesive narrative of movement by linking detections of the same object across a video sequence. This process assigns a persistent identity (ID) to each object, resulting in a set of complete trajectories.

The dominant approach for this task is the tracking-by-detection paradigm. Instead of trying to find and track objects simultaneously, it separates the problem into a continuous loop that cycles through three main steps for each new frame of a video.

1. **Predict:** For every object currently being tracked, a motion model is used to predict its new position in the next frame. A Kalman filter is commonly used for this step. It’s an optimal estimation algorithm that can predict a future state based on past measurements, effectively modeling the object’s momentum and smoothing out noisy detections.
2. **Associate:** Next, the system tries to match the new detections in the current frame with the predicted locations of existing tracks. This is an assignment problem, typically solved with the Hungarian algorithm. The “cost” of matching a detection to a track is calculated based on factors like:
 - *Geometric Proximity:* How close are the predicted and detected bounding boxes are, This is often measured using IoU.
 - *Appearance Similarity:* For more advanced trackers, a small CNN extracts an “appearance embedding” for each object. The similarity between these embeddings provides a powerful cue to re-identify an object even after it has been occluded or has moved unpredictably.
3. **Manage Tracks:** Finally, the tracker updates the status of all tracks. Matched tracks are updated with their new positions. New tracks are created for detections that couldn’t be associated with any existing track. And tracks that are consistently not found for several frames are terminated.

We have included a few multi-object tracking methods that are very widely used in the literature in table A.2

Table A.2: A few multi-object trackers

Tracker	Mode	Brief description
SORT (Bewley et al., 2016)	Online	Very lightweight: constant-velocity Kalman filter for motion; IoU + Hungarian for association. Fast but brittle under occlusion.
DeepSORT (Wojke et al., 2017)	Online	Extends SORT with a CNN-based appearance embedding to maintain IDs through overlaps and re-entries.
ByteTrack (Zhang et al., 2021)	Online	Two-stage association: link high-confidence detections first, then attach low-confidence boxes to boost recall without heavy re-ID; real-time friendly.
FairMOT (Zhang et al., 2020)	Online	Single backbone jointly predicts detection heatmaps and re-ID features, balancing precision and identity consistency.

A.2.5 Additional Foundational Algorithms

In this section, we provide concise descriptions of several foundational computer vision algorithms that are instrumental to our method.

A.2.5.1 Canny Edge Detection

The Canny algorithm detects object boundaries in five stages: (1) smooth the image with a Gaussian filter to suppress noise; (2) compute intensity gradients (e.g., with a Sobel filter) to get edge magnitude and direction; (3) apply non-maximum suppression to thin edges by keeping only local maxima along the gradient direction; (4) use a double threshold to mark strong versus weak edge pixels; and (5) perform hysteresis tracking to connect weak pixels to strong ones if they form a contiguous edge path. The result is a clean, one-pixel-wide edge map that is robust to noise, though its quality depends on the blur scale and the two thresholds.

A.2.5.2 Shi–Tomasi Corner Detection

The Shi–Tomasi method finds ”good features to track” by examining the structure tensor, M , in a small window around each pixel:

$$M = \sum_{(x,y) \in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

Let λ_1 and λ_2 be the eigenvalues of M . Shi–Tomasi uses the score $R = \min(\lambda_1, \lambda_2)$ and accepts points where R is above a threshold. Corners, which have high texture in two directions, yield two large eigenvalues, making them stable for tracking. Edges yield one large and one small eigenvalue, while flat regions yield two small eigenvalues.

A.2.5.3 Line Segment Detector (LSD)

LSD is an algorithm designed to extract straight line segments from an image without requiring significant parameter tuning. It operates by grouping pixels with a nearly constant gradient direction into thin, rectangular regions. It then fits a line to each region and validates it with an a-contrario test (based on the Number of False Alarms) to keep only those segments that are statistically very unlikely to have arisen from noise. The algorithm runs in linear time, produces sub-pixel accurate endpoints, and performs particularly well on man-made scenes with many straight boundaries, such as roads, curbs, and building facades.

A.2.5.4 Lucas–Kanade (LK) Optical Flow

Lucas–Kanade estimates how a small image patch moves between two consecutive frames. It assumes (i) the patch keeps the same appearance (brightness constancy), (ii) the motion between frames is small, and (iii) all pixels in the patch move roughly the same way. Practically, LK looks at how pixel intensities change across the patch (spatial gradients) and from one frame to the next (temporal change), then finds the 2D shift (u, v) that best explains these changes in a least-squares sense. The estimate is reliable only if the patch has texture in two directions (e.g., a corner); otherwise the aperture problem makes motion ambiguous. A coarse-to-fine image pyramid is often used so the method can cope with larger motions. In our context, LK is typically applied to a sparse set of good features (e.g., Shi–Tomasi corners) to track them over time.

A.3 Camera geometry

A single perspective image is not a metric map. Parallel lines appear to meet, angles and lengths are distorted, and even ratios of lengths are not preserved. From pixels alone, we can recover directions and incidence (which line meets which) but not absolute distances or speeds. Any measurement pipeline therefore needs geometry that handles “parallel meets at infinity” consistently and a calibration step that restores metric meaning.

A.3.1 Camera Intrinsics and Characteristics

To understand how a camera translates the 3D world into a 2D image, we use the pinhole camera model. This model simplifies the complex optics of a camera into two fundamental sets of parameters: the intrinsic parameters, which describe the camera’s internal properties, and the extrinsic parameters, which describe its position and orientation in the world. This model is illustrated in figure A.4, in which we can see a world point X project along the ray through the camera center C onto the image plane at pixel $q = (u, v)$, and the camera parameters.

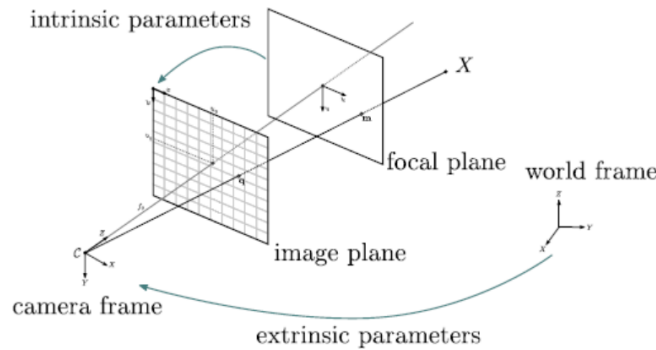


Figure A.4: Representative illustration of a pinhole camera model(openMVG, n.d.).

A.3.1.1 Intrinsic Parameters: The Camera’s Internal Fingerprint.

The *intrinsic* parameters describe the unique, internal characteristics of the camera and lens assembly. They define how the camera forms an image, independent of where it is placed in the world. One can think of them as the camera’s fixed “factory settings” (unless the lens or zoom are changed). These properties are bundled into a 3×3 matrix called the calibration matrix, K :

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

The elements of this matrix have specific meanings:

- **Focal Length (f_x, f_y):** These values represent the "zoom" of the camera, measured in units of pixels. They determine the field of view; a larger focal length results in a narrower, more zoomed-in view. We have two values to account for the possibility of non-square pixels on the sensor, but for most modern cameras, $f_x \approx f_y$.
- **Principal Point (c_x, c_y):** This is the true optical center of the image in pixel coordinates. It's the point where the lens's central axis intersects the sensor. While one might assume this is the exact center of the image, it's often slightly offset due to manufacturing tolerances.
- **Skew (s):** The skew parameter models any non-orthogonality between the x and y axes of the pixel sensor. For virtually all modern digital cameras, the pixels are perfectly rectangular, so this value is assumed to be zero.

Knowing these intrinsic parameters is the crucial first step in any measurement task. They allow us to correct for the inherent properties of the camera's lens and sensor before we can begin to reason about the 3D world it has captured.

A.3.1.2 Extrinsic Parameters: The Camera's Place in the World.

While intrinsics describe **how** the camera sees, the *extrinsic* parameters describe **from where** it sees. They define the camera's pose—its position and orientation—relative to a fixed world coordinate system. This is what you change when you physically move or rotate the camera. The extrinsics consist of two parts:

- **Rotation (R):** A 3×3 matrix that describes the camera's orientation (e.g., its pan, tilt, and roll).
- **Translation (t):** A 3×1 vector that specifies the camera's physical position in 3D space.

A.3.1.3 The Full Projection Matrix: From World to Pixels.

By combining both intrinsic and extrinsic parameters, we can form the complete camera **projection matrix**, P . This 3×4 matrix is the full recipe for mapping any 3D point in the world to its corresponding 2D pixel location in the image:

$$P = K [R \mid t]$$

This equation elegantly describes the entire imaging process. A 3D point is first transformed from the world's coordinate system into the camera's coordinate system using the rotation and translation ($[R \mid t]$), and then this camera-centric view is projected onto the 2D sensor plane using the camera's internal properties (K).

A.3.2 Perspective Correction: Restoring Metric Scale

As established, a raw camera image is not a metric map. To perform any real-world measurement, we must first correct for the distortions of perspective. This process, often called camera calibration, is a central problem that our pipeline must solve.

A variety of techniques exist for this purpose, ranging from highly manual to nearly automatic. Below, we review the most common approaches and evaluate their suitability for our scenario.

A.3.2.1 Calibration Techniques

Table A.3: Common approaches to calibrate traffic cameras.

Approach (refs)	Core idea(s)	Notes (inputs / pros / limits)
DLT (Direct Linear Transform) (Abdel-Aziz and Karara, 1971)	Map 3D→2D via a linear projection matrix solved by least squares.	Inputs: ≥ 4 known 3D–2D correspondences. Pros: Fundamental, simple, general. Limits: Manual point picking; time-consuming; not scalable to city-wide CCTV.
Zhang’s planar method (Zhang, 2000)	Multi-view planar pattern (checkerboard) to estimate intrinsics/extrinsics and distortion.	Inputs: Printed checkerboard in several poses. Pros: Accurate; standard; well-tooled. Limits: Impractical for surveillance networks; drift over time; large FOV needs big boards.
Vanishing-point based (Caprile and Torre, 1990)	Parallel line families $\Rightarrow$ vanishing points; constrain horizon/orientation; derive road-plane homography.	Inputs: Sufficient straight-line structure; robust line grouping (Hough/RANSAC). Pros: No physical pattern; scene-driven; automatable. Limits: No absolute translation/scale from VPs alone; brittle in clutter/noise.
Deep-learning based (Bogdan et al., 2018)	CNN regresses calibration parameters directly from image content (often with horizon/VP priors).	Inputs: Trained model; diverse labeled data/domain coverage. Pros: Fully automatic; adapts to complex scenes. Limits: Data-hungry; harder to validate; possible “black-box” failure modes.

Given the constraints and goals of our project, deploying a scalable and automated solution is paramount. Manual methods like DLT and Zhang’s are therefore impractical. While deep learning is a promising direction, it introduces significant complexity and data requirements.

Therefore, we have chosen to focus our implementation on the **vanishing point-based method**. Despite its challenges, it offers the best trade-off for our needs: a powerful, geometry-grounded approach with a clear path toward automating the calibration process for existing, uncalibrated roadside cameras. The following sections will give the needed background definitions to understand our implementation.

A.3.2.2 Foundational Geometric Concepts for Calibration

To understand how we can automate camera calibration using scene geometry, it is essential to first review the mathematical framework that makes it possible. This section introduces the core principles of projective geometry, homogeneous coordinates, and vanishing points, and explains how they apply to traffic scenes under the Manhattan-world assumption.

A.3.2.2.1 Projective Geometry and Homogeneous Coordinates In standard Euclidean geometry, parallel lines never intersect. While true in theory, this creates practical and algebraic challenges in computer vision when modeling perspective projection, where parallel lines in the world visibly converge in an image. Projective geometry extends Euclidean geometry to elegantly handle this, providing a unified framework where all parallel lines do meet at a well-defined "point at infinity."

The practical tool for this is **homogeneous coordinates**. In this system, a 2D Cartesian point (x, y) is represented by a 3-component vector $(x, y, 1)$. More generally, any triplet (kx, ky, kw) represents the same point for any non-zero scalar k . To convert back to Cartesian coordinates, we simply divide the first two components by the third, w .

This representation allows us to define points at infinity.

- If $w \neq 0$, the vector corresponds to a **finite point** in the Cartesian plane.
- If $w = 0$, the vector corresponds to a **point at infinity**, also known as an **ideal point**.

This formalism is incredibly powerful because it allows us to treat all points—finite or infinite—within the same algebraic system. For instance, the intersection of any two lines (represented as 3-component vectors) can be computed using the vector cross product. When this is applied to two parallel lines, the resulting intersection point is a homogeneous vector with its third component $w = 0$. This elegantly confirms that parallel lines meet at a point at infinity, resolving the "degenerate" case of Euclidean geometry.

A.3.2.2.2 Vanishing Points: The Image of Infinity The abstract concept of a point at infinity has a very concrete and visible meaning in computer vision: it is a **vanishing point (VP)**.

When a 3D scene is projected onto a 2D image by a camera, all the parallel lines in the 3D world that share the same direction appear to converge towards a single point in the image. This point of convergence is precisely the vanishing point. It is, in projective terms, the **image of the ideal point** associated with that family of parallel lines. This concept is simply illustrated in figure A.5.

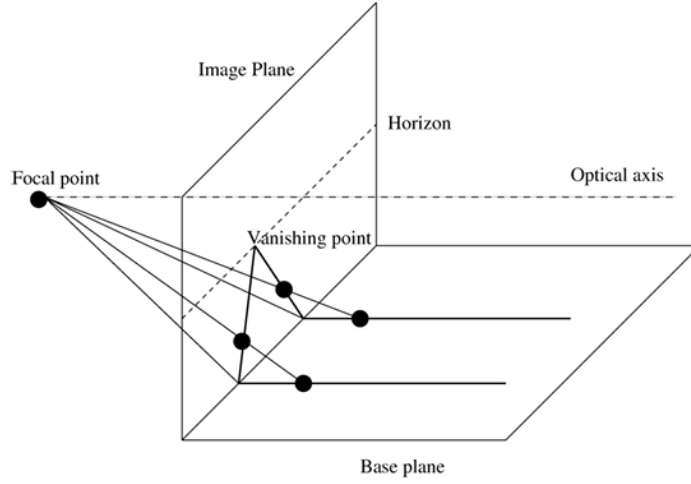


Figure A.5: illustrative figure of a vanishing point on the image plane vs. the base plane (world plane)) (Mian, n.d.).

A.3.2.2.3 The Manhattan-World Assumption for Roads General scenes can be geometrically complex, but many urban environments, including traffic scenes, adhere to a simplifying principle known as the **Manhattan-world assumption**. This assumption posits that the scene is dominated by three mutually orthogonal dominant directions. For a typical road scene, these are:

- A direction $\mathbf{u}$ aligned with the flow of traffic.
- A direction $\mathbf{v}$ running across the lanes, perpendicular to traffic.
- A vertical direction $\mathbf{w}$.

Under perspective projection, these three orthogonal directions give rise to three corresponding vanishing points in the image. The two vanishing points associated with the road plane, u and v , conveniently define the **horizon line** in the image, which is the projection of the line at infinity of the ground plane.

A.3.2.2.4 Application to Perspective Correction These geometric principles directly enable our calibration and measurement pipeline. By identifying two orthogonal vanishing points on the road plane (u and v), we can locate the horizon line. This information powerfully constrains the camera's orientation relative to the road. With this orientation known, we can compute a projective transformation, known as a **homography** (H), that can "un-distort" the perspective view of the road into a metrically-correct, top-down, bird's-eye view.

A.4 Conclusion

In this appendix, we have reviewed the foundational concepts that underpin our entire solution. We began by outlining the high-level computer vision tasks—classification and tracking—that define what we aim to achieve with video data. We then explored the essential principles of camera and projective geometry, which provide the mathematical framework required to perform accurate metric measurements from a 2D image.

These concepts are not merely theoretical background; they are the fundamental building blocks upon which our practical pipeline is designed, implemented, and evaluated. A clear grasp

of these principles is therefore key to understanding the methodological choices and results presented in the main body of this report.